GoodGraph

Методичка по научной графике на Python

Принципы · нормативы · примеры «плохо/хорошо»

Версия 0.1 · ноябрь 2026

Содержание

Раздел 1. Принципы качественной научной графики 3
Раздел 2. Python-инструментарий и настройка среды 15
Раздел 3. Требования к научной графике: цвет, шрифты, размеры и форматы 28
Раздел 4. Основные виды научных графиков и области применения 37
5.01 Линейный график: динамика во времени 43
5.02 Ступенчатый график: события и дискретные изменения 47
5.03 Area chart: площадь как величина 51
5.04 Small multiples: много панелей, одна логика 55
5.05 Scatter: связь двух непрерывных величин 59
5.06 Hexbin: плотность тысяч точек 63
5.07 Contour: изолинии плотности 67
5.08 LOESS: нелинейный тренд 71
5.09 Гистограмма: распределение одной величины 75
5.10 ECDF: функция распределения 79
5.11 Boxplot: сравнение распределений по группам 83
5.12 Violin: форма распределения 88
5.13 Strip: все точки по группам 92
5.14 Dot + CI: оценка и неопределённость 96
5.15 Bar: сравнение величин 100
5.16 Forest: эффект по подгруппам 104
5.17 Slope: изменение между двумя точками 108
5.18 Paired dot: индивидуальные изменения 112
5.19 Bland–Altman: согласие методов 116
5.20 Stacked bar: состав целого 120
5.21 Bar 100 %: нормированный состав 124
5.22 Pie: части целого 127
5.23 Treemap: иерархический состав 130
5.24 Heatmap: матрица значений 133
5.25 Clustermap: heatmap с кластеризацией 135
5.26 Corr matrix: корреляционная матрица 138
5.27 Kaplan–Meier: выживаемость 140
5.28 Cumulative incidence: конкурирующие риски 142
5.29 ROC: чувствительность и специфичность 144
5.30 Precision–recall: редкий исход 146
5.31 Calibration: соответствие прогноза и реальности 148
5.32 Caterpillar: упорядоченные оценки 150
5.33 Funnel: асимметрия и малые исследования 152
6. Заключение 154
7. Публикация 159
8. Источники 164
9. Приложения 169

Раздел 1. Принципы качественной научной графики

График в научной работе — не украшение и не «картинка к тексту». Это способ рассуждения о данных, их представления и интерпретации: читатель должен за несколько секунд увидеть закономерность, оценить её величину и понять, насколько ей можно доверять. Поэтому разговор о графике начинается не с библиотек и не с цветов, а с вопроса: **что именно читатель должен понять и какое решение принять?**

Что является главным при создании графиков? Сначала идут принципы, соблюдение которых обеспечивает смысл графика:

- график должен быть правдивым (честным, fair);
- необходимо, чтобы график погружал читателя в контекст исследования;
- график должен быть понятным и не создавать ложной неопределённости;
- должна быть выдержана иерархия объектов графика.

Лишь затем следуют нормативы и оформление: размеры, шрифты, цвет, единицы, подписи. Они носят подчинённый характер и служат главной цели: **чтобы читатель мог понять, что и как он должен делать с графиком**.

В этой методичке мы постараемся привести как неудачные примеры графики, так и удачные с комментариями: что и почему изменилось.

> В примерах все данные синтетические. Они правдоподобны по единицам и величинам, но не описывают реальных пациентов.

1.1 «Правильный» и «качественный»: два разных требования

Полезно развести два прилагательных, которые часто используют как синонимы.

Правильный график (valid, truthful) — тот, который не искажает данные и не нарушает нормы:

- масштаб осей соответствует типу кодирования;
- статистика показана корректно (есть *n*, разброс, определение ошибок);
- единицы измерения указаны;
- подпись самодостаточна;
- рисунок соответствует требованиям журнала или ГОСТ.

Качественный график (effective, craft) — тот, который решает задачу читателя:

- сообщение считывается за несколько секунд;
- визуальная иерархия ведёт взгляд от главного к второстепенному;
- палитра доступна при дальтонизме и в чёрно-белой печати;
- шрифты и размеры комфортны читателю в конечном физическом размере;
- отдельный график согласован с остальными рисунками работы;
- результат воспроизводим по данным и коду.

Эти требования независимы. График может быть правильным, но бесполезным: аккуратные столбики с *n*, SD и единицами, по которым невозможно ничего сравнить.

И наоборот: эффектный график с градиентами и тенями может быть красивым, но лживым.

Методичка — о том, как выдержать оба требования одновременно.

Главное. График проектируется от вопроса читателя и структуры данных, а не от возможностей библиотеки.

1.2 Смысловые принципы

1.2.1 Сначала сообщение, потом график

Перед первой строкой кода сформулируйте одно предложение:

> «После этого графика читатель должен понять, что препарат В снижает САД примерно вдвое сильнее плацебо, и что разброс между пациентами велик».

Если предложение не формулируется, график, скорее всего, не нужен: данные лучше показать таблицей. Если предложений два, это два графика или две панели A и B.

Из этого следуют практические правила:

- **Заголовок — вывод, а не тема.** «Результаты исследования» сообщает ноль информации. «Препарат В снижает САД вдвое сильнее плацебо» — сообщает вывод и заодно объясняет, что читать на графике.
- **Один график — одна мысль.** Если хочется показать и динамику, и сравнение групп, и корреляцию, — эти три мысли можно показать на трёх панелях, а не в одном комбинированном графике.
- **Главное — крупнее и контрастнее.** Основная серия или эффект должны визуально доминировать; вспомогательные — уходить на второй план.
- **Читатель не обязан знать контекст.** График должен работать вместе с подписью, но не вместо неё.

Заметим, что заголовок внутри рисунка — спорная территория. Для слайдов, отчётов и внутренних документов он полезен. Для журнальной публикации многие редакции предпочитают не дублировать заголовок в изображении и в подписи: содержательный вывод переносят в подпись (caption). Проверяйте гайд журнала; но в этой методичке заголовок сопровождает графики как дидактический приём.

1.2.2 Форму определяют данные

Вид графика — не вкусовое решение. Он определяется:

- типом переменных (категориальные, непрерывные, временные, парные);
- числом наблюдений и групп;
- наличием распределения, выбросов, цензурирования;
- задачей: сравнить, показать связь, распределение, динамику, состав.

Минимальная таблица соответствия:

Задача	**Рекомендуемая форма**	**Почему**
Сравнить несколько категорий	точка с доверительным интервалом, полоса, столбцы	положение читается точнее длины
Показать распределение	гистограмма, ECDF, box, violin, strip	виден разброс и форма, а не только среднее
Связь двух непрерывных	диаграмма рассеяния, hexbin, contour	видна форма зависимости и выбросы
Динамика во времени	линия; small multiples для многих серий	время должно идти по оси X
Доля целого	stacked bar, 100 % bar; круговая — только для 3–5 долей	углы и площади читаются хуже длины
Матрица значений	heatmap с ограниченной палитрой	цвет кодирует величину, но требует шкалы
Парные измерения	slope chart, paired dot plot	видно направление изменения у каждого объекта
Оценки и неопределённость	forest plot	компактно показывает эффект и ДИ
Выживаемость	Kaplan–Meier с числом рисков	цензурирование нельзя игнорировать
Диагностическая точность	ROC-кривая, precision–recall	зависит от prevalence
Многомерные наблюдения	small multiples, PCA/biplot (осторожно)	уменьшение размерности меняет интерпретацию

Подробный разбор видов графиков — в разделе 4, примеры с кодом — в разделе 5.

Существует несколько подходов к визуализации данных: один из них носит императивный характер, другой — скорее описывает график как диалог с читателем, то есть является декларативным.

Полезно помнить **«грамматику графики»** (Wilkinson, 1999; реализована, например, в библиотеке ggplot2): данные → эстетические отображения → геометрия → шкалы → грани.

Очень популярная библиотека Matplotlib (Python) работает императивно («нарисуй линию здесь»), ggplot2 и plotnine — декларативно («свяжи x и y, сгруппируй по цвету»).

Оба подхода допустимы; декларативный лучше защищает от несогласованности, императивный — даёт полный контроль над процессом визуализации.

1.2.3 Масштаб

Возможно, самым частым источником визуальной лжи является масштаб, и он же самый простой для исправления.

Полезно запомнить: **если столбцы кодируют длину, то** длина обязана начинаться с нуля. Если ось Y обрезана, читатель сравнивает не значения, а обрезки. Пример из раздела 1.4: средние 4,2 и 9,0 мм рт. ст., ось начинается с 3,5. Визуально препарат B «в 7,5 раза больше» плацебо, хотя в данных разница в 2,1 раза.

Можно даже рассчитать коэффициент искажения (lie factor, Tufte):

> **LF** = (5,46 / 0,73) / (8,96 / 4,23) ≈ 3,5.

Значения LF заметно выше 1 или заметно ниже 1 означают искажение; Tufte предлагал считать допустимым диапазон примерно 0,95–1,05.

Точки и линии кодируют положение. Для них нулевая отметка не обязательна: если весь смысл в изменении на 2–3 единицы, растянутая ось помогает увидеть тренд. Но у этого способа представления есть условия:

- ось явно подписана, диапазон не скрывает важный контекст;
- читателю не навязывают выводы: обрезка оси не маскирует выбросы и не создаёт ложный «обвал» или «взлёт» данных;
- для сравнения нескольких серий используются одинаковые шкалы, если это возможно.

Логарифмическая шкала допустима для величин, которые меняются мультипликативно (титры антител, вирусная нагрузка, концентрации), и только для положительных значений. Она обязана быть подписана: «лог. шкала, основание 10». Логарифм не «улучшает» график — он меняет вопрос, на который тот отвечает, поскольку логарифмы измеряют пропорции (удвоение данных, снижение концентрации в 10 раз и т.п.).

Разрыв оси допустим как крайняя мера для выбросов, с явными метками разрыва. Для столбцов — недопустим: он ломает длину.

Две оси Y на одном графике почти всегда создают ложную корреляцию: две произвольно масштабированные величины всегда можно «подогнать» так, чтобы линии пересеклись в нужном месте. Если без этого нельзя обойтись, кодируйте оси цветом, подписывайте единицы и объясняйте (хотя бы для себя), почему две панели хуже.

Пропорции. Для временных рядов удачный приём — «banking to 45°»: выбирать соотношение сторон так, чтобы типичный наклон тренда был около 45°. Слишком широкий график сглаживает изменения, слишком узкий — драматизирует. Для сравнения наклонов в scatter-plot полезна квадратная область.

1.2.4 Неопределённость, распределение и *n*

Box plot (коробчатый график, «ящик с усами») — это способ компактно показать распределение данных. Джон Тьюки предложил его как компактную «упаковку»: медиану, квартили и выбросы. Но это не универсальный график: при небольшом *n* он может скрывать форму распределения. У столбиков с усами ошибок есть ироничное прозвище — *dynamite plot*: они скрывают и распределение, и число наблюдений. Weissgerber и соавторы (2015) показали, как часто такие графики искажают картину в биомедицинских статьях.

Если вы используете box plot, столбики с усами или похожие сводки, крайне желательно показывать:

- ***n* для каждой группы** — объём выборки, в подписи или на оси.
- **Разброс**: отдельные точки при небольшом *n*, box/violin или strip при большем, SD/CI — как дополнение, а не замена.
- **Определение ошибок**: SD — разброс наблюдений, 95 % ДИ — точность оценки; это разные вещи, которые часто путают.

Обозначение	Что характеризует	Как читать
SD	разброс наблюдений вокруг среднего	«насколько разные пациенты»
SEM	точность выборочного среднего, SD/√*n*	«насколько точно измерено среднее», не разброс
95 % ДИ	неопределённость среднего	при повторных выборках накрывал бы истинное среднее в 95 % случаев

Частая ошибка — подписать «mean ± SEM», а в тексте интерпретировать как «разброс». Ещё чаще — использовать 95 % ДИ как «диапазон, в котором лежат 95 % данных»: это неверно, для этого нужен интервал предсказания или перцентили.

Для парных измерений (до/после у одного пациента) столбики независимых групп теряют связь между наблюдениями. Постарайтесь показывать пары: slope chart или paired dot plot.

При скошенном распределении (не симметричном, с длинным хвостом) предпочтительнее медиана и межквартильный интервал, а не среднее и SD.

И наконец: **эффект и его неопределённость важнее p-значения**. «$p < 0{,}05$» не говорит, насколько велик эффект и насколько он клинически значим. Показывайте разницу средних, отношение рисков, hazard ratio — с 95 % ДИ.

1.2.5 Контекст и воспроизводимость

График без контекста — головоломка, поэтому важно показывать контекст. Минимальный набор:

- единицы измерения на осях и в подписи;
- *n* (объём выборки) и способ формирования групп;
- что означают дополнительные обозначения в виде ошибок или полос;
- статистический тест или модель, если они упомянуты;
- программное обеспечение и версия (для воспроизводимости);
- доступность данных и кода, если это допускает журнал.

Дополнительно:

- **Согласованность групп.** Один и тот же цвет и порядок групп во всех рисунках статьи. Читатель не должен каждый раз заново расшифровывать легенду.
- **Нормализация.** «Относительно контроля» — относительно какого контроля, что именно делили, что делали с выбросами или пропусками? Всё это — в подпись.
- **Пропуски.** Если данные исключены, покажите сколько и почему.
- **Версии.** «Построено в Python 3.12, matplotlib 3.11, seaborn 0.13» — полезная строка в подписи или разделе supplementary. Иногда даже небольшая разница в версиях библиотек заметно влияет на картину (*это, увы, обратная сторона свободного программного обеспечения*).

1.2.6 Иерархия, минимализм и декор

Эдвард Тафти ввёл два полезных понятия: **data-ink ratio** (доля «чернил», которая кодирует данные) и **chartjunk** (декор, не несущий информации). Естественно, следует увеличивать долю «осмысленных» чернил и сокращать бесполезный графический мусор. Практический вывод: **убирайте всё, что не помогает читать данные**.

Что обычно лишнее:

- тени, псевдо-3D, градиенты, не кодирующие величину (практически всегда);
- тяжёлая сетка по обеим осям и полная рамка;
- легенда, дублирующая подписи оси (один из дублей нужно убрать);
- декоративные иконки, фоны, текстуры;
- подписи «Рисунок 1» внутри изображения (они и так будут в тексте публикации);
- несколько акцентных цветов без семантики, то есть без расшифровки.

Но минимализм — не самоцель. Убрать единицы, *n* или подпись оси — это не минимализм, а потеря информации. Хороший график не «пустой», а **иерархичный**:

1. данные — самый контрастный слой;
2. оси, подписи, аннотации — средний;
3. сетка, рамки, фон — самый бледный.

Полезные приёмы:

- **прямые подписи** вместо легенды, если серий немного;
- **один акцентный цвет** для главного вывода, серый — для контекста;
- **лёгкая сетка** только по той оси, где она помогает читать значения;
- **белое пространство** как способ группировки, а не бесполезная «пустота».

1.2.7 Доступность

Следует напомнить: около 8 % мужчин европейского происхождения имеют нарушения цветового зрения (протанопия или дейтеранопия); среди женщин доля ниже, но не нулевая. В зале на 250 человек, по оценке

Color Universal Design, может быть больше десяти человек с дальтонизмом. Рецензент тоже может быть одним из них.

Лучше учитывать этот факт, чем не делать этого. Правила:

- **Не кодировать смысл только цветом.** Дублируйте формой маркера, типом линии, прямой подписью, штриховкой.
- **Использовать палитры, проверенные на дальтонизм** (практически обязательно): Okabe–Ito, Paul Tol, viridis/cividis для последовательных шкал.
- **Проверять в оттенках серого.** Если после обесцвечивания серии неразличимы — добавить форму или подпись.
- **Контраст.** Для текста — не ниже 4,5:1, для графических объектов — не ниже 3:1 (WCAG 2.1).
- **Кегль в конечном размере.** Шрифт 6 pt, который «нормально смотрится» на экране в 200 %, в журнале превратится в нечитаемую полосу.
- **Alt-текст** для веб-версий и доступных PDF.

1.2.8 Согласованность

Серия рисунков в статье должна выглядеть как один документ, а не как коллаж из разных статей. Единый визуальный язык:

- одна гарнитура и 2–3 кегля;
- одна палитра и одно соответствие «цвет → группа»;
- одинаковые толщины линий, размеры маркеров, стиль сетки;
- одинаковое расположение меток панелей (A, B, C) — слева сверху;
- одинаковый десятичный разделитель и формат дат;
- одинаковые единицы и сокращения.

Если на рисунке 1 плацебо серое, на рисунке 2 оно тоже серое. Если на рисунке 3 появился новый препарат — он получает новый цвет, а не переиспользует цвет старого.

Программно это можно реализовать через единую функцию настройки параметров всех графиков для публикации.

1.2.9 Проверка «пять секунд — тридцать секунд — пять минут»

Быстрый тест перед отправкой:

1. **Пять секунд.** Что читатель понял? Если ответ «какие-то столбики» — график не работает.
2. **Тридцать секунд.** Можно ли прочитать значения, единицы, *n* и понять, что означают усы? Если нет — не хватает подписей.
3. **Пять минут.** Можно ли восстановить, как получены данные, что такое ошибки, есть ли выбросы и не противоречит ли график тексту? Если нет — подпись не самодостаточна.

1.3 Нормативы и оформление

1.3.1 Какие документы задают правила

Журнал → ГОСТ → требования организации → проект

Иерархия проста: **требования журнала важнее ГОСТ**, если вы отправляете статью. ГОСТ и локальные шаблоны — для диссертаций, отчётов о НИР и внутренних документов.

Документ	Что регулирует	Комментарий
Гайд журнала (Instructions for Authors)	размеры, шрифты, форматы, число рисунков, captions	всегда проверяйте перед отправкой
ICMJE Recommendations	общие нормы медицинских журналов, подготовка рукописи	icmje.org
EQUATOR Network	отчётные стандарты (CONSORT, STROBE, PRISMA, ARRIVE)	equator-network.org
COPE	публикационная этика, целостность изображений	publicationethics.org
ГОСТ 7.32-2017	отчёт о НИР: структура, рисунки, подписи	docs.cntd.ru

Документ	Что регулирует	Комментарий
ГОСТ 2.105-95	ЕСКД: общие требования к текстовым документам	docs.cntd.ru
ГОСТ 8.417-2002	единицы величин	docs.cntd.ru

Отчётные стандарты важны не только для текста. CONSORT требует схему потока участников, STROBE — аккуратного описания выборки, PRISMA — блок-схему отбора исследований, ARRIVE — деталей экспериментов на животных. Это тоже «научная графика», и к ней предъявляются те же требования, что и к остальным рисункам.

Целостность изображений. Нельзя (крайне нежелательно) использовать такую редактуру изображений, как selectively enhance, замазывать, склеивать фрагменты без явных границ, менять яркость так, чтобы исчезали детали. Любая обработка — в подписи или supplementary. Это прямая норма COPE и ICMJE, а не «стилистика».

1.3.2 Размер, разрешение, формат

Проектируйте в конечном размере. Откройте требования журнала к публикации, найдите ширину колонки текста (а если необходимо — двух колонок). Если рисунок будет шириной 85 мм, не рисуйте его в 16 дюймов и не уменьшайте в Word: шрифты и толщины превратятся в лотерею.

Напомним также, что экран ЭВМ выдаёт цвета RGB (цвета «разноцветных горящих лампочек»), а печать графиков на бумаге использует CMYK (цвет чернил, которые выдаются в виде четырёх пигментов), поэтому для правильного отображения графиков на бумаге необходимо использовать CMYK.

Параметр	Типичное требование	Комментарий
Ширина	85–90 мм (1 колонка), 170–183 мм (2 колонки)	Nature: 89 и 183 мм
Шрифт	sans-serif, 7–10 pt в конечном размере	нижняя граница 5–7 pt у отдельных журналов
Линии	0,25–1 pt	тоньше 0,25 pt может исчезнуть при печати
Растр	300 dpi для фото, 600–1200 dpi для штриховых	TIFF/PNG, без JPEG для линий
Вектор	PDF, EPS, SVG, AI	предпочтительно для графиков и схем
Цвет	RGB/sRGB для онлайн, CMYK для печати	конвертировать в конце, проверять gamut
Поля	3–5 мм	чтобы подписи не обрезались

Практические следствия:

- **Не скриншот.** Экспорт из Python, R, GraphPad, Origin — но не снимок экрана: скриншот теряет точность векторной графики и разрешение.
- **Шрифты — в кривые или встраивать.** Для SVG удобно включить прорисовку шрифтов в кривые `svg.fonttype = "path"`; для PDF — `pdf.fonttype = 42`, чтобы текст оставался редактируемым, но не подменялся при просмотре.
- **Не масштабировать непропорционально.** Растянутый по ширине график меняет восприятие наклонов и пропорций.
- **Проверять в 100 %.** Откройте итоговый PDF при реальном размере и посмотрите, читаются ли деления и легенда.

1.3.3 Шрифты

Для рисунков почти всегда лучше **sans-serif**: Arial, Helvetica, Liberation Sans, DejaVu Sans. Они плотнее и лучше читаются в мелком кегле, чем засечные гарнитуры.

Элемент	Ориентир
Заголовок внутри рисунка	9–10 pt
Подписи осей	8–9 pt
Деления и подписи делений	7–8 pt
Легенда	7–8 pt
Подпись к рисунку (caption)	8–9 pt, обычно в тексте, не в изображении

Правила:

- одна гарнитура на весь рисунок и, желательно, на всю статью;

- не писать подписи КАПСОМ — это замедляет чтение;
- не делать всё жирным: жирный — только для акцента;
- не использовать тонкие начертания (Light, Thin) в мелком кегле;
- проверить кириллицу и специальные символы (это частая проблема): μ, ±, ≥, ≤, →, °C (решение — использовать шрифты с поддержкой кириллицы, LaTeX или Typst);
- математические переменные — курсивом (*p*, *n*, *M*), единицы — прямым;
- не смешивать в одном рисунке разные семейства шрифтов без причины.

1.3.4 Цвет

Цвет должен **кодировать смысл**, а не оживлять картинку.

Три типа палитр:

- **качественная** — для категорий (группы, препараты, регионы);
- **последовательная** — для упорядоченных значений (концентрация, время);
- **дивергентная** — для отклонений от нуля/нормы (изменение, разница).

Базовый набор, который стоит запомнить, — палитра Okabe–Ito:

Цвет	HEX	Разумная роль
оранжевый	`#E69F00`	категория 1
небесно-голубой	`#56B4E9`	категория 2
сине-зелёный	`#009E73`	категория 3
жёлтый	`#F0E442`	категория 4 (только с тёмным контуром)
синий	`#0072B2`	основной результат
вермилион	`#D55E00`	акцент/второй результат
красновато-фиолетовый	`#CC79A7`	категория 5
чёрный	`#000000`	текст, оси, а не серии

Правила:

- не больше 6–8 категориальных цветов; больше — объединяйте или делайте small multiples;
- не используйте такие палитры, как «радуга» и `jet`: она создаёт ложные границы и не воспринимается равномерно;
- не кодируйте противоположные категории только красным и зелёным;
- серый — для контроля, фона и «прочего»; он не должен конкурировать с данными;
- проверяйте рисунок в градациях серого и/или через симулятор дальтонизма;
- фон — белый (для печати), без тёмных тем;
- контраст графических объектов — не ниже 3:1.

Для последовательных шкал используйте `viridis`, `cividis`, `Blues`, `Greens`; для дивергентных — `RdBu_r`, `PuOr`, `BrBG` с симметричным диапазоном относительно нуля. Центр дивергентной шкалы должен совпадать с содержательным нулём, а не с серединой диапазона данных.

1.3.5 Единицы, числа, статистика

- **СИ и общепринятые медицинские единицы.** «мм рт. ст.» — внесистемная, но допустимая в медицине единица; «мг/кг», «ммоль/л», «уд/мин».
- **Пробел между числом и единицей:** 8,9 мм рт. ст., 10 мг, 5 %, 37 °C. Исключения — знаки градусов угла и некоторые типографские случаи. Часто имеет смысл ставить неразрывный пробел.
- **Десятичный разделитель.** В русском тексте — запятая, в англоязычных журналах — точка. Главное — единообразие в пределах работы. В коде и CSV обычно точка; в подписях рисунка — по языку публикации.
- **Ведущий ноль:** 0,05, а не ,05 (как принято у программистов, но не принято в научных работах).
- **Тысячи:** 12 000, а не 12000, в тексте; в осях можно 12 000 или 12 тыс. Как и для единиц измерения, здесь желательно использовать автоматическое форматирование или неразрывный пробел.
- **Средние:** $M \pm SD$; медиана и IQR: Me [Q1; Q3]; доли: *n* (%).
- **p-значение:** $p = 0{,}032$, а не «p<0.05» без точного значения, если журнал не требует иначе. Курсив для *p*, *n*, *M*.
- **Сокращения** расшифровывайте при первом упоминании в подписи.

1.3.6 Подписи к рисункам

Подпись — не «название файла», а самостоятельный текст. Хорошая структура:

1. **Что показано** — одно предложение с выводом или темой.
2. **Как показано** — группы, *n*, что означают точки/столбцы/усы.
3. **Определения** — ошибки, сокращения, статистический тест.
4. **Источник/ПО** — данные, код, версия, при необходимости.

Плохо:

> Рисунок 1. Результаты.

Хорошо:

> **Рисунок 1.** Снижение систолического АД через 12 недель терапии. Точки — индивидуальные значения пациентов, маркеры — среднее, усы — 95 % доверительный интервал (t-распределение). Плацебо — *n* = 48, препарат А 10 мг — *n* = 51, препарат В 20 мг — *n* = 49. Штриховая линия — отсутствие изменения. Данные синтетические; код — `01_principles/scripts/good_plot.py`.

Формальные детали:

- в ГОСТ 7.32 подпись обычно начинается с «Рисунок 1 — Название» (с тире);
- в англоязычных журналах — «Figure 1. …»;
- подпись располагается под рисунком, а не внутри изображения;
- точка в конце подписи — по стилю журнала;
- ссылка в тексте — до появления рисунка: «(рисунок 1)» или «(Fig. 1)».

1.3.7 Чек-лист перед отправкой

- ☐ Заголовок/подпись сообщают вывод, а не «результаты».
- ☐ Тип графика соответствует задаче и структуре данных.
- ☐ Столбцы начинаются с нуля; обрезка осей объяснена.
- ☐ Логарифмическая шкала подписана.
- ☐ Нет двух осей Y без крайней необходимости.
- ☐ Показаны *n*, разброс и определение ошибок.
- ☐ Единицы указаны на осях и в подписи.
- ☐ Цвета различимы при дальтонизме и в градациях серого.
- ☐ Шрифт ≥ 7 pt в конечном размере; кириллица и символы не потеряны.
- ☐ Ширина соответствует колонке журнала (85/170 мм или гайд).
- ☐ Вектор или растр ≥ 300 dpi; нет скриншотов.
- ☐ Верхняя и правая рамки, тяжёлая сетка, тени и 3D убраны.
- ☐ Легенда не дублирует подписи и не закрывает данные.
- ☐ Подпись самодостаточна: группы, *n*, тест, сокращения.
- ☐ Рисунок согласован с другими рисунками по палитре и шрифтам.
- ☐ Нет искажений изображения и необъявленной обработки.
- ☐ Файлы названы по схеме журнала; формат и цветовой профиль проверены.
- ☐ Рисунок проверен в PDF при 100 % масштабе.

1.4 Один результат — два графика

Ниже — один и тот же набор данных: 148 пациентов, три группы, изменение САД через 12 недель. Слева — «плохая» версия, справа — «правильная». Обе построены по одному CSV; различается только код визуализации.

1.4.1 Плохой график

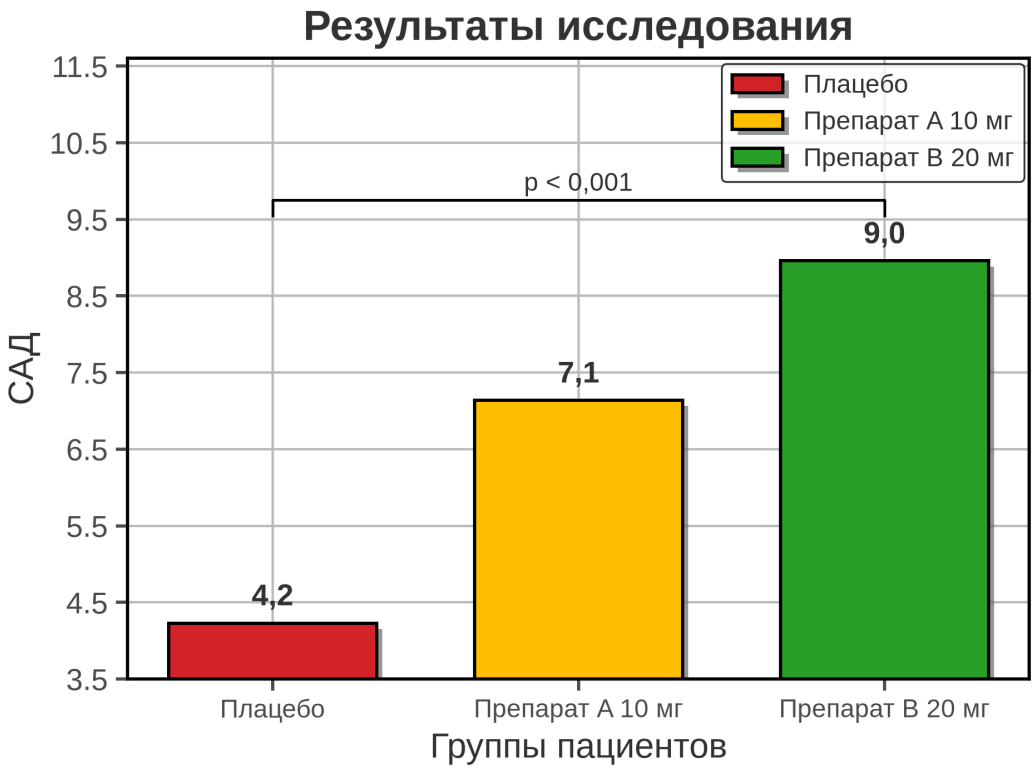


Рисунок 1.1. Плохой график: обрезанная ось, светофорная палитра, тени, тяжёлая сетка и легенда внутри поля данных.

Код: `01_principles/scripts/bad_plot.py`. Данные: `data/00_dataset.csv`.

1.4.2 Что не так

1. **Обрезанная ось.** Столбцы начинаются с 3,5 мм рт. ст., а не с нуля. Визуальное отношение В/плацебо — около 7,5, реальное — 2,1. Lie factor ≈ 3,5.
2. **Светофорная палитра.** Красный — жёлтый — зелёный не только неразличим при протанопии/дейтеранопии, но и навязывает оценку «плохо — средне — хорошо», которой в данных нет.
3. **Тени и псевдо-3D.** Тень не кодирует ни одну переменную; она лишь утяжеляет нижнюю часть столбцов и мешает сравнивать высоту.
4. **Тяжёлая сетка по обеим осям.** Сетка конкурирует с данными; вертикальные линии вообще не помогают читать столбцы.
5. **Полная рамка и жирные линии.** Визуальный шум, который не несёт смысла.
6. **Легенда дублирует подписи.** Названия групп уже написаны под столбцами; легенда занимает место и закрывает часть данных.
7. **Заголовок «Результаты исследования».** Он не сообщает, что именно показано и какой вывод должен сделать читатель.
8. **Нет единиц, *n*, разброса и определения ошибок.** «САД» — это что: исходное, конечное, изменение? В каких единицах? Сколько пациентов?
9. **Только средние.** Без индивидуальных точек и ДИ нельзя понять, насколько велик разброс и насколько устойчива разница.
10. **p < 0,001 без теста и эффекта.** Значимость не заменяет величину эффекта; не указано, какой тест применялся и какие группы сравнивались.

1.4.3 Правильный график

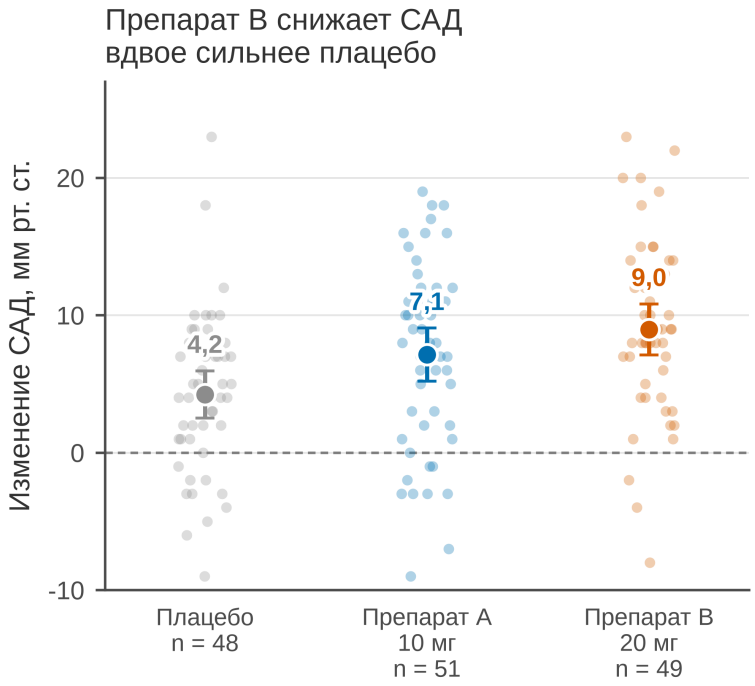


Рисунок 1.2. Правильный график: индивидуальные точки, среднее и 95 % ДИ, нулевая линия, палитра Okabe–Ito, прямые подписи.

Код: `01_principles/scripts/good_plot.py`.

1.4.4 Что изменилось

1. **Точечная оценка вместо длины.** Средние показаны точками, а не столбцами; обрезать ось больше не нужно, ложной разницы не возникает.
2. **Нулевая линия.** Штриховая линия на нуле показывает отсутствие эффекта и помогает оценить клиническую значимость.
3. **Индивидуальные наблюдения.** Полупрозрачные точки показывают разброс, выбросы и то, что у части пациентов САД выросло.
4. **Среднее и 95 % ДИ.** Видна неопределённость оценки; в подписи указано, что это t-интервал.
5. **Семантическая палитра.** Серый — контроль, синий — препарат А, вермилион — препарат В. Цвета различимы при дальтонизме, а подписи групп дублируют цвет.
6. **Прямые подписи и *n*.** Названия групп и число пациентов — прямо под осью; легенда не нужна.
7. **Заголовок-вывод.** «Препарат В снижает САД вдвое сильнее плацебо» сразу задаёт вопрос, на который отвечает график.
8. **Единицы и десятичная запятая.** «Изменение САД, мм рт. ст.» — без догадок; числа в русской типографике.
9. **Иерархия.** Лёгкая сетка только по Y, нет верхней и правой рамок, данные контрастнее осей.
10. **Журнальный размер.** 85 мм в ширину, кегли 7–9,5 pt; рисунок спроектирован в конечном размере, а не уменьшен из «слайда».

1.4.5 Было и стало

Что было	Почему плохо	Что сделано
ось Y от 3,5	длина столбца искажена, LF ≈ 3,5	точки + нулевая линия
красный/жёлтый/зелёный	ложная семантика, CVD	Okabe–Ito: серый/синий/вермилион
тени, 3D	декор, искажает площадь	плоские маркеры
сетка по X и Y	шум, конкуренция с данными	сетка только по Y, бледная
полная рамка	визуальный шум	только левая и нижняя оси
легенда	дублирует подписи, закрывает данные	прямые подписи под осью
«Результаты исследования»	нет сообщения	заголовок-вывод
нет единиц и *n*	невозможно прочитать	единицы, *n* в подписях
только средние	скрыт разброс	точки + среднее + 95 % ДИ
p < 0,001	нет теста и эффекта	эффект и ДИ; тест — в подписи

Пара замечаний:

- График показывает изменение от исходного уровня, но не заменяет статистическую модель. Для сравнения групп с поправкой на исходное САД нужен ANCOVA или смешанная модель; рисунок этого не «исправляет».
- При $n \approx 50$ точек на группу strip plot может выглядеть шумно. Альтернативы — box/violin с точками, half-violin, или среднее с ДИ на фоне распределения. Выбор зависит от сообщения и от того, что важнее: форма распределения или компактность.

1.4.6 Как воспроизвести

```
# из корня репозитория
uv sync
uv run python 01_principles/scripts/make_data.py
uv run python 01_principles/scripts/bad_plot.py
uv run python 01_principles/scripts/good_plot.py
```

Вот как выглядит «правильный» код:

```
from goodgraph import apply_style, figure_size, save_figure

apply_style()
fig, ax = plt.subplots(
    figsize=figure_size("single", ratio=0.90),
    layout="constrained",
)

# ... точки, средние, 95 % ДИ ...

ax.set_ylabel("Изменение САД, мм рт. ст.")
ax.set_title("Препарат В снижает САД\nвдвое сильнее плацебо", loc="left")
save_figure(fig, "01_good_dot_ci")  # images/01_good_dot_ci.svg + .png
```

Доверительный интервал считается через t-распределение, а не «среднее ± 2 SE»:

```
from scipy import stats

se = sd / np.sqrt(n)
half_width = stats.t.ppf(0.975, df=n - 1) * se
ci_low, ci_high = mean - half_width, mean + half_width
```

Для больших n разница с нормальным приближением мала, но t-интервал корректнее при $n \approx 50$ и менее.

1.5 Итоги раздела

Положения, которые стоит запомнить:

1. График — аргумент, а не украшение; сначала сообщение, потом код.
2. Тип графика определяется задачей и структурой данных.
3. Столбцы — с нуля; точки и линии — могут без нуля, но с объяснением.
4. Логарифм, разрыв оси и две оси Y — только осознанно и с подписями.
5. Показывайте n, разброс и неопределённость; определяйте, что такое усы.
6. Эффект и 95 % ДИ важнее p-значения.
7. Убирайте chartjunk, но не путайте минимализм с потерей информации.
8. Цвет — смысл; палитра должна работать при дальтонизме и в сером.
9. Подпись самодостаточна; рисунок согласован с остальными.
10. Проектируйте в конечном размере и проверяйте в PDF при 100 %.

Далее мы рассмотрим инструменты создания графиков. В разделе 2 разберём, как это реализуется в Python с использованием распространённых библиотек: matplotlib, seaborn, plotnine, а также настройку среды через менеджер `uv` и типичные архитектурные решения для воспроизводимых графиков.

Источники к разделу

- Tufte E. R. *The Visual Display of Quantitative Information*. Graphics Press, 1983. — edwardtufte.com
- Cleveland W. S., McGill R. Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods // *JASA*. 1984. Vol. 79, № 387. P. 531–554. — doi:10.1080/01621459.1984.10478080
- Wilke C. O. *Fundamentals of Data Visualization*. O’Reilly, 2019. — clauswilke.com/dataviz
- Cairo A. *The Truthful Art: Data, Charts, and Maps for Communication*. New Riders, 2016. — thefunctionalart.com
- Schwabish J. *Better Data Visualizations: A Guide for Scholars, Researchers, and Wonks*. Columbia University Press, 2021. — policyviz.com
- Okabe M., Ito K. Color Universal Design (CUD): How to Make Figures and Presentations that are Friendly to Colorblind People. 2008. — jfly.uni-koeln.de/color
- ColorBrewer 2.0 — colorbrewer2.org
- Weissgerber T. L. et al. Beyond Bar and Line Graphs: Time for a New Data Presentation Paradigm // *PLoS Biology*. 2015. Vol. 13, № 4. e1002128. — doi:10.1371/journal.pbio.1002128
- ICMJE Recommendations — icmje.org
- EQUATOR Network — equator-network.org
- Nature. Formatting guide — nature.com
- W3C. WCAG 2.1, Understanding SC 1.4.11 Non-text Contrast — w3.org
- ГОСТ 7.32-2017 — docs.cntd.ru
- ГОСТ 2.105-95 — docs.cntd.ru
- ГОСТ 8.417-2002 — docs.cntd.ru

Раздел 2. Python-инструментарий и настройка среды

В разделе 1 мы обсудили основные принципы научной графики. Каким должен быть график? Честным, читаемым, доступным и воспроизводимым. Настало время для выьора инструментария, средств для рисования графиков.

Теперь — чем его рисовать. Безусловно, существует множество как бесплатных, так и платных программ для построения графиков. Например, всегда можно использовать возможности электронных таблиц с «живой» графикой, реагирующей на изменения данных. Имеются такие монстры, как OriginLab, MatLab, GNUPlot, ParaView и многие другие, Каждый с собственными возможностями и даже характером. Стоило бы отметить, что графики могут быть либо сгенерированы как статичные картинки, либо отражать представленные данные «в реальном времени, вживую» по принципу WYSIWYG, Сам сгенерированный график может представлять собой как статичную картинку, так и иерархию редактируемых объектов.

По мнению авторов, хотя живая графика является более наглядной, ее качество уступает сгенерированной статичной картинке. Зачастую, особнно при сложной визуализации данных, живая графика может оказаться даже более медленной, а усилия по ее настройке могут быть не меньшими, чем написание небольшого программного кода. С другой стороны, живая графика может быть очень полезна при визуализации данных, которые изменяются со временем.

Работа с данными часто предполагает работу в команде и необходимость встраиваться в команду и ее рабочие процессы, а также выдавать стабильный результат в повторяющихся проектах. В таких условиях программируемая генерация графики оказывается очень полезной. Стандартом де-факто в аналитике данных является использование языка программирования Python. В свою очередь, Python, несмотря на огромное разнообразие модулей и библиотек, предлагает несколько наиболее распространенных основных модулей для построения графиков, В частности, это matplotlib и seaborn.

Matplotlib — это библиотека для построения графиков, которая позволяет создавать графики в Python, а seaborn - ее дполнение для построения прежде всего статистическихграфиков.

Задача этого раздела — не выучить все функции matplotlib, а понять, **как устроен стек** и **как получить воспроизводимый результат** из любого места: с ноутбука, из CI, с другого компьютера.

Хорошая новость: инструменты не определяют качество графика. Плохой график можно сделать и в matplotlib, и в ggplot2. Но разные библиотеки по-разному заставляют думать о данных. Matplotlib даёт полный контроль и требует дисциплины. Seaborn берёт на себя статистику и группы. Plotnine заставляет описать график как композицию слоёв. Выбор — не религия, а вопрос задачи.

> Все примеры раздела используют собственный набор `data/02_python_dataset.csv` (HbA1c, 132 пациента). Данные синтетические и предназначены только для иллюстративных целей.

2.1 Карта экосистемы

Инструмент	Роль	Документация	Когда нужен
Python 3.12	язык и рантайм	docs.python.org	всегда
uv	окружения, зависимости, версии Python	docs.astral.sh/uv	всегда
NumPy	массивы, векторизованная математика	numpy.org/doc	расчёты, генерация данных
pandas	таблицы, groupby, CSV	pandas.pydata.org	загрузка и агрегация
SciPy	статистика, распределения, тесты	docs.scipy.org	доверительные интервалы, тесты
matplotlib	рендеринг, Figure/Axes, artists	matplotlib.org	полный контроль, финальная правка
seaborn	статистические графики, long-format	seaborn.pydata.org	EDA, группы, распределения
plotnine	грамматика ggplot2, фасеты, слои	plotnine.org	многопанельные и декларативные графики
adjustText	подписи без наложений	adjusttext.readthedocs.io	точечные подписи

Инструмент	Роль	Документация	Когда нужен
Pillow	растр, композит, конвертация	pillow.readthedocs.io	сборка панелей, проверка PNG

Стек удобно представлять так:

```
numpy / pandas / scipy
        ↓  данные и статистика
matplotlib (Figure, Axes, artists)
        ↓  рендеринг
seaborn / plotnine
        ↓  грамматика, статистика, фасеты
goodgraph (rcParams, палитры, размеры, save)
        ↓
SVG / PNG / PDF
```

Дело в том, что **seaborn и plotnine в итоге рисуют через matplotlib**. Поэтому понимание `Figure`/`Axes`, `rcParams` и `savefig` полезно даже тем, кто пишет только на plotnine. И наоборот: matplotlib не мешает использовать seaborn для статистики, а потом дорабатывать оси вручную.

2.2 Три подхода к одному графику

2.2.1 matplotlib: императивный контроль

Matplotlib — низкоуровневый движок. Вы создаёте `Figure` (холст), `Axes` (область с осями) и добавляете artists: линии, точки, текст, стрелки, патчи. Каждый элемент — объект с координатами и свойствами.

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(3.35, 2.8), layout="constrained")

for group, part in df.groupby("group"):
    ax.scatter(
        part["baseline_hba1c_pct"],
        part["followup_hba1c_pct"],
        s=12,
        alpha=0.5,
        label=group,
    )

ax.set_xlabel("HbA1c до, %")
ax.set_ylabel("HbA1c после, %")
ax.legend(frameon=False)
fig.savefig("scatter.svg")
```

Сильные стороны:

- полный контроль над каждым элементом;
- аннотации, врезки, стрелки, произвольные artists;
- предсказуемое поведение при экспорте;
- огромная экосистема примеров.

Слабые стороны:

- многословно;
- статистика и группировка — вручную;
- легко получить несогласованные рисунки, если каждый скрипт настраивает оси по-своему;
- глобальное состояние `pyplot` (`plt.plot`, `plt.xlim`) может «протечь» из одного рисунка в другой.

Практическое правило: в скриптах используйте **объектный API** (`fig, ax = plt.subplots()`), а не `plt.plot`. Глобальный `pyplot` удобен в интерактивной консоли, но плохо предсказуем в воспроизводимом пайплайне.

2.2.2 seaborn: данные и статистика

Seaborn — надстройка над matplotlib, ориентированная на таблицы данных. Вы передаёте `data=` и имена колонок, а не массивы; `hue`, `style`, `size`, `col`, `row` автоматически кодируют группы и создают фасеты.

```
import seaborn as sns

sns.scatterplot(
    data=df,
    x="baseline_hba1c_pct",
    y="followup_hba1c_pct",
    hue="group",
    style="group",
    ax=ax,
)
```

Сильные стороны:

- статистические преобразования «из коробки»: средние, доверительные интервалы, регрессии, KDE;
- аккуратные умолчания для категориальных групп;
- быстрый EDA и разведочные графики;
- интеграция с matplotlib через `ax=`.

Слабые стороны:

- меньше контроля над отдельными artists;
- темы seaborn (`sns.set_theme`) **перезаписывают** rcParams matplotlib;
- API статистики менялся между версиями (`ci` → `errorbar`), поэтому примеры из старых статей могут не работать;
- легко получить «красиво, но не по журнальному»: крупные шрифты, лишние линии, тени.

Практическое правило: seaborn — для исследования и быстрых статистических сводок; финальную правку удобнее делать в matplotlib.

2.2.3 plotnine: грамматика графики

Plotnine — реализация грамматики графики (Wilkinson; ggplot2) для Python. График описывается декларативно: данные → эстетические отображения → геометрия → шкалы → координаты → фасеты → тема.

```
from plotnine import *

(
    ggplot(df, aes("baseline_hba1c_pct", "followup_hba1c_pct", color="group"))
    + geom_point(alpha=0.5)
    + labs(x="HbA1c до, %", y="HbA1c после, %")
    + theme_bw()
)
```

Сильные стороны:

- декларативность: вы описываете **что**, а не **как**;
- фасеты (`facet_wrap`, `facet_grid`) и согласованные шкалы;
- легко менять геометрию, не переписывая код;
- хорошо ложится на многопанельные статьи и воспроизводимые отчёты.

Слабые стороны:

- требует tidy/long-формата данных;
- меньше контроля над отдельными artists, чем в matplotlib;
- своя система тем, не полностью совпадающая с rcParams;
- часть возможностей ggplot2 отличается или называется иначе;
- ошибки в слоях иногда появляются только на этапе отрисовки.

Практическое правило: plotnine — для структурированных многопанельных графиков и случаев, где важна декларативная воспроизводимость.

2.2.4 Сравнение

Критерий	matplotlib	seaborn	plotnine
Уровень	низкий	средний	высокий
Стиль	императивный	дата-ориентированный	декларативный
Контроль над artists	максимальный	средний	средний/низкий
Статистика «из коробки»	нет	да	частично
Фасеты	вручную	`col`/`row`	`facet_wrap`/`facet_grid`
Кривая обучения	средняя	низкая	средняя
Лучше всего для	финальной правки, аннотаций	EDA, групп, распределений	структуры, multi-panel

Эти средства не исключают друг друга. Типичный пайплайн: seaborn или plotnine для черновика, matplotlib для финальной правки, `goodgraph` — для единого стиля.

2.3 Один набор данных — три библиотеки

Чтобы сравнение не осталось теоретическим, один и тот же набор (HbA1c до и после 12 недель, 132 пациента) построен трижды. Код — `02_python/scripts/compare_libraries.py`. Вы едва ли заметите разницу между всеми тремя графиками, хотя они сипользуют различные библиотеки

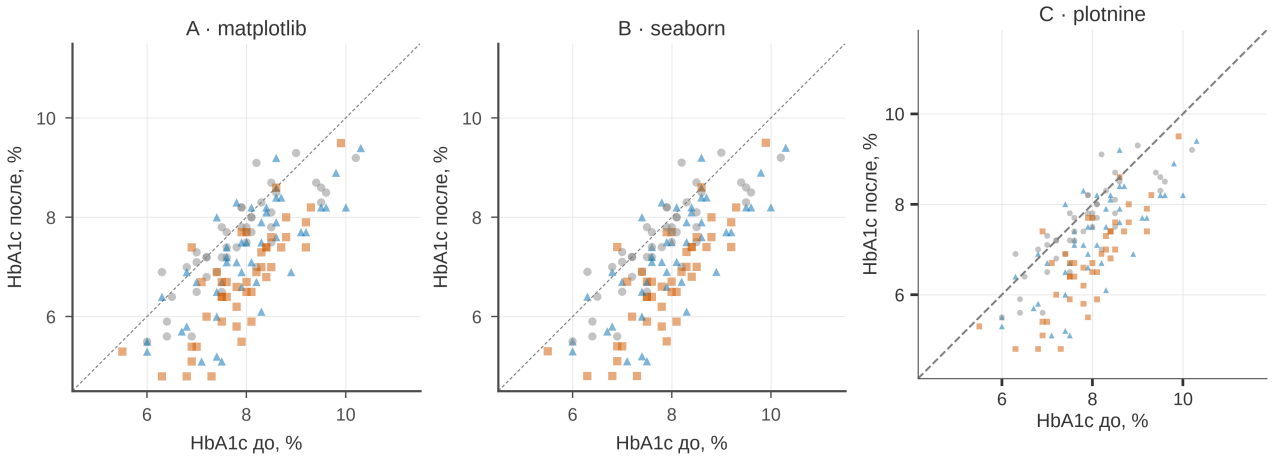


Рисунок 2.1. Один и тот же набор данных в трёх библиотеках: A — matplotlib, B — seaborn, C — plotnine. Серые круги — плацебо, синие треугольники — препарат A 10 мг, вермилионовые квадраты — препарат B 20 мг; штриховая линия — равенство HbA1c до и после. Данные — `data/02_python_dataset.csv`.

Что здесь важно:

- **Общие пределы и деления.** Все панели используют `xlim/ylim = 4,5–11,5` и деления 6/8/10. Иначе сравнение было бы нечестным.
- **Общая палитра и формы.** Цвет и форма маркера заданы один раз и повторены во всех библиотеках. Это принцип согласованности из раздела 1.
- **Разный объём кода.** Matplotlib требует явного цикла по группам; seaborn — одного вызова с `hue`/`style`; plotnine — описания слоёв и темы. Но результат один и тот же, потому что стиль вынесен наружу.
- **Размер панели 54 × 62 мм.** Три панели по 54 мм с промежутками дают ширину около 170 мм. Это пример проектирования в конечном размере, а не «нарисуем большое и уменьшим».

Ключевые фрагменты:

```
# matplotlib: цикл по группам
for group in GROUP_ORDER:
    part = df.loc[df["group"] == group]
    ax.scatter(part["baseline_hba1c_pct"], part["followup_hba1c_pct"],
               marker=GROUP_MARKERS[group], color=GROUP_COLORS[group])

# seaborn: одна функция, группы через hue/style
sns.scatterplot(data=df, x="baseline_hba1c_pct", y="followup_hba1c_pct",
                hue="group", style="group", ax=ax)

# plotnine: слои через «+»
(
    ggplot(df, aes("baseline_hba1c_pct", "followup_hba1c_pct",
                   color="group", shape="group"))
    + geom_point(alpha=0.5)
    + geom_abline(slope=1, intercept=0, linetype="dashed")
    + coord_fixed(ratio=1, xlim=(80, 215), ylim=(80, 215))
    + theme_bw()
)
```

Нетрудно видеть, что seaborn строит график минимальным количеством кода. В plotnine это делается с помощью слоев. Наиболее многословной конструкцией отличается matplotlib.

2.4 uv: окружение и воспроизводимость

2.4.1 Что такое uv

Использование Python-проектов предполагает, что вы используете интерпретатор языка, подгружаете нужные вам библиотеки через Интернет, и пишете свой код, в расчете на определенную версию интерпретатора и пакетов. Часто для обслуживания проектов используются менеджеры пакетов, например `pip` — менеджер пакетов от Python.

В последнее время получил распространение `uv` — менеджер пакетов и окружений от Astral, написанный на Rust. Он заменяет связку `pip` + `venv` + `pip-tools` + `pyenv` и умеет:

- создавать виртуальные окружения;
- разрешать и фиксировать зависимости (`uv.lock`);
- скачивать и переключать версии Python;
- запускать скрипты и инструменты (`uv run`, `uvx`);
- работать в CI и на чужих машинах.

Документация: docs.astral.sh/uv. В этом репозитории используется uv 0.12 и Python 3.12; точные версии пакетов зафиксированы в `uv.lock`.

2.4.2 Установка менеджера пакетов UV

```
# Linux / macOS
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows PowerShell
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# Если уже есть pipx или Homebrew
pipx install uv
brew install uv

uv --version
```

Если нужной версии Python нет:

```
uv python install 3.12
uv python list
```

uv скачает интерпретатор в свой каталог и не тронет системный Python.

2.4.3 Новый проект

```
mkdir my-figure && cd my-figure
uv init --python 3.12
uv add matplotlib seaborn plotnine pandas numpy scipy
uv add --dev ruff pytest
uv run python -c "import matplotlib; print(matplotlib.__version__)"
```

Что появилось:

- `pyproject.toml` — метаданные проекта и зависимости;
- `.python-version` — закреплённая версия Python;
- `uv.lock` — точное дерево зависимостей с хешами;
- `.venv/` — виртуальное окружение (не коммитится).

`uv add` одновременно обновляет `pyproject.toml`, `uv.lock` и окружение. `uv sync` приводит окружение в соответствие с lock-файлом.

2.4.4 `pyproject.toml` этого репозитория

Сокращённый вид:

```
[project]
name = "goodgraph"
version = "0.1.0"
requires-python = ">=3.12,<3.13"
dependencies = [
    "numpy>=2.0",
    "pandas>=2.2",
    "matplotlib>=3.9",
    "seaborn>=0.13",
    "plotnine>=0.13",
    "scipy>=1.13",
]

[dependency-groups]
dev = ["ruff>=0.6", "pytest>=8.0"]

[tool.uv]
package = true
```

Полная версия с `[build-system]` и `[tool.hatch.build.targets.wheel]` — в `pyproject.toml`.

Что здесь важно:

- `requires-python` задаёт **диапазон**, а `.python-version` — конкретную версию для разработки;
- `uv.lock` фиксирует точные версии; его нужно коммитить;
- `[tool.uv] package = true` и `src/`-layout означают, что пакет `goodgraph` устанавливается в окружение редактируемым (`pip install -e`);
- `[dependency-groups] dev` — инструменты разработки, которые не нужны читателю в рантайме.

2.4.5 Повседневные команды

Команда	Что делает
`uv sync`	создаёт/обновляет `.venv` по `uv.lock`
`uv sync --frozen`	не меняет lock, падает при расхождении — для CI
`uv add pandas`	добавляет зависимость и обновляет lock
`uv add --dev ruff`	добавляет в dev-группу
`uv remove pandas`	удаляет зависимость
`uv lock`	пересчитывает lock
`uv lock --upgrade`	обновляет версии в пределах ограничений
`uv run python script.py`	запускает в окружении проекта
`uv run --with rich script.py`	добавляет пакет только на запуск

Команда	Что делает
`uv python pin 3.12`	фиксирует версию Python
`uv tree`	показывает дерево зависимостей
`uv pip list`	список установленного
`uvx ruff check .`	запускает инструмент без установки в проект

`uv run` не требует активации окружения. Если привычнее `source .venv/bin/activate`, это тоже работает; но в скриптах и CI `uv run` надёжнее — он гарантирует нужное окружение.

2.4.6 Одноразовые скрипты и PEP 723

Если скрипт не входит в проект, зависимости можно объявить прямо в нём:

```
# /// script
# requires-python = ">=3.12"
# dependencies = ["matplotlib", "pandas"]
# ///

import matplotlib.pyplot as plt
import pandas as pd
```

Запуск:

```
uv run plot.py
```

`uv` создаст временное окружение с указанными пакетами и выполнит скрипт. Это удобно для примеров в статье или для утилиты, которую не хочется тащить в основной `pyproject.toml`. Спецификация — PEP 723.

2.4.7 Воспроизводимость и кеш

Правила:

- коммитить `pyproject.toml`, `uv.lock`, `.python-version`;
- не коммитить `.venv/`;
- в CI использовать `uv sync --frozen` и `uv run ...`;
- обновлять зависимости осознанно: `uv lock --upgrade`, затем прогнать тесты и пересобрать рисунки;
- фиксировать системные зависимости отдельно: шрифты, Cairo, LaTeX, Graphviz — uv их не устанавливает.

В песочнице или CI с read-only домашним каталогом задайте локальные пути:

```
export UV_CACHE_DIR="$PWD/.uv-cache"
export UV_PYTHON_INSTALL_DIR="$PWD/.uv-python"
export MPLCONFIGDIR="$PWD/.mplconfig"
```

В этом репозитории такие каталоги уже добавлены в `.gitignore`.

2.5 Анатомия воспроизводимого графика

2.5.1 Четыре слоя

```
data/02_python_dataset.csv
      ↓  make_data.py: seed, генерация, CSV
      ↓  plot.py: pandas/scipy, агрегация
      ↓  goodgraph.style: rcParams, палитра, размер
      ↓  matplotlib / seaborn / plotnine
      ↓  save_figure: SVG + PNG
images/01_good_dot_ci.svg, .png
```

Принципы:

1. **Данные не знают о графике.** `make_data.py` пишет CSV и не импортирует matplotlib.
2. **График не генерирует данные.** `plot.py` читает CSV; если данных нет — падает с понятной ошибкой.

3. **Стиль — отдельный слой.** Один `apply_style()` и одна палитра на всю методичку.
4. **Экспорт — отдельная функция.** `save_figure()` сохраняет SVG и PNG, закрывает фигуру и печатает пути.

2.5.2 Каркас скрипта

```python
from __future__ import annotations

import matplotlib

matplotlib.use("Agg")  # без дисплея

import matplotlib.pyplot as plt

from goodgraph import apply_style, figure_size, save_figure
from goodgraph.paths import DATA_DIR

apply_style()
fig, ax = plt.subplots(
    figsize=figure_size("single", ratio=0.8),
    layout="constrained",
)
# ... данные и слои ...
save_figure(fig, "02_example")
```

`figure_size("single")` возвращает размер в дюймах для 85 мм; `save_figure` сохраняет `images/02_example.svg` и `images/02_example.png` с dpi 600 и закрывает фигуру. Если нужен точный физический размер, не включайте `tight=True`: обрезка полей меняет ширину.

2.5.3 Что задаёт `apply_style`

Параметр	Значение	Зачем
`font.family`	Liberation Sans, Arial, DejaVu Sans	кириллица, метрики Arial
`font.size`	9 pt (paper)	журнальный кегль
`axes.spines.top/right`	False	меньше визуального шума
`axes.grid`	True, только Y	помогает читать значения
`grid.color`	`#E6E6E6`	сетка не конкурирует с данными
`axes.prop_cycle`	Okabe–Ito	доступная палитра по умолчанию
`svg.fonttype`	`path`	текст не «поплывёт» в SVG
`pdf.fonttype`	42	TrueType в PDF, текст редактируем
`savefig.dpi`	600	достаточный растр
`axes.unicode_minus`	False	совместимость со шрифтами

2.5.4 Порядок применения стилей

Порядок важен, потому что настройки перекрывают друг друга:

1. значения по умолчанию matplotlib;
2. `plt.style.use("...")`, если он используется;
3. `sns.set_theme(...)` — **сбрасывает** многие rcParams;
4. `apply_style(...)` — должен идти последним;
5. точечные переопределения в конкретном графике.

Если вызвать `sns.set_theme()` после `apply_style()`, шрифты, рамки и сетка могут вернуться к умолчаниям seaborn. Если нужна тема seaborn — передайте её параметры в `apply_style` или настройте rcParams после `set_theme`.

Plotnine живёт в своей системе тем. `theme(text=element_text(...))` не читает все rcParams matplotlib; задавайте шрифт и размеры явно.

2.6 Разбор `compare_libraries.py`

Скрипт `02_python/scripts/compare_libraries.py` устроен так:

1. **Общие константы.** `WIDTH_MM = 54`, `HEIGHT_MM = 62`, `LIM_MIN = 80`, `LIM_MAX = 215`, `TICKS = [80, 120, 160, 200]`. Одинаковые пределы — обязательное условие честного сравнения.
2. **Палитра и формы.** `GROUP_COLORS` и `GROUP_MARKERS` заданы один раз; цвет дублируется формой, чтобы панели читались в сером.
3. **`load_data()`.** Читает `data/02_python_dataset.csv` и проверяет, что все группы на месте.
4. **`style_axes()`.** Общая функция для matplotlib и seaborn: пределы, деления, линия равенства, подписи, сетка, аспект.
5. **Три функции построения.** `plot_matplotlib`, `plot_seaborn`, `plot_plotnine` — каждая самодостаточна и сохраняет свою пару SVG/PNG.
6. **`main()`.** Последовательно вызывает три функции.

Почему это важно: если каждая панель настраивает пределы сама, сравнение превращается в сравнение разных графиков. Общая функция `style_axes` фиксирует контракт.

2.7 Типичные ошибки и как их исправить

2.7.1 `plt.show()` в скрипте

```
# плохо: скрипт зависит от дисплея
plt.show()

# хорошо: Agg + сохранение
matplotlib.use("Agg")
fig.savefig("figure.png", dpi=600)
```

`show()` полезен в интерактивной консоли; в скрипте, CI и на сервере он либо блокирует выполнение, либо падает без дисплея.

2.7.2 Глобальное состояние `pyplot`

```
# плохо: неясно, какая фигура сохраняется
plt.plot(x, y)
plt.title("...")
plt.savefig("a.png")
plt.plot(x2, y2)
plt.savefig("b.png")  # на самом деле обе линии на одной фигуре

# хорошо: явные объекты
fig, ax = plt.subplots()
ax.plot(x, y)
fig.savefig("a.png")
plt.close(fig)
```

2.7.3 `sns.set_theme()` после `apply_style()`

```
# плохо: тема seaborn перезапишет шрифты и рамки
apply_style()
sns.set_theme(style="whitegrid")

# хорошо: сначала тема, потом свой стиль
sns.set_theme(style="whitegrid")
apply_style()
```

2.7.4 `tight_layout` и `constrained_layout` вместе

```
# плохо: конфликтующие движки компоновки
fig, ax = plt.subplots(layout="constrained")
fig.tight_layout()

# хорошо: один движок
fig, ax = plt.subplots(layout="constrained")
```

2.7.5 `bbox_inches="tight"` и точная ширина

```
# плохо для журнальной ширины: итоговый размер может отличаться от 85 мм
fig.savefig("figure.svg", bbox_inches="tight")

# хорошо: размер задан figsize, поля — полями
fig.savefig("figure.svg")
```

`tight` удобен для слайдов и препринтов, но не для точного соответствия колонке журнала.

2.7.6 Шрифт не найден

```
from matplotlib import font_manager

print(font_manager.findfont("Liberation Sans"))
```

Если шрифт не найден, matplotlib молча подставит DejaVu Sans и предупредит. Проверяйте `findfont` и наличие кириллицы. В Linux установите `fonts-liberation`, в Windows используйте Arial.

2.7.7 SVG с текстом или с кривыми

- `svg.fonttype = "path"` — текст превращается в кривые: вид стабилен, но текст нельзя редактировать.
- `svg.fonttype = "none"` — текст остаётся текстом: удобно для правки, но зависит от шрифтов у читателя.

Для журнала обычно выбирают `path`; для внутренней правки — `none`. В этом проекте по умолчанию `path`.

2.7.8 Единицы plotnine

```
# плохо: дюймы и миллиметры перепутаны
plot.save("figure.png", width=85, height=60)

# хорошо: перевести миллиметры в дюймы
plot.save("figure.png", width=85 / 25.4, height=60 / 25.4, dpi=600)
```

`plotnine.save` принимает ширину и высоту в дюймах, как и matplotlib.

2.7.9 Данные не в tidy-формате

Plotnine ожидает long-формат: одна строка — одно наблюдение, колонки — переменные. Широкая таблица `group, before, after` подойдёт для matplotlib, но для plotnine её нужно привести к `group, time, value` через `melt`.

2.7.10 DPI и цветовой профиль

- DPI влияет только на растр; для SVG/PDF он не важен.
- RGB → CMYK лучше конвертировать в конце, после утверждения макета; проверьте, что цвета не «провалились» в печати.
- Прозрачный фон (`transparent=True`) почти никогда не нужен для журнала: белый фон предсказуемее.

2.8 Чек-лист раздела 2

- ☐ `uv sync --frozen` работает на чистой машине.
- ☐ `pyproject.toml`, `uv.lock`, `.python-version` закоммичены.
- ☐ `.venv/` не закоммичен.
- ☐ Скрипт использует `matplotlib.use("Agg")`.
- ☐ Данные и построение разделены: `make_data.py` ≠ `plot.py`.
- ☐ `apply_style()` вызывается один раз и последним.
- ☐ Нет `sns.set_theme()` после `apply_style()`.
- ☐ Пределы осей и палитра заданы явно, а не «по умолчанию».
- ☐ Сохраняются SVG и PNG; размер — в миллиметрах, не «как получилось».
- ☐ Шрифт найден, кириллица и символы (μ, ±, ≥) отображаются.
- ☐ `uv run python ...` работает из корня репозитория.
- ☐ `uv run ruff check .` проходит.
- ☐ Повторный запуск даёт те же данные и рисунки (фиксированный seed).

2.9 Итоги

- **matplotlib** — фундамент и полный контроль; используйте объектный API.
- **seaborn** — быстрый доступ к статистике и группам; не заменяет matplotlib, а дополняет его.
- **plotnine** — грамматика графики и фасеты; требует tidy-данных и отдельной темы.
- **uv** — воспроизводимость: версия Python, lock-файл, `uv run`, `uv sync --frozen`.
- **goodgraph** — единый стиль и экспорт; это контракт между разделами.

Дальше — раздел 3: требования к цвету, шрифтам, размерам и форматам. Там мы разберём палитры подробнее, научимся проверять рисунок на дальтонизм и печать и соберём таблицу «параметр → значение → источник».

Источники к разделу

- Matplotlib documentation — matplotlib.org
- Matplotlib: Quick start guide — matplotlib.org
- Matplotlib: Artists — matplotlib.org
- Seaborn documentation — seaborn.pydata.org
- Seaborn tutorial — seaborn.pydata.org/tutorial
- Plotnine documentation — plotnine.org
- Plotnine reference — plotnine.org/reference
- uv documentation — docs.astral.sh/uv
- uv: Working on projects — docs.astral.sh/uv/guides/projects
- uv: Running scripts — docs.astral.sh/uv/guides/scripts
- PEP 723: Inline script metadata — peps.python.org/pep-0723
- Wilkinson L. *The Grammar of Graphics*. 2nd ed. Springer, 2005. — doi:10.1007/0-387-28695-0
- ggplot2 documentation — ggplot2.tidyverse.org
- Rougier N. P., Droettboom M., Bourne P. E. Ten Simple Rules for Better Figures // *PLoS Computational Biology*. 2014. Vol. 10, № 9. e1003833. — doi:10.1371/journal.pcbi.1003833
- pandas documentation — pandas.pydata.org
- NumPy documentation — numpy.org/doc
- SciPy documentation — docs.scipy.org
- adjustText documentation — adjusttext.readthedocs.io
- Pillow documentation — pillow.readthedocs.io

Раздел 3. Требования к научной графике: цвет, шрифты, размеры и форматы

В разделе 1 мы сформулировали принципы, в разделе 2 — выбрали инструменты. Теперь наступает самая практичная часть: **конкретные требования**, которые можно проверить до отправки статьи. Не «сделайте красиво», а «ширина 85 мм, кегль не меньше 7 pt, контраст не ниже 3:1, шрифт встроен».

Требования удобно разделить на три уровня:

1. **Смысл** — что именно кодирует цвет, размер, положение.
2. **Доступность** — увидят ли это при дальтонизме, в сером, на печати.
3. **Производство** — дойдёт ли файл до редакции без потерь.

Гайд журнала всегда важнее любых рекомендаций ниже. Если журнал требует RGB, 300 dpi и TIFF — делайте RGB, 300 dpi и TIFF, даже если раздел советует вектор. Но если гайд молчит, значения по умолчанию из этого раздела помогут не изобретать велосипед.

3.1 Цвет: язык, а не украшение

3.1.1 Что кодирует цвет

Цвет в научном графике выполняет одну из четырёх ролей:

- **категория** — группа, препарат, регион, тип события;
- **величина** — концентрация, время, доза, интенсивность;
- **отклонение** — разница с нулём, нормой или контролем;
- **акцент** — главный вывод на фоне серого контекста.

Цвет — мощный, но неточный канал. Точные значения читаются по положению и длине, а цвет хорош для **группировки** и **поиска**. Поэтому не стоит кодировать цветом то, что читатель должен сравнивать количественно: для этого есть оси, точки и подписи.

Три практических следствия:

- цвет должен иметь смысл: если убрать цвет, график не должен потерять информацию (или должен потерять только второстепенную);
- одно и то же соответствие «цвет → группа» сохраняется во всех рисунках;
- цвет не единственный носитель смысла: форма, тип линии, подпись дублируют его.

3.1.2 Три типа палитр

Тип	Вопрос	Примеры	Правило
Качественная	«Какая это категория?»	Okabe–Ito, Paul Tol bright/muted, ColorBrewer Set2	6–8 цветов, без порядка, без радуги
Последовательная	«Насколько велика величина?»	viridis, cividis, Blues, Greens	монотонная светлота, равномерные шаги
Дивергентная	«Насколько отличается от нуля?»	RdBu_r, PuOr, BrBG	два полюса, нейтральная середина в смысловом нуле

Типичная ошибка — использовать последовательную шкалу для категорий («группа 1 — светло-синяя, группа 2 — тёмно-синяя»): читатель начинает искать порядок и величину там, где их нет. Обратная ошибка — раскрасить упорядоченную величину качественными цветами: пропадает направление изменения.

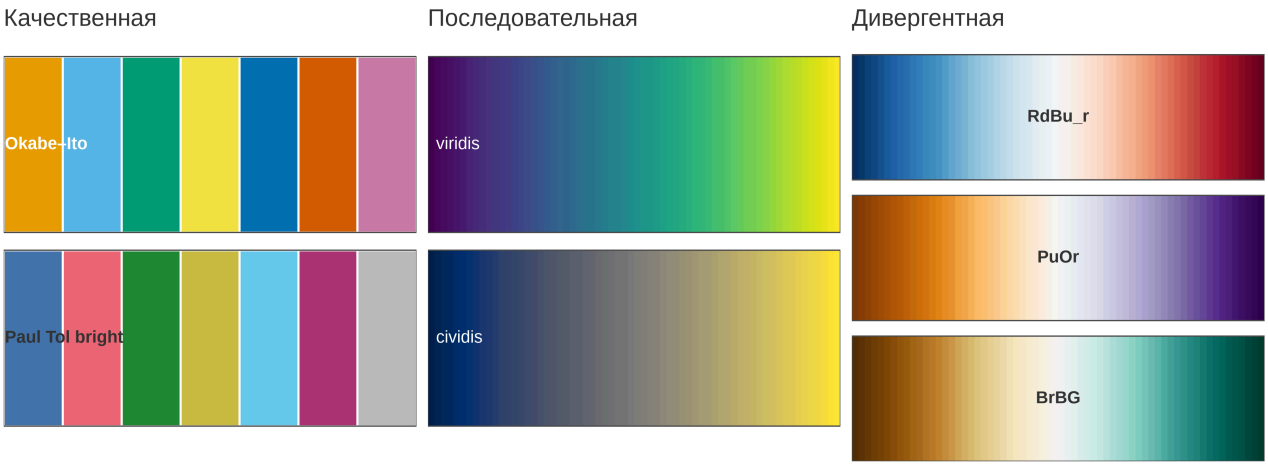


Рисунок 3.1. Три типа палитр: качественная (Okabe–Ito, Paul Tol bright), последовательная (viridis, cividis) и дивергентная (RdBu_r, PuOr, BrBG). Качественная отвечает на вопрос «кто это», последовательная — «сколько», дивергентная — «насколько отличается от нуля».

3.1.3 Качественные палитры

Базовый набор — **Okabe–Ito**. Он различим при протанопии и дейтеранопии и хорошо работает в полиграфии.

Цвет	HEX	Роль
оранжевый	`#E69F00`	категория 1
небесно-голубой	`#56B4E9`	категория 2
сине-зелёный	`#009E73`	категория 3
жёлтый	`#F0E442`	категория 4, только с тёмным контуром
синий	`#0072B2`	основной результат
вермилион	`#D55E00`	акцент/второй результат
красновато-фиолетовый	`#CC79A7`	категория 5
чёрный	`#000000`	текст, оси, а не серии

Дополнительные варианты:

- **Paul Tol bright/muted** — спокойные наборы для больших заливок и слайдов;
- **ColorBrewer** — проверенные качественные, последовательные и дивергентные схемы, в том числе для печати;
- **табличные** палитры (например, `tab10`) удобны, но не оптимизированы под дальтонизм; не полагайтесь на них в финальной версии.

Правила:

- не больше 6–8 категориальных цветов; больше — объединяйте редкие категории в «прочее» или делайте small multiples;
- серый — для контроля, фона и «прочего»; он не должен конкурировать с данными;
- один акцентный цвет на главный вывод; остальное — сдержанное;
- не используйте красный и зелёный как единственную пару «плохо/хорошо»;
- проверяйте палитру в сером и через симулятор дальтонизма.

3.1.4 Последовательные и дивергентные шкалы

Последовательная шкала должна:

- монотонно менять светлоту — тогда она читается в сером;
- быть равномерной по восприятию — без ложных границ;
- подходить для дальтоников;
- иметь подписанную цветовую шкалу с единицами.

`viridis` и `cividis` — хорошие значения по умолчанию. `cividis` специально спроектирован для дальтонизма. `Blues` и `Greens` хороши для печати, но следите за контрастом крайних значений.

```python
import matplotlib as mpl

cmap = mpl.colormaps["viridis"]
# Для дискретных значений:
cmap = mpl.colormaps["viridis"].resampled(5)
```

Почему не `jet` и не «радуга»:

- светлота меняется немонотонно, возникают ложные границы;
- разные шаги данных выглядят как разные по величине;
- жёлтый и голубой на белом фоне теряют контраст;
- шкала плохо читается при дальтонизме и в сером.

Дивергентная шкала нужна, когда есть **смысловой ноль**: изменение, разница, отклонение от нормы. Правила:

- центр шкалы — ровно ноль (или другая содержательная точка);
- пределы симметричны: −3…+3, а не −1…+5;
- два полюса разного цвета, середина — нейтральная;
- не используйте дивергентную шкалу для данных без нуля.

Белая середина `RdBu_r` на белом фоне может «исчезнуть». Если это мешает, добавьте тонкую контурную линию или выберите середину с лёгким оттенком.

3.1.5 Доступность: дальтонизм и серый

Около 8 % мужчин европейского происхождения имеют нарушения цветового зрения; среди женщин доля ниже, но не нулевая. Наиболее часты протанопия и дейтеранопия (красный/зелёный), реже — тританопия (синий/жёлтый).

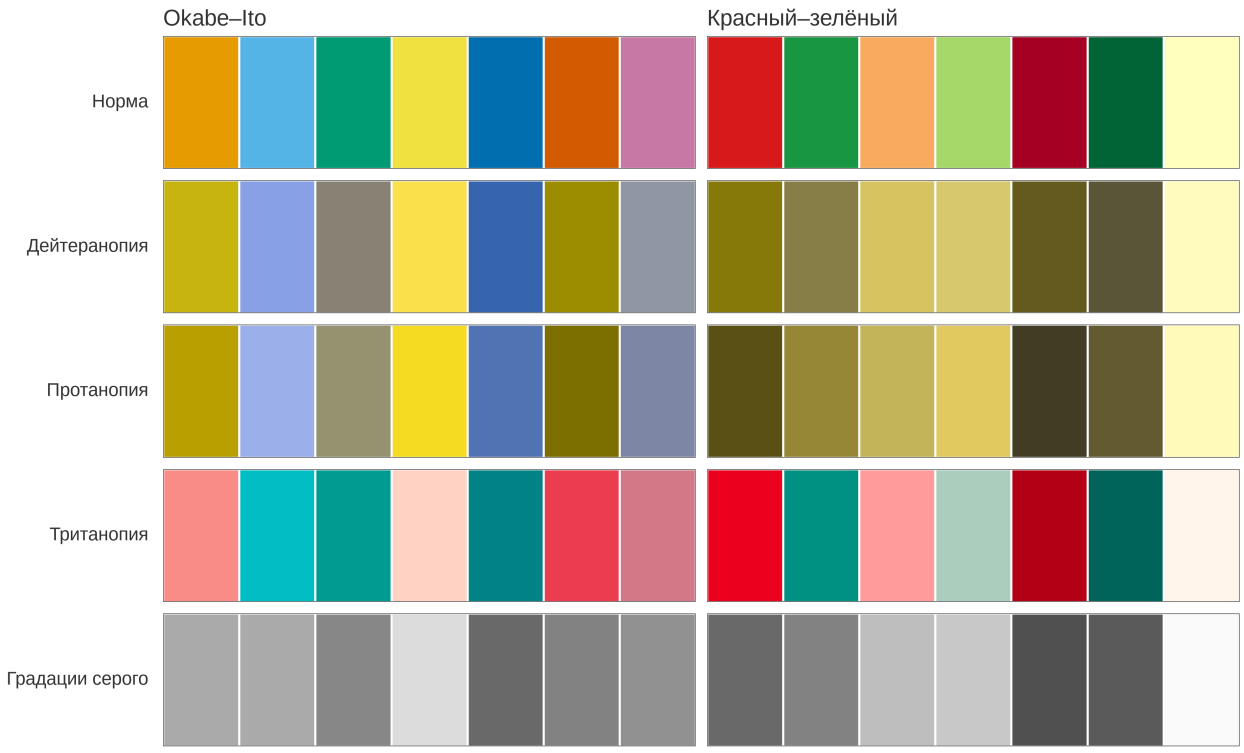


Рисунок 3.2. Симуляция дальтонизма (Machado et al., 2009; severity 100 %). Слева — Okabe–Ito, справа — палитра «красный–зелёный». В нижних рядах видно, что красные и зелёные пары сливаются, а Okabe–Ito сохраняет больше различий. В сером ряду цвет исчезает полностью — форму и подписи нужно дублировать.

Что делать:

- **дублируйте цвет формой, типом линии, штриховкой или прямой подписью**;
- не кодируйте противоположные категории только красным и зелёным;
- проверяйте рисунок в градациях серого;
- проверяйте контраст: текст — не ниже 4,5:1, графические объекты — не ниже 3:1 (WCAG 2.1);

- используйте симуляторы (Coblis, Color Oracle, Photoshop, матрицы Machado) и, в идеале, покажите рисунок человеку с дальтонизмом.

Серый — не «отсутствие цвета», а полноценный канал. В сером графике работают положение, форма, толщина линии и подписи. Если после обесцвечивания серии неразличимы — добавьте форму или подпись.

3.1.6 Цвет в печати

Экран и бумага — разные устройства.

- **sRGB** — для онлайн-публикации и большинства журнальных PDF.
- **CMYK** — для офсетной печати; охват уже, насыщенные зелёные и синие могут «провалиться».
- Конвертируйте в CMYK **в конце**, после утверждения макета, и проверяйте цветопробу.
- Не полагайтесь на светлые оттенки для тонких линий: на бумаге они исчезают.
- Мелкий текст — 100 % K, а не составной «богатый чёрный»: иначе возможны проблемы с приводкой.
- Не используйте прозрачность как единственный способ показать неопределённость: при печати она может стать грязной.

3.1.7 Чек-лист цвета

- ☐ Цвет кодирует смысл, а не украшает.
- ☐ Тип палитры соответствует задаче: категория / величина / отклонение.
- ☐ Категориальных цветов не больше 6–8.
- ☐ Нет радуги и `jet`; последовательная шкала монотонна по светлоте.
- ☐ Дивергентная шкала центрирована в смысловом нуле и симметрична.
- ☐ Красный и зелёный не единственные носители смысла.
- ☐ Рисунок читается в градациях серого.
- ☐ Контраст текста ≥ 4,5:1, графики ≥ 3:1.
- ☐ Цвета согласованы между рисунками.
- ☐ Проверена цветопроба или CMYK-версия, если журнал печатный.

3.2 Шрифты

3.2.1 Гарнитура

Для рисунков почти всегда лучше **sans-serif**: Arial, Helvetica, Liberation Sans, DejaVu Sans. Они плотнее и лучше читаются в мелком кегле, чем засечные гарнитуры. Основной текст статьи может быть serif — это нормально; важно, чтобы рисунок не спорил с текстом.

Правила:

- одна гарнитура на рисунок, максимум две на статью;
- не использовать декоративные и рукописные шрифты;
- проверять метрическую совместимость: Liberation Sans близок к Arial, поэтому вёрстка не «поедет» при замене;
- встраивать шрифт в PDF или переводить текст в кривые в SVG.

3.2.2 Кегли

Кегль задаётся **в конечном физическом размере**. Если рисунок будет 85 мм, не рисуйте его в 16 дюймов и не уменьшайте в Word: 9 pt превратятся в лотерею.

Элемент	**Ориентир**
Заголовок внутри рисунка	9–10 pt
Подписи осей	8–9 pt
Деления и подписи делений	7–8 pt
Легенда	7–8 pt
Метки панелей A/B/C	9–10 pt, bold
Подпись к рисунку (caption)	8–9 pt, в тексте

Практический минимум — **7 pt**. Некоторые журналы допускают 5–7 pt, но это уже риск: в печати и на плохом экране такой текст расплывается. 6 pt — ниже практического минимума; используйте только как крайнюю меру.



Рисунок 3.3. Один и тот же текст при 6–12 pt в конечном размере 85 мм. В PDF рисунок показан 1:1; в HTML он может масштабироваться, поэтому ориентируйтесь на подписи кеглей, а не на размер картинки на экране.

3.2.3 Начертания, регистр, математика

- Regular — по умолчанию; bold — только для акцента.
- Не писать подписи КАПСОМ: это замедляет чтение.
- Не подчёркивать и не добавлять тени/обводки ради «красоты».
- Математические переменные — курсивом: *p*, *n*, *M*, *SD*.
- Единицы — прямым: мм рт. ст., мг, %, °C.
- Не смешивать курсив и прямой шрифт в одном обозначении без причины.
- Десятичный разделитель — по языку публикации: запятая в русском тексте, точка в англоязычном; главное — единообразие.

3.2.4 Кириллица и специальные символы

Проверяйте не только буквы, но и символы:

- μ, ±, ≥, ≤, →, ≈, ×, Δ, °C;
- минус и дефис — разные символы; в осях допустим обычный минус, но он не должен превращаться в дефис;
- неразрывный пробел между числом и единицей: 8,9 мм, 10 мг, 5 %.

Если шрифт не найден, matplotlib молча подставит DejaVu Sans и предупредит. Проверяйте:

```
from matplotlib import font_manager

print(font_manager.findfont("Liberation Sans"))
```

Для проверки глифов в шрифте можно использовать `fontTools`:

```
from fontTools.ttLib import TTFont

font = TTFont("path/to/font.ttf")
cmap = font.getBestCmap()
print(all(ord(ch) in cmap for ch in "μ±≥≤→≈×Δ°C"))
```

3.2.5 Композиция текста

- Подписи осей — параллельно осям, не под углом 45° без необходимости.
- Единицы — в подписи оси: «САД, мм рт. ст.», а не «САД (мм рт. ст.)» и не отдельной строкой в легенде.
- Деления не должны налезать друг на друга; при необходимости уменьшите их число, а не кегль.
- Легенда — тем же кеглем, что и деления; без рамки, если позволяют оси.
- Прямые подписи лучше легенды, если серий немного.
- Не располагайте подпись поверх данных без белой подложки или ореола.

3.2.6 Чек-лист шрифтов

- ☐ Одна гарнитура, sans-serif для рисунка.
- ☐ Кегли 7–10 pt в конечном размере.
- ☐ Нет КАПСА и декоративных эффектов.
- ☐ Математика курсивом, единицы прямым.
- ☐ Кириллица и символы (μ, ±, ≥, →, °C) отображаются.
- ☐ Десятичный разделитель единообразен.
- ☐ Подписи не налезают друг на друга и не обрезаны.
- ☐ Шрифт встроен в PDF или переведён в кривые в SVG.

3.3 Размеры и композиция

3.3.1 Ширина колонки

Ширина	Типичное применение
85 мм	одна колонка
114 мм	полторы колонки
170 мм	две колонки
180 мм	максимальная ширина полосы

Nature ориентирует на 89 и 183 мм; другие журналы — на 85–90 и 170–183 мм. Проверяйте гайд. Проектируйте сразу в конечной ширине: `figsize` в matplotlib задаётся в дюймах, поэтому 85 мм = 3,35 дюйма, 170 мм = 6,69.

3.3.2 Поля и отступы

- Внешние поля — 3–5 мм, чтобы подписи не обрезались.
- Внутренние отступы между панелями — 2–4 мм.
- Не используйте `bbox_inches="tight"` для журнального рисунка: он меняет физический размер.
- `layout="constrained"` предпочтительнее ручного `subplots_adjust`, если нет особых требований.
- Проверяйте, что подписи осей и легенда не выходят за холст.

3.3.3 Пропорции

- Временной ряд — широкий: 16:9 или 3:2, чтобы тренд читался.
- Scatter — ближе к квадрату, особенно если сравниваете наклоны.
- Столбцы — не слишком широкие: расстояние между ними важнее толщины.
- Не растягивайте рисунок непропорционально: это меняет восприятие наклонов и площадей.
- Для временных рядов полезно «banking to 45°»: подобрать аспект так, чтобы типичный наклон был около 45°.

3.3.4 Панели и выравнивание

- Метки A, B, C — слева сверху, одним кеглем и в одном месте.
- Одинаковые пределы осей, если панели сравниваются.
- Общая легенда — для всех панелей, а не отдельная в каждой.
- Выравнивайте области построения, а не только внешние рамки.
- Не дублируйте подписи осей на каждой панели, если они общие.

3.3.5 Чек-лист размеров

- ☐ Ширина соответствует колонке журнала.
- ☐ Рисунок спроектирован в конечном размере, а не уменьшен.
- ☐ Поля 3–5 мм, ничего не обрезано.
- ☐ Панели выровнены, метки A/B/C на месте.
- ☐ Пропорции не искажены.
- ☐ `bbox_inches="tight"` не используется для точной ширины.

3.4 Форматы и разрешение

3.4.1 Вектор или растр

Формат	Тип	Когда использовать	Ограничения
PDF	вектор	графики, схемы, финальная отправка	проверьте встраивание шрифтов
SVG	вектор	веб, правка в Inkscape/Illustrator	текст может стать кривыми

Формат	Тип	Когда использовать	Ограничения
EPS	вектор	старые редакции, LaTeX	устаревает, проверяйте шрифты
AI	вектор	правка в Illustrator	не все журналы принимают
TIFF	растр	фото, микроскопия	большой файл; LZW без потерь
PNG	растр	веб, скриншоты	не для печати больших фото
JPEG	растр	только фото	артефакты на линиях и тексте

На практике: **линии, текст и графики — вектор; фото — растр**. Не вставляйте скриншот графика в статью: это растр с потерями и без масштабирования.

3.4.2 Разрешение

- Фото и микроскопия — 300 dpi в конечном размере.
- Штриховая графика, если журнал требует растр, — 600–1200 dpi.
- Эффективное разрешение = пиксели / дюймы конечного размера. Например, 2000 px при ширине 3,35″ дают ≈ 597 dpi.
- Не увеличивайте растр интерполяцией: это не добавляет деталей.
- Проверяйте, что после масштабирования в Word разрешение не упало.

3.4.3 Цветовые профили

- sRGB IEC61966-2.1 — безопасный профиль для онлайн-публикации.
- Adobe RGB — шире, но не все редакции его принимают.
- CMYK — для печати; конвертируйте в конце и проверяйте цветопробу.
- Встраивайте профиль в TIFF/JPEG; для PDF используйте OutputIntent, если журнал просит.
- Не конвертируйте RGB → CMYK → RGB многократно: цвета деградируют.

3.4.4 Шрифты в файлах

- PDF: `pdf.fonttype = 42` — TrueType, текст остаётся текстом и встроен.
- SVG: `svg.fonttype = "path"` — текст в кривые, вид стабилен; `"none"` — текст остаётся текстом, но зависит от шрифтов.
- EPS: проверяйте, что шрифты встроены или заменены.
- Перед отправкой откройте PDF в другом просмотрщике и проверьте, что шрифты не подменились.

3.4.5 Метаданные и именование

- Имена файлов — по схеме журнала: `Figure1.pdf`, `fig_01.svg` и т. п.
- Не включайте дату и версию в имя финального файла, если журнал просит иначе; версию храните в репозитории.
- Подпись — отдельным текстом, не внутри изображения.
- Удаляйте дату из SVG-метаданных, если готовите воспроизводимый архив: иначе файл меняется при каждом сохранении.
- Для архивации подходит PDF/A-2b, если журнал его принимает.

3.4.6 Чек-лист форматов

- ☐ Вектор для линий и текста, растр для фото.
- ☐ 300 dpi для фото, 600–1200 dpi для штриховки.
- ☐ Нет JPEG для графиков и текста.
- ☐ Шрифты встроены или переведены в кривые.
- ☐ Цветовой профиль соответствует журналу.
- ☐ Нет скриншотов и апскейла.
- ☐ Имя файла и формат соответствуют требованиям.
- ☐ PDF проверен в другом просмотрщике.

3.5 Сводная таблица требований

Параметр	Значение по умолчанию	Как проверить
Ширина	85 / 114 / 170 мм	`figsize`, PDF при 100 %
Кегль	7–10 pt	линейка в PDF, `fontsize`
Линии	0,25–1 pt	PDF при 100 %
Растр	300 dpi фото, 600+ dpi штрих	размер в пикселях / дюймы
Вектор	PDF/SVG/EPS	текст и линии не пиксельные

Параметр	Значение по умолчанию	Как проверить
Цвет	sRGB онлайн, CMYK печать	профиль, цветопроба
Поля	3–5 мм	PDF, отсутствие обрезки
Контраст	4,5:1 текст, 3:1 графика	WCAG-калькулятор
Подпись	вне изображения, самодостаточная	чтение без текста статьи
Единицы	СИ, пробел перед единицей	вычитка

3.6 Чек-лист раздела 3

- ☐ Палитра соответствует задаче и проверена на дальтонизм.
- ☐ В сером графике сохраняется смысл.
- ☐ Контраст текста и графики достаточен.
- ☐ Шрифт sans-serif, кегли 7–10 pt.
- ☐ Кириллица и специальные символы отображаются.
- ☐ Ширина соответствует журналу; поля не обрезаны.
- ☐ Панели выровнены, метки A/B/C на месте.
- ☐ Вектор/растр выбран по типу содержимого.
- ☐ Разрешение достаточно; нет апскейла и скриншотов.
- ☐ Шрифты встроены; PDF проверен в другом просмотрщике.
- ☐ Цветовой профиль соответствует требованиям.
- ☐ Подпись самодостаточна и не встроена в изображение.

3.7 Итоги

- Цвет — это канал кодирования, а не декор: тип палитры зависит от задачи.
- Качественная палитра — 6–8 цветов, без радуги, с проверкой на CVD.
- Последовательная шкала — монотонная светлота; дивергентная — ноль в центре.
- Шрифт — sans-serif, 7–10 pt в конечном размере, без КАПСА и эффектов.
- Ширина — 85/114/170 мм; проектируйте сразу в конечном размере.
- Вектор для линий и текста, растр для фото; 300/600+ dpi.
- Шрифты встраиваются, профиль проверяется, подпись остаётся текстом.

Дальше — раздел 4: основные виды научных графиков и области их применения. Там мы соберём таблицу «задача → график → ограничения» и перейдём к практике в разделе 5.

Источники к разделу

- Okabe M., Ito K. Color Universal Design (CUD) — jfly.uni-koeln.de/color
- Paul Tol’s colour schemes — personal.sron.nl/ pault
- ColorBrewer 2.0 — colorbrewer2.org
- Viridis colormap — bids.github.io/colormap
- Matplotlib: Choosing colormaps — matplotlib.org
- Machado G. M., Oliveira M. M., Fernandes L. A. F. A Physiologically-based Model for Simulation of Color Vision Deficiency // *IEEE TVCG*. 2009. Vol. 15, № 6. P. 1291–1298. — doi:10.1109/TVCG.2009.113
- W3C. WCAG 2.1: Contrast (Minimum) — w3.org
- W3C. WCAG 2.1: Non-text Contrast — w3.org
- Wilke C. O. *Fundamentals of Data Visualization*. O’Reilly, 2019. — clauswilke.com/dataviz
- Cairo A. *The Truthful Art*. New Riders, 2016. — thefunctionalart.com
- Nature. Formatting guide — nature.com
- ICMJE Recommendations — icmje.org
- ГОСТ 7.32-2017 — docs.cntd.ru
- ГОСТ 8.417-2002 — docs.cntd.ru
- Matplotlib: Fonts — matplotlib.org
- FontTools documentation — fonttools.readthedocs.io

Раздел 4. Основные виды научных графиков и области применения

Выбор графика — первый проектный шаг, и его почти нельзя исправить оформлением. Если для сравнения групп выбран график распределения, читатель не увидит сравнения; если для динамики — столбцы по категориям, он не увидит времени. Здесь — **карта**, а не учебник: мы разбираем семейства графиков, их вопросы и ограничения, а подробные разборы с данными, «плохими» и «правильными» версиями будут в разделе 5.

> В разделе 5 каждый **отдельный график** получит свою папку, свой генератор `make_data.py` и свой набор `data/NN_dataset.csv`. Один «вид» на карте — это семейство; внутри него несколько графиков (например, линия, step, area и small multiples), и каждый станет отдельным разделом. Данные одного графика не переиспользуются в другом.

4.1 Пять вопросов перед выбором графика

Полезно задать себе пять вопросов — в этом порядке.

1. **Какой вопрос я задаю?** Сравнить группы, описать распределение, показать связь, динамику, состав, структуру, время до события, качество теста?
2. **Какие переменные?** Категориальные, непрерывные, временные, парные, цензурированные?
3. **Сколько наблюдений и групп?** 10 точек или 10 000? Одна группа или двадцать?
4. **Что важнее читателю?** Точное значение, ранг, форма распределения, неопределённость, наличие выбросов?
5. **Что читатель должен уметь прочитать?** Величину эффекта, разброс, доверительный интервал, порог, число рисков?

Если ответ на первый вопрос не формулируется, график, скорее всего, не нужен: данные лучше показать таблицей.

4.2 Карта семейств

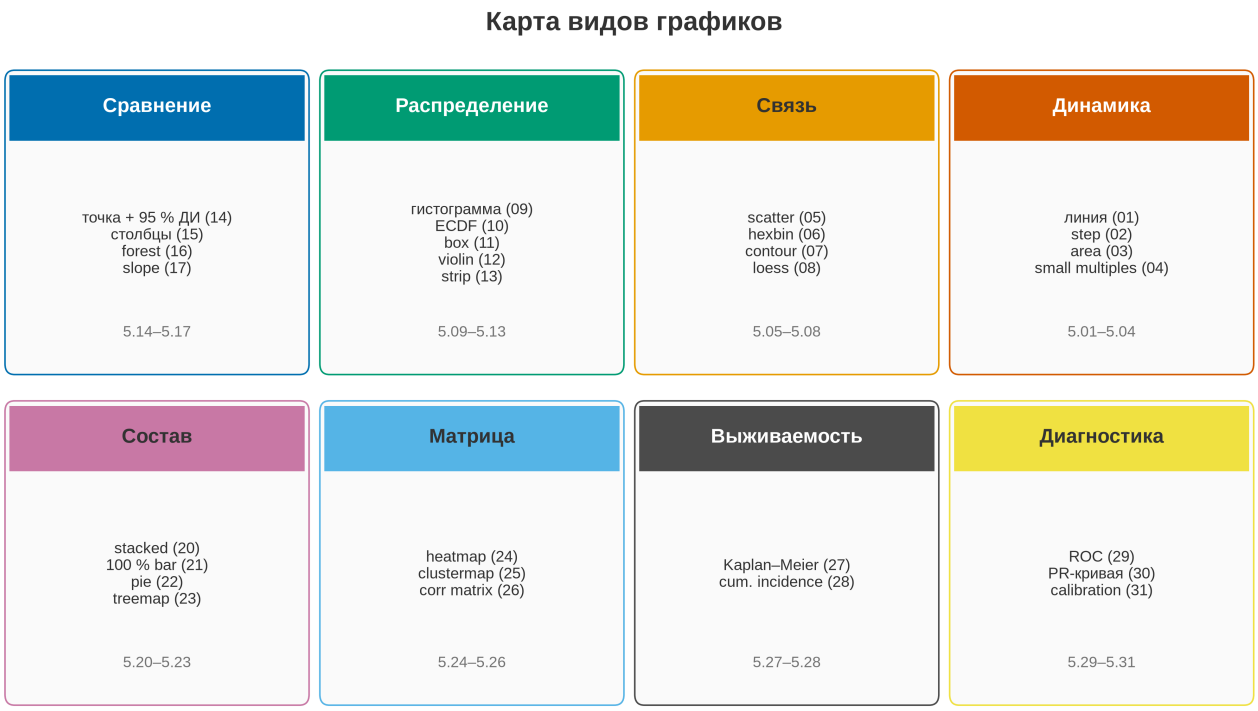


Рисунок 4.1. Карта семейств: вопрос, примеры графиков и номер раздела 5, где вид будет разобран подробно. Карта не заменяет таблицу выбора — она показывает связи между вопросами и формами.

Семейство	Вопрос	Графики (раздел 5)	Ограничения
Сравнение	«Кто больше и насколько?»	точка + 95 % ДИ (5.14), столбцы (5.15), forest (5.16), slope (5.17)	столбцы скрывают распределение; нужна нулевая база
Распределение	«Как устроена выборка?»	гистограмма (5.09), ECDF (5.10), box (5.11), violin (5.12), strip (5.13)	ширина бинов; box скрывает бимодальность; violin требует *n*
Связь	«Есть ли зависимость?»	scatter (5.05), hexbin (5.06), contour (5.07), loess (5.08)	overplotting; корреляция ≠ причинность
Динамика	«Как меняется во времени?»	линия (5.01), step (5.02), area (5.03), small multiples (5.04)	нерегулярные интервалы; нельзя соединять категории
Состав	«Из чего состоит целое?»	stacked bar (5.20), 100 % bar (5.21), pie (5.22), treemap (5.23)	части должны суммироваться; pie — максимум 5 долей
Матрица	«Где значения велики или малы?»	heatmap (5.24), clustermap (5.25), correlation matrix (5.26)	масштаб, кластеризация, colorbar
Выживаемость	«Когда происходит событие?»	Kaplan–Meier (5.27), cumulative incidence (5.28)	цензурирование, at-risk, конкурирующие риски
Диагностика	«Насколько хорош тест?»	ROC (5.29), precision–recall (5.30), calibration (5.31)	prevalence, порог, доверительные интервалы
Парные измерения	«Как меняется у каждого?»	slope chart (5.17), paired dot (5.18), Bland–Altman (5.19)	парность, пропущенные пары
Оценки эффекта	«Какой эффект и насколько он точен?»	forest (5.16), caterpillar (5.32), funnel (5.33)	лог-шкала для отношений, гетерогенность

4.3 Семейства подробнее

4.3.1 Сравнение

Вопрос: «Кто больше, насколько и насколько уверенно?»

Хорошие формы:

- **точка + 95 % ДИ** — лучше столбцов, потому что положение читается точнее длины, а интервал показывает неопределённость;
- **forest plot** — для нескольких оценок и эффектов;
- **slope chart** — для парных измерений;
- **столбцы** — только если нулевая база обязательна и распределение не главное.

На что смотреть: показывать эффект и ДИ, а не только *p*; не обрезать ось у столбцов; не скрывать *n* и разброс. Подробно — 5.14, 5.15, 5.16, 5.17.

4.3.2 Распределение

Вопрос: «Как устроена выборка: центр, разброс, форма, выбросы?»

Формы:

- **гистограмма** — форма распределения, но зависит от ширины бинов;
- **ECDF** — точное чтение квантилей и сравнение выборок;
- **box** — компактно, но скрывает бимодальность и размер выборки;
- **violin** — форма, но требует достаточного *n* и может вводить в заблуждение при малых группах;
- **strip / swarm** — отдельные точки, лучше при малых *n*;
- **raincloud** — комбинация, но не для каждого журнала.

Подробно — 5.09–5.13.

4.3.3 Связь

Вопрос: «Меняется ли одна величина вместе с другой?»

Формы:

- **scatter** — базовая форма; при больших *n* — **hexbin** или **contour**;
- **loess/regression** — тренд, но не заменяет точки;
- **log-шкалы** — если связь мультипликативная;
- **small multiples** — если есть третья категориальная переменная.

На что смотреть: не путать корреляцию с причинностью, показывать *n* и выбросы, не использовать две оси Y. Подробно — 5.05–5.08.

4.3.4 Динамика

Вопрос: «Как величина меняется во времени?»

Формы:

- **линия** — регулярные интервалы, одна или несколько серий;
- **step** — измерения «до/после» или дискретные события;
- **area** — накопленный итог, но не для сравнения многих серий;
- **small multiples** — много серий или групп;
- **individual trajectories** — если важна гетерогенность.

На что смотреть: время — по оси X; не соединяйте линии через категории; показывайте пропуски и нерегулярность; не начинайте ось с произвольного значения без объяснения. Подробно — 5.01–5.04.

4.3.5 Состав

Вопрос: «Из чего состоит целое и как доли меняются?»

Формы:

- **stacked bar** — состав по группам;
- **100 % bar** — сравнение долей;
- **pie** — только 3–5 долей, и только если сумма долей действительно 100 %;
- **treemap** — иерархический состав;
- **alluvial/Sankey** — потоки, но требуют осторожности.

На что смотреть: части должны суммироваться в целое; круговая диаграмма плохо читается при многих долях и почти всегда проигрывает полосам. Подробно — 5.20–5.23.

4.3.6 Матрица и многомерность

Вопрос: «Где значения велики, где структура, где корреляции?»

Формы:

- **heatmap** — матрица значений; обязательна цветовая шкала с единицами;
- **clustermap** — порядок строк/столбцов подсказывает структуру, но кластеризация субъективна;
- **correlation matrix** — только как дополнение, не как доказательство;
- **PCA/biplot** — уменьшение размерности, но интерпретация меняется;
- **small multiples / parallel coordinates** — альтернатива «радуге».

Подробно — 5.24–5.26.

4.3.7 Выживаемость

Вопрос: «Когда происходит событие и как различаются группы?»

Формы:

- **Kaplan–Meier** — кривые выживаемости;
- **cumulative incidence** — конкурирующие риски;
- **forest plot** — hazard ratio;
- **at-risk table** — число под риском по времени.

На что смотреть: цензурирование, число рисков, доверительные интервалы, log-rank/Cox, проверка пропорциональности. Кривые без at-risk table и без числа событий читаются плохо. Подробно — 5.27–5.28.

4.3.8 Диагностика

Вопрос: «Насколько хорош тест и при каком пороге?»

Формы:

- **ROC** — чувствительность/специфичность, но зависит от prevalence;
- **precision–recall** — лучше при сильном дисбалансе классов;
- **calibration** — соответствие предсказанных и наблюдаемых вероятностей;
- **decision curve** — клиническая полезность.

На что смотреть: указывать prevalence, порог, ДИ, число наблюдений; AUROC — не единственная метрика. Подробно — 5.29–5.31.

4.3.9 Парные измерения

Вопрос: «Как меняется каждый объект?»

Формы:

- **slope chart** — линии «до/после» для каждого пациента;
- **paired dot plot** — пары точек;
- **Bland–Altman** — согласие методов, а не корреляция.

На что смотреть: не анализировать парные данные как независимые группы; показывать пропущенные пары. Подробно — 5.17–5.19.

4.3.10 Оценки эффекта и неопределённость

Вопрос: «Какой эффект и насколько он точен?»

Формы:

- **forest plot** — оценки и ДИ для нескольких групп/исследований;
- **caterpillar** — много оценок;
- **funnel** — публикационное смещение, но только при достаточном числе исследований.

На что смотреть: эффект + ДИ, лог-шкала для отношений (OR, RR, HR), гетерогенность, а не только *p*. Подробно — 5.16, 5.32–5.33.

4.4 Когда таблица лучше графика

График не всегда выигрывает. Таблица лучше, если:

- значений мало (3–8) и нужна точная выборка;
- единицы разные и график требует нескольких осей;
- важны точные цифры, а не форма;
- категорий слишком много для читаемой палитры;
- данные неполные или несопоставимые.

Практическое правило: **если таблица отвечает на вопрос быстрее и точнее, делайте таблицу**. График — для закономерностей, сравнений и формы, а не для переписывания чисел.

4.5 Антипаттерны выбора

Антипаттерн	Почему плохо	Альтернатива
3D-столбцы и 3D-круг	перспектива искажает длину/площадь	2D
Круговая с 8–12 долями	углы и площади читаются плохо	полосы, stacked bar
Столбцы с SD/SEM	скрывают распределение и *n*	точки, box, violin
Обрезанная ось у столбцов	искажает длину	нулевая база или точки
Линия по категориям	навязывает порядок	столбцы/точки
Две оси Y	ложная корреляция	две панели или индекс
Scatter на 100 000 точек	overplotting	hexbin, density, contour
ROC без prevalence	AUROC не отражает клиническую пользу	PR, calibration
Kaplan–Meier без at-risk	хвост кривой не интерпретируется	at-risk table
Heatmap без шкалы	цвет не читается	colorbar, contour
Radar chart	площадь/угол искажают	small multiples, bar
Word cloud	площадь не пропорциональна	bar
100 % stacked на 20 категорий	невозможно сравнить	small multiples

4.6 Что дальше: как устроен раздел 5

Раздел 5 — практическая часть. Каждый вид графика живёт в своей папке:

```
05_charts/01_line/
├── README.md
├── line.md
├── line.typ
└── scripts/
    ├── make_data.py   # свой генератор → data/01_dataset.csv
    ├── bad_plot.py    # неудачный график → images/05_01_bad_line.*
    └── good_plot.py   # правильный график → images/05_01_good_line.*
```

В каждом разборе будет:

1. сфера применения и типичные вопросы;
2. реалистичные синтетические данные с единицами и диапазонами;
3. неудачный график и разбор ошибок;
4. правильный график и объяснение улучшений;
5. чек-лист и связь с нормативами.

План:

Семейство	**Разделы 5**
Динамика	5.01 линия, 5.02 step, 5.03 area, 5.04 small multiples
Связь	5.05 scatter, 5.06 hexbin, 5.07 contour, 5.08 loess
Распределение	5.09 гистограмма, 5.10 ECDF, 5.11 box, 5.12 violin, 5.13 strip
Сравнение	5.14 точка + ДИ, 5.15 столбцы, 5.16 forest, 5.17 slope
Состав	5.20 stacked bar, 5.21 100 % bar, 5.22 pie, 5.23 treemap
Матрица	5.24 heatmap, 5.25 clustermap, 5.26 correlation matrix
Выживаемость	5.27 Kaplan–Meier, 5.28 cumulative incidence
Диагностика	5.29 ROC, 5.30 precision–recall, 5.31 calibration
Парные	5.17 slope, 5.18 paired dot, 5.19 Bland–Altman
Оценки эффекта	5.16 forest, 5.32 caterpillar, 5.33 funnel

Полный индекс с папками и данными — в `05_charts/README.md`. Порядок может меняться по запросу. Каждый график получит **свой** генератор и **свой** датасет; переиспользование CSV между графиками запрещено.

4.7 Чек-лист выбора графика

- ☐ Вопрос сформулирован одним предложением.
- ☐ Тип переменных определён: категориальные, непрерывные, время, парные, цензурированные.
- ☐ Известны n и число групп.
- ☐ Понятно, что важнее: точное значение, форма, неопределённость.
- ☐ Выбрано семейство, а не «любимый тип графика».
- ☐ Проверены ограничения семейства.
- ☐ Рассмотрена альтернатива в виде таблицы.
- ☐ Цвет и форма не единственные носители смысла.
- ☐ График выдержит конечную ширину и печать.
- ☐ Указан раздел 5, где вид будет разобран подробно.

4.8 Итоги

- Тип графика определяется вопросом и структурой данных, а не привычкой.
- Сравнение, распределение, связь, динамика, состав, матрица, выживаемость, диагностика, парные измерения и оценки эффекта — разные семейства с разными ограничениями.
- Столбцы — не универсальный график; круговая — не универсальный график состава.
- Если таблица точнее и быстрее, выбирайте таблицу.
- Раздел 5 даст полный цикл для каждого из 33 графиков: данные → плохо → разбор → хорошо, с отдельным генератором и датасетом.

Источники к разделу

- Wilke C. O. *Fundamentals of Data Visualization*. O’Reilly, 2019. — clauswilke.com/dataviz
- Cairo A. *The Truthful Art*. New Riders, 2016. — thefunctionalart.com
- Tufte E. R. *The Visual Display of Quantitative Information*. Graphics Press, 1983. — edwardtufte.com
- Cleveland W. S., McGill R. Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods // *JASA*. 1984. Vol. 79, № 387. P. 531–554. — doi:10.1080/01621459.1984.10478080
- Weissgerber T. L. et al. Beyond Bar and Line Graphs: Time for a New Data Presentation Paradigm // *PLoS Biology*. 2015. Vol. 13, № 4. e1002128. — doi:10.1371/journal.pbio.1002128
- From Data to Viz — data-to-viz.com
- Financial Times Visual Vocabulary — github.com/Financial-Times/chart-doctor
- Data Viz Project — datavizproject.com
- CONSORT 2010 — consort-statement.org
- STROBE — strobe-statement.org
- PRISMA — prisma-statement.org

5.01 Линейный график: динамика во времени

Линейный график — самый частый в научных статьях и один из самых обманчивых. Он кажется простым: время по X, значение по Y, точки соединили. Но именно здесь легко спрятать неопределённость, раздуть эффект масштабом и потерять внимание читателя к главному.

Это первый практический разбор. Данные, «плохая» и «правильная» версии используют один и тот же CSV; различается только визуализация.

5.01.1 Сфера применения

Линейный график отвечает на вопрос: **как величина меняется во времени или вдоль упорядоченной оси?**

Где это пригодится:

- повторные измерения у одних и тех же пациентов (до/через 4/8/12 недель);
- динамика биомаркеров, АД, ЧСС, глюкозы, HbA1c;
- кривые роста и дозозависимые эффекты;
- временные ряды в эпидемиологии и организации здравоохранения;
- сравнение нескольких групп во времени.

Когда линия **не** подходит:

- ось X — неупорядоченные категории (пол, регион, препарат): линия навязывает порядок и расстояние;
- серий слишком много (больше 5–7): лучше small multiples;
- интервалы времени неравномерны, а вы строите равномерные позиции — это искажает скорость изменения;
- показаны только средние, а разброс и *n* важны для вывода;
- данные цензурированы (выживаемость) — нужен Kaplan–Meier, а не линия.

5.01.2 Данные

Сценарий: гипотетическое 12-недельное исследование, 120 пациентов с артериальной гипертензией, три группы. САД измеряют на 0, 4, 8 и 12-й неделе.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`group`	группа терапии	—
`week`	неделя измерения (0, 4, 8, 12)	недели
`sbp_mm_hg`	систолическое АД	мм рт. ст.

n = 40 в каждой группе, 480 строк в long-формате. Данные синтетические, но правдоподобные по единицам и величинам: плацебо снижает САД примерно до 148, препарат A — до 139, препарат B — до 132 мм рт. ст.

Генератор: `05_charts/01_line/scripts/make_data.py → data/01_dataset.csv`.

5.01.3 Плохой график

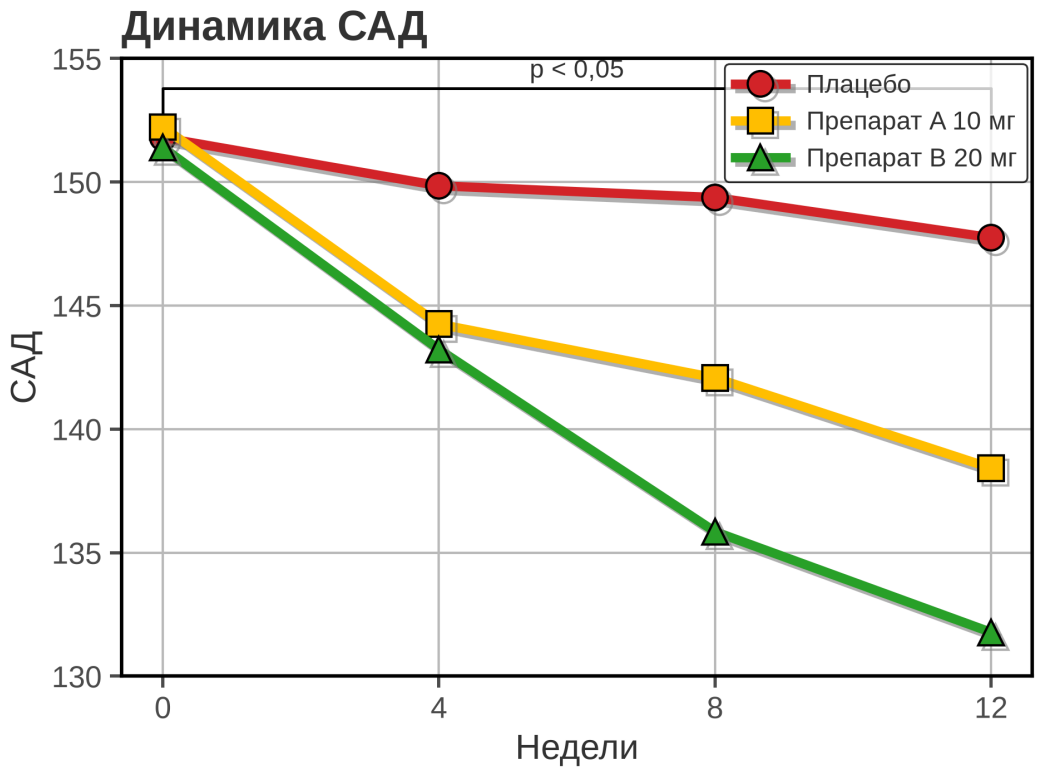


Рисунок 5.1.1. Плохой линейный график: обрезанная ось, светофорные цвета, тени, тяжёлая сетка и легенда поверх данных.

Код: `05_charts/01_line/scripts/bad_plot.py`.

Что не так:

1. **Обрезанная ось Y (130–155).** Для линии нулевая база не обязательна, но диапазон не должен обрезать данные и создавать ложный драматизм. Здесь препарат В почти упирается в нижнюю границу, а плацебо выглядит почти плоским; читатель переоценивает разницу.
2. **Светофорная палитра.** Красный — жёлтый — зелёный неразличим при протанопии/дейтеранопии и навязывает оценку «плохо — средне — хорошо».
3. **Толстые линии, тени и крупные маркеры.** Тень не кодирует данные; она утяжеляет линии и мешает сравнивать наклоны.
4. **Тяжёлая сетка по обеим осям и полная рамка.** Сетка конкурирует с данными, вертикальные линии не помогают читать недели.
5. **Заголовок «Динамика САД».** Он не сообщает, что именно произошло и какой вывод должен сделать читатель.
6. **Нет единиц, *n* и неопределённости.** «САД» — это что: исходное, конечное, среднее? В каких единицах? Сколько пациентов?
7. **Только средние.** Без доверительных интервалов или полос разброса нельзя понять, насколько устойчива разница.
8. **Легенда внутри поля данных.** Она дублирует подписи и закрывает часть линий.
9. **Брекет «p < 0,05» без теста и эффекта.** Значимость не заменяет величину эффекта; не указано, какой тест и какие группы сравнивались.

Дополнительно: равномерные позиции 0/4/8/12 здесь корректны, потому что интервалы равны. Если бы интервалы были неравными (0, 2, 12, 52 недели), точки нужно ставить по реальному времени, а не через равные шаги.

5.01.4 Правильный график

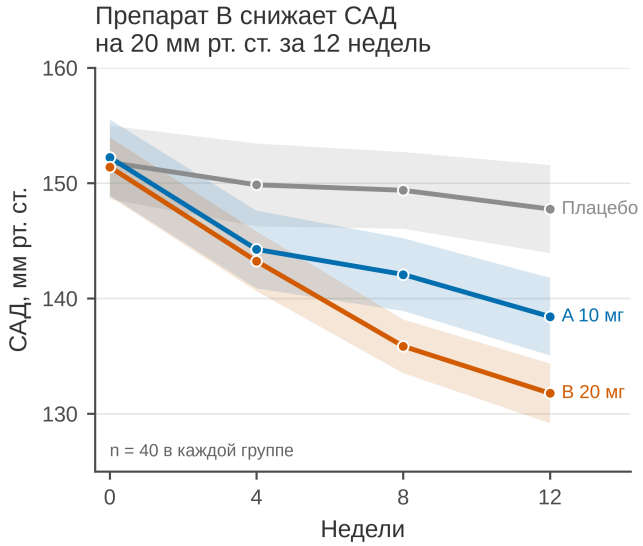


Рисунок 5.1.2. Правильный линейный график: среднее и 95 % ДИ, палитра Okabe–Ito, прямые подписи и заголовок-вывод.

Код: `05_charts/01_line/scripts/good_plot.py`.

Что изменилось:

1. **Ширина 85 мм, высота 72 мм.** Рисунок спроектирован в конечном журнальном размере, а не уменьшен из слайда.
2. **Среднее и 95 % ДИ.** Полосы показывают неопределённость; видно, где различия уверенные, а где интервалы пересекаются.
3. **Палитра Okabe–Ito.** Серый — контроль, синий — препарат А, вермилион — препарат В; цвета различимы при дальтонизме.
4. **Прямые подписи в конце линий.** Легенда не нужна; читатель сразу видит, какая линия какой группе соответствует.
5. **Заголовок-вывод.** «Препарат В снижает САД на 20 мм рт. ст. за 12 недель» — сразу понятно, что читать.
6. **Единицы и *n*.** «САД, мм рт. ст.» и «n = 40 в каждой группе»; в подписи можно добавить определение ДИ.
7. **Ось Y 125–160.** Достаточный контекст: видны исходный уровень, конечные значения и величина снижения, без обрезки «под эффект».
8. **Лёгкая сетка только по Y.** Верхняя и правая рамки убраны; данные контрастнее осей.
9. **Маркеры только в измеренных точках.** Линия соединяет реальные недели 0/4/8/12, а не произвольные позиции.

5.01.5 Было и стало

Что было	Почему плохо	Что сделано
ось Y 130–155	масштаб раздувает различия	125–160, виден контекст
красный/жёлтый/зелёный	ложная семантика, CVD	Okabe–Ito: серый/синий/вермилион
тени, толстые линии	декор, мешает сравнивать наклоны	линии 1,8 pt, маркеры 4 pt
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
легенда	дублирует подписи	прямые подписи
«Динамика САД»	нет сообщения	заголовок-вывод
нет единиц и *n*	невозможно прочитать	единицы, *n*
только средние	скрыта неопределённость	среднее + 95 % ДИ
p < 0,05	нет теста и эффекта	эффект и ДИ; тест — в подписи

5.01.6 Чек-лист линейного графика

- ☐ Время (или упорядоченная ось) — по X, значения — по Y.

- ☐ Интервалы на оси X соответствуют реальному времени.
- ☐ Ось Y не обрезает данные и не создаёт ложного драматизма.
- ☐ Показаны среднее и 95 % ДИ (или individual trajectories).
- ☐ Указаны *n* и единицы.
- ☐ Цвет не единственный носитель смысла.
- ☐ Легенда не закрывает данные; лучше прямые подписи.
- ☐ Заголовок сообщает вывод, а не «динамику».
- ☐ Нет 3D, теней и тяжёлой сетки.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.01.7 Как воспроизвести

```
uv sync
uv run python 05_charts/01_line/scripts/make_data.py
uv run python 05_charts/01_line/scripts/bad_plot.py
uv run python 05_charts/01_line/scripts/good_plot.py
```

Вот как выглядит «правильный» код:

```
fig, ax = plt.subplots(
    figsize=figure_size("single", height_mm=72),
    layout="constrained",
)
ax.fill_between(x, part["ci_low"], part["ci_high"], color=color, alpha=0.15)
ax.plot(x, part["mean"], color=color, marker="o", linewidth=1.8)
ax.annotate(GROUP_SHORT[group], (x[-1] + 0.08, part["mean"].iloc[-1]))
save_figure(fig, "05_01_good_line")
```

5.01.8 Что дальше

Следующий график — **5.02 step**: ступенчатая функция для событий, измерений «до/после» и дискретных изменений. У него будет свой генератор и свой `data/02_dataset.csv`.

Источники

- Cleveland W. S., McGill R. Graphical Perception // *JASA*. 1984. — doi:10.1080/01621459.1984.10478080
- Weissgerber T. L. et al. Beyond Bar and Line Graphs // *PLoS Biology*. 2015. — doi:10.1371/journal.pbio.1002128
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz

5.02 Ступенчатый график: события и дискретные изменения

Ступенчатый график — это линия, у которой значение **сохраняется** между событиями и меняется скачком в момент события. Он нужен там, где процесс по своей природе дискретен: доза препарата, скорость инфузии, статус, тариф, число коек, порог.

Главная ошибка — соединить такие точки наклонной линией. Тогда читатель видит плавное изменение, которого в данных не было.

5.02.1 Сфера применения

Ступенчатая функция отвечает на вопрос: **в какой момент значение изменилось и каким оно оставалось до следующего события?**

Где это пригодится:

- титрование дозы препарата (варфарин, инсулин, антибиотик);
- скорость инфузии (мкг/кг/мин, мл/ч);
- изменение статуса (госпитализирован/выписан, жив/умер — но для выживаемости нужен Kaplan–Meier);
- тарифы, оклады, налоговые ставки;
- число занятых коек или доступных аппаратов;
- ступенчатые протоколы и пороговые правила.

Когда ступенчатый график **не** подходит:

- величина меняется непрерывно (концентрация, температура, АД);
- ось X — неупорядоченные категории;
- нужно показать распределение или неопределённость, а не момент изменения;
- данных слишком мало для шага — тогда лучше точки.

5.02.2 Данные

Сценарий: три пациента на варфарине, 12 недель. Дозу меняют на визитах раз в неделю, шагом 0,5 мг/сут; INR измеряют на каждом визите, целевой диапазон — 2,0–3,0.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`week`	неделя визита (0–12)	недели
`dose_mg`	доза варфарина	мг/сут
`inr`	международное нормализованное отношение	—

39 строк: 3 пациента × 13 визитов. Дозы — 3,0–7,5 мг/сут; у P01 доза стабильна, у P02 и P03 — титруется. Это не исследование, а иллюстрация индивидуального титрования; для групповых выводов нужны другие графики и статистика.

Генератор: `05_charts/02_step/scripts/make_data.py → data/02_dataset.csv`.

5.02.3 Плохой график

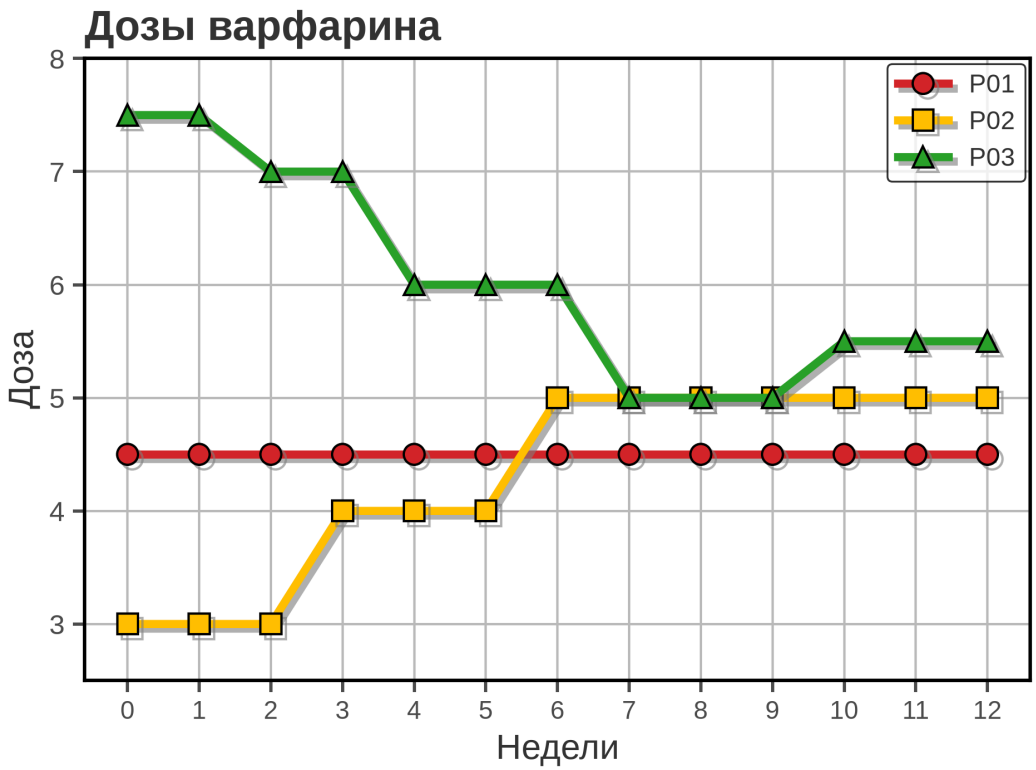


Рисунок 5.2.1. Плохой ступенчатый график: наклонные линии вместо ступеней, обрезанная ось, светофорные цвета и тени.

Код: `05_charts/02_step/scripts/bad_plot.py`.

Что не так:

1. **Наклонные линии вместо ступеней.** Доза не меняется плавно между визитами; `ax.plot` создаёт ложное впечатление непрерывного титрования. Это главная ошибка.
2. **Обрезанная ось Y (2,5–8).** Масштаб раздувает различия; нулевая база для дозы не обязательна, но диапазон должен показывать реальный масштаб.
3. **Светофорная палитра.** Красный — жёлтый — зелёный неразличим при протанопии/дейтеранопии и навязывает оценку «плохо — средне — хорошо».
4. **Толстые линии, тени и крупные маркеры.** Декор мешает видеть моменты изменения.
5. **Тяжёлая сетка по обеим осям и полная рамка.** Шум, который конкурирует с данными.
6. **Заголовок «Дозы варфарина».** Нет сообщения: что происходит с дозами и почему это важно.
7. **Нет единиц, числа пациентов и контекста INR.** «Доза» — в чём? Сколько пациентов? Почему дозу меняли?
8. **Легенда внутри поля данных.** Она дублирует подписи и закрывает линии.

5.02.4 Правильный график

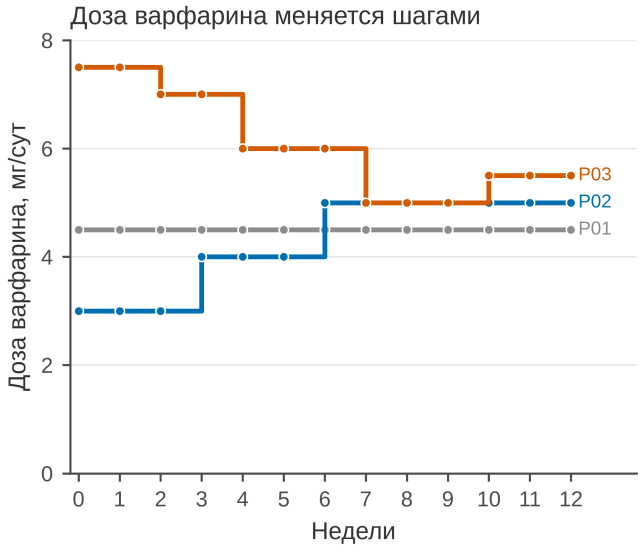


Рисунок 5.2.2. Правильный ступенчатый график: steps-post, маркеры визитов, прямые подписи и ось 0–8 мг/сут.

Код: `05_charts/02_step/scripts/good_plot.py`.

Что изменилось:

1. **`ax.step(..., where="post").`** Доза сохраняется до следующего визита; вертикальные сегменты показывают скачок, горизонтальные — период постоянной дозы.
2. **Маркеры в моменты визитов.** Видно, где именно измеряли и меняли дозу, а не абстрактную кривую.
3. **Прямые подписи P01/P02/P03.** Легенда не нужна; конец линии сразу подписан.
4. **Ось Y 0–8 мг/сут.** Виден реальный масштаб доз, без обрезки «под эффект».
5. **Единицы и недели.** «Доза варфарина, мг/сут», недели 0–12.
6. **Заголовок-вывод.** «Доза варфарина меняется шагами» объясняет, что читать.
7. **Палитра Okabe–Ito.** Серый — P01, синий — P02, вермилион — P03.
8. **Лёгкая сетка только по Y.** Верхняя и правая рамки убраны.
9. **85 мм в ширину.** Рисунок спроектирован в конечном размере.

INR не вынесен на вторую ось: две оси Y создают ложную корреляцию. Если INR важен, сделайте отдельную панель или таблицу; в подписи укажите целевой диапазон 2,0–3,0.

5.02.5 Было и стало

Что было	Почему плохо	Что сделано
`ax.plot`	плавное изменение, которого не было	`ax.step(where="post")`
ось Y 2,5–8	масштаб раздувает различия	0–8 мг/сут
красный/жёлтый/зелёный	ложная семантика, CVD	Okabe–Ito
тени, толстые линии	декор	линии 1,8 pt, маркеры 3,5 pt
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
легенда	дублирует подписи	прямые подписи
«Дозы варфарина»	нет сообщения	заголовок-вывод
нет единиц и *n*	невозможно прочитать	единицы, число пациентов в подписи
INR на второй оси	ложная корреляция	INR в подписи/отдельной панели

5.02.6 Чек-лист ступенчатого графика

- ☐ Величина действительно постоянна между событиями.
- ☐ Использован `step`, а не `plot` с наклонными линиями.
- ☐ Выбрано `where="post"` или `where="pre"` осознанно.
- ☐ Маркеры стоят в моменты измерения/изменения.
- ☐ Ось Y показывает реальный масштаб, без обрезки «под эффект».
- ☐ Указаны единицы, число пациентов и контекст.

- ☐ Прямые подписи лучше легенды.
- ☐ Нет двух осей Y.
- ☐ Цвет не единственный носитель смысла.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.02.7 Как воспроизвести

```
uv sync
uv run python 05_charts/02_step/scripts/make_data.py
uv run python 05_charts/02_step/scripts/bad_plot.py
uv run python 05_charts/02_step/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
ax.step(
    weeks,
    doses[patient].to_numpy(),
    where="post",
    color=color,
    linewidth=1.8,
    marker="o",
    markersize=3.5,
)
```

`where="post"` означает: значение `y[i]` держится от `x[i]` до `x[i+1]`, затем происходит скачок. Если событие фиксируется в конце интервала, проверьте, не нужен ли `where="pre"`.

5.02.8 Что дальше

Следующий график — **5.03 area**: область под линией для накопленных и составных величин. У него будет свой генератор и свой `data/03_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `step` — matplotlib.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com

5.03 Area chart: площадь как величина

Area chart — это линия с заливкой под ней. Он оправдан только тогда, когда **площадь является смыслом**: площадь под кривой концентрации (AUC), накопленная экспозиция, суммарная нагрузка, cumulative burden. Если заливка просто «украшает» линию, она добавляет чернила, но не информацию.

Главное: **заливка area chart должна начинаться с нуля**. Иначе площадь перестаёт быть пропорциональна величине, которую она якобы кодирует.

5.03.1 Сфера применения

Area chart отвечает на вопрос: **какая суммарная величина накоплена за период?**

Где это пригодится:

- площадь под кривой концентрация–время (AUC) в фармакокинетике;
- накопленная экспозиция лекарства или излучения;
- суммарная нагрузка (число случаев, койко-дни, затраты) за период;
- cumulative incidence, если это не кривая выживаемости (для выживаемости — Kaplan–Meier, 5.27);
- stacked area для состава во времени, но с осторожностью: наложение областей затрудняет сравнение (для состава чаще лучше stacked bar, 5.20).

Когда area chart **не** подходит:

- площадь не несёт смысла, а заливка нужна «для красоты» — используйте линию;
- несколько перекрывающихся областей: одна заливка закрывает другую;
- ось Y не начинается с нуля;
- нужно точное сравнение значений — линия или точки читаются лучше;
- категории по X не упорядочены.

5.03.2 Данные

Сценарий: 40 пациентов получают условный препарат внутрь в дозе 500 мг. Концентрацию в плазме измеряют через 0, 1, 2, 4, 6, 8, 12 и 24 часа. AUC 0–24 — интегральная экспозиция, мг·ч/л.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`time_h`	время после приёма	часы
`conc_mg_l`	концентрация в плазме	мг/л

320 строк: 40 пациентов × 8 моментов. Средний AUC 0–24 — 80,7 ± 18,2 мг·ч/л; пик концентрации — около 8–9 мг/л через 1–2 часа; к 24 часам — около 0,5 мг/л.

Генератор: `05_charts/03_area/scripts/make_data.py → data/03_dataset.csv`.

5.03.3 Плохой график

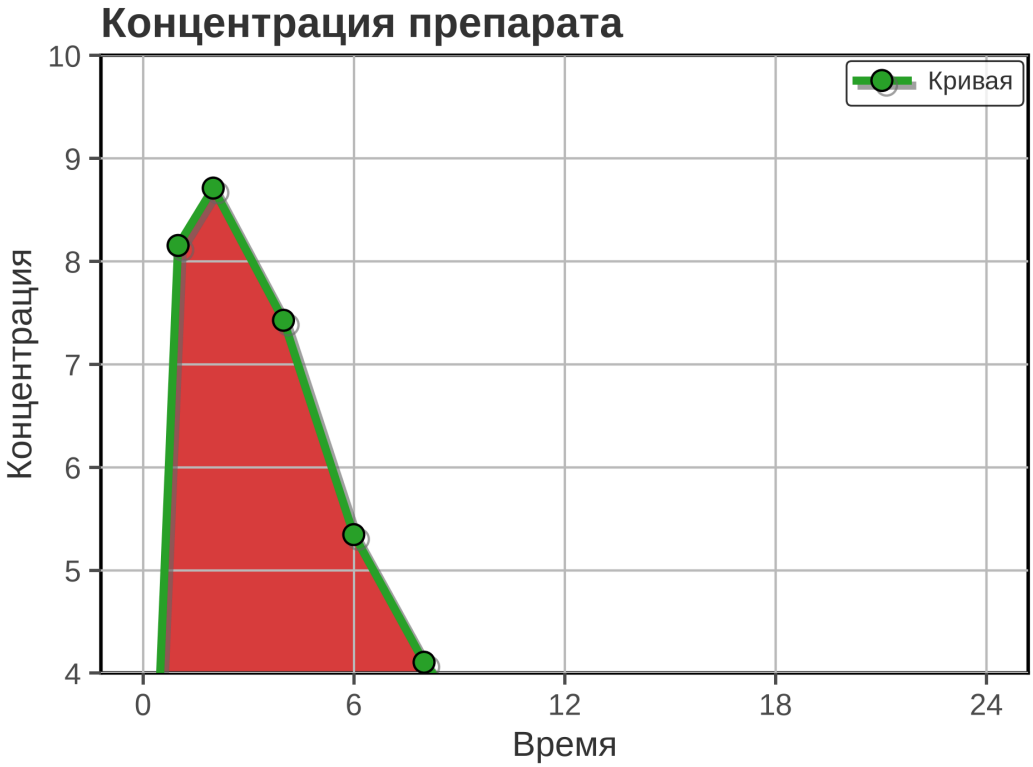


Рисунок 5.3.1. Плохой area chart: заливка от 4 вместо нуля, обрезанная ось, красная заливка и зелёная линия, тени, тяжёлая сетка.

Код: `05_charts/03_area/scripts/bad_plot.py`.

Что не так:

1. **Заливка начинается с y = 4, а не с нуля.** Площадь больше не пропорциональна AUC; график показывает не интеграл, а обрезок. Это главная ошибка.
2. **Обрезанная ось Y (4–10).** Пик и хвост искажены; нижняя часть кривой и базовая линия скрыты.
3. **Красная заливка и зелёная линия.** Светофорная пара неразличима при протанопии/дейтеранопии; тёмная заливка конкурирует с линией.
4. **Тени и крупные маркеры.** Декор, который не кодирует данные.
5. **Тяжёлая сетка по обеим осям и полная рамка.** Сетка просвечивает через заливку и создаёт шум.
6. **Заголовок «Концентрация препарата».** Не сообщает, что площадь — это экспозиция и какова её величина.
7. **Нет единиц, *n* и AUC.** «Концентрация» — в чём? Сколько пациентов? Чему равна площадь?
8. **Легенда из одной строки «Кривая».** Она не несёт информации.

5.03.4 Правильный график

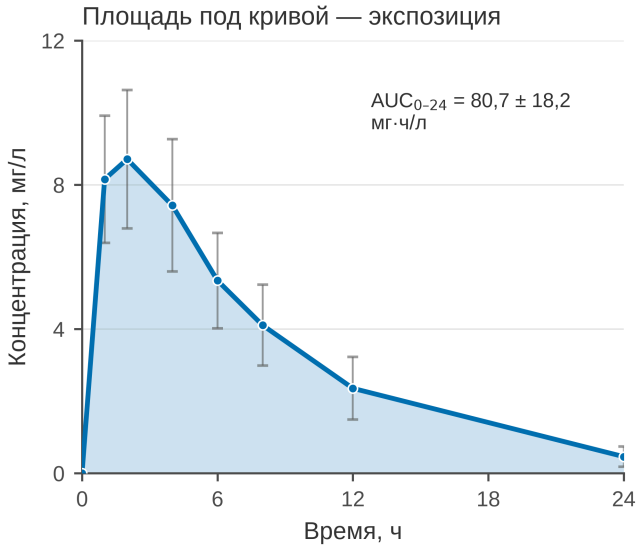


Рисунок 5.3.2. Правильный area chart: заливка от нуля, подпись AUC, среднее ± SD, палитра Okabe–Ito, ось 0–12 мг/л.

Код: `05_charts/03_area/scripts/good_plot.py`.

Что изменилось:

1. **Заливка от 0.** Площадь под кривой действительно равна AUC; базовая линия видна.
2. **Подпись AUC 0–24 = 80,7 ± 18,2 мг·ч/л.** Площадь превращается в число, которое можно интерпретировать.
3. **Ось Y 0–12 мг/л.** Видны пик, хвост и нулевая база; нет обрезки «под эффект».
4. **Среднее ± SD в точках.** Видна неопределённость; заливка показывает интеграл, а не заменяет разброс.
5. **Палитра Okabe–Ito.** Один синий цвет для одной серии; ничего лишнего.
6. **Единицы и *n*.** «Концентрация, мг/л», «Время, ч», *n* = 40 в подписи.
7. **Заголовок-вывод.** «Площадь под кривой — экспозиция» объясняет, зачем здесь заливка.
8. **Лёгкая сетка только по Y, без верхней и правой рамок.**
9. **85 мм в ширину.** Рисунок спроектирован в конечном размере.

Если площадь не нужна, эти же данные лучше показать линией без заливки: меньше чернил, та же информация. Area chart — не «линия с красивым фоном», а график интеграла.

5.03.5 Было и стало

Что было	Почему плохо	Что сделано
заливка от 4	площадь не равна AUC	заливка от 0
ось Y 4–10	обрезаны пик и хвост	0–12 мг/л
красная заливка + зелёная линия	CVD, шум	один синий цвет
тени, крупные маркеры	декор	линия 1,8 pt, маркеры 3,5 pt
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
легенда «Кривая»	бесполезна	без легенды
«Концентрация препарата»	нет сообщения	заголовок-вывод
нет AUC и единиц	площадь не читается	AUC, мг·ч/л, *n*
только среднее	нет неопределённости	среднее ± SD

5.03.6 Чек-лист area chart

- ☐ Площадь действительно является смыслом (AUC, накопление).
- ☐ Заливка начинается с нуля.
- ☐ Ось Y не обрезает нижнюю часть площади.
- ☐ Подписана величина площади (AUC, сумма, интеграл).

- ☐ Показана неопределённость (SD/CI или отдельные кривые).
- ☐ Указаны единицы, *n*, время.
- ☐ Нет перекрывающихся непрозрачных областей.
- ☐ Заливка не конкурирует с линией и сеткой.
- ☐ Если площадь не нужна — выбран line chart.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.03.7 Как воспроизвести

```
uv sync
uv run python 05_charts/03_area/scripts/make_data.py
uv run python 05_charts/03_area/scripts/bad_plot.py
uv run python 05_charts/03_area/scripts/good_plot.py
```

Ключевые фрагменты:

```
ax.fill_between(time, 0, mean, color=PRIMARY_BLUE, alpha=0.18, linewidth=0)
ax.plot(time, mean, color=PRIMARY_BLUE, marker="o", linewidth=1.8)

auc = np.trapezoid(part["conc_mg_l"], part["time_h"])
```

`np.trapezoid` считает площадь трапеций; единицы AUC — мг·ч/л, потому что концентрация в мг/л умножается на время в часах.

5.03.8 Что дальше

Следующий график — **5.04 small multiples**: несколько панелей с общей шкалой для многомерных данных. У него будет свой генератор и свой `data/04_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `fill_between` — matplotlib.org
- NumPy: `trapezoid` — numpy.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com

5.04 Small multiples: много панелей, одна логика

Small multiples (малые кратные) — это ряд небольших графиков с одинаковой структурой и, как правило, общей шкалой. Вместо того чтобы рисовать шесть линий на одной оси и надеяться, что читатель их различит, мы показываем шесть панелей: сравнение идёт по форме и уровню, а не по цвету.

Принцип прост: **одна панель — одна серия; общая шкала — контракт сравнения.** Если у панелей разные оси, сравнение становится косметикой.

5.04.1 Сфера применения

Small multiples отвечают на вопрос: **как различаются группы, если смотреть на них по отдельности, но по одной шкале?**

Где это пригодится:

- динамика показателя по 5–20 отделениям, регионам, центрам;
- распределения по группам (гистограммы или box в facets);
- сезонность по месяцам для многих категорий;
- сравнение центров в многоцентровом исследовании;
- проверка согласованности трендов без наложения линий.

Когда small multiples **не** нужны:

- серий 2–3: обычная линия с прямыми подписями читается лучше;
- нужно точное сравнение двух значений — точки/таблица;
- панели не помещаются в журнальную ширину;
- шкалы всё равно придётся делать разными — тогда это уже не small multiples, а набор разных графиков.

5.04.2 Данные

Сценарий: шесть отделений больницы, 12 месяцев. Показатель — потребление антибиотиков в DDD на 100 койко-дней. Зимой потребление выше, летом ниже.

Колонка	Смысл	Единица
`department`	отделение	—
`month`	месяц (1–12)	—
`ddd_per_100_bed_days`	потребление антибиотиков	DDD/100 койко-дней

72 строки: 6 отделений × 12 месяцев. Средние уровни: ОРИТ — 109,4, пульмонология — 80,1, хирургия — 69,4, терапия — 55,9, урология — 49,3, кардиология — 45,7. ОРИТ потребляет в 2,4 раза больше кардиологии.

Генератор: `05_charts/04_small_multiples/scripts/make_data.py` → `data/04_dataset.csv`.

5.04.3 Плохой график

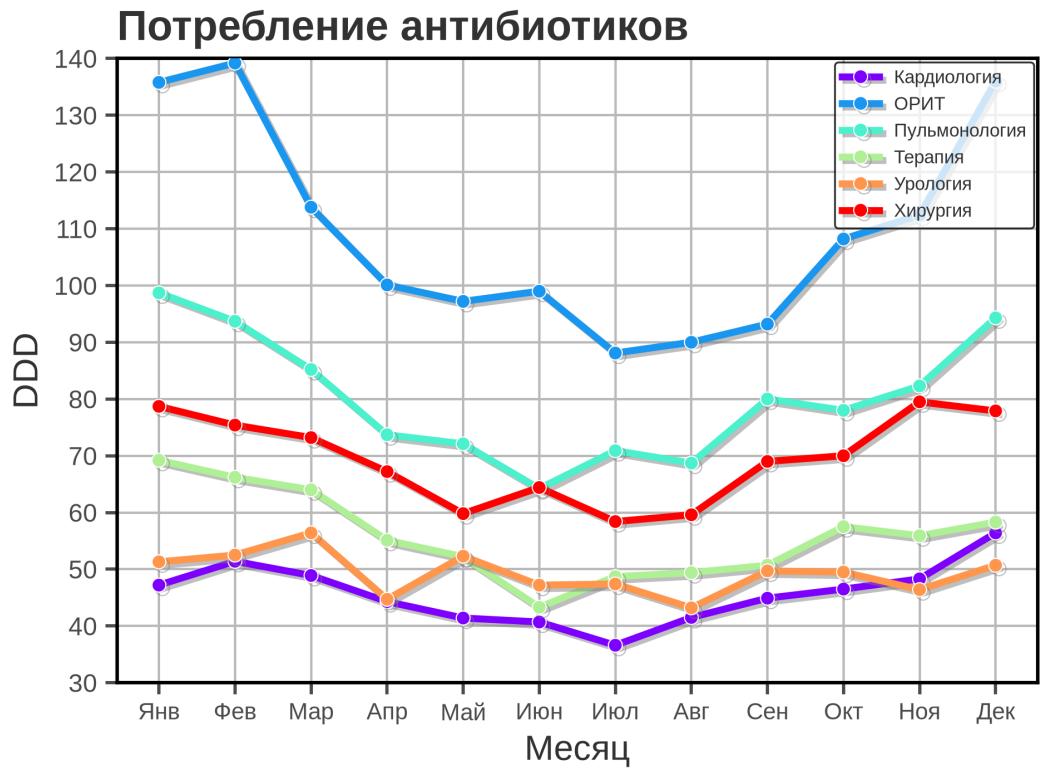


Рисунок 5.4.1. Плохой график: шесть отделений на одной оси, радужная палитра, обрезанная ось, тяжёлая сетка.

Код: `05_charts/04_small_multiples/scripts/bad_plot.py`.

Что не так:

1. **Шесть серий на одной оси.** Линии перекрываются, особенно в области 45–80 DDD; сравнить отделения невозможно.
2. **Радужная палитра.** Шесть цветов неразличимы при дальтонизме, а легенда заставляет постоянно переводить взгляд.
3. **Обрезанная ось Y (30–140).** Низкие значения искажены; разница между кардиологией и урологией выглядит больше, чем есть.
4. **Толстые линии, тени и крупные маркеры.** Декор усиливает overplotting.
5. **Тяжёлая сетка по обеим осям и полная рамка.** Сетка конкурирует с шестью линиями.
6. **Заголовок «Потребление антибиотиков».** Не сообщает, что сравниваются отделения и кто лидирует.
7. **Нет единиц и n.** «DDD» — это что? Сколько отделений и месяцев?
8. **Легенда из шести строк** закрывает верхнюю часть данных.

5.04.4 Правильный график

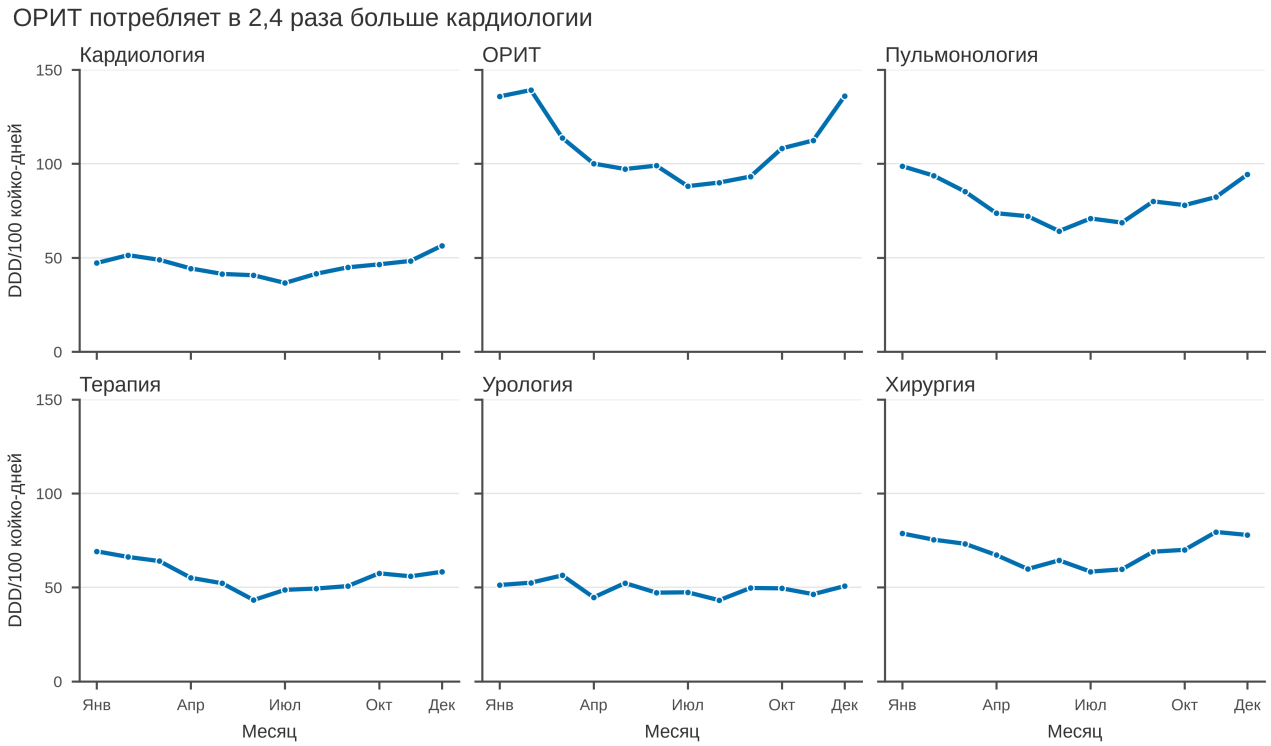


Рисунок 5.4.2. Правильные малые кратные: 2×3 панели, общая ось 0–150, один цвет, подписи отделений.

Код: `05_charts/04_small_multiples/scripts/good_plot.py`.

Что изменилось:

1. **2×3 панели.** Каждое отделение — своя панель; линии не перекрываются.
2. **Общая ось Y 0–150.** Панели честно сравнимы; видно, что ОРИТ выше всех, а кардиология ниже всех.
3. **Один цвет.** Сравнение идёт по форме и уровню, а не по легенде.
4. **Общие деления X.** Янв/Апр/Июл/Окт/Дек; подписи только на нижнем ряду, чтобы не дублировать.
5. **Подписи осей по краям.** «DDD/100 койко-дней» — только слева, «Месяц» — только снизу.
6. **Заголовок-вывод.** «ОРИТ потребляет в 2,4 раза больше кардиологии» сразу задаёт сообщение.
7. **Лёгкая сетка по Y, без верхней и правой рамок.**
8. **170 мм.** Две журнальные колонки; панели не сжаты в одну.

Порядок панелей — по алфавиту; если нужен содержательный порядок (например, по уровню потребления), задайте его явно и сохраняйте во всех рисунках.

5.04.5 Было и стало

Что было	Почему плохо	Что сделано
6 линий на одной оси	overplotting	2×3 панели
радужная палитра	CVD, легенда	один синий цвет
ось Y 30–140	искажает низкие значения	0–150
толстые линии, тени	декор	линии 1,6 pt, маркеры 2,6 pt
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
легенда	закрывает данные	подписи панелей
«Потребление антибиотиков»	нет сообщения	заголовок-вывод
нет единиц и *n*	невозможно прочитать	единицы, 6 отделений × 12 месяцев
разные шкалы	нельзя сравнивать	общая шкала

5.04.6 Чек-лист small multiples

- ☐ Панели действительно сравнимы: общая шкала X и Y.
- ☐ Одна панель — одна серия; нет перекрытия.

- ☐ Порядок панелей осмысленный и одинаковый во всех рисунках.
- ☐ Подписи осей не дублируются на каждой панели.
- ☐ Один цвет, если нет смыслового повода для разных.
- ☐ Заголовки панелей короткие и однозначные.
- ☐ Есть общий заголовок-вывод.
- ☐ Указаны единицы и *n*.
- ☐ Панели не сжаты: минимум 45–50 мм на панель.
- ☐ Если серий 2–3, рассмотрена обычная линия.

5.04.7 Как воспроизвести

```
uv sync
uv run python 05_charts/04_small_multiples/scripts/make_data.py
uv run python 05_charts/04_small_multiples/scripts/bad_plot.py
uv run python 05_charts/04_small_multiples/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
fig, axes = plt.subplots(
    2, 3,
    figsize=figure_size("double", height_mm=100),
    sharex=True,
    sharey=True,
    layout="constrained",
)
for ax, department in zip(axes.ravel(), departments, strict=True):
    ax.plot(x, pivot[department], color=PRIMARY_BLUE)
    ax.set_title(department, loc="left")
```

`sharex=True` и `sharey=True` — это и есть контракт сравнения. Если приходится делать `sharey=False`, подпишите разные шкалы явно и объясните, почему они разные.

5.04.8 Что дальше

Следующий график — **5.05 scatter**: связь двух непрерывных величин, выбросы и плотность. У него будет свой генератор и свой `data/05_dataset.csv`.

Источники

- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Cairo A. *The Truthful Art*. New Riders, 2016. — thefunctionalart.com
- Matplotlib: `subplots` — matplotlib.org

5.05 Scatter: связь двух непрерывных величин

Scatter plot — базовая форма для связи двух непрерывных переменных. Он показывает не только тренд, но и разброс, выбросы, гетерогенность и плотность наблюдений. Поэтому scatter честнее коэффициента корреляции: коэффициент сжимает всю картину в одно число.

На практике: **точки важнее линии.** Линия регрессии — дополнение к данным, а не замена им. Если на графике видны только линия и *r*, читатель не увидит ни формы, ни выбросов.

5.05.1 Сфера применения

Scatter отвечает на вопрос: **связаны ли две непрерывные величины и как выглядит эта связь?**

Где это пригодится:

- связь биомаркера и клинического показателя (BMI и глюкоза, CRP и тяжесть);
- корреляция двух методов измерения (но для согласия — Bland–Altman, 5.19);
- зависимость дозы и эффекта;
- поиск выбросов и гетерогенности;
- проверка линейности перед регрессией.

Когда scatter **не** подходит:

- одна из переменных категориальная — нужны точки/столбцы;
- время по X — нужна линия (5.01);
- точек слишком много — overplotting, нужны hexbin (5.06), contour (5.07) или плотность;
- нужно показать согласие методов, а не корреляцию — Bland–Altman (5.19);
- обе переменные — доли, сумма которых фиксирована (compositional data).

5.05.2 Данные

Сценарий: 180 взрослых на профилактическом осмотре. Измерены BMI (кг/м²) и глюкоза натощак (ммоль/л).

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`bmi_kg_m2`	индекс массы тела	кг/м²
`glucose_mmol_l`	глюкоза натощак	ммоль/л

Средний BMI — 27,5 ± 4,1 кг/м²; глюкоза — 5,7 ± 1,0 ммоль/л; корреляция Пирсона $r = 0{,}52$ ($p < 0{,}001$). Связь положительная, но не жёсткая: вокруг линии большой разброс.

Генератор: `05_charts/05_scatter/scripts/make_data.py` → `data/05_dataset.csv`.

5.05.3 Плохой график

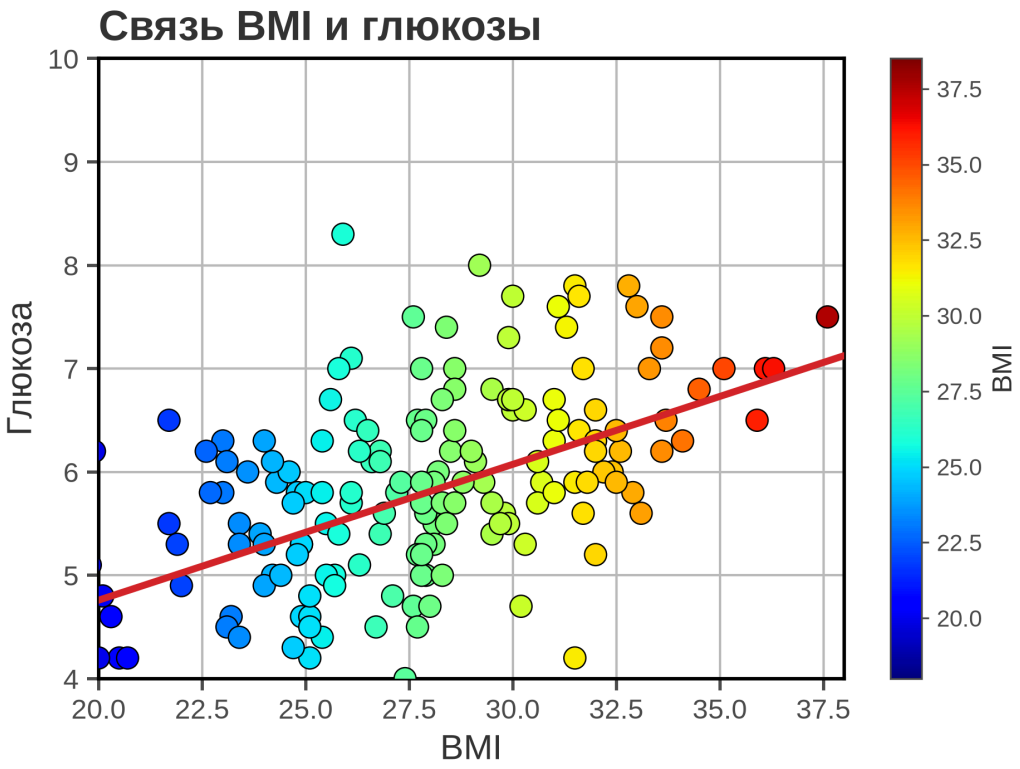


Рисунок 5.5.1. Плохой scatter: обрезанные оси, точки окрашены по BMI, крупные маркеры, линия без статистики.

Код: `05_charts/05_scatter/scripts/bad_plot.py`.

Что не так:

1. **Обрезанные оси (20–38 и 4–10).** Часть точек и хвосты скрыты; форма связи и выбросы искажены.
2. **Цвет точек по BMI.** BMI уже на оси X; цвет дублирует ось и добавляет радужную шкалу, которая не несёт новой информации.
3. **Крупные непрозрачные маркеры с чёрной обводкой.** Overplotting: в плотных областях точки сливаются, нельзя оценить плотность.
4. **Красная линия регрессии без доверительной полосы и статистики.** Не указаны *r*, *n*, наклон; линия выглядит как «истина».
5. **Тяжёлая сетка по обеим осям и полная рамка.** Шум, который конкурирует с точками.
6. **Заголовок «Связь BMI и глюкозы».** Не сообщает направление и силу связи.
7. **Нет единиц и *n*.** «BMI» и «Глюкоза» — в чём? Сколько людей?
8. **Цветовая шкала BMI** дублирует ось X и занимает место.

5.05.4 Правильный график

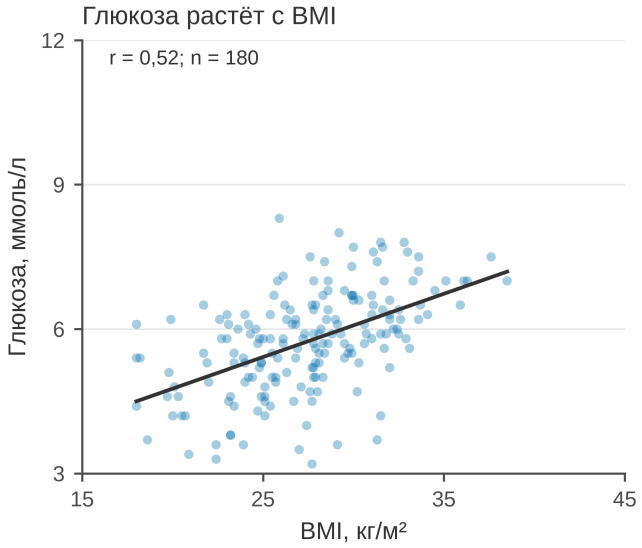


Рисунок 5.5.2. Правильный scatter: полный диапазон, полупрозрачные точки, линия регрессии, r и n.

Код: `05_charts/05_scatter/scripts/good_plot.py`.

Что изменилось:

1. **Полный диапазон осей.** 15–45 кг/м² и 3–12 ммоль/л; все точки видны, включая хвосты.
2. **Полупрозрачные точки одного цвета.** `alpha=0.35`, размер 12: видна плотность и не теряются отдельные наблюдения.
3. **Линия регрессии только в диапазоне данных.** Без экстраполяции за пределы наблюдений.
4. **Подпись *r* = 0,52; *n* = 180.** Связь quantified; читатель видит и форму, и число.
5. **Единицы на осях.** «BMI, кг/м²», «Глюкоза, ммоль/л».
6. **Заголовок-вывод.** «Глюкоза растёт с BMI» задаёт направление.
7. **Лёгкая сетка только по Y, без верхней и правой рамок.**
8. **85 мм.** Рисунок спроектирован в конечном размере.

Корреляция не равна причинности: BMI и глюкоза могут быть связаны через возраст, питание, физическую активность. Scatter показывает ассоциацию, а не механизм.

5.05.5 Было и стало

Что было	Почему плохо	Что сделано
оси 20–38 / 4–10	точки и хвосты обрезаны	15–45 / 3–12
цвет по BMI	двойное кодирование, радуга	один синий цвет
крупные непрозрачные точки	overplotting	s=12, alpha=0,35
линия без статистики	не quantifies связь	r, n, линия в диапазоне
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
«Связь BMI и глюкозы»	нет сообщения	заголовок-вывод
нет единиц и *n*	невозможно прочитать	единицы, *n*
colorbar BMI	дублирует ось	без colorbar

5.05.6 Чек-лист scatter

- ☐ Обе переменные непрерывные.
- ☐ Оси не обрезают данные без объяснения.
- ☐ Есть *n* и единицы.
- ☐ Точки не сливаются: alpha/размер/hexbin.
- ☐ Линия регрессии не выходит за диапазон данных.
- ☐ Указаны *r* (или наклон) и неопределённость.
- ☐ Цвет не дублирует ось без смысла.
- ☐ Выбросы видны, а не скрыты.

- ☐ Корреляция не подана как причинность.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.05.7 Как воспроизвести

```
uv sync
uv run python 05_charts/05_scatter/scripts/make_data.py
uv run python 05_charts/05_scatter/scripts/bad_plot.py
uv run python 05_charts/05_scatter/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
ax.scatter(x, y, s=12, color=PRIMARY_BLUE, alpha=0.35, linewidths=0)
slope, intercept, r_value, p_value, _ = stats.linregress(x, y)
line_x = np.array([x.min(), x.max()])
ax.plot(line_x, intercept + slope * line_x, color="#333333", linewidth=1.4)
```

Если точек станет больше нескольких тысяч, scatter превратится в сплошное пятно; тогда нужны hexbin или contour.

5.05.8 Что дальше

Следующий график — **5.06 hexbin**: плотность тысяч точек без overplotting. У него будет свой генератор и свой `data/06_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Cleveland W. S., McGill R. Graphical Perception // *JASA*. 1984. — doi:10.1080/01621459.1984.10478080
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com
- Matplotlib: `scatter` — matplotlib.org

5.06 Hexbin: плотность тысяч точек

Hexbin — это scatter, у которого точки агрегированы в шестиугольные бины, а цвет показывает число наблюдений в бине. Он нужен там, где обычный scatter превращается в сплошное пятно: тысячи точек, плотное ядро, длинные хвосты.

Важно помнить: hexbin показывает **плотность**, а не отдельного пациента. Цвет — это число наблюдений в бине, а не значение признака.

5.06.1 Сфера применения

Hexbin отвечает на вопрос: **где наблюдений много, а где мало, и какова форма облака при больших n?**

Где это пригодится:

- 5 000–1 000 000 точек в scatter;
- плотность биомедицинских показателей (HbA1c и глюкоза, BMI и АД);
- кластеры и гетерогенность в больших когортах;
- проверка формы распределения перед регрессией;
- визуализация больших выгрузок без потери плотности.

Когда hexbin **не** нужен:

- точек мало (до нескольких сотен) — scatter честнее;
- нужно прочитать отдельное наблюдение;
- одна из переменных категориальная;
- важна точная форма хвоста, а бинование её огрубляет;
- нужна связь, а не плотность — scatter или contour.

5.06.2 Данные

Сценарий: 8 000 пациентов, HbA1c (%) и глюкоза натощак (ммоль/л). Связь сильная, но с разбросом; при 8 000 точек обычный scatter превращается в пятно.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`hba1c_pct`	гликированный гемоглобин	%
`glucose_mmol_l`	глюкоза натощак	ммоль/л

Средний HbA1c — 6,3 ± 0,9 %; глюкоза — 7,5 ± 1,8 ммоль/л; корреляция Пирсона r = 0,83.

Генератор: `05_charts/06_hexbin/scripts/make_data.py → data/06_dataset.csv`.

5.06.3 Плохой график

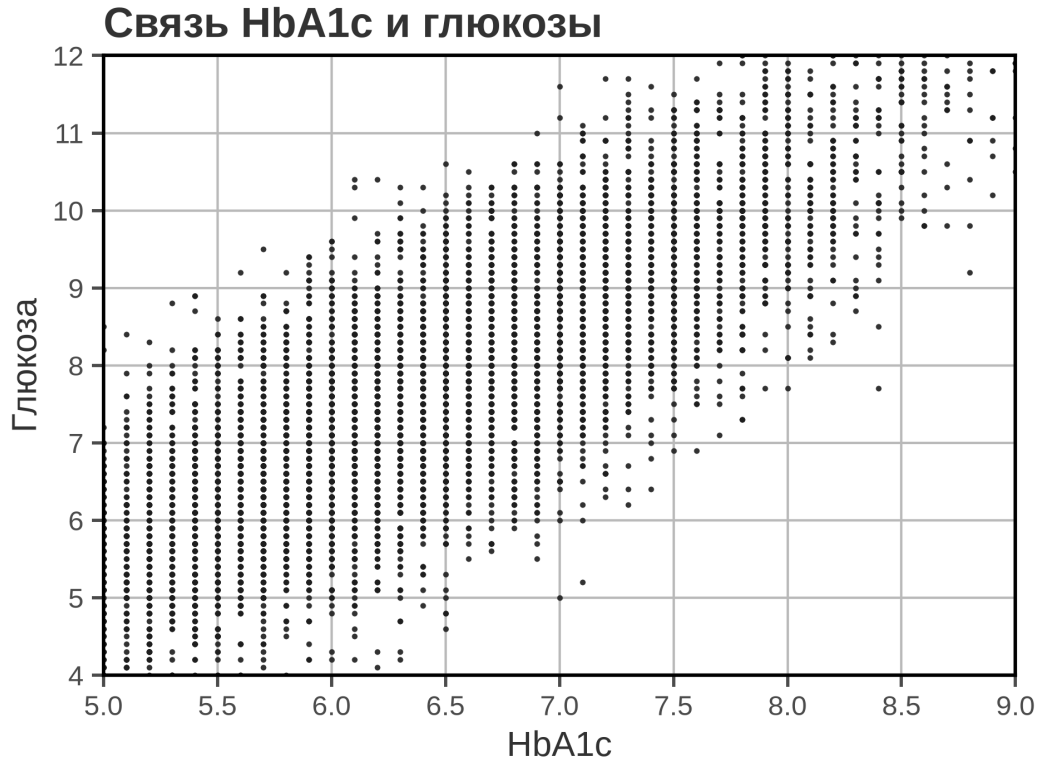


Рисунок 5.6.1. Плохой пример: 8 000 непрозрачных точек, overplotting, обрезанные оси, тяжёлая сетка.

Код: `05_charts/06_hexbin/scripts/bad_plot.py`.

Что не так:

1. **8 000 непрозрачных точек.** Вместо плотности — чёрное пятно; в центре точки сливаются, хвосты теряются.
2. **Обрезанные оси (5–9 и 4–12).** Часть наблюдений и хвосты распределения скрыты.
3. **Тяжёлая сетка по обеим осям и полная рамка.** Сетка конкурирует с плотным облаком.
4. **Заголовок «Связь HbA1c и глюкозы».** Не сообщает, что важно именно распределение плотности.
5. **Нет единиц, n и r.** Непонятно, сколько наблюдений и какова сила связи.
6. **Чёрный цвет не кодирует плотность.** Читатель не видит, где наблюдений больше.

5.06.4 Правильный график

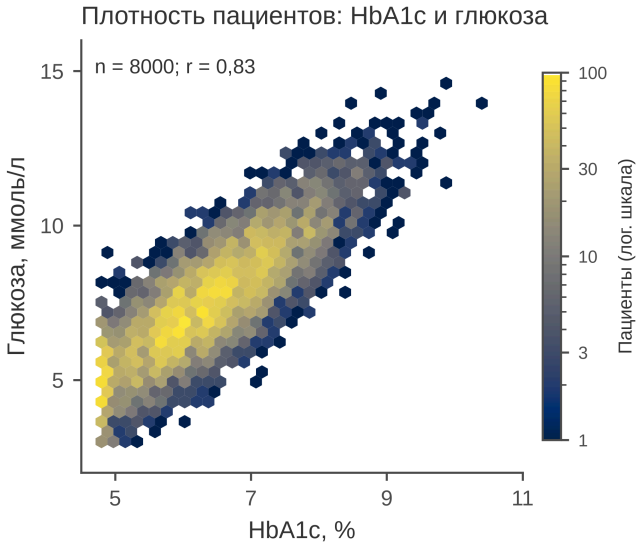


Рисунок 5.6.2. Правильный hexbin: плотность в логарифмической шкале, colorbar 1–100, полный диапазон, n и r.

Код: `05_charts/06_hexbin/scripts/good_plot.py`.

Что изменилось:

1. **Hexbin вместо scatter.** Цвет кодирует число пациентов в шестиугольнике; плотность видна.
2. **Логарифмическая шкала плотности.** Видны и плотное ядро, и разреженные хвосты; линейная шкала «съела» бы хвосты.
3. **`mincnt=1`.** Пустые бины остаются белыми, а не окрашиваются в цвет «нуля».
4. **Полный диапазон осей.** 4,5–11 % и 2–16 ммоль/л; данные не обрезаны.
5. **Подписанный colorbar.** «Пациенты (лог. шкала)», деления 1/3/10/30/100 — цвет читается как число.
6. **Единицы, n и r.** «HbA1c, %», «Глюкоза, ммоль/л», n = 8000, r = 0,83.
7. **`cividis`.** Последовательная шкала, читаемая при дальтонизме.
8. **85 мм.** Рисунок спроектирован в конечном размере.

Hexbin не заменяет scatter для отдельных наблюдений: он показывает плотность. Если нужен конкретный пациент или выброс, hexbin его не подпишет.

5.06.5 Было и стало

Что было	Почему плохо	Что сделано
8 000 точек scatter	overplotting	hexbin
оси 5–9 / 4–12	хвосты обрезаны	4,5–11 / 2–16
чёрный цвет	не кодирует плотность	цвет = число пациентов
линейная шкала	хвосты не видны	лог. шкала
пустые бины окрашены	ложные «нули»	`mincnt=1`
сетка по X и Y	шум	без сетки
полная рамка	визуальный шум	только левая и нижняя оси
«Связь HbA1c и глюкозы»	нет сообщения	заголовок о плотности
нет единиц и n	невозможно прочитать	единицы, n, r
нет colorbar	цвет не читается	colorbar с делениями

5.06.6 Чек-лист hexbin

- ☐ Точек действительно много; scatter уже не читается.
- ☐ Понятно, что цвет кодирует число наблюдений, а не значение.
- ☐ Указаны единицы и n.
- ☐ Оси не обрезают данные.
- ☐ `mincnt=1`, чтобы пустые бины не выглядели «нулём».
- ☐ Шкала плотности (линейная/логарифмическая) подписана.
- ☐ Colorbar с делениями и единицами.
- ☐ `gridsize` не слишком крупный и не слишком мелкий.
- ☐ Для точных значений используется scatter/contour.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.06.7 Как воспроизвести

```
uv sync
uv run python 05_charts/06_hexbin/scripts/make_data.py
uv run python 05_charts/06_hexbin/scripts/bad_plot.py
uv run python 05_charts/06_hexbin/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
hexbin = ax.hexbin(
    x, y,
    gridsize=32,
    cmap="cividis",
    mincnt=1,
    bins="log",
    linewidths=0,
)
cbar = fig.colorbar(hexbin, ax=ax, fraction=0.05, pad=0.02, shrink=0.85)
cbar.set_label("Пациенты (лог. шкала)")
```

`bins="log"` включает логарифмическую нормализацию цвета; `mincnt=1` оставляет пустые бины белыми. Если нужно среднее значение в бине, используйте `reduce_C_function=np.mean` и подпишите, что кодирует цвет.

5.06.8 Что дальше

Следующий график — **5.07 contour**: изолинии плотности или поверхности отклика. У него будет свой генератор и свой `data/07_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `hexbin` — matplotlib.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com
- Cairo A. *The Truthful Art*. New Riders, 2016. — thefunctionalart.com

5.07 Contour: изолинии плотности

Contour plot рисует линии равного значения: плотности, вероятности, поверхности отклика. В отличие от hexbin, контурная карта выглядит гладкой и показывает форму облака, но поэтому она может «придумать» структуру, которой нет в данных.

На практике: **сначала плотность, потом линии.** Контурные линии — это изолинии оценки плотности; без colorbar и подписи уровней они ничего не значат.

5.07.1 Сфера применения

Contour отвечает на вопрос: **где плотность (или значение поверхности) одинакова и какова форма двумерного распределения?**

Где это пригодится:

- 2D-плотность больших выборок (возраст и САД, BMI и глюкоза);
- поверхность отклика (доза × время → эффект);
- доверительные области и эллипсы;
- топография/визуализация пространственных полей;
- сглаженная альтернатива hexbin.

Когда contour **не** нужен:

- точек мало — контурная оценка неустойчива;
- нужно прочитать отдельное наблюдение;
- важна локальная структура, а не гладкая форма — hexbin/scatter;
- уровни выбраны произвольно и не подписаны;
- сглаживание скрывает реальные разрывы.

5.07.2 Данные

Сценарий: 10 000 взрослых, возраст (лет) и систолическое АД (мм рт. ст.). САД растёт с возрастом, разброс тоже растёт.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`age_years`	возраст	лет
`sbp_mm_hg`	систолическое АД	мм рт. ст.

Средний возраст — 51,8 ± 14,7 лет; САД — 125,0 ± 17,3 мм рт. ст.; корреляция r = 0,49. Порог 140 мм рт. ст. отмечен на графике.

Генератор: `05_charts/07_contour/scripts/make_data.py` → `data/07_dataset.csv`.

5.07.3 Плохой график

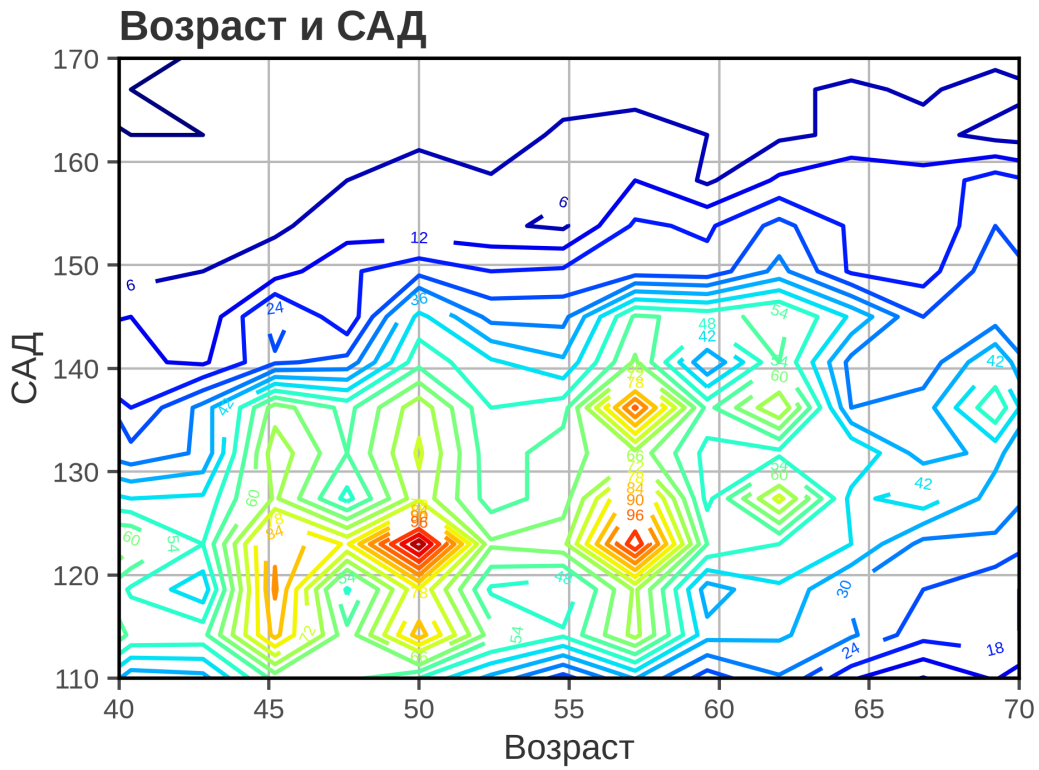


Рисунок 5.7.1. Плохой contour: радужная палитра, 20 уровней, налезающие подписи, обрезанные оси.

Код: `05_charts/07_contour/scripts/bad_plot.py`.

Что не так:

1. **Палитра `jet` и 20 уровней.** Ложные границы, «радуга», неразличимая при дальтонизме.
2. **Подписи уровней налезают друг на друга.** Читать карту невозможно.
3. **Обрезанные оси (40–70 и 110–170).** Хвосты распределения скрыты.
4. **Тяжёлая сетка по обеим осям и полная рамка.** Шум поверх и без того шумной карты.
5. **Заголовок «Возраст и САД».** Не сообщает, что показана плотность и где порог.
6. **Нет единиц, *n*, colorbar и порога 140.** Цвет и уровни не читаются.
7. **Слишком мелкие бины.** Карта выглядит «рябой»: шум гистограммы принимается за структуру.

5.07.4 Правильный график

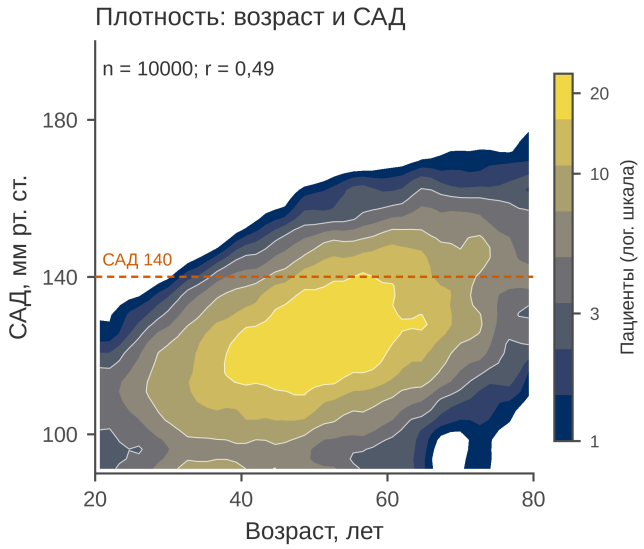


Рисунок 5.7.2. Правильный contour: сглаженная плотность, cividis, colorbar 1–20, порог 140, n и r.

Код: `05_charts/07_contour/scripts/good_plot.py`.

Что изменилось:

1. **Сглаженная 2D-плотность.** `gaussian_filter` убирает шум бинов, не стирая облако; структура читается.
2. **`cividis` и 9 уровней.** Последовательная доступная шкала, без ложных границ.
3. **Полный диапазон осей.** 20–80 лет и 90–200 мм рт. ст.
4. **Подписанный colorbar.** «Пациенты (лог. шкала)», деления 1/3/10/20.
5. **Порог 140 мм рт. ст.** Выделен пунктиром и подписью.
6. **Единицы, *n* и *r*.** «Возраст, лет», «САД, мм рт. ст.», $n = 10000$, $r = 0{,}49$.
7. **Без тяжёлой сетки.** Контурные линии — единственная вспомогательная графика.
8. **85 мм.** Рисунок спроектирован в конечном размере.

Контурная карта — это **оценка** плотности. Сглаживание и число уровней — параметры модели, а не «истина». Если структура важна, сравните contour с hexbin или scatter.

5.07.5 Было и стало

Что было	Почему плохо	Что сделано
`jet`, 20 уровней	ложные границы, CVD	`cividis`, 9 уровней
подписи всех уровней	налезают	без подписей, colorbar
оси 40–70 / 110–170	хвосты обрезаны	20–80 / 90–200
мелкие бины	шум как структура	сглаживание
сетка по X и Y	шум	без сетки
полная рамка	визуальный шум	только левая и нижняя оси
«Возраст и САД»	нет сообщения	заголовок о плотности
нет единиц и *n*	невозможно прочитать	единицы, *n*, *r*
нет colorbar	цвет не читается	colorbar 1–20
нет порога 140	клинический контекст потерян	пунктир 140

5.07.6 Чек-лист contour

- ☐ Понятно, что такое «уровень» и что кодирует цвет.
- ☐ Указаны единицы и *n*.
- ☐ Оси не обрезают данные.
- ☐ Уровни подписаны или есть colorbar.
- ☐ Палитра последовательная, не `jet`.
- ☐ Сглаживание не уничтожает реальную структуру.
- ☐ Подписи уровней не налезают.
- ☐ Есть клинический/смысловой порог, если он важен.
- ☐ Для локальной структуры рассмотрен hexbin.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.07.7 Как воспроизвести

```
uv sync
uv run python 05_charts/07_contour/scripts/make_data.py
uv run python 05_charts/07_contour/scripts/bad_plot.py
uv run python 05_charts/07_contour/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
hist, xedges, yedges = np.histogram2d(
    age, sbp, bins=(45, 45), range=[[20, 80], [90, 200]]
)
smooth = gaussian_filter(hist, sigma=1.2)
levels = np.logspace(0, np.log10(smooth.max()), 9)
ax.contourf(xx, yy, smooth.T, levels=levels, cmap="cividis")
```

`sigma` и число уровней — параметры сглаживания. Меняйте их осознанно и проверяйте, что вывод не зависит от выбранного сглаживания.

5.07.8 Что дальше

Следующий график — **5.08 loess**: нелинейный тренд на scatter с доверительной полосой. У него будет свой генератор и свой `data/08_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `contourf` — matplotlib.org
- SciPy: `gaussian_filter` — docs.scipy.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com

5.08 LOESS: нелинейный тренд

LOESS (locally estimated scatterplot smoothing) — локальная регрессия: для каждой точки строится свой полином по соседям. Он не предполагает глобальной формы связи и поэтому хорошо показывает насыщение, пороги и изгибы, которые линейная прямая пропускает.

Но LOESS — это **описание тренда**, а не модель причинности. Он зависит от ширины окна (`frac`), плохо ведёт себя на краях и не годится для экстраполяции.

5.08.1 Сфера применения

LOESS отвечает на вопрос: **как выглядит нелинейный тренд, если не навязывать ему форму?**

Где это пригодится:

- доза–эффект с насыщением (статин и ЛПНП, препарат и АД);
- проверка линейности перед регрессией;
- нелинейные биомаркерные ассоциации;
- сглаживание временных рядов (но для времени чаще line/step);
- визуальная проверка порогов и плато.

Когда LOESS **не** нужен:

- точек мало (десятки) — сглаживание неустойчиво;
- нужна предсказательная модель или экстраполяция;
- важна причинная интерпретация;
- данные имеют резкие разрывы, которые нельзя сглаживать;
- `frac` выбран так, что исчезает реальная структура.

5.08.2 Данные

Сценарий: 300 пациентов, фиксированные дозы статина 0, 5, 10, 20, 40 и 80 мг/сут. Эффект — снижение ЛПНП (%), насыщается с ростом дозы.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`dose_mg`	доза статина	мг/сут
`ldl_reduction_pct`	снижение ЛПНП	%

Средние по дозам: 0 мг — 1,7 %, 5 мг — 17,6 %, 10 мг — 26,4 %, 20 мг — 34,5 %, 40 мг — 41,8 %, 80 мг — 47,7 %. Рост замедляется: это и есть насыщение.

Генератор: `05_charts/08_loess/scripts/make_data.py` → `data/08_dataset.csv`.

5.08.3 Плохой график

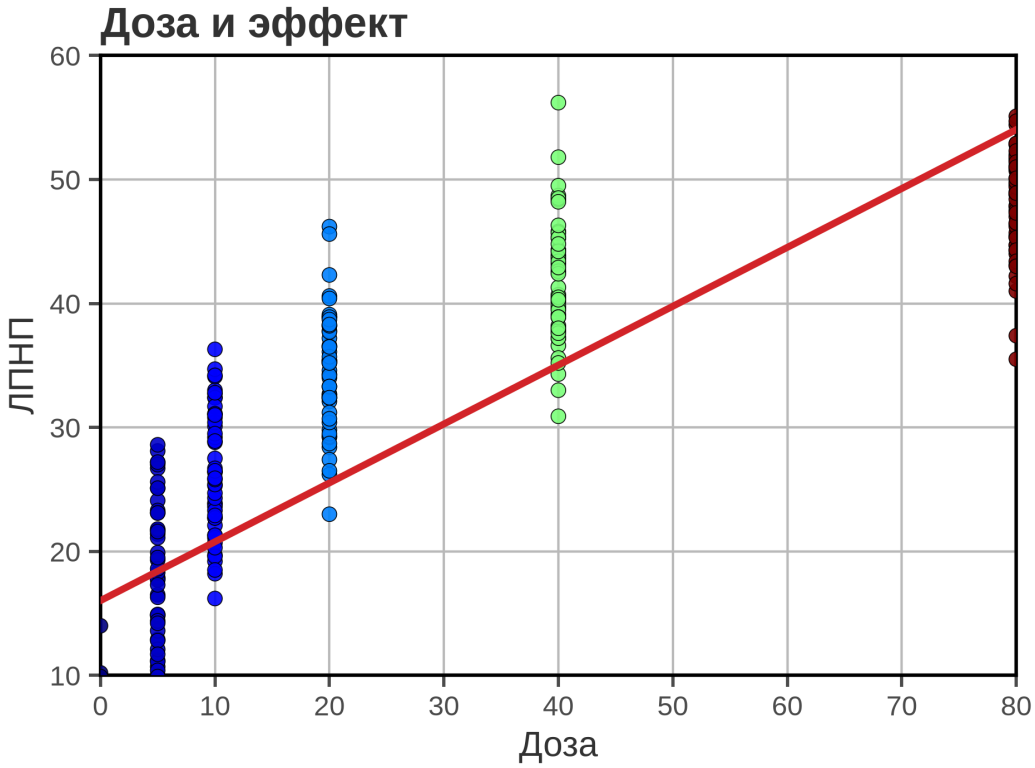


Рисунок 5.8.1. Плохой пример: линейная прямая на данных с насыщением, точки окрашены по дозе, обрезанная ось.

Код: `05_charts/08_loess/scripts/bad_plot.py`.

Что не так:

1. **Линейная регрессия на нелинейных данных.** Прямая не описывает плато: на 80 мг она завышает эффект, на 10–20 мг — занижает.
2. **Цвет точек по дозе.** Доза уже на оси X; цвет дублирует ось и добавляет радужную шкалу.
3. **Обрезанная ось Y (10–60).** Низкие дозы и хвост скрыты.
4. **Крупные непрозрачные точки, тяжёлая сетка, полная рамка.**
5. **Заголовок «Доза и эффект».** Не сообщает, что эффект насыщается.
6. **Нет единиц, n, доверительной полосы и описания модели.**

5.08.4 Правильный график

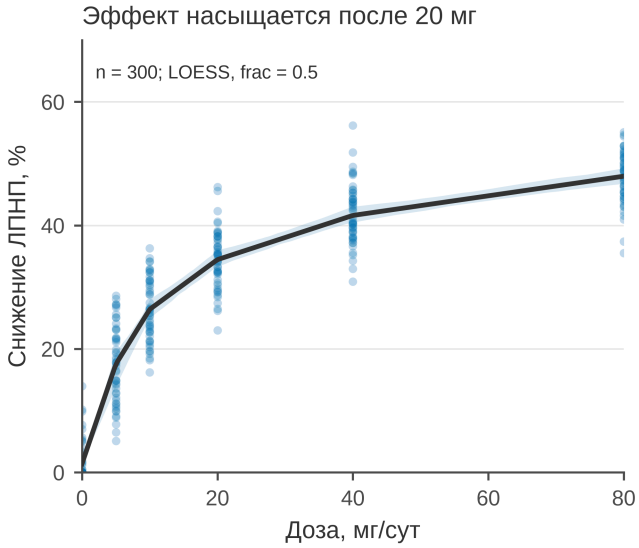


Рисунок 5.8.2. Правильный LOESS: точки, локальная кривая, bootstrap-полоса 95 %, полный диапазон.

Код: `05_charts/08_loess/scripts/good_plot.py`.

Что изменилось:

1. **LOESS вместо прямой.** Видно быстрое снижение ЛПНП до 20 мг и выполаживание после.
2. **Bootstrap-полоса 95 %.** Показана неопределённость кривой; на краях она шире.
3. **Полный диапазон осей.** 0–80 мг и 0–70 %.
4. **Полупрозрачные точки одного цвета.** Видна плотность, нет двойного кодирования.
5. **Единицы, n и `frac`.** «Доза, мг/сут», «Снижение ЛПНП, %», *n* = 300, `frac = 0.5`.
6. **Заголовок-вывод.** «Эффект насыщается после 20 мг» задаёт сообщение.
7. **Лёгкая сетка только по Y, без верхней и правой рамок.**
8. **85 мм.** Рисунок спроектирован в конечном размере.

`frac` — доля наблюдений в локальном окне. Маленький `frac` даёт извилистую кривую, большой — слишком гладкую. Показывайте выбранное значение и проверяйте устойчивость.

5.08.5 Было и стало

Что было	Почему плохо	Что сделано
линейная прямая	не описывает плато	LOESS
цвет по дозе	двойное кодирование	один цвет
ось Y 10–60	низкие дозы обрезаны	0–70
крупные точки	шум	s=10, alpha=0,25
нет CI	не видна неопределённость	bootstrap 95 %
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
«Доза и эффект»	нет сообщения	заголовок-вывод
нет единиц и n	невозможно прочитать	единицы, n, frac

5.08.6 Чек-лист LOESS

- ☐ Понятно, что кривая — описание тренда, а не модель причинности.
- ☐ Указан `frac` (или span) и число точек.
- ☐ Показана неопределённость (bootstrap/CI).
- ☐ Оси не обрезают данные.
- ☐ Нет экстраполяции за пределы данных.
- ☐ Краевые эффекты не интерпретируются как реальные.
- ☐ Точки видны, а не закрыты линией.
- ☐ Есть единицы и *n*.
- ☐ Сравнение с линейной моделью/Spline/GAM осознано.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.08.7 Как воспроизвести

```
uv sync
uv run python 05_charts/08_loess/scripts/make_data.py
uv run python 05_charts/08_loess/scripts/bad_plot.py
uv run python 05_charts/08_loess/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
from statsmodels.nonparametric.smoothers_lowess import lowess

fit = lowess(y, x, frac=0.5, return_sorted=True)
grid = np.linspace(0, 80, 200)
mean_line = np.interp(grid, fit[:, 0], fit[:, 1])

# bootstrap-полоса
boots = []
for _ in range(200):
    idx = rng.integers(0, len(x), len(x))
    boot = lowess(y[idx], x[idx], frac=0.5, return_sorted=True)
    boots.append(np.interp(grid, boot[:, 0], boot[:, 1]))
low, high = np.percentile(boots, [2.5, 97.5], axis=0)
```

Bootstrap здесь оценивает неопределённость **кривой**, а не отдельных пациентов. Для формального вывода о дозе–эффекте нужна параметрическая модель (Emax, spline, GAM) и клиническое обоснование.

5.08.8 Что дальше

Следующий график — **5.09 histogram**: распределение одной непрерывной величины. У него будет свой генератор и свой `data/09_dataset.csv`.

Источники

- Cleveland W. S. Robust Locally Weighted Regression and Smoothing Scatterplots // *JASA*. 1979. — doi:10.1080/01621459.1979.10482531
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Statsmodels: LOWESS — statsmodels.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com

5.09 Гистограмма: распределение одной величины

Гистограмма показывает, как распределена одна непрерывная величина: центр, разброс, форму, скошенность и выбросы. Она отвечает на вопрос «как устроена выборка», а не «как связаны две переменные».

Главная ловушка — **выбор ширины бина**. Слишком широкие бины скрывают форму, слишком узкие — превращают шум в «структуру». Гистограмма — это семейство изображений, а не одно «истинное».

5.09.1 Сфера применения

Гистограмма отвечает на вопрос: **как распределена величина и где её центр, разброс, хвосты?**

Где это пригодится:

- распределение биомаркера (витамин D, CRP, глюкоза);
- проверка нормальности/скошенности перед тестом;
- поиск выбросов и многомодальности;
- описание выборки до сравнения групп;
- контроль качества измерений.

Когда гистограмма **не** подходит:

- наблюдений мало (десятки) — лучше точки/strip;
- нужно точное значение или квантиль — ECDF (5.10);
- несколько групп — box/violin/small multiples;
- важна форма хвоста, а бины её огрубляют;
- ось X обрезана и хвост скрыт.

5.09.2 Данные

Сценарий: 1 200 взрослых, уровень 25(OH)D (нг/мл). Распределение сдвинуто влево и имеет правый хвост.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`vitamin_d_ng_ml`	25(OH)D	нг/мл

Медиана — 23,4 нг/мл; IQR — 16,5–32,5; дефицит (< 20 нг/мл) — у 38 %; диапазон 3–70 нг/мл.

Генератор: `05_charts/09_histogram/scripts/make_data.py → data/09_dataset.csv`.

5.09.3 Плохой график

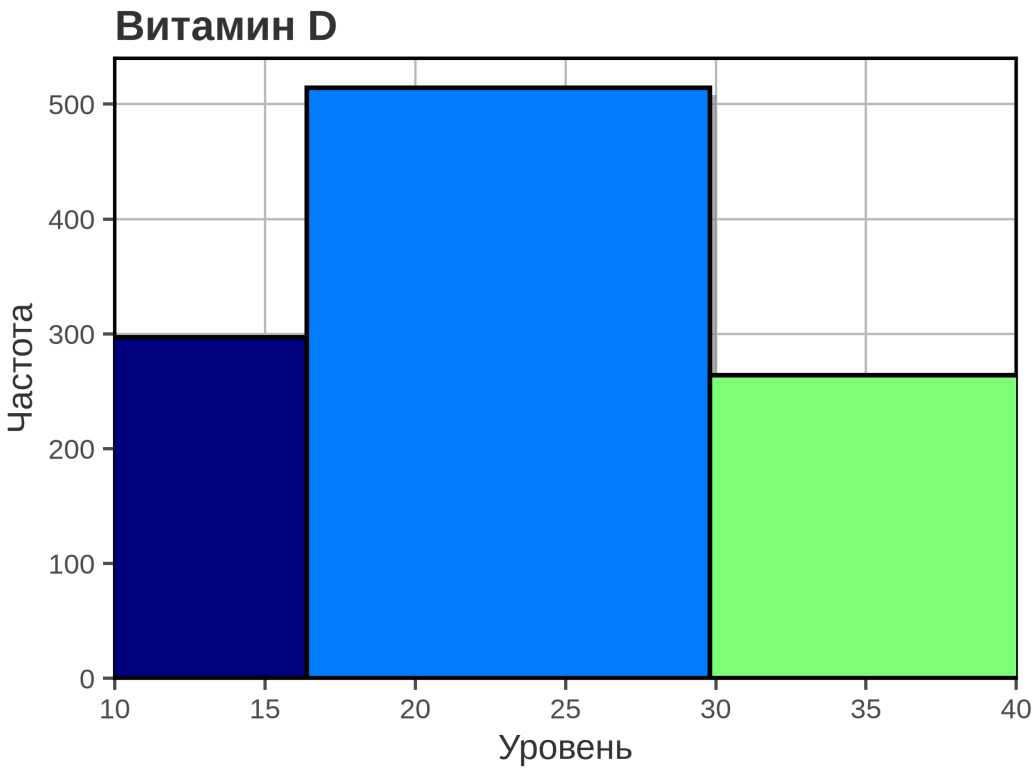


Рисунок 5.9.1. Плохая гистограмма: пять бинов, обрезанная ось 10–40, радужные столбцы, тени, нет единиц и порога.

Код: `05_charts/09_histogram/scripts/bad_plot.py`.

Что не так:

1. **Пять бинов.** Форма распределения и правый хвост исчезли; осталось три столбца.
2. **Обрезанная ось X (10–40).** Низкие и высокие значения не видны.
3. **Радужные столбцы, чёрные края и тени.** Декор, который не кодирует данные.
4. **Тяжёлая сетка по обеим осям и полная рамка.**
5. **Заголовок «Витамин D».** Не сообщает, что важно: дефицит, медиана, форма.
6. **Нет единиц, n, медианы и порога 20.** Гистограмма не читается как клинический результат.

5.09.4 Правильный график

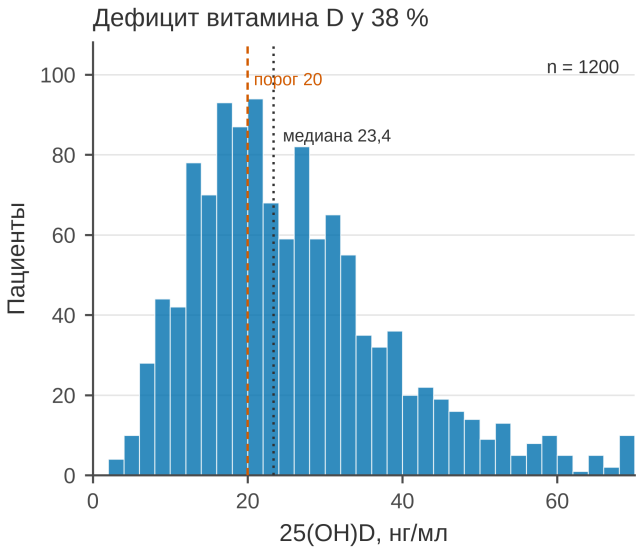


Рисунок 5.9.2. Правильная гистограмма: бины по 2 нг/мл, полный диапазон, порог 20, медиана, n и доля дефицита.

Код: `05_charts/09_histogram/scripts/good_plot.py`.

Что изменилось:

1. **Ширина бина 2 нг/мл.** Видна форма, правый хвост и мода около 18–24 нг/мл.
2. **Полный диапазон 0–70 нг/мл.** Ничего не обрезано.
3. **Один цвет, белые края, без теней.**
4. **Порог дефицита 20 нг/мл** выделен пунктиром.
5. **Медиана 23,4 нг/мл** отмечена и подписана.
6. **n = 1200 и доля дефицита 38 %** в заголовке.
7. **Единицы, лёгкая сетка только по Y, без верхней и правой рамок.**
8. **85 мм.** Рисунок спроектирован в конечном размере.

Проверьте устойчивость: постройте гистограмму с шириной 1, 2 и 5 нг/мл. Если вывод меняется, он не должен держаться на одном выборе бинов.

5.09.5 Было и стало

Что было	Почему плохо	Что сделано
5 бинов	форма и хвост скрыты	бины по 2 нг/мл
ось X 10–40	хвосты обрезаны	0–70
радуга, тени	декор	один цвет
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
«Витамин D»	нет сообщения	заголовок о дефиците
нет единиц и n	невозможно прочитать	единицы, n
нет порога и медианы	клинический контекст потерян	порог 20, медиана
counts без подписи	непонятно, что такое Y	«Пациенты»

5.09.6 Чек-лист гистограммы

- ☐ Указана единица измерения.
- ☐ Ось X не обрезает хвосты.
- ☐ Ширина бина осмысленна; проверена устойчивость.
- ☐ Указаны *n* и, если важно, медиана/IQR.
- ☐ Есть клинический порог, если он существенен.
- ☐ Один цвет, без 3D и теней.
- ☐ Сетка не конкурирует с данными.
- ☐ Понятно, что по Y: пациенты или плотность.
- ☐ Для малых *n* рассмотрены точки/ECDF.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.09.7 Как воспроизвести

```
uv sync
uv run python 05_charts/09_histogram/scripts/make_data.py
uv run python 05_charts/09_histogram/scripts/bad_plot.py
uv run python 05_charts/09_histogram/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
bins = np.arange(0, 72, 2)
counts, _, _ = ax.hist(
    values, bins=bins,
    color=PRIMARY_BLUE, alpha=0.8,
    edgecolor="white", linewidth=0.4,
)
ax.axvline(20, color=ACCENT_VERMILLION, linestyle="--", linewidth=0.9)
ax.axvline(np.median(values), color="#333333", linestyle=":", linewidth=0.9)
```

`bins` можно задать числом или массивом границ. Массив границ предпочтительнее: он фиксирует ширину и облегчает сравнение нескольких выборок.

5.09.8 Что дальше

Следующий график — **5.10 ECDF**: эмпирическая функция распределения, точное чтение квантилей. У него будет свой генератор и свой `data/10_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `hist` — matplotlib.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com
- Weissgerber T. L. et al. Beyond Bar and Line Graphs // *PLoS Biology*. 2015. — doi:10.1371/journal.pbio.1002128

5.10 ECDF: функция распределения

ECDF (empirical cumulative distribution function) показывает, какая доля наблюдений не превышает каждое значение X. В отличие от гистограммы, здесь нет бинов: кривая строится по всем точкам и позволяет точно прочитать медиану, квартили и любой перцентиль.

ECDF — это ступенчатая функция. Каждый скачок — одно или несколько наблюдений; высота ступени — накопленная доля.

5.10.1 Сфера применения

ECDF отвечает на вопрос: **какая доля пациентов имеет значение ≤ x и где проходят квантили?**

Где это пригодится:

- распределение длительности госпитализации, времени до выписки;
- сравнение двух групп по всей кривой, а не только по средним;
- чтение медианы, IQR и перцентилей;
- проверка сдвига распределения;
- малые и средние выборки, где гистограмма капризна.

Когда ECDF **не** подходит:

- время до события с цензурированием — Kaplan–Meier (5.27);
- нужно показать плотность/форму — histogram/KDE;
- групп слишком много — кривые перекрываются (нужны small multiples);
- нужна одна сводка, а не распределение — dot/box.

5.10.2 Данные

Сценарий: 1 200 госпитализаций, 600 плановых и 600 экстренных. Длительность — дни; распределение скошено вправо.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`admission_type`	плановое/экстренное	—
`los_days`	длительность госпитализации	дни

Медиана: плановые — 6 дней, экстренные — 8,5 дня. Средние: 6,2 и 9,4. Правый хвост — до 20–25 дней.

Генератор: `05_charts/10_ecdf/scripts/make_data.py → data/10_dataset.csv`.

5.10.3 Плохой график

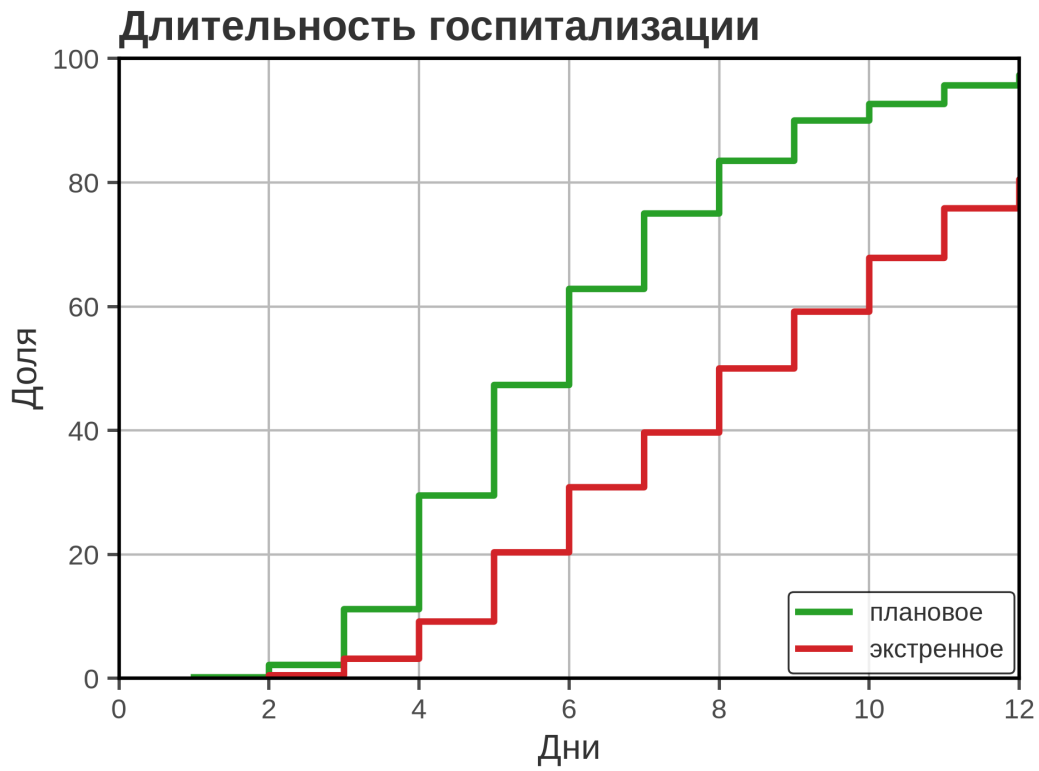


Рисунок 5.10.1. Плохая ECDF: обрезанная ось 0–12, красно-зелёные ступени, легенда, нет медиан и n.

Код: `05_charts/10_ecdf/scripts/bad_plot.py`.

Что не так:

1. **Ось X обрезана до 12 дней.** Правый хвост и половина кривой скрыты; экстренные госпитализации выглядят короче, чем есть.
2. **Красно-зелёная пара.** Неразличима при протанопии/дейтеранопии.
3. **Легенда.** Заставляет переводить взгляд; для двух групп лучше прямые подписи.
4. **Толстые ступени, тяжёлая сетка, полная рамка.**
5. **Нет медиан, квартилей, единиц и n.**
6. **Заголовок «Длительность госпитализации».** Не сообщает, что экстренные длиннее.

5.10.4 Правильный график

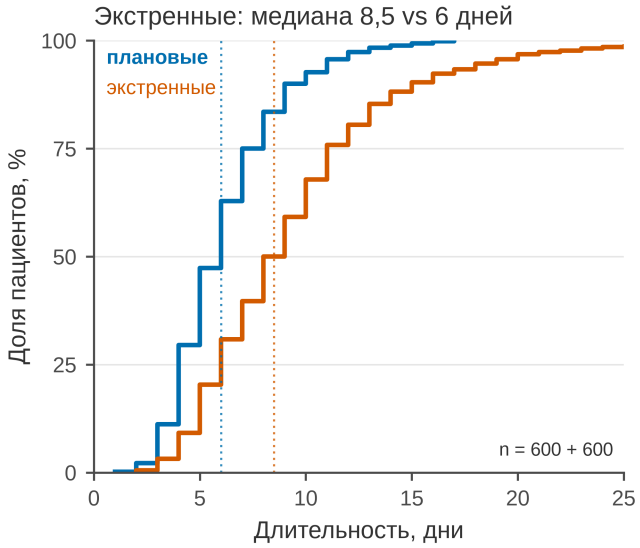


Рисунок 5.10.2. Правильная ECDF: полный диапазон 0–25, синий и вермиллион, медианы, прямые подписи, n.

Код: `05_charts/10_ecdf/scripts/good_plot.py`.

Что изменилось:

1. **Полный диапазон 0–25 дней.** Обе кривые видны целиком, включая хвост.
2. **Синий и вермиллион.** Различимы при CVD; без легенды, с прямыми подписями.
3. **Медианы отмечены пунктиром.** 6 и 8,5 дня; разница видна на уровне 50 %.
4. **Ось Y в процентах.** «Доля пациентов, %» читается без пересчёта.
5. **n = 600 + 600.** Понятно, сколько наблюдений в группах.
6. **Заголовок-вывод.** «Экстренные: медиана 8,5 vs 6 дней».
7. **Лёгкая сетка только по Y, без верхней и правой рамок.**
8. **85 мм.** Рисунок спроектирован в конечном размере.

ECDF читается так: найдите на оси X интересующий срок, поднимитесь до кривой и посмотрите долю на оси Y. Медиана — точка, где кривая пересекает 50 %.

5.10.5 Было и стало

Что было	Почему плохо	Что сделано
ось X 0–12	хвост обрезан	0–25
красный/зелёный	CVD	синий/вермиллион
легенда	лишний поиск	прямые подписи
толстые ступени	шум	1,7 pt
нет медиан	не читаются квантили	пунктирные медианы
нет единиц и n	невозможно прочитать	«дни», n = 600 + 600
«Длительность…»	нет сообщения	заголовок с медианами
сетка по X и Y	шум	сетка только по Y

5.10.6 Чек-лист ECDF

- ☐ Ось X не обрезает хвост.
- ☐ Указаны единицы и *n*.
- ☐ Подписаны группы (прямо или легендой).
- ☐ Медиана/квартили отмечены, если они важны.
- ☐ Ось Y — доля или процент, ясно что именно.
- ☐ Нет лишних цветов и толстых ступеней.
- ☐ Для цензурированных данных выбран Kaplan–Meier.
- ☐ Сравнение групп не сводится к средним.
- ☐ Кривые не перекрываются из-за числа групп.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.10.7 Как воспроизвести

```
uv sync
uv run python 05_charts/10_ecdf/scripts/make_data.py
uv run python 05_charts/10_ecdf/scripts/bad_plot.py
uv run python 05_charts/10_ecdf/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
def ecdf(values):
    xs = np.sort(values)
    ys = np.arange(1, len(xs) + 1) / len(xs) * 100
    return xs, ys

xs, ys = ecdf(values)
ax.step(xs, ys, where="post", color=PRIMARY_BLUE, linewidth=1.7)
ax.axvline(np.median(values), color=PRIMARY_BLUE, linestyle=":", linewidth=0.8)
```

`where="post"` важен: ступень держится до следующего значения. Для сравнения групп используйте одинаковые оси и прямые подписи.

5.10.8 Что дальше

Следующий график — **5.11 box plot**: сравнение распределений по группам. У него будет свой генератор и свой `data/11_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `step` — matplotlib.org
- Weissgerber T. L. et al. Beyond Bar and Line Graphs // *PLoS Biology*. 2015. — doi:10.1371/journal.pbio.1002128
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com

5.11 Boxplot: сравнение распределений по группам

Boxplot — компактная сводка распределения: медиана, межквартильный размах (IQR), усы и выбросы. Он хорош для сравнения 2–5 групп, но **сам по себе скрывает форму** распределения и размер выборки.

Поэтому в этом разделе показаны четыре варианта:

1. плохой boxplot;
2. boxplot + отдельные точки;
3. violin + box + точки;
4. boxplot, совмещённый с гистограммой.

5.11.1 Сфера применения

Boxplot отвечает на вопрос: **как различаются медианы и разброс между группами?**

Где это пригодится:

- сравнение биомаркера по 2–5 группам (активность, тяжесть, центр);
- проверка сдвига и разброса до формального теста;
- поиск выбросов;
- компактная сводка в таблице рисунков;
- сравнение до/после (но для парных данных — 5.18).

Когда boxplot **не** достаточен:

- нужно увидеть форму/модальность — violin или histogram;
- групп много — small multiples;
- выборка маленькая — точки важнее коробки;
- нужны точные квантили — ECDF (5.10);
- данные цензурированы — Kaplan–Meier (5.27).

5.11.2 Данные

Сценарий: 450 взрослых, три уровня физической активности — низкая, умеренная, высокая. Показатель — BMI (кг/м²), по 150 человек в группе.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`activity`	уровень активности	—
`bmi_kg_m2`	индекс массы тела	кг/м²

Медианы: низкая — 29,4; умеренная — 27,8; высокая — 24,4 кг/м². Есть правый хвост и отдельные выбросы.

Генератор: `05_charts/11_box/scripts/make_data.py → data/11_dataset.csv`.

5.11.3 Плохой график

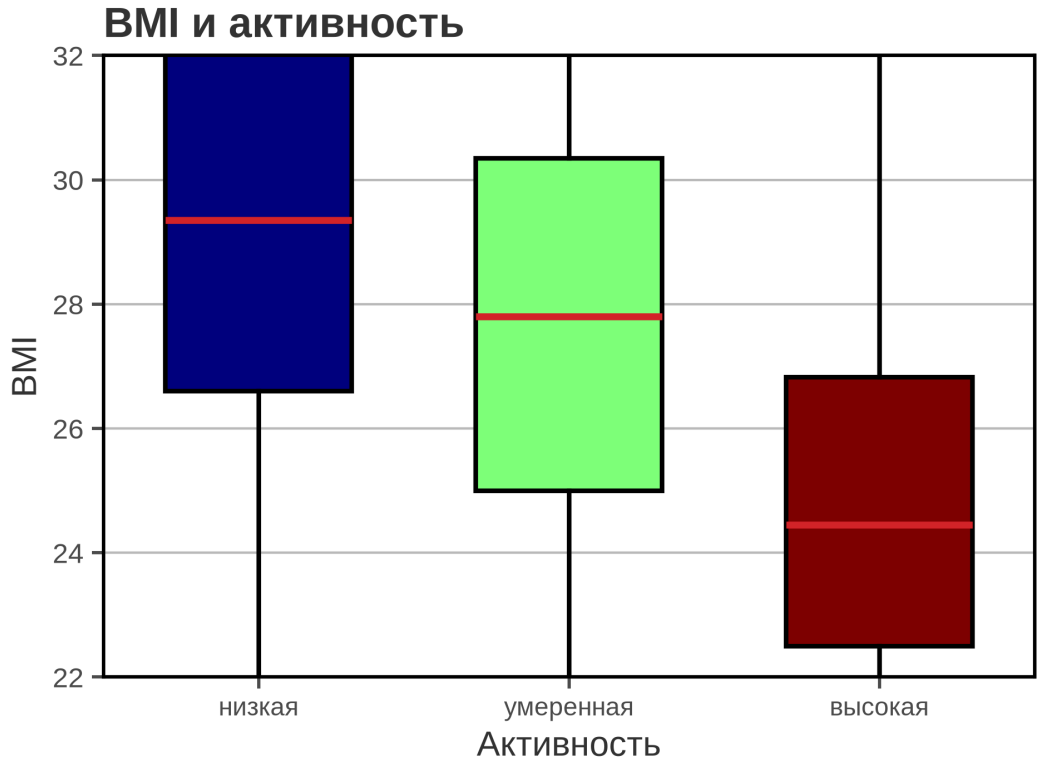


Рисунок 5.11.1. Плохой boxplot: обрезанная ось 22–32, радужные ящики, нет точек, тяжёлая сетка.

Код: `05_charts/11_box/scripts/bad_plot.py`.

Что не так:

1. **Ось Y обрезана до 22–32.** Выбросы и хвосты скрыты; высокая группа выглядит «обрезанной».
2. **Радужные ящики, красные медианы и выбросы.** Декор вместо данных; CVD-небезопасно.
3. **Нет отдельных точек.** Не видно формы, размера групп и того, что за пределами усов.
4. **Тяжёлая сетка, полная рамка, толстые линии.**
5. **Заголовок «BMI и активность».** Не сообщает направление и величину различий.
6. **Нет единиц и *n*.**

5.11.4 Boxplot + точки

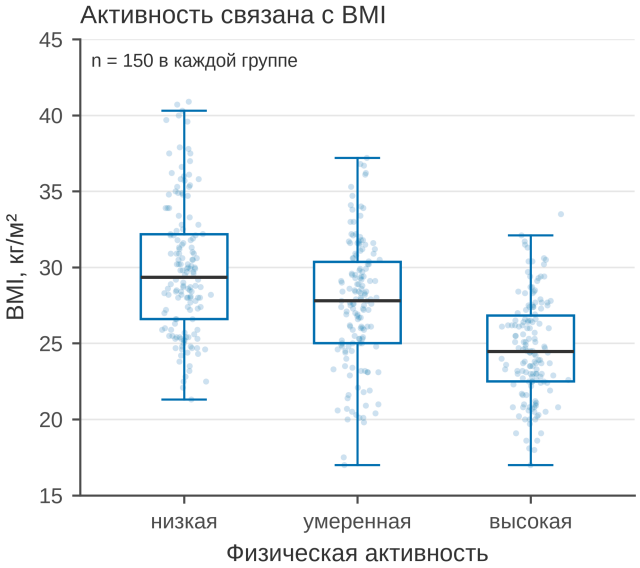


Рисунок 5.11.2. Правильный boxplot с точками: полный диапазон 15–45, jitter, n = 150, единицы.

Код: `05_charts/11_box/scripts/good_box_points.py`.

Что изменилось:

1. **Полный диапазон 15–45 кг/м².** Видны хвосты и выбросы.
2. **Белые ящики с синим контуром, тёмная медиана.** Один цвет, без радуги.
3. **Все 450 точек с jitter.** Видна плотность, форма и размер групп; выбросы не прячутся.
4. **Единицы, *n* = 150, заголовок-вывод.**
5. **Лёгкая сетка только по Y, 85 мм.**

Точки не заменяют boxplot: они показывают, что внутри коробки, а коробка даёт сводку. Вместе они честнее, чем boxplot отдельно.

5.11.5 Violin + box + точки

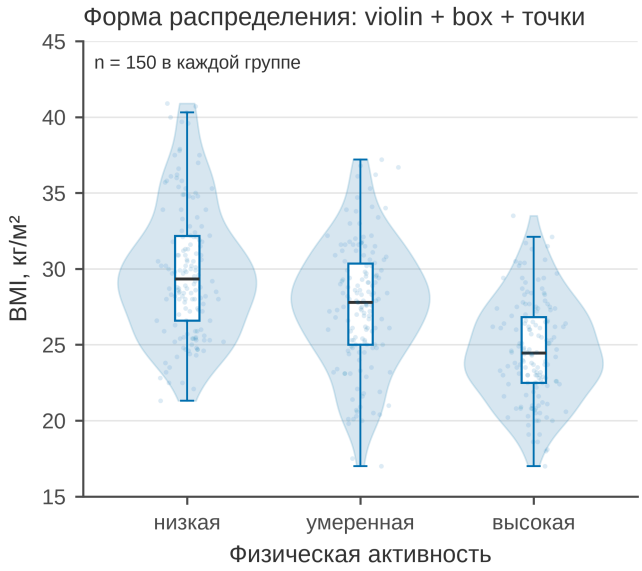


Рисунок 5.11.3. Violin + box + точки: форма распределения, медиана и исходные наблюдения.

Код: `05_charts/11_box/scripts/good_violin_box.py`.

Что показывает:

1. **Violin** — оценку плотности: где значения встречаются чаще.
2. **Box** — медиану, IQR и усы.
3. **Точки** — исходные наблюдения.

Это «raincloud»-подход: форма, сводка и данные одновременно. Он хорош для 3 групп и умеренных выборок. Но violin — это KDE: она может «продлить» хвост за пределы реальных данных и создать ложную плавность. При малых *n* доверяйте точкам больше, чем контуру.

5.11.6 Boxplot + гистограмма

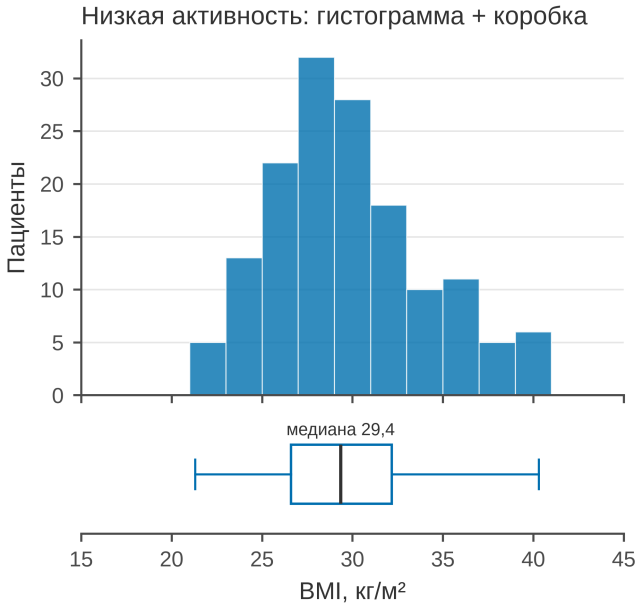


Рисунок 5.11.4. Совмещённый график: гистограмма сверху, горизонтальный boxplot снизу, общая ось BMI.

Код: `05_charts/11_box/scripts/good_box_hist.py`.

Что показывает:

1. **Гистограмма сверху** — форму распределения (бины по 2 кг/м²).
2. **Boxplot снизу** — медиану, IQR, усы и выбросы.
3. **Общая ось X** — гистограмма и коробка выровнены.

Этот вариант полезен, когда нужно показать и форму, и сводку для одной переменной или одной группы. Медиана 29,4 кг/м² подписана прямо над коробкой.

5.11.7 Было и стало

Что было	Почему плохо	Что сделано
ось Y 22–32	выбросы и хвосты скрыты	15–45
радуга, красные медианы	декор, CVD	один синий цвет
нет точек	не видна форма	jitter-точки
толстые линии	шум	0,8–1,2 pt
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
«BMI и активность»	нет сообщения	заголовок-вывод
нет единиц и n	невозможно прочитать	единицы, n = 150
только коробка	форма скрыта	1. violin / histogram

5.11.8 Чек-лист boxplot

- ☐ Ось Y не обрезает усы и выбросы.
- ☐ Указаны единицы и *n* (по группам).
- ☐ Медиана и IQR читаются; усы объяснены.
- ☐ Точки показаны, если *n* небольшое или важна форма.
- ☐ Для формы добавлен violin или histogram.
- ☐ Цвет не дублирует группы без смысла.
- ☐ Нет 3D, теней и радуги.
- ☐ Сетка не конкурирует с данными.
- ☐ Для многих групп — small multiples.
- ☐ Рисунок проверен в конечном размере.

5.11.9 Как воспроизвести

```
uv sync
uv run python 05_charts/11_box/scripts/make_data.py
uv run python 05_charts/11_box/scripts/bad_plot.py
uv run python 05_charts/11_box/scripts/good_box_points.py
uv run python 05_charts/11_box/scripts/good_violin_box.py
uv run python 05_charts/11_box/scripts/good_box_hist.py
```

Ключевые фрагменты:

```
# boxplot + точки
ax.boxplot(data, positions=[1, 2, 3], widths=0.5, patch_artist=True,
           showfliers=False, boxprops={"facecolor": "white"})
for position, values in zip([1, 2, 3], data, strict=True):
    ax.scatter(position + rng.normal(0, 0.06, len(values)), values,
               s=5, alpha=0.18, linewidths=0)

# violin + box
ax.violinplot(data, positions=[1, 2, 3], widths=0.8, showextrema=False)

# histogram + horizontal box
fig, (ax_hist, ax_box) = plt.subplots(2, 1, sharex=True,
                                      gridspec_kw={"height_ratios": [3, 1]})
ax_box.boxplot([values], orientation="horizontal", positions=[0.5])
```

5.11.10 Что дальше

Следующий график — **5.12 violin**: отдельный разбор скрипичной диаграммы, её преимуществ и ловушек. У него будет свой генератор и свой `data/12_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Weissgerber T. L. et al. Beyond Bar and Line Graphs // *PLoS Biology*. 2015. — doi:10.1371/journal.pbio.1002128
- Matplotlib: `boxplot` — matplotlib.org
- Matplotlib: `violinplot` — matplotlib.org

5.12 Violin: форма распределения

Violin plot — это зеркальная оценка плотности (KDE). Ширина скрипки показывает, как часто встречаются значения; форма — модальность, скошенность, хвосты. В отличие от boxplot, violin не прячет форму за коробкой.

Но violin — это **оценка**, а не исходные данные. Ширина зависит от bandwidth, хвосты могут выходить за реальный диапазон, а при малых *n* контур становится фантазией. Поэтому внутри скрипки нужна сводка: квартильные линии или коробка.

5.12.1 Сфера применения

Violin отвечает на вопрос: **как выглядит форма распределения в каждой группе?**

Где это пригодится:

- сравнение 2–5 групп по непрерывной величине;
- поиск модальности/двугорбости;
- проверка формы перед выбором теста;
- замена boxplot, когда форма важна;
- презентация распределений в больших выборках.

Когда violin **не** нужен:

- *n* меньше 20 — лучше точки/strip;
- нужны точные квантили — ECDF;
- групп много — small multiples;
- данные цензурированы — Kaplan–Meier;
- важна только медиана/IQR — boxplot.

5.12.2 Данные

Сценарий: 600 пациентов, три стадии ХБП (1–2, 3, 4–5). Показатель — eGFR (мл/мин/1,73 м²), по 200 человек в группе.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`ckd_stage`	стадия ХБП	—
`sex`	пол	—
`egfr_ml_min_1_73m2`	СКФ	мл/мин/1,73 м²

Медианы: 88,2; 44,0; 20,8 мл/мин/1,73 м². Распределения сдвигаются вниз, форма меняется.

Генератор: `05_charts/12_violin/scripts/make_data.py → data/12_dataset.csv`.

5.12.3 Плохой график

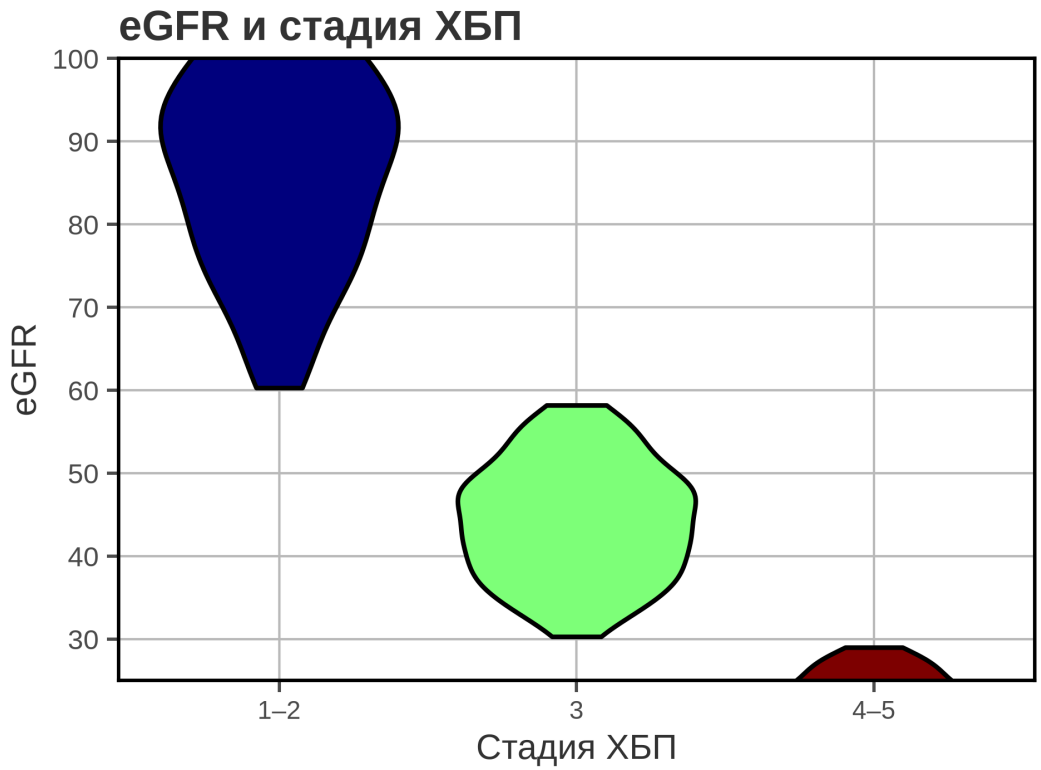


Рисунок 5.12.1. Плохой violin: обрезанная ось 25–100, радужные тела, нет медиан и точек.

Код: `05_charts/12_violin/scripts/bad_plot.py`.

Что не так:

1. **Ось Y обрезана до 25–100.** Стадия 4–5 и верхний хвост стадии 1–2 скрыты; форма искажена.
2. **Радужные тела и толстые чёрные контуры.** Декор, CVD.
3. **Нет медианы, квартилей и точек.** Видна форма, но нет сводки и исходных наблюдений.
4. **Тяжёлая сетка, полная рамка.**
5. **Заголовок «eGFR и стадия ХБП».** Не сообщает направление.
6. **Нет единиц и *n*.**

5.12.4 Violin с квартильными линиями

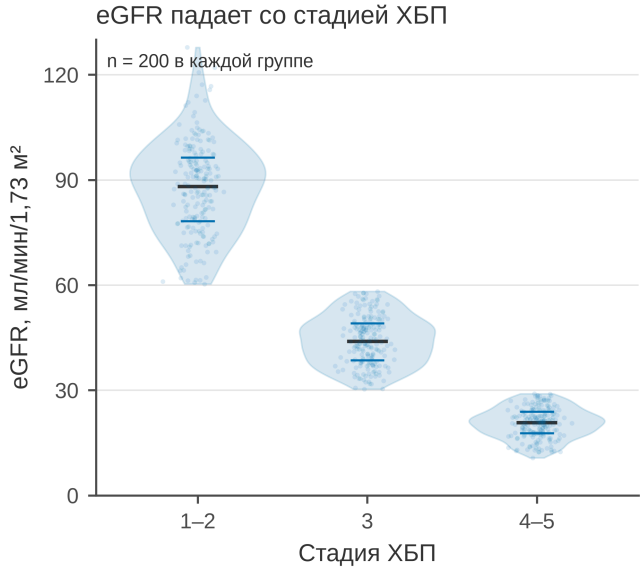


Рисунок 5.12.2. Правильный violin: полный диапазон 0–130, квартильные линии, точки, n = 200.

Код: `05_charts/12_violin/scripts/good_violin_quart.py`.

Что изменилось:

1. **Полный диапазон 0–130.** Все стадии и хвосты видны.
2. **Один цвет, без радуги.**
3. **Квартили и медиана — горизонтальными линиями.** Медиана толще, Q1/Q3 тоньше.
4. **Точки с jitter.** Видны наблюдения и размер групп.
5. **Единицы, *n* = 200, заголовок-вывод.**
6. **Лёгкая сетка по Y, 85 мм.**

5.12.5 Violin с внутренней коробкой

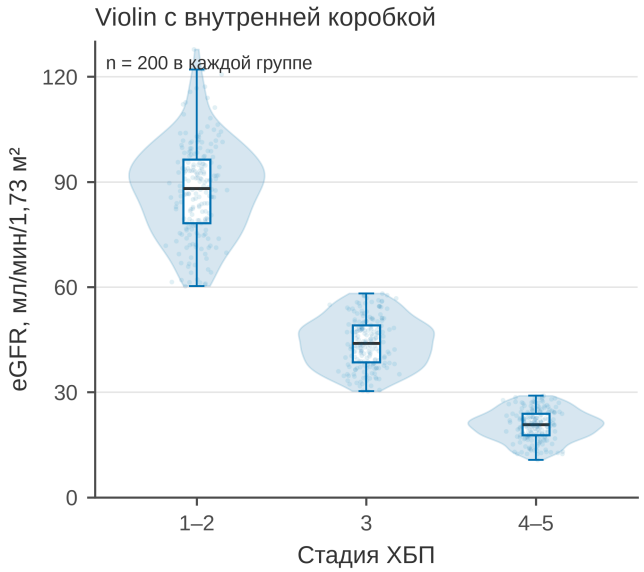


Рисунок 5.12.3. Violin с прямоугольной коробкой внутри: медиана, IQR, усы и точки.

Код: `05_charts/12_violin/scripts/good_violin_box.py`.

Это версия «violin + прямоугольники»: внутри каждой скрипки — мини-коробка с медианой, IQR и усами. Она полезна, когда нужно одновременно показать форму (violin) и классическую сводку (box).

На что смотреть: коробка внутри должна быть узкой, иначе она закрывает скрипку. Усы и выбросы можно не показывать, если точки уже нанесены; но тогда это нужно указать в подписи.

5.12.6 Было и стало

Что было	Почему плохо	Что сделано
ось Y 25–100	стадия 4–5 обрезана	0–130
радуга, толстые контуры	декор, CVD	один синий цвет
нет медиан	нет сводки	квартильные линии / коробка
нет точек	не видно наблюдений	jitter-точки
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
«eGFR и стадия ХБП»	нет сообщения	заголовок-вывод
нет единиц и n	невозможно прочитать	единицы, n = 200
только violin	нет сводки	violin + quart / box

5.12.7 Чек-лист violin

- ☐ Ось Y не обрезает хвосты.
- ☐ Указаны единицы и *n*.
- ☐ Внутри есть медиана/IQR (линии или коробка).
- ☐ Точки показаны, если *n* небольшое.
- ☐ Bandwidth/trim не скрывают структуру.
- ☐ Нет радуги и тяжёлых контуров.

- ❐ Для малых *n* выбран strip/dot.
- ❐ Для многих групп — small multiples.
- ❐ Форма не интерпретируется как «точная» плотность.
- ❐ Рисунок проверен в конечном размере 85 мм.

5.12.8 Как воспроизвести

```
uv sync
uv run python 05_charts/12_violin/scripts/make_data.py
uv run python 05_charts/12_violin/scripts/bad_plot.py
uv run python 05_charts/12_violin/scripts/good_violin_quart.py
uv run python 05_charts/12_violin/scripts/good_violin_box.py
```

Ключевые фрагменты:

```
violins = ax.violinplot(data, positions=[1, 2, 3], widths=0.8,
                        showextrema=False, points=100)
for body in violins["bodies"]:
    body.set_facecolor(PRIMARY_BLUE)
    body.set_alpha(0.15)

# квартильные линии
q1, median, q3 = np.percentile(values, [25, 50, 75])
ax.hlines([q1, q3], position - 0.1, position + 0.1, color=PRIMARY_BLUE)
ax.hlines(median, position - 0.12, position + 0.12, color="#333333", linewidth=1.3)

# коробка внутри
ax.boxplot(data, positions=[1, 2, 3], widths=0.16, patch_artist=True,
           showfliers=False, boxprops={"facecolor": "white"})
```

5.12.9 Что дальше

Следующий график — **5.13 strip**: отдельные точки по группам без коробки. У него будет свой генератор и свой `data/13_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Hintze J. L., Nelson R. D. Violin Plots: A Box Plot-Density Trace Synergism // *The American Statistician*. 1998. — doi:10.1080/00031305.1998.10480559
- Matplotlib: `violinplot` — matplotlib.org
- Weissgerber T. L. et al. Beyond Bar and Line Graphs // *PLoS Biology*. 2015. — doi:10.1371/journal.pbio.1002128

5.13 Strip: все точки по группам

Strip plot (jitter plot) показывает каждое наблюдение: точки располагаются по группам, а небольшой случайный сдвиг по X не даёт им слиться. Это самый честный способ сравнить небольшие выборки: видны разброс, выбросы и форма, а не только медиана.

Jitter — не декор и не «шум». Он должен быть случайным, но воспроизводимым (seed), и не настолько большим, чтобы точки уходили в соседнюю группу.

5.13.1 Сфера применения

Strip отвечает на вопрос: **как выглядят все наблюдения в группах и где медиана?**

Где это пригодится:

- небольшие выборки (10–50 на группу);
- сравнение 2–5 групп без коробки;
- показ выбросов и гетерогенности;
- замена bar/box, когда важны отдельные пациенты;
- проверка, не скрывает ли среднее бимодальность.

Когда strip **не** нужен:

- точек тысячи — overplotting, нужны hexbin/contour;
- нужны точные квантили — ECDF;
- важна только сводка — box/dot;
- группы сильно разного размера — точки могут перегрузить ось;
- jitter скрывает реальную структуру (например, дискретные значения).

5.13.2 Данные

Сценарий: 90 пациентов, три схемы обезболивания — НПВС, опиоид, комбинация. Через 24 часа — боль по ВАШ (0–100 мм), по 30 человек в группе.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`analgesia`	схема обезболивания	—
`vas_mm`	боль по ВАШ	мм

Медианы: НПВС — 56, опиоид — 45,5, комбинация — 32,5 мм.

Генератор: `05_charts/13_strip/scripts/make_data.py → data/13_dataset.csv`.

5.13.3 Плохой график

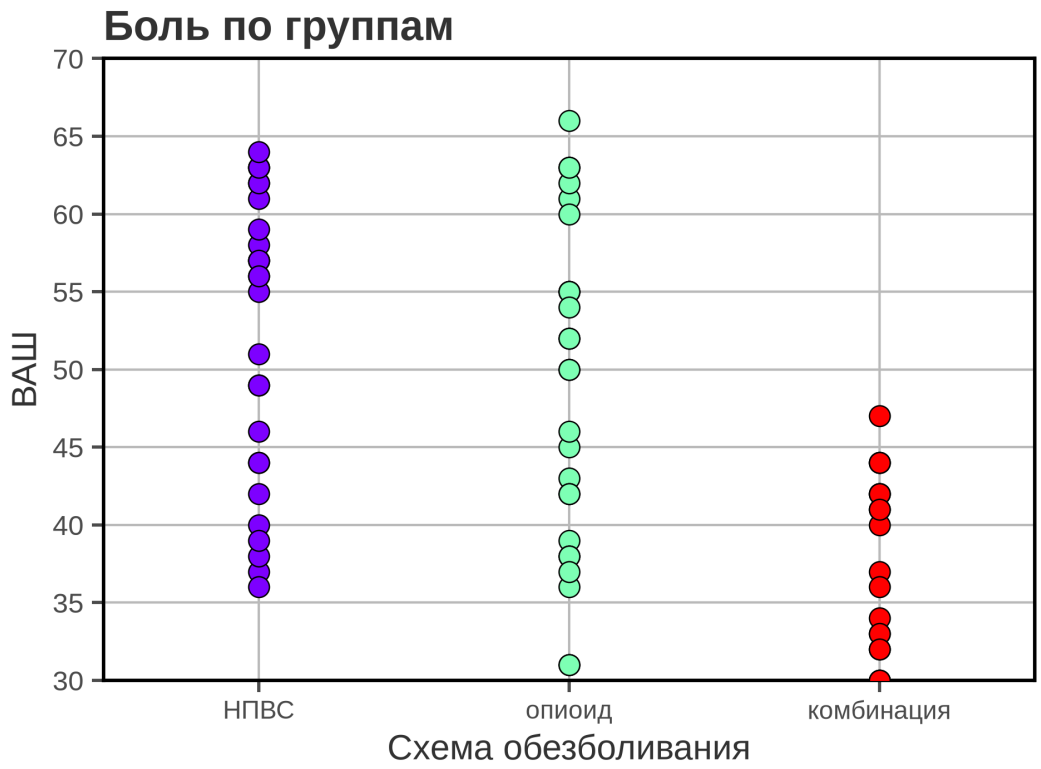


Рисунок 5.13.1. Плохой strip: точки без jitter, радуга, обрезанная ось 30–70, нет медиан и n.

Код: `05_charts/13_strip/scripts/bad_plot.py`.

Что не так:

1. **Нет jitter.** 30 точек в каждой группе стоят строго по одной вертикали и сливаются; видно не распределение, а плотность наложения.
2. **Радужные точки по группам.** Цвет дублирует ось X, CVD.
3. **Обрезанная ось Y (30–70).** Низкие и высокие значения скрыты.
4. **Крупные непрозрачные точки, тяжёлая сетка, полная рамка.**
5. **Заголовок «Боль по группам».** Не сообщает, какая схема лучше.
6. **Нет медиан, единиц и n.**

5.13.4 Правильный график

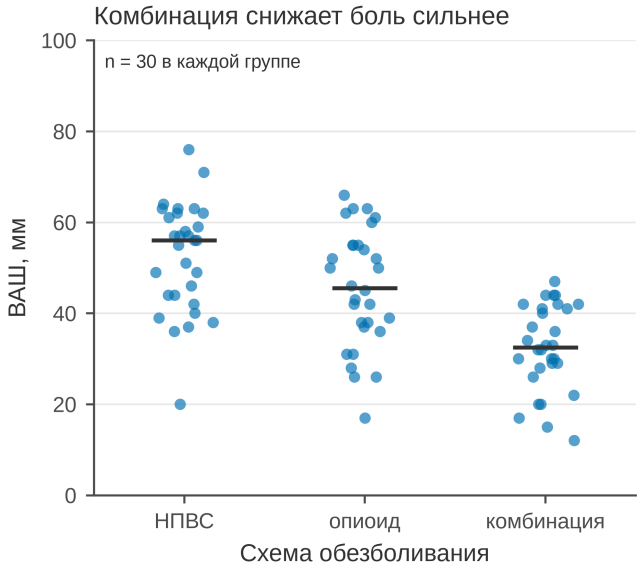


Рисунок 5.13.2. Правильный strip: jitter, один цвет, полный диапазон 0–100, медианы, n = 30.

Код: `05_charts/13_strip/scripts/good_plot.py`.

Что изменилось:

1. **Jitter по X.** Все 30 точек видны отдельно; jitter воспроизводимый.
2. **Один цвет, alpha = 0,65.** Нет двойного кодирования.
3. **Полный диапазон 0–100 мм.**
4. **Медиана — горизонтальная линия** в каждой группе.
5. **n = 30, единицы, заголовок-вывод.**
6. **Лёгкая сетка только по Y, 85 мм.**

Strip особенно хорош, когда *n* небольшое: читатель видит каждого пациента, а не только сводку. При больших *n* точки всё равно сливаются — тогда нужны box/violin/hexbin.

5.13.5 Было и стало

Что было	Почему плохо	Что сделано
нет jitter	точки сливаются	jitter 0,09
радуга	CVD, двойное кодирование	один цвет
ось Y 30–70	хвосты скрыты	0–100
крупные точки	шум	s=18, alpha=0,65
нет медиан	нет сводки	горизонтальные медианы
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
«Боль по группам»	нет сообщения	заголовок-вывод
нет единиц и n	невозможно прочитать	«мм», n = 30

5.13.6 Чек-лист strip

- ❒ Jitter есть и он воспроизводимый.
- ❒ Jitter не переносит точки в чужую группу.
- ❒ Ось Y не обрезает хвосты.
- ❒ Указаны единицы и *n*.
- ❒ Медиана/среднее показаны, если нужна сводка.
- ❒ Цвет не дублирует ось X.
- ❒ Точек не слишком много; иначе box/hexbin.
- ❒ Дискретные значения не «размазаны» jitter’ом.
- ❒ Группы одинаково читаемы.
- ❒ Рисунок проверен в конечном размере 85 мм.

5.13.7 Как воспроизвести

```
uv sync
uv run python 05_charts/13_strip/scripts/make_data.py
uv run python 05_charts/13_strip/scripts/bad_plot.py
uv run python 05_charts/13_strip/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
for position, group in enumerate(GROUPS, start=1):
    values = df.loc[df["analgesia"] == group, "vas_mm"].to_numpy()
    jitter = rng.normal(0.0, 0.09, len(values))
    ax.scatter(position + jitter, values,
               s=18, color=PRIMARY_BLUE, alpha=0.65, linewidths=0)
    ax.hlines(np.median(values), position - 0.18, position + 0.18,
              color="#333333", linewidth=1.5)
```

`rng.normal(0, 0.09)` — компромисс: достаточно, чтобы развести точки, но не настолько, чтобы смешать группы. Seed фиксируйте в подписи или коде.

5.13.8 Что дальше

Следующий график — **5.14 dot + CI**: точка оценки и доверительный интервал. У него будет свой генератор и свой `data/14_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Weissgerber T. L. et al. Beyond Bar and Line Graphs // *PLoS Biology*. 2015. — doi:10.1371/journal.pbio.1002128
- Matplotlib: `scatter` — matplotlib.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com

5.14 Dot + CI: оценка и неопределённость

Dot + CI — это точка оценки и доверительный интервал вокруг неё. Такой график отвечает не только «сколько», но и «насколько точно». Он честнее столбца с одной высотой: видно, что оценка — случайная величина с неопределённостью.

Разница принципиальная: **CI — это не SD**. Стандартное отклонение описывает разброс наблюдений, доверительный интервал — точность оценки. Для доли используют интервал Уилсона, для среднего — t-интервал или бутстрэп.

5.14.1 Сфера применения

Dot + CI отвечает на вопрос: **какова оценка в каждой группе и насколько она точна?**

Где это пригодится:

- доля осложнений/смертности по центрам;
- средние показатели по больницам с 95 % CI;
- результаты подгрупп перед forest plot (5.16);
- сравнение центров без ложного ранжирования;
- оценка эффекта с неопределённостью.

Когда dot + CI **не** нужен:

- нужны все наблюдения — strip/box;
- нужна одна сводка без сравнения — таблица;
- групп много и CI пересекаются — forest plot;
- оценка без интервала — описательная статистика;
- время до события — Kaplan–Meier.

5.14.2 Данные

Сценарий: 600 пациентов, шесть центров, по 100 человек. Исход — осложнение (1/0). Для каждой доли — 95 % ДИ Уилсона.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`center`	центр	—
`complication`	осложнение (1/0)	—

Доли: 9–19 %; общая — 14,7 %. Интервалы широкие (≈ 5–28 %) из-за $n = 100$ на центр.

Генератор: `05_charts/14_dot_ci/scripts/make_data.py → data/14_dataset.csv`.

5.14.3 Плохой график

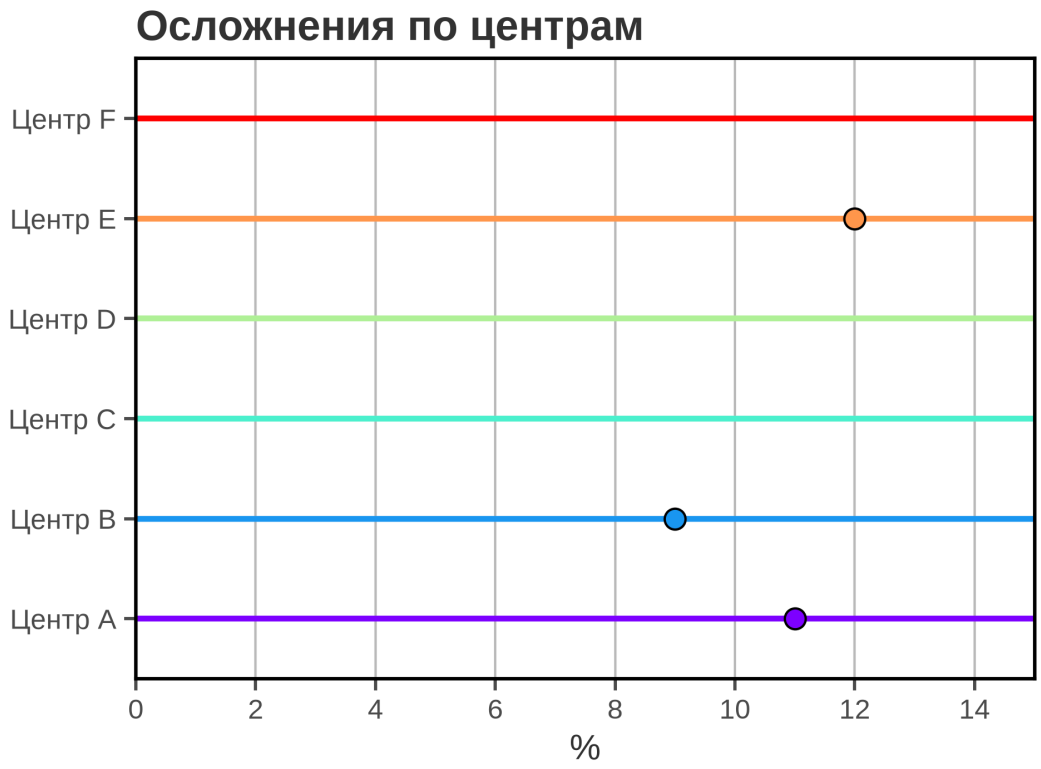


Рисунок 5.14.1. Плохой dot + CI: вместо ДИ — SD, обрезанная ось 0–15, радуга, нет общей доли и n.

Код: `05_charts/14_dot_ci/scripts/bad_plot.py`.

Что не так:

1. **Вместо 95 % ДИ — стандартное отклонение.** Усы выглядят как «интервалы», но это разброс бинарного исхода; их нельзя интерпретировать как точность оценки.
2. **Ось X обрезана до 0–15 %.** Центры с долей 18–19 % вообще не видны.
3. **Радужные точки и усы.** CVD, декор.
4. **Тяжёлая сетка, полная рамка.**
5. **Нет общей доли, n и единиц.**
6. **Заголовок «Осложнения по центрам».** Не сообщает, что показана неопределённость.

5.14.4 Правильный график

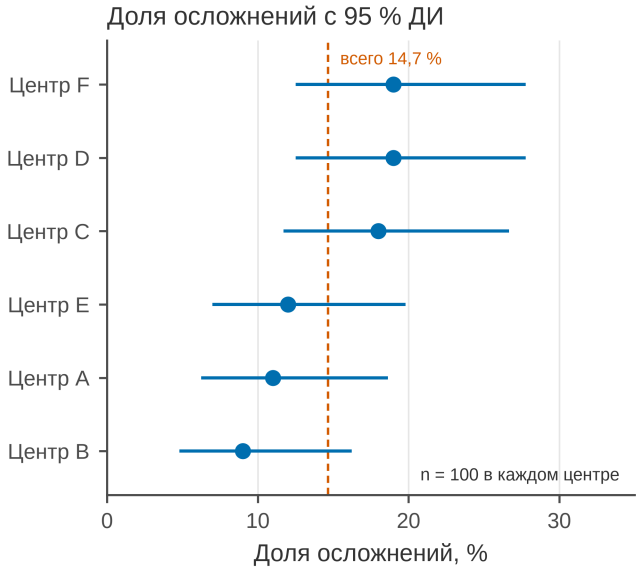


Рисунок 5.14.2. Правильный dot + CI: 95 % ДИ Уилсона, полный диапазон 0–35, общая доля, n = 100.

Код: `05_charts/14_dot_ci/scripts/good_plot.py`.

Что изменилось:

1. **95 % ДИ Уилсона** для каждой доли — корректная неопределённость.
2. **Полный диапазон 0–35 %.** Все центры видны.
3. **Один цвет.** Точки и усы не конкурируют.
4. **Общая доля 14,7 %** — пунктирная линия и подпись.
5. **Сортировка по доле.** Легче читать, но CI показывают, что различия неочевидны.
6. **n = 100 в каждом центре, единицы, 85 мм.**

Пересекающиеся интервалы — важный сигнал: нельзя ранжировать центры без формального сравнения. Dot + CI как раз показывает, что разница между 9 % и 19 % при $n = 100$ может быть случайной.

5.14.5 Было и стало

Что было	Почему плохо	Что сделано
SD вместо CI	не та неопределённость	95 % ДИ Уилсона
ось X 0–15	центры 18–19 % скрыты	0–35
радуга	CVD, декор	один цвет
нет общей доли	нет ориентира	линия 14,7 %
нет сортировки	трудно читать	сортировка по доле
сетка по X и Y	шум	сетка только по X
полная рамка	визуальный шум	только левая и нижняя оси
«Осложнения по центрам»	нет сообщения	заголовок о 95 % ДИ
нет n	непонятна точность	n = 100

5.14.6 Чек-лист dot + CI

- ☐ Указано, что такое интервал: 95 % CI, а не SD/SE.
- ☐ Для долей — Уилсон/точный интервал, не нормальное приближение.
- ☐ Ось не обрезает интервалы.
- ☐ Указаны единицы и *n* по группам.
- ☐ Есть общая оценка/референс, если она нужна.
- ☐ Сортировка не выдаёт себя за статистический вывод.
- ☐ Пересечение CI не игнорируется.
- ☐ Цвет не дублирует ось.
- ☐ Сетка не конкурирует с усами.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.14.7 Как воспроизвести

```
uv sync
uv run python 05_charts/14_dot_ci/scripts/make_data.py
uv run python 05_charts/14_dot_ci/scripts/bad_plot.py
uv run python 05_charts/14_dot_ci/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
def wilson_ci(successes, n, z=1.96):
    p = successes / n
    denominator = 1 + z**2 / n
    center = (p + z**2 / (2 * n)) / denominator
    half = z * np.sqrt(p * (1 - p) / n + z**2 / (4 * n**2)) / denominator
    return (center - half) * 100, (center + half) * 100

low, high = wilson_ci(successes, n)
ax.errorbar(p, y, xerr=[[p - low], [high - p]], fmt="o", color=PRIMARY_BLUE)
```

Для средних используйте t-интервал или бутстрэп; для долей Уилсон устойчивее нормального приближения при малых *n* и крайних долях.

5.14.8 Что дальше

Следующий график — **5.15 bar**: столбцы для сравнения величин и их типичные ошибки. У него будет свой генератор и свой `data/15_dataset.csv`.

Источники

- Wilson E. B. Probable Inference, the Law of Succession, and Statistical Inference // *JASA*. 1927. — doi:10.1080/01621459.1927.10502953
- Cumming G. *Understanding the New Statistics*. Routledge, 2012. — routledge.com
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `errorbar` — matplotlib.org

5.15 Bar: сравнение величин

Bar chart сравнивает **агрегированные величины** по категориям: число госпитализаций, затраты, койко-дни, объём выборки. Это не график распределения и не график тренда: для распределения — histogram/box, для времени — line.

Главное: **нулевая база обязательна**. Длина столбца должна быть пропорциональна величине; обрезанная ось превращает небольшую разницу в «драму».

5.15.1 Сфера применения

Bar отвечает на вопрос: **какая категория больше и насколько?**

Где это пригодится:

- число пациентов/госпитализаций по отделениям;
- затраты или койко-дни по программам;
- объём выборки по центрам;
- сравнение немногих категорий (2–8);
- рейтинг, где важна абсолютная величина.

Когда bar **не** нужен:

- нужно распределение — histogram/box/violin;
- динамика во времени — line;
- много категорий — horizontal bar или dot;
- важна точность оценки — dot + CI;
- части целого — stacked bar 100 % (5.21), не pie (5.22).

5.15.2 Данные

Сценарий: пять отделений больницы, число госпитализаций за год. Величины — счётчики, поэтому столбцы уместны.

Колонка	Смысл	Единица
`department`	отделение	—
`admissions`	госпитализаций за год	число

Терапия — 1 240, хирургия — 980, кардиология — 860, урология — 540, офтальмология — 420. Всего 4 040.

Генератор: `05_charts/15_bar/scripts/make_data.py → data/15_dataset.csv`.

5.15.3 Плохой график

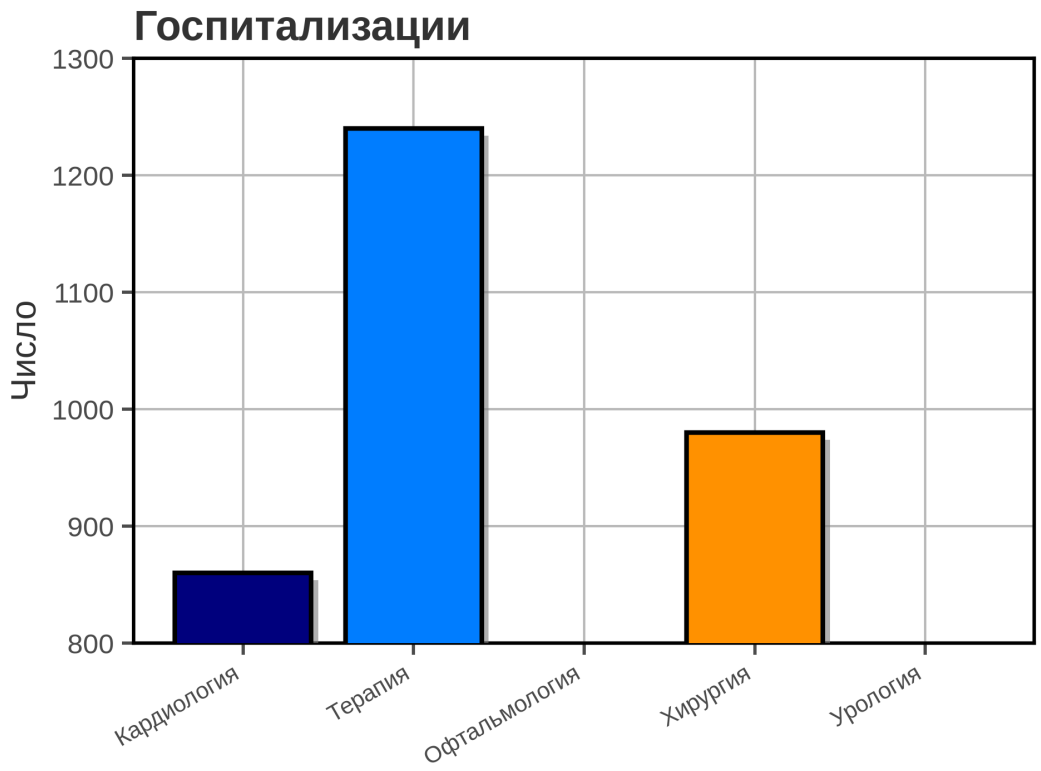


Рисунок 5.15.1. Плохой bar: обрезанная ось 800–1300, радужные столбцы, тени, нет значений.

Код: `05_charts/15_bar/scripts/bad_plot.py`.

Что не так:

1. **Ось Y обрезана до 800–1300.** Нулевая база скрыта; столбцы 540 и 420 вообще не видны, а разница между 980 и 1 240 преувеличена.
2. **Радужные столбцы, чёрные края и тени.** Декор, CVD.
3. **Отделения не отсортированы.** Глаз не может ранжировать.
4. **Тяжёлая сетка по обеим осям и полная рамка.**
5. **Заголовок «Госпитализации».** Не сообщает вывод.
6. **Нет значений на столбцах, единиц и n.**

5.15.4 Правильный график

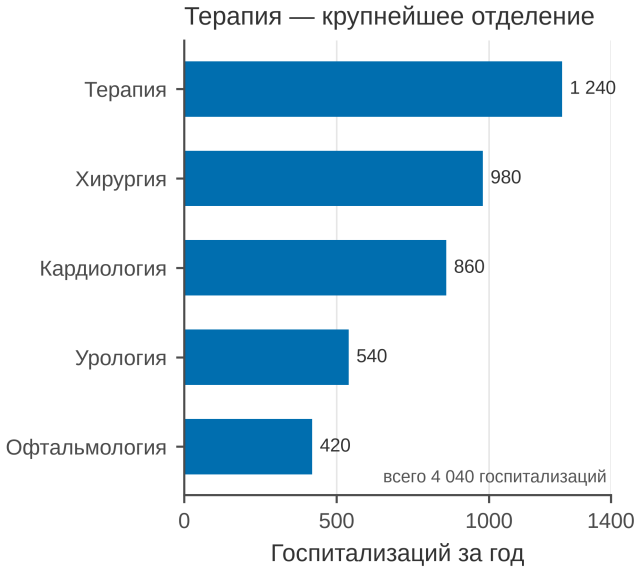


Рисунок 5.15.2. Правильный bar: горизонтальные сортированные столбцы, нулевая база, значения, всего 4 040.

Код: `05_charts/15_bar/scripts/good_plot.py`.

Что изменилось:

1. **Горизонтальные столбцы.** Длинные названия отделений читаются без поворота.
2. **Сортировка по величине.** Ранжирование очевидно.
3. **Нулевая база.** Ось X начинается с 0; длины пропорциональны.
4. **Значения подписаны** прямо у столбцов.
5. **Один цвет, без теней и 3D.**
6. **Единицы, общее число 4 040, заголовок-вывод, 85 мм.**

Bar — не «универсальный график». Если нужна форма распределения, он её не покажет; если нужна динамика, он её скроет.

5.15.5 Было и стало

Что было	Почему плохо	Что сделано
ось Y 800–1300	нулевая база скрыта	0–1400
радуга, тени	декор	один цвет
без сортировки	трудно ранжировать	сортировка
вертикально	подписи наклонены	горизонтально
нет значений	нужно оценивать на глаз	подписи значений
сетка по X и Y	шум	сетка только по X
полная рамка	визуальный шум	только левая и нижняя оси
«Госпитализации»	нет вывода	заголовок-вывод
нет единиц и n	непонятно, что считать	«за год», всего 4 040

5.15.6 Чек-лист bar

- ☐ Ось начинается с нуля.
- ☐ Длина пропорциональна величине.
- ☐ Категории отсортированы, если нет смыслового порядка.
- ☐ Значения подписаны или легко читаются.
- ☐ Указаны единицы и период.
- ☐ Один цвет, без 3D/теней.
- ☐ Длинные подписи — горизонтально.
- ☐ Для распределения выбран histogram/box.
- ☐ Для тренда выбрана line.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.15.7 Как воспроизвести

```
uv sync
uv run python 05_charts/15_bar/scripts/make_data.py
uv run python 05_charts/15_bar/scripts/bad_plot.py
uv run python 05_charts/15_bar/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
df = pd.read_csv(...).sort_values("admissions")
bars = ax.barh(df["department"], df["admissions"], color=PRIMARY_BLUE)
for bar in bars:
    value = int(bar.get_width())
    ax.text(value + 25, bar.get_y() + bar.get_height() / 2, f"{value:,}")
ax.set_xlim(0, 1400)
```

`barh` удобнее для длинных названий; сортировка и подписи значений делают график самодостаточным.

5.15.8 Что дальше

Следующий график — **5.16 forest**: оценки эффектов и доверительные интервалы для многих подгрупп. У него будет свой генератор и свой `data/16_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com
- Matplotlib: `barh` — matplotlib.org

5.16 Forest: эффект по подгруппам

Forest plot — это ряд оценок эффекта с доверительными интервалами, выстроенных по вертикали. Он показывает не только «где эффект», но и «насколько он точен» в каждой подгруппе.

Для отношений (OR, RR, HR) ось **обязательно логарифмическая**: только тогда OR = 0,5 и OR = 2 симметричны относительно 1. Линия «нет эффекта» — OR = 1 — обязательна.

5.16.1 Сфера применения

Forest отвечает на вопрос: **каков эффект в подгруппах и перекрывают ли интервалы единицу?**

Где это пригодится:

- подгрупповой анализ RCT;
- мета-анализ нескольких исследований;
- OR/RR/HR по центрам и стратам;
- проверка гетерогенности эффекта;
- честное представление неопределённости.

Когда forest **не** нужен:

- одна оценка — dot + CI;
- нужны исходные распределения — box/strip;
- нет общего эффекта и осмысленной группировки;
- подгруппы выбраны post hoc без поправки;
- эффект выражен в абсолютных единицах без лог-шкалы.

5.16.2 Данные

Сценарий: RCT, 800 пациентов (400 лечение / 400 контроль), исход — осложнение. Эффект — OR с 95 % CI Уолфа; восемь подгрупп и общая оценка.

Колонка	Смысл	Единица
`subgroup`	подгруппа	—
`n_treatment`	n в группе лечения	—
`events_treatment`	события в лечении	—
`n_control`	n в контроле	—
`events_control`	события в контроле	—

Общая OR — 0,62 (95 % ДИ 0,46–0,84). В подгруппах — 0,56–0,67; у женщин, ≥65 лет и ХБП интервал пересекает 1.

Генератор: `05_charts/16_forest/scripts/make_data.py → data/16_dataset.csv`.

5.16.3 Плохой график

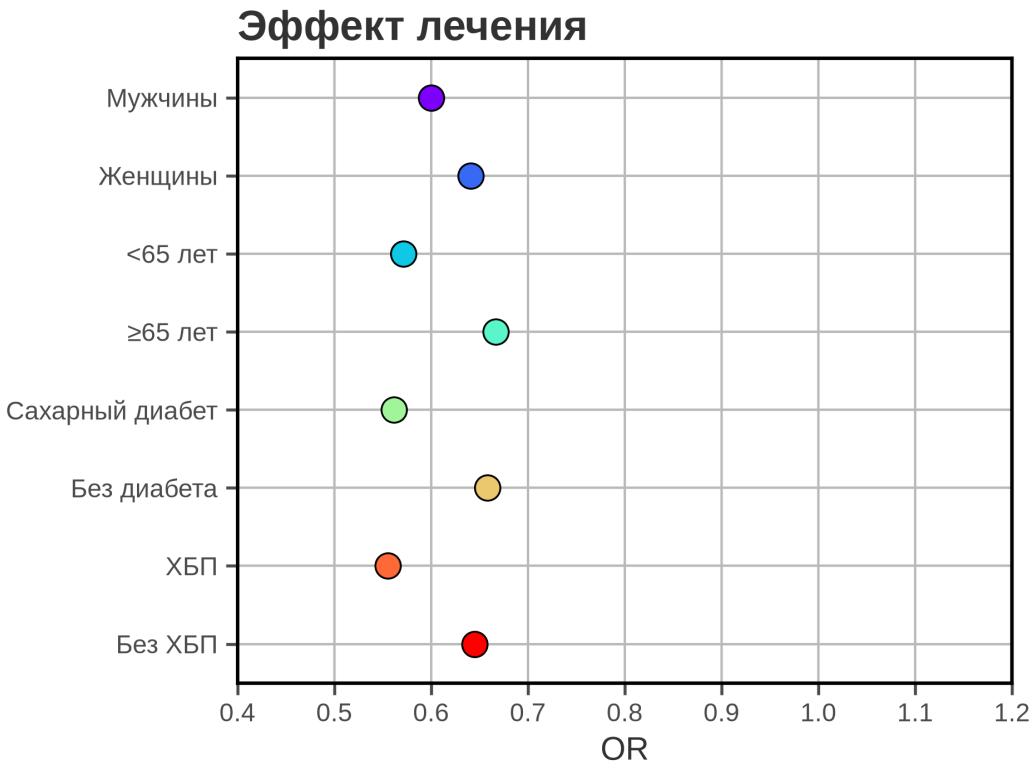


Рисунок 5.16.1. Плохой forest: только точки OR, линейная ось, нет ДИ, линии 1 и общей оценки.

Код: `05_charts/16_forest/scripts/bad_plot.py`.

Что не так:

1. **Нет доверительных интервалов.** Точка без CI не показывает неопределённость; подгруппы выглядят точнее, чем есть.
2. **Линейная ось OR.** Расстояния между 0,5 и 0,7 и между 1,0 и 1,2 не сопоставимы; лог-шкала обязательна.
3. **Нет линии OR = 1.** Непонятно, где «нет эффекта».
4. **Радужные точки, тяжёлая сетка, полная рамка.**
5. **Нет общей оценки и n.**
6. **Заголовок «Эффект лечения».** Не сообщает направление и неопределённость.

5.16.4 Правильный график

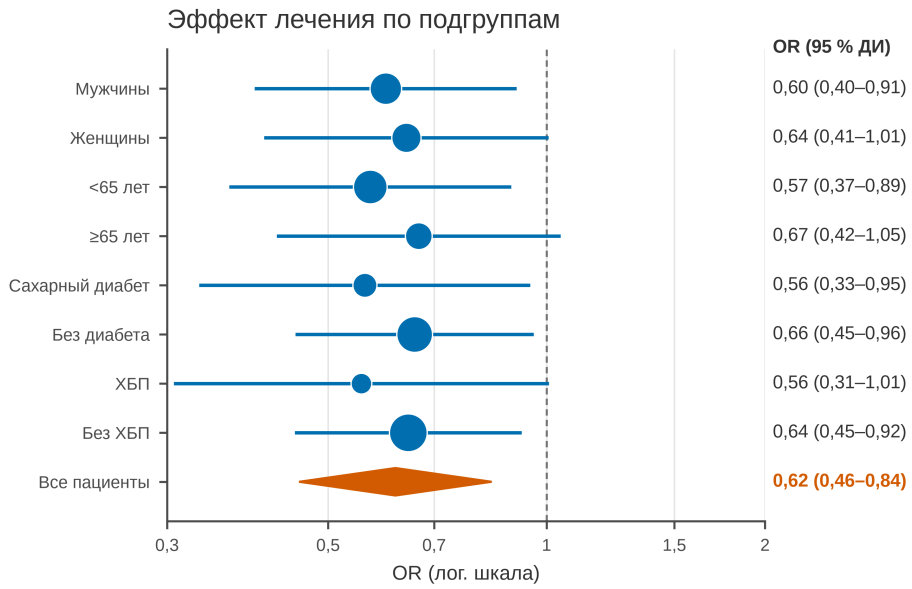


Рисунок 5.16.2. Правильный forest: лог-ось 0,3–2, 95 % ДИ, линия 1, размер по n, ромб общей оценки.

Код: `05_charts/16_forest/scripts/good_plot.py`.

Что изменилось:

1. **95 % ДИ для каждой подгруппы.**
2. **Логарифмическая ось 0,3–2,0.** OR симметричны относительно 1.
3. **Линия OR = 1** — референс «нет эффекта».
4. **Размер квадрата пропорционален n.** Видно, где оценка точнее.
5. **Ромб общей оценки** внизу; OR 0,62 (0,46–0,84).
6. **Справа — OR (95 % ДИ), слева — подгруппы.**
7. **Один цвет, лёгкая сетка, 170 мм.**

CI, пересекающие 1, — не «отсутствие эффекта», а недостаток точности. Не ранжируйте подгруппы по p-value и не делайте причинных выводов из подгруппового анализа.

5.16.5 Было и стало

Что было	Почему плохо	Что сделано
нет CI	не видна неопределённость	95 % ДИ
линейная ось	OR искажены	лог-ось 0,3–2
нет линии 1	нет референса	пунктир OR = 1
радуга	CVD, декор	один цвет
нет размера по n	не видна точность	размер квадрата
нет общей оценки	нет итога	ромб
нет таблицы справа	нужно считать в уме	OR (95 % ДИ)
«Эффект лечения»	нет сообщения	заголовок о подгруппах
нет n	непонятна точность	n в данных

5.16.6 Чек-лист forest

- ☐ Для OR/RR/HR — логарифмическая ось.
- ☐ Есть линия «нет эффекта» (1).
- ☐ Показаны 95 % ДИ, а не только точки.
- ☐ Размер маркера отражает вес/n.
- ☐ Есть общая оценка (ромб/линия).
- ☐ Подгруппы не ранжированы по значимости.

- ❐ Указаны n и единицы эффекта.
- ❐ CI, пересекающие 1, не поданы как «нет эффекта».
- ❐ Подгрупповой анализ не подан как причинный.
- ❐ Рисунок проверен в конечном размере.

5.16.7 Как воспроизвести

```
uv sync
uv run python 05_charts/16_forest/scripts/make_data.py
uv run python 05_charts/16_forest/scripts/bad_plot.py
uv run python 05_charts/16_forest/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
odds = (a / b) / (c / d)
se = np.sqrt(1/a + 1/b + 1/c + 1/d)
low = np.exp(np.log(odds) - 1.96 * se)
high = np.exp(np.log(odds) + 1.96 * se)

ax.set_xscale("log")
ax.axvline(1, color="#777777", linestyle="--")
ax.hlines(y, low, high, color=PRIMARY_BLUE)
ax.scatter(odds, y, s=n_total / 3, color=PRIMARY_BLUE)
```

Для мета-анализа вместо OR Уолфа используйте модель с фиксированным/случайным эффектом и вес исследований.

5.16.8 Что дальше

Следующий график — **5.17 slope**: изменение показателя между двумя точками для многих групп. У него будет свой генератор и свой `data/17_dataset.csv`.

Источники

- Cumming G. *Understanding the New Statistics*. Routledge, 2012. — routledge.com
- Cochrane Handbook, Chapter 10 — training.cochrane.org
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `hlines` — matplotlib.org

5.17 Slope: изменение между двумя точками

Slope plot соединяет две точки — «до» и «после» — для нескольких групп. Наклон линии показывает направление и величину изменения, а набор линий — гетерогенность эффекта.

Это график **парных агрегатов** (средних по группам), а не spaghetti из сотен индивидуальных траекторий: при больших *n* линии сливаются.

5.17.1 Сфера применения

Slope отвечает на вопрос: **как изменился показатель в каждой группе и насколько различается эффект?**

Где это пригодится:

- средние до/после программы по клиникам;
- парные измерения до/после лечения;
- изменение доли/риска по центрам;
- сравнение эффекта вмешательства в 2–8 группах;
- наглядная величина сдвига.

Когда slope **не** нужен:

- больше 8–10 групп — spaghetti/labels;
- нужны индивидуальные траектории — paired dot (5.18);
- нет парных измерений;
- нужно распределение — box/violin;
- динамика многих временных точек — line.

5.17.2 Данные

Сценарий: 360 пациентов, шесть клиник, по 60 человек. Вес до и через 12 недель программы снижения веса.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`clinic`	клиника	—
`weight_before_kg`	вес до	кг
`weight_after_kg`	вес после	кг

Среднее снижение — 6,7 кг. Клиника C — лидер: −11,2 кг; клиника D — минимум: −3,8 кг. Все клиники снизили вес.

Генератор: `05_charts/17_slope/scripts/make_data.py → data/17_dataset.csv`.

5.17.3 Плохой график

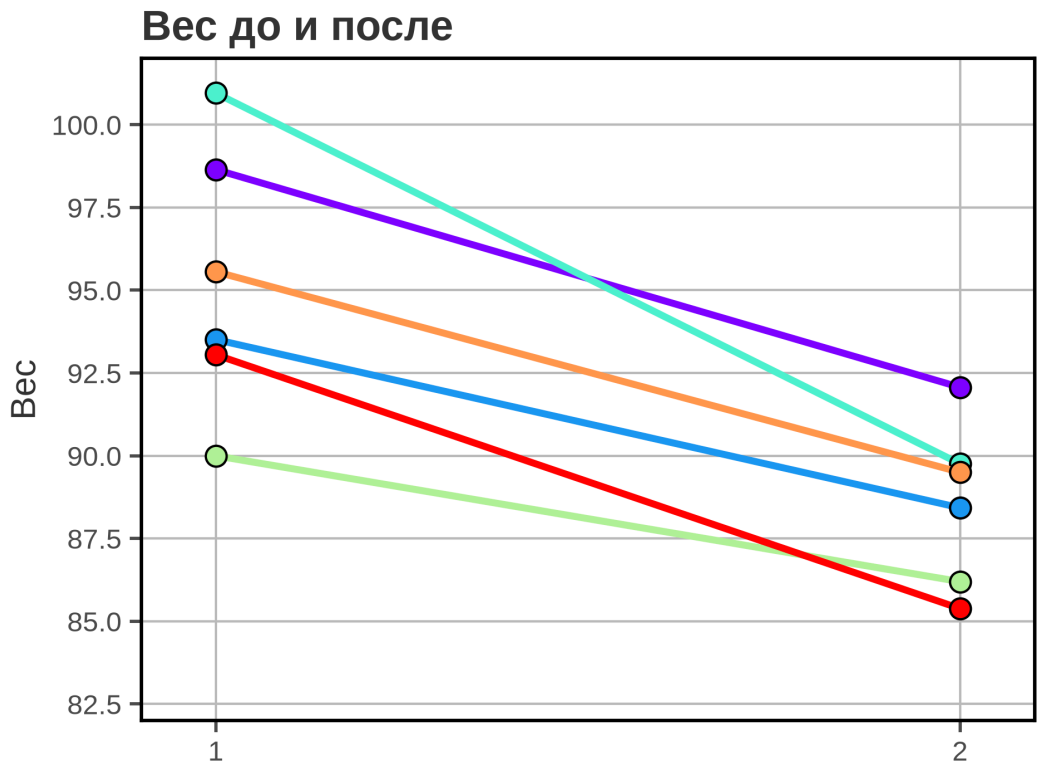


Рисунок 5.17.1. Плохой slope: шесть радужных линий без подписей, обрезанная ось 82–102, нет единиц и n.

Код: `05_charts/17_slope/scripts/bad_plot.py`.

Что не так:

1. **Шесть радужных линий без подписей.** Непонятно, где какая клиника; цвет не помогает.
2. **Ось Y обрезана до 82–102.** Нижние значения и размах скрыты.
3. **Нет подписей «до/после» и единиц.**
4. **Тяжёлая сетка, полная рамка.**
5. **Заголовок «Вес до и после».** Не сообщает вывод.
6. **Нет n и изменения в кг.**

5.17.4 Правильный график



Рисунок 5.17.2. Правильный slope: серые линии, акцент на лидере, подписи до/после, 120 мм.

Код: `05_charts/17_slope/scripts/good_plot.py`.

Что изменилось:

1. **Серые линии для всех, акцент на лидере.** Видно общий паттерн и лучший результат.
2. **Прямые подписи слева и справа** с лидер-линиями: клиника и вес до, вес после и изменение.
3. **Полный диапазон 80–106 кг.**
4. **Подписи «До/После», единицы, n = 60.**
5. **Заголовок-вывод:** все клиники снизили вес в среднем на 6,7 кг.
6. **120 мм** — хватает места для подписей.

Slope показывает **направление и величину** изменения, но не статистическую значимость. Для вывода о различии эффектов нужны парный тест/модель и доверительные интервалы.

5.17.5 Было и стало

Что было	Почему плохо	Что сделано
радуга без подписей	непонятно, где кто	серые + акцент + подписи
ось Y 82–102	размах скрыт	80–106
нет «до/после»	непонятны точки	подписи осей
нет единиц	«вес» без кг	«Вес, кг»
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
«Вес до и после»	нет вывода	заголовок-вывод
нет n	непонятна точность	n = 60

5.17.6 Чек-лист slope

- ☐ Понятно, что слева «до», справа «после».
- ☐ Каждая линия подписана.

- ☐ Ось не обрезает диапазон.
- ☐ Указаны единицы и *n*.
- ☐ Линия не тонет в радуге.
- ☐ Акцент оправдан (лидер/аутлайер).
- ☐ Для многих групп — не spaghetti.
- ☐ Для индивидуальных пар — paired dot.
- ☐ Нет ложной причинности.
- ☐ Рисунок проверен в конечном размере.

5.17.7 Как воспроизвести

```
uv sync
uv run python 05_charts/17_slope/scripts/make_data.py
uv run python 05_charts/17_slope/scripts/bad_plot.py
uv run python 05_charts/17_slope/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
ax.plot([0, 1], [before, after], color="#8A8A8A", marker="o")
ax.annotate(f"{clinic}  {before:.0f}", xy=(0, before), xytext=(-0.1, label_y),
            ha="right", arrowprops={"arrowstyle": "-", "color": "#BBBBBB"})
ax.set_xticks([0, 1])
ax.set_xticklabels(["До", "После"])
```

Подписи можно развести по вертикали, сохраняя порядок; стрелки свяжут подпись с точкой.

5.17.8 Что дальше

Следующий график — **5.18 paired dot**: индивидуальные парные изменения. У него будет свой генератор и свой `data/18_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com
- Matplotlib: `annotate` — matplotlib.org

5.18 Paired dot: индивидуальные изменения

Paired dot — это линии, соединяющие два измерения одного и того же пациента: «до» и «после». В отличие от slope plot средних, здесь виден **каждый** пациент: у кого эффект большой, у кого маленький, у кого его нет.

График хорош для 10–50 пар. При сотнях пар линии превращаются в spaghetti; тогда нужны средние/CI или распределение разниц.

5.18.1 Сфера применения

Paired dot отвечает на вопрос: **как изменился показатель у каждого пациента и каково среднее изменение?**

Где это пригодится:

- до/после лечения в одной группе;
- парные измерения биомаркера;
- индивидуальный ответ на вмешательство;
- проверка, не скрывает ли среднее разнородность;
- малые выборки, где важна каждая пара.

Когда paired dot **не** нужен:

- наблюдения непарные;
- пар слишком много (spaghetti);
- нужны только средние — slope/dot+CI;
- нужна динамика многих точек — line;
- нужно распределение — histogram/ECDF.

5.18.2 Данные

Сценарий: 30 пациентов, гемоглобин (г/л) до и через 8 недель лечения анемии. Парные наблюдения.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`hemoglobin_before`	гемоглобин до	г/л
`hemoglobin_after`	гемоглобин после	г/л

Среднее до — 93,4 г/л; после — 105,8 г/л; средний прирост — +12,4 ± 5,8 г/л. У всех 30 пациентов уровень вырос, но величина прироста различается.

Генератор: `05_charts/18_paired_dot/scripts/make_data.py → data/18_dataset.csv`.

5.18.3 Плохой график

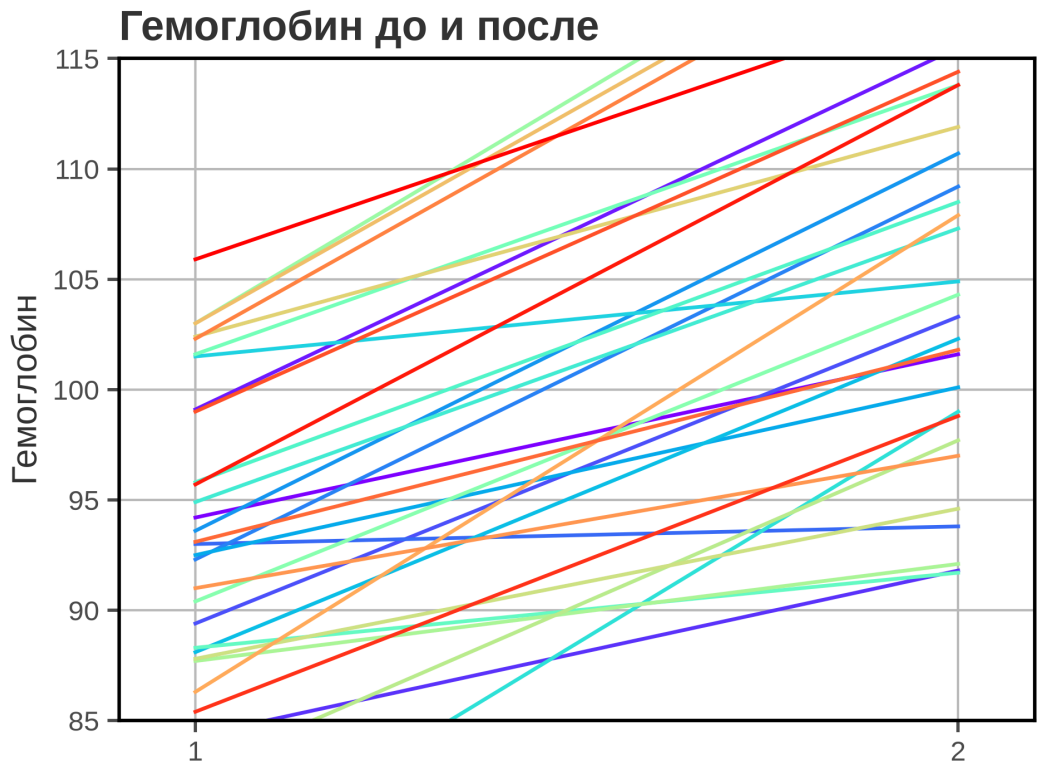


Рисунок 5.18.1. Плохой paired dot: 30 радужных линий без точек и средних, обрезанная ось 85–115.

Код: `05_charts/18_paired_dot/scripts/bad_plot.py`.

Что не так:

1. **30 радужных линий.** Непонятно, где отдельный пациент; цвет не помогает.
2. **Нет точек и средних.** Видны только линии, нет сводки.
3. **Ось Y обрезана до 85–115.** Высокие значения после лечения скрыты.
4. **Тяжёлая сетка, полная рамка.**
5. **Заголовок «Гемоглобин до и после».** Не сообщает вывод.
6. **Нет единиц, n и среднего изменения.**

5.18.4 Правильный график

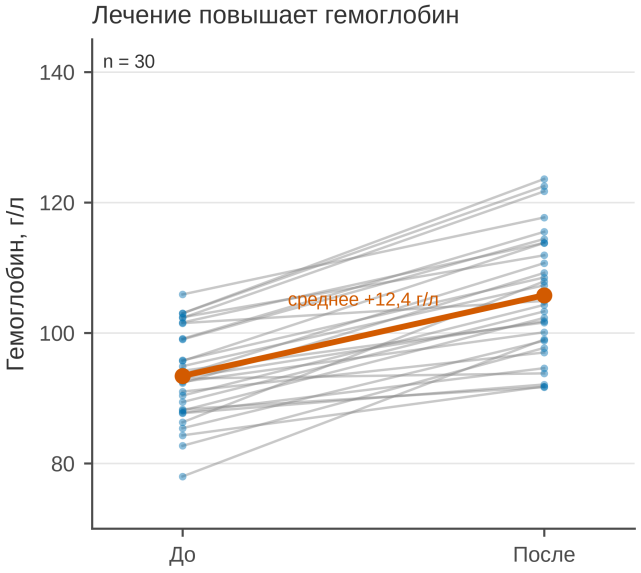


Рисунок 5.18.2. Правильный paired dot: серые линии, точки, среднее изменение +12,4 г/л, полный диапазон.

Код: `05_charts/18_paired_dot/scripts/good_plot.py`.

Что изменилось:

1. **Серые линии для отдельных пациентов.** Видна каждая пара, но линии не конкурируют.
2. **Точки на обоих концах.** Видны исходные значения и выбросы.
3. **Среднее до/после — цветная толстая линия.** Подписано «среднее +12,4 г/л».
4. **Полный диапазон 70–145 г/л.**
5. **«До/После», единицы, n = 30.**
6. **Заголовок-вывод, 85 мм.**

Paired dot показывает индивидуальную изменчивость, но не статистическую значимость. Для вывода нужен парный тест (Уилкоксон/парный t) и CI среднего изменения.

5.18.5 Было и стало

Что было	Почему плохо	Что сделано
радужные линии	непонятно, где кто	серые линии
нет точек	не видно значений	точки на концах
нет средних	нет сводки	цветная средняя линия
ось Y 85–115	высокие значения скрыты	70–145
нет единиц	«гемоглобин» без г/л	«г/л»
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
«Гемоглобин до и после»	нет вывода	заголовок-вывод
нет n	непонятна точность	n = 30

5.18.6 Чек-лист paired dot

- ☐ Пары действительно парные.
- ☐ Видны отдельные линии и точки.
- ☐ Среднее/медиана изменения показаны.
- ☐ Ось не обрезает диапазон.
- ☐ Указаны единицы и *n*.
- ☐ Линии не радужные без смысла.
- ☐ При больших *n* — не spaghetti.
- ☐ Есть парный тест/CI для вывода.
- ☐ Выбросы не скрыты.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.18.7 Как воспроизвести

```
uv sync
uv run python 05_charts/18_paired_dot/scripts/make_data.py
uv run python 05_charts/18_paired_dot/scripts/bad_plot.py
uv run python 05_charts/18_paired_dot/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
for b, a in zip(before, after, strict=True):
    ax.plot([0, 1], [b, a], color="#8A8A8A", alpha=0.45, linewidth=0.9)
    ax.scatter([0, 1], [b, a], s=8, color=PRIMARY_BLUE, alpha=0.45)

ax.plot([0, 1], [before.mean(), after.mean()],
        color=ACCENT_VERMILLION, linewidth=2.2, marker="o")
```

Для формального вывода используйте `scipy.stats.wilcoxon` или `ttest_rel` и CI среднего изменения.

5.18.8 Что дальше

Следующий график — **5.19 Bland–Altman**: согласие двух методов измерения. У него будет свой генератор и свой `data/19_dataset.csv`.

Источники

- Bland J. M., Altman D. G. Statistical Methods for Assessing Agreement between Two Methods of Clinical Measurement // *The Lancet*. 1986. — doi:10.1016/S0140-6736(86)90837-8
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `plot/scatter` — matplotlib.org

5.19 Bland–Altman: согласие методов

Bland–Altman отвечает на вопрос: **насколько два метода измерения согласуются?** По X — среднее двух методов, по Y — разница между ними. Горизонтальная линия — средняя разница (bias), пунктиры — пределы согласия (limits of agreement, LoA = bias ± 1,96 SD).

Это не корреляция. Два метода могут коррелировать почти идеально и при этом систематически расходиться; BA показывает именно согласие.

5.19.1 Сфера применения

BA отвечает на вопрос: **где проходит средняя разница и в каких пределах лежат 95 % расхождений?**

Где это пригодится:

- новый анализатор против референсного;
- глюкометр против лаборатории;
- два способа измерения АД;
- согласие шкал/опросников;
- проверка, можно ли заменить метод.

Когда BA **не** нужен:

- сравниваются не методы, а группы;
- нужна корреляция/ассоциация — scatter (5.05);
- нужны только средние — dot + CI (5.14);
- разница зависит от масштаба — нужна лог-трансформация;
- нет парных измерений.

5.19.2 Данные

Сценарий: 150 пациентов, глюкоза измерена лабораторным анализатором и глюкометром. Разница = лаборатория − глюкометр.

Колонка	Смысл	Единица
`patient_id`	идентификатор пациента	—
`lab_glucose`	лабораторный анализ	ммоль/л
`meter_glucose`	глюкометр	ммоль/л

Bias = +0,35 ммоль/л; 95 % LoA = −1,09…+1,79 ммоль/л. Глюкометр систематически занижает, но расхождения укладываются в широкие пределы.

Генератор: `05_charts/19_bland_altman/scripts/make_data.py → data/19_dataset.csv`.

5.19.3 Плохой график

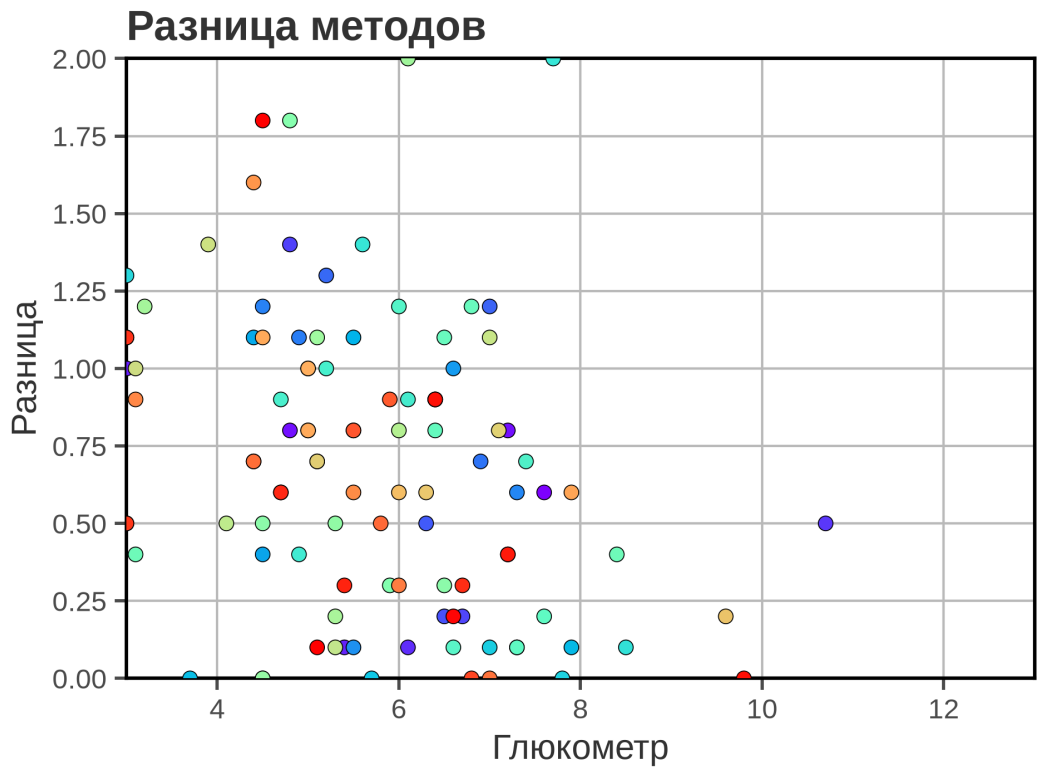


Рисунок 5.19.1. Плохой BA: по X один метод, нет bias и LoA, обрезанная ось 0–2, радуга.

Код: `05_charts/19_bland_altman/scripts/bad_plot.py`.

Что не так:

1. **По X — один метод, а не среднее двух.** Это не BA; разница может коррелировать с масштабом.
2. **Нет bias и LoA.** Непонятно, где средняя разница и границы согласия.
3. **Ось Y обрезана до 0–2.** Отрицательные разницы скрыты; картина согласия искажена.
4. **Радужные точки, тяжёлая сетка, полная рамка.**
5. **Заголовок «Разница методов».** Не сообщает вывод.
6. **Нет единиц и n.**

5.19.4 Правильный график



Рисунок 5.19.2. Правильный BA: среднее по X, разница по Y, bias +0,35 и LoA −1,09…+1,79.

Код: `05_charts/19_bland_altman/scripts/good_plot.py`.

Что изменилось:

1. **По X — среднее двух методов.**
2. **Bias +0,35 и 95 % LoA −1,09…+1,79** — линии и подписи.
3. **Полный диапазон разниц −2,5…+2,5.**
4. **Один цвет, полупрозрачные точки.**
5. **Единицы, n = 150, заголовок-вывод.**
6. **Лёгкая сетка по Y, 85 мм.**

Bias — систематическое расхождение; LoA — случайное. Если LoA шире клинически допустимого, методы нельзя считать взаимозаменяемыми, даже если bias мал.

5.19.5 Было и стало

Что было	Почему плохо	Что сделано
по X один метод	это не BA	среднее двух методов
нет bias/LoA	нет согласия	линии и подписи
ось Y 0–2	отрицательные скрыты	−2,5…+2,5
радуга	CVD, декор	один цвет
сетка по X и Y	шум	сетка только по Y
полная рамка	визуальный шум	только левая и нижняя оси
«Разница методов»	нет вывода	заголовок с bias
нет единиц и n	невозможно прочитать	единицы, n = 150

5.19.6 Чек-лист Bland–Altman

- ☐ По X — среднее двух методов.
- ☐ По Y — разница (указано, какой метод из какого вычитается).
- ☐ Показаны bias и 95 % LoA.
- ☐ Ось Y не обрезает разницы.
- ☐ Указаны единицы и *n*.
- ☐ Проверена пропорциональная bias (наклон).
- ☐ LoA сопоставлены с клинически допустимыми.
- ☐ Нет корреляции вместо согласия.
- ☐ Выбросы видны.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.19.7 Как воспроизвести

```
uv sync
uv run python 05_charts/19_bland_altman/scripts/make_data.py
uv run python 05_charts/19_bland_altman/scripts/bad_plot.py
uv run python 05_charts/19_bland_altman/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
mean = (lab + meter) / 2
diff = lab - meter
bias = diff.mean()
loa = 1.96 * diff.std(ddof=1)

ax.scatter(mean, diff, s=14, alpha=0.45)
ax.axhline(bias, color="#333333")
ax.axhline(bias - loa, color=ACCENT_VERMILLION, linestyle="--")
ax.axhline(bias + loa, color=ACCENT_VERMILLION, linestyle="--")
```

Если разброс разницы растёт с уровнем, постройте BA для логарифмов или добавьте регрессию разницы на среднее.

5.19.8 Что дальше

Следующий график — **5.20 stacked bar**: состав целого по категориям. У него будет свой генератор и свой `data/20_dataset.csv`.

Источники

- Bland J. M., Altman D. G. Statistical Methods for Assessing Agreement between Two Methods of Clinical Measurement // *The Lancet*. 1986. — doi:10.1016/S0140-6736(86)90837-8
- Giavarina D. Understanding Bland Altman Analysis // *Biochemia Medica*. 2015. — doi:10.11613/BM.2015.015
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz

5.20 Stacked bar: состав целого

Stacked bar показывает не только общий объём, но и вклад категорий. Он отвечает на вопрос: **из чего состоит целое и как состав меняется между группами?**

Главное ограничение: сравнивать сегменты, которые начинаются не от нуля, трудно. Поэтому stacked bar хорош для **2–4 категорий**, а для точного сравнения долей лучше 100 % stacked bar (5.21) или dot.

5.20.1 Сфера применения

Stacked bar отвечает на вопрос: **каков общий объём и из каких частей он состоит?**

Где это пригодится:

- исходы госпитализации (выписан/переведён/умер);
- структура затрат по программам;
- возрастные группы в когорте;
- степени тяжести по отделениям;
- состав выборки по центрам.

Когда stacked bar **не** нужен:

- нужно сравнить отдельные сегменты — grouped bar/dot;
- частей много (5+) — 100 % stacked или таблица;
- важна динамика — line;
- части не складываются в целое;
- нулевая база недоступна.

5.20.2 Данные

Сценарий: четыре отделения, три исхода — выписан, переведён, умер.

Колонка	Смысл	Единица
`department`	отделение	—
`outcome`	исход	—
`patients`	число пациентов	человек

Всего 3 180 пациентов. В ОРИТ переведены 44 % — больше, чем в других отделениях.

Генератор: `05_charts/20_stacked_bar/scripts/make_data.py → data/20_dataset.csv`.

5.20.3 Плохой график

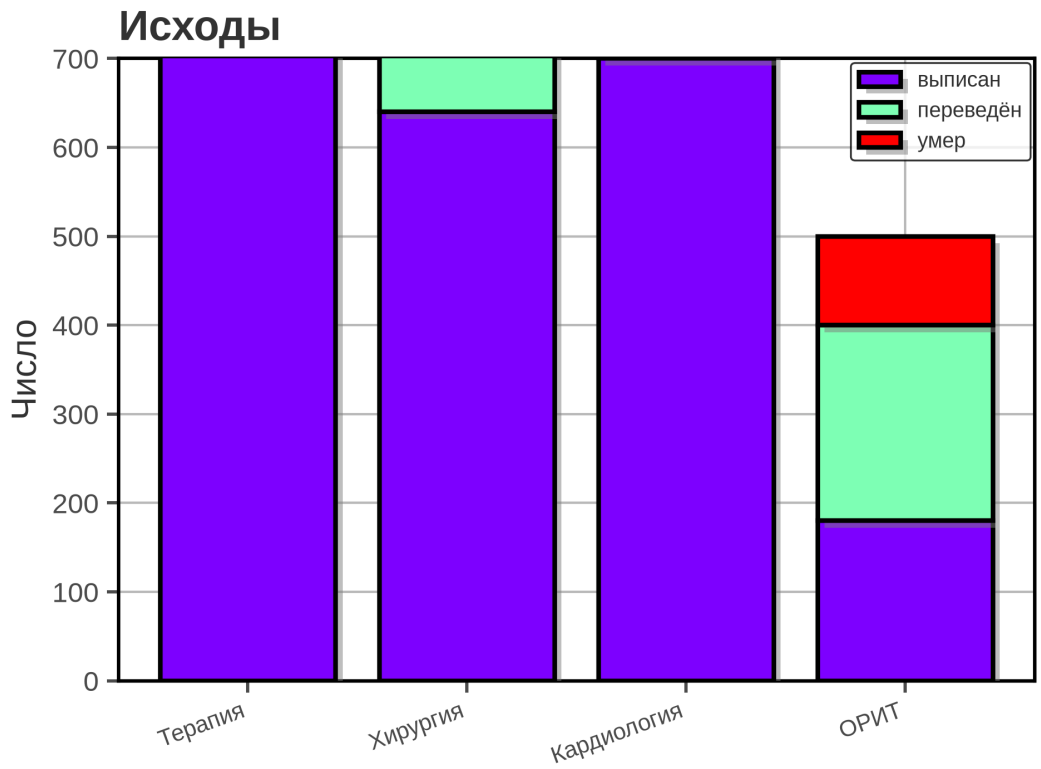


Рисунок 5.20.1. Плохой stacked bar: обрезанная ось 700, радужные сегменты, тени, нет значений.

Код: `05_charts/20_stacked_bar/scripts/bad_plot.py`.

Что не так:

1. **Ось Y обрезана до 700.** Столбцы Терапии и Кардиологии обрезаны; общий объём не виден.
2. **Радужные сегменты, чёрные края, тени, «3D».**
3. **Нет подписей значений и общего числа.**
4. **Тяжёлая сетка, полная рамка.**
5. **Заголовок «Исходы».** Не сообщает вывод.
6. **Нет единиц и n.**

5.20.4 Правильный график

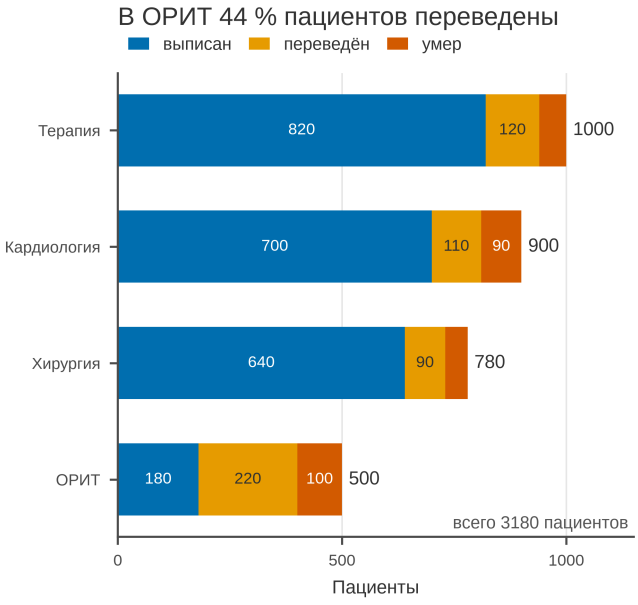


Рисунок 5.20.2. Правильный stacked bar: горизонтальные столбцы, сортировка, палитра Okabe–Ito, значения и итоги.

Код: `05_charts/20_stacked_bar/scripts/good_plot.py`.

Что изменилось:

1. **Горизонтально и отсортировано по общему числу.**
2. **Нулевая база и полный масштаб 0–1150.**
3. **Палитра Okabe–Ito** — различима при CVD.
4. **Значения внутри сегментов, итоги справа.**
5. **Легенда сверху, единицы, всего 3 180.**
6. **Заголовок-вывод:** в ОРИТ 44 % переведены.

Stacked bar показывает состав, но не заменяет сравнение долей: для этого используйте 100 % stacked bar или dot + CI.

5.20.5 Было и стало

Что было	Почему плохо	Что сделано
ось Y 0–700	столбцы обрезаны	0–1150
радуга, тени	декор, CVD	Okabe–Ito
без сортировки	трудно читать	сортировка по итогу
нет значений	оценка на глаз	подписи сегментов
нет итогов	не виден объём	итоги справа
сетка по X и Y	шум	сетка только по X
полная рамка	визуальный шум	только левая и нижняя оси
«Исходы»	нет вывода	заголовок-вывод
нет n	непонятен масштаб	всего 3 180

5.20.6 Чек-лист stacked bar

- ☐ Нулевая база.
- ☐ Частей 2–4, иначе 100 % stacked/таблица.
- ☐ Палитра различима при CVD.
- ☐ Значения/итоги подписаны.
- ☐ Указаны единицы и общее *n*.
- ☐ Порядок сегментов одинаков во всех столбцах.
- ☐ Сортировка по итогу или смыслу.
- ☐ Нет 3D и теней.
- ☐ Для сравнения долей — 100 % stacked/dot.
- ☐ Рисунок проверен в конечном размере 85 мм.

5.20.7 Как воспроизвести

```
uv sync
uv run python 05_charts/20_stacked_bar/scripts/make_data.py
uv run python 05_charts/20_stacked_bar/scripts/bad_plot.py
uv run python 05_charts/20_stacked_bar/scripts/good_plot.py
```

Вот как это выглядит в коде:

```
left = np.zeros(len(pivot))
for outcome, color in zip(OUTCOMES, COLORS, strict=True):
    ax.barh(pivot.index, pivot[outcome], left=left,
            color=color, label=outcome)
    left += pivot[outcome].to_numpy()
```

Сегменты должны идти в одном порядке; иначе сравнение столбцов становится ловушкой.

5.20.8 Что дальше

Следующий график — **5.21 bar 100 %**: нормированный состав. У него будет свой генератор и свой `data/21_dataset.csv`.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com
- Matplotlib: `barh` — matplotlib.org

5.21 Bar 100 %: нормированный состав

100 % stacked bar показывает **доли**, а не абсолютные числа: каждый столбец нормирован к 100 %. Он хорош, когда нужно сравнить состав групп разного размера, но скрывает общий объём — поэтому *n* нужно указывать рядом.

5.21.1 Сфера применения

- сравнение структуры по группам разного размера;
- доли исходов, степеней тяжести, возрастных групп;
- 2–4 категории;
- когда абсолютные итоги не важны.

Не подходит, если нужны абсолютные объёмы, частей больше 5 или разница долей мала.

5.21.2 Данные

4 возрастные группы × 3 степени тяжести, по 200 пациентов.

Колонка	Смысл	Единица
`age_group`	возрастная группа	—
`severity`	лёгкая/средняя/тяжёлая	—
`patients`	число пациентов	человек

Доля тяжёлой степени: 10 % (<40), 20 % (40–59), 35 % (60–74), 57 % (75+).

Генератор: `05_charts/21_bar_100/scripts/make_data.py` → `data/21_dataset.csv`.

5.21.3 Плохой график

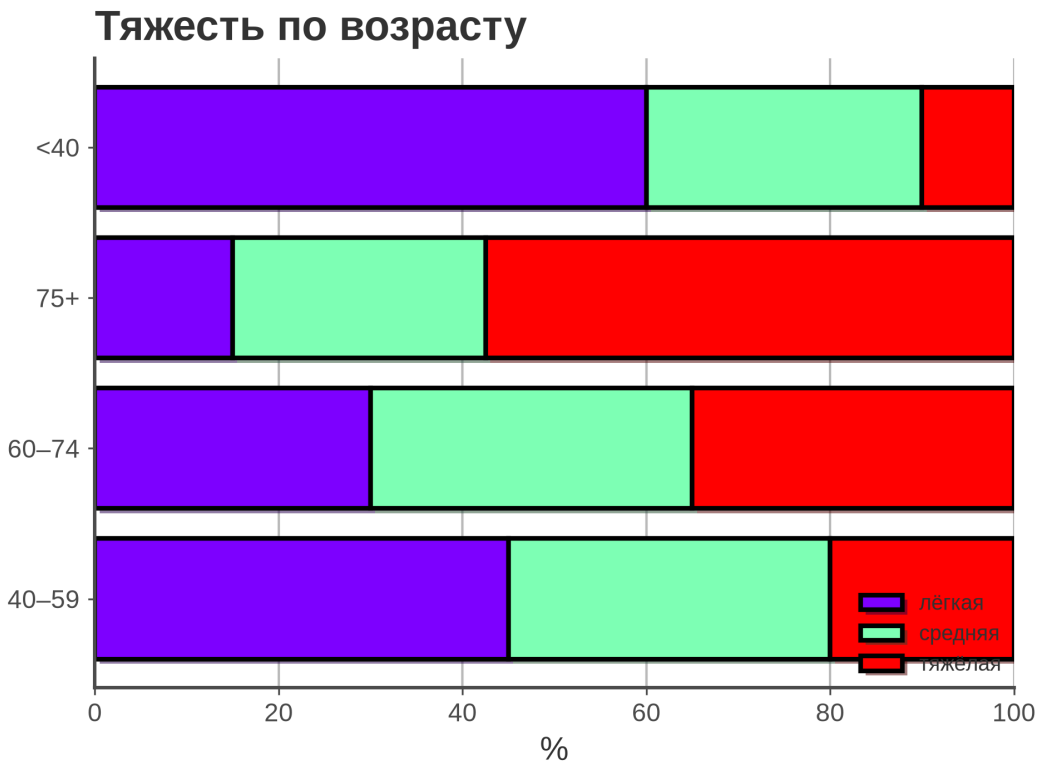


Рисунок 5.21.1. Плохой 100 % bar: радуга, тени, нет подписей и n.

Код: `05_charts/21_bar_100/scripts/bad_plot.py`.

Ошибки: радужные сегменты, тени, нет процентов внутри, нет *n*, тяжёлая сетка, заголовок «Тяжесть по возрасту» без вывода.

5.21.4 Правильный график

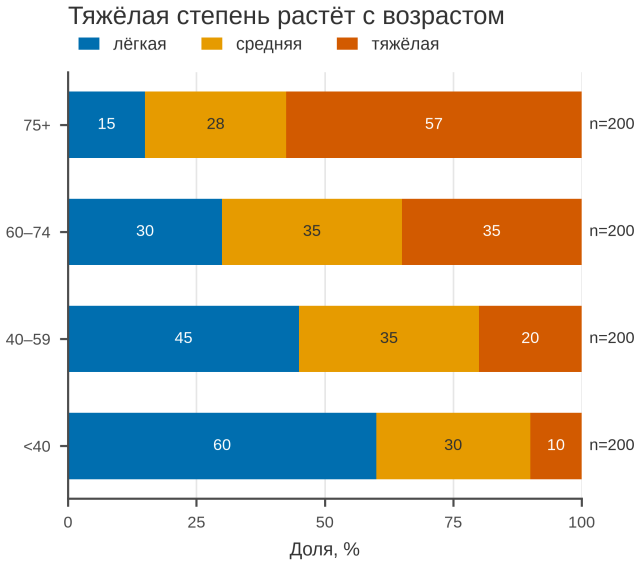


Рисунок 5.21.2. Правильный 100 % bar: доли подписаны, n справа, палитра Okabe–Ito.

Код: `05_charts/21_bar_100/scripts/good_plot.py`.

Улучшения: горизонтально, порядок возрастов, палитра Okabe–Ito, проценты внутри сегментов, n = 200 справа, заголовок-вывод, 85 мм.

5.21.5 Было и стало

Было	Стало
радуга, тени	Okabe–Ito
нет процентов	подписи долей
нет n	n = 200
нет вывода	«Тяжёлая степень растёт с возрастом»

5.21.6 Чек-лист

- ☐ Столбцы нормированы к 100 %.
- ☐ Указаны доли и *n*.
- ☐ 2–4 сегмента.
- ☐ Порядок сегментов одинаков.
- ☐ Палитра CVD-безопасна.
- ☐ Нет 3D/теней.
- ☐ Заголовок-вывод.
- ☐ Проверено в 85 мм.

5.21.7 Как воспроизвести

```
uv sync
uv run python 05_charts/21_bar_100/scripts/make_data.py
uv run python 05_charts/21_bar_100/scripts/bad_plot.py
uv run python 05_charts/21_bar_100/scripts/good_plot.py
```

```
share = pivot.div(pivot.sum(axis=1), axis=0) * 100
ax.barh(share.index, share["тяжёлая"], left=..., color="#D55E00")
```

5.21.8 Что дальше

5.22 Pie: части целого и почему pie редко бывает лучшим выбором.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `barh` — matplotlib.org

5.22 Pie: части целого

Pie показывает доли целого. Он уместен только для **3–4 частей**, когда нужно подчеркнуть «примерно половина/четверть», а не сравнивать близкие значения. Для точного сравнения лучше bar/dot.

5.22.1 Сфера применения

- 3–4 части целого;
- одна аудитория, один вывод;
- доли, которые легко узнать (50 %, 25 %);
- презентация, а не анализ.

Не подходит: много частей, близкие доли, отрицательные значения, несколько кругов, точное сравнение.

5.22.2 Данные

400 повторных госпитализаций, 4 причины.

Причина	n	Доля
Сердечная недостаточность	160	40 %
Инфекции	100	25 %
Кровотечения	80	20 %
Прочее	60	15 %

Генератор: `05_charts/22_pie/scripts/make_data.py → data/22_dataset.csv`.

5.22.3 Плохой график

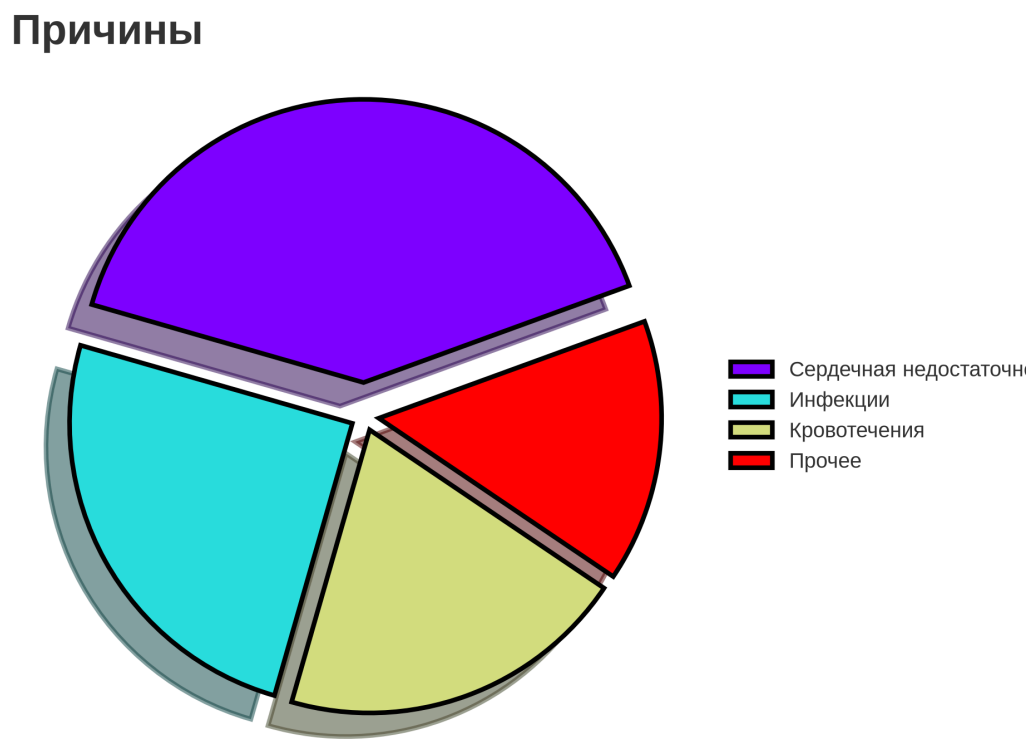


Рисунок 5.22.1. Плохая pie: 3D-тени, explode, радуга, легенда.

Код: `05_charts/22_pie/scripts/bad_plot.py`.

Ошибки: 3D-тени и explode искажают площадь, радуга, легенда вместо подписей, нет процентов, заголовок «Причины».

5.22.4 Правильный график



Рисунок 5.22.2. Правильная pie: 4 доли, подписи с n и %, без 3D.

Код: `05_charts/22_pie/scripts/good_plot.py`.

Улучшения: 4 доли, старт на 12 часах, подписи «причина, n (%)», белые границы, без 3D, заголовок-вывод, 95 мм.

5.22.5 Было и стало

Было	Стало
3D, explode	плоские доли
радуга	палитра Okabe–Ito
легенда	прямые подписи
нет %	n и %
«Причины»	«40 % — сердечная недостаточность»

5.22.6 Чек-лист

- ☐ 3–4 доли, сумма 100 %.
- ☐ Подписи с n и %.
- ☐ Нет 3D, explode, теней.
- ☐ Палитра различима.
- ☐ Старт на 12 часах, по часовой.
- ☐ Для сравнения предложен bar.
- ☐ Заголовок-вывод.
- ☐ Проверено в 95 мм.

5.22.7 Как воспроизвести

```
uv sync
uv run python 05_charts/22_pie/scripts/make_data.py
uv run python 05_charts/22_pie/scripts/bad_plot.py
uv run python 05_charts/22_pie/scripts/good_plot.py
```

```
ax.pie(values, labels=labels, colors=COLORS, startangle=90,
       counterclock=False, wedgeprops={"edgecolor": "white"})
```

5.22.8 Что дальше

5.23 Treemap: иерархический состав.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com

5.23 Treemap: иерархический состав

Treemap кодирует величину **площадью** прямоугольников. Он хорош для иерархического состава (программы → подпрограммы), но плохо подходит для точного сравнения: человек слабо оценивает площади.

5.23.1 Сфера применения

- 5–20 категорий, составляющих целое;
- иерархия (программа → направление);
- акцент на крупнейших вкладчиках;
- презентация структуры затрат.

Не подходит: мало категорий (bar), точное сравнение, отрицательные значения, динамика.

5.23.2 Данные

Затраты по шести программам, всего 1 200 млн руб.

Программа	Млн руб.
Кардиология	320
Онкология	280
Терапия	210
Хирургия	180
Педиатрия	120
Прочее	90

Кардиология и онкология — 50 % затрат.

Генератор: `05_charts/23_treemap/scripts/make_data.py` → `data/23_dataset.csv`.

5.23.3 Плохой график

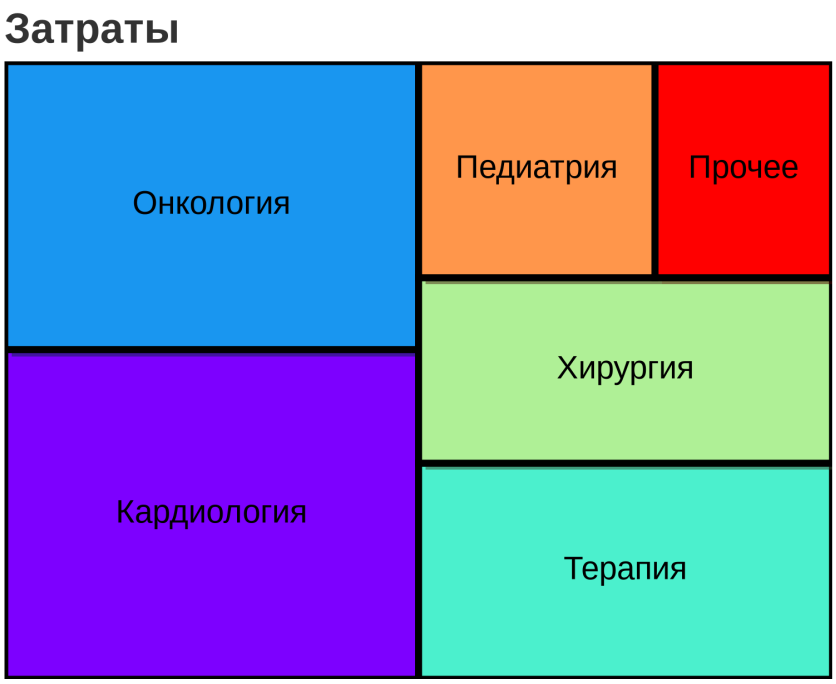


Рисунок 5.23.1. Плохой treemap: радуга, толстые границы, тени, нет значений.

Код: `05_charts/23_treemap/scripts/bad_plot.py`.

Ошибки: радуга, толстые чёрные границы, тени, только названия без значений, нет процентов и итога.

5.23.4 Правильный график

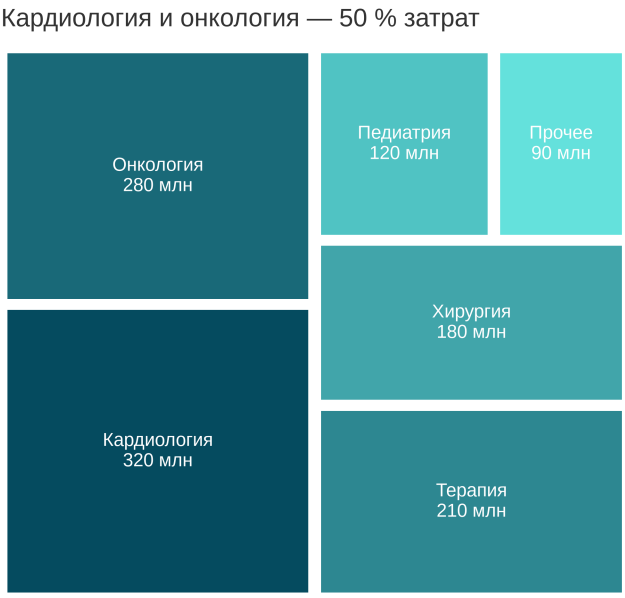


Рисунок 5.23.2. Правильный treemap: последовательная палитра, подписи с млн руб., без границ.

Код: `05_charts/23_treemap/scripts/good_plot.py`.

Улучшения: сортировка, последовательная палитра, подписи «программа
1. млн руб.», без границ и теней, заголовок-вывод, 85 мм.

5.23.5 Было и стало

Было	Стало
радуга	последовательная палитра
толстые границы	без границ
тени	плоско
нет значений	подписи с млн
нет вывода	«50 % затрат»

5.23.6 Чек-лист

- ☐ Площадь пропорциональна величине.
- ☐ 5–20 категорий.
- ☐ Подписи с значениями.
- ☐ Последовательная палитра.
- ☐ Нет 3D/теней.
- ☐ Крупнейшие видны.
- ☐ Заголовок-вывод.
- ☐ Проверено в 85 мм.

5.23.7 Как воспроизвести

```
uv sync
uv run python 05_charts/23_treemap/scripts/make_data.py
uv run python 05_charts/23_treemap/scripts/bad_plot.py
uv run python 05_charts/23_treemap/scripts/good_plot.py
```

```
squarify.plot(sizes=df["cost_mln_rub"], label=labels,
              color=COLORS, ax=ax, pad=True)
```

5.23.8 Что дальше

5.24 Heatmap: матрица значений.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Squarify — github.com/laserson/squarify

5.24 Heatmap: матрица значений

Heatmap кодирует значение цветом в сетке «строка × столбец». Он хорош для матриц 5×5–50×50, но требует **упорядочивания**, **подписей** и **colorbar**: иначе цвет не читается.

5.24.1 Сфера применения

- время ожидания по отделениям и часам;
- показатели по центрам и месяцам;
- матрица различий/ошибок;
- тепловая карта загрузки.

Не подходит: мало значений (bar), точное сравнение (таблица), непрерывные двумерные данные (hexbin/contour).

5.24.2 Данные

5 отделений × 6 часов, среднее время ожидания (мин).

Колонка	Смысл	Единица
`department`	отделение	—
`hour`	час	ч
`wait_min`	ожидание	мин

Хирургия — самое долгое ожидание; пик — 10–12 часов.

Генератор: `05_charts/24_heatmap/scripts/make_data.py → data/24_dataset.csv`.

5.24.3 Плохой график

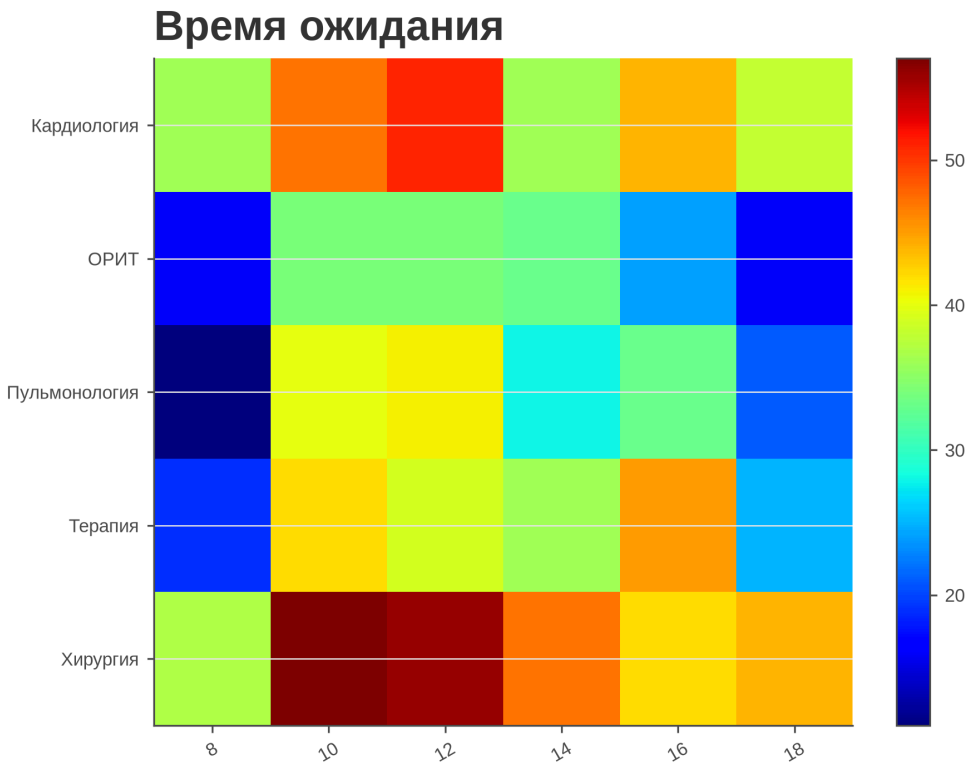


Рисунок 5.24.1. Плохая heatmap: jet, нет значений и единиц, без сортировки.

Код: `05_charts/24_heatmap/scripts/bad_plot.py`.

Ошибки: `jet`, нет чисел в ячейках, нет единиц у colorbar, строки не упорядочены, заголовок «Время ожидания» без вывода.

5.24.4 Правильный график

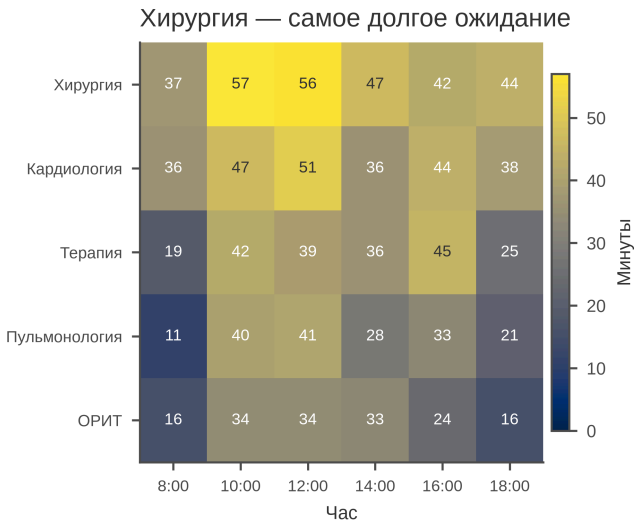


Рисунок 5.24.2. Правильная heatmap: cividis, значения в ячейках, colorbar «Минуты», сортировка.

Код: `05_charts/24_heatmap/scripts/good_plot.py`.

Улучшения: `cividis`, значения в ячейках, colorbar «Минуты», строки отсортированы по среднему, подписи часов, заголовок-вывод, 85 мм.

5.24.5 Чек-лист

- ☐ Строки/столбцы упорядочены осмысленно.
- ☐ Значения подписаны или есть colorbar с единицами.
- ☐ Палитра последовательная/дивергентная по смыслу.
- ☐ Нет `jet`.
- ☐ Указаны единицы и *n*.
- ☐ Ячейки не искажены (aspect auto/equal осознанно).
- ☐ Заголовок-вывод.
- ☐ Проверено в 85 мм.

5.24.6 Как воспроизвести

```
uv sync
uv run python 05_charts/24_heatmap/scripts/make_data.py
uv run python 05_charts/24_heatmap/scripts/bad_plot.py
uv run python 05_charts/24_heatmap/scripts/good_plot.py
```

5.24.7 Что дальше

5.25 Clustermap: heatmap с кластеризацией.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Matplotlib: `imshow` — matplotlib.org

5.25 Clustermap: heatmap с кластеризацией

Clustermap — это heatmap, у которого строки и столбцы переставлены иерархической кластеризацией. Он показывает блоки/кластеры, но **не доказывает** их существование: дендрограмма — результат выбранной метрики и linkage.

5.25.1 Сфера применения

- экспрессия генов;
- биомаркеры по пациентам;
- матрицы расстояний;
- поиск блоков в больших матрицах.

Не подходит: малые матрицы, точное сравнение, данные без смысла кластеризации.

5.25.2 Данные

10 генов × 12 пациентов, экспрессия. Два кластера: G01–G05 выше у S01–S06, G06–G10 — у S07–S12.

Генератор: `05_charts/25_clustermap/scripts/make_data.py → data/25_dataset.csv`.

5.25.3 Плохой график

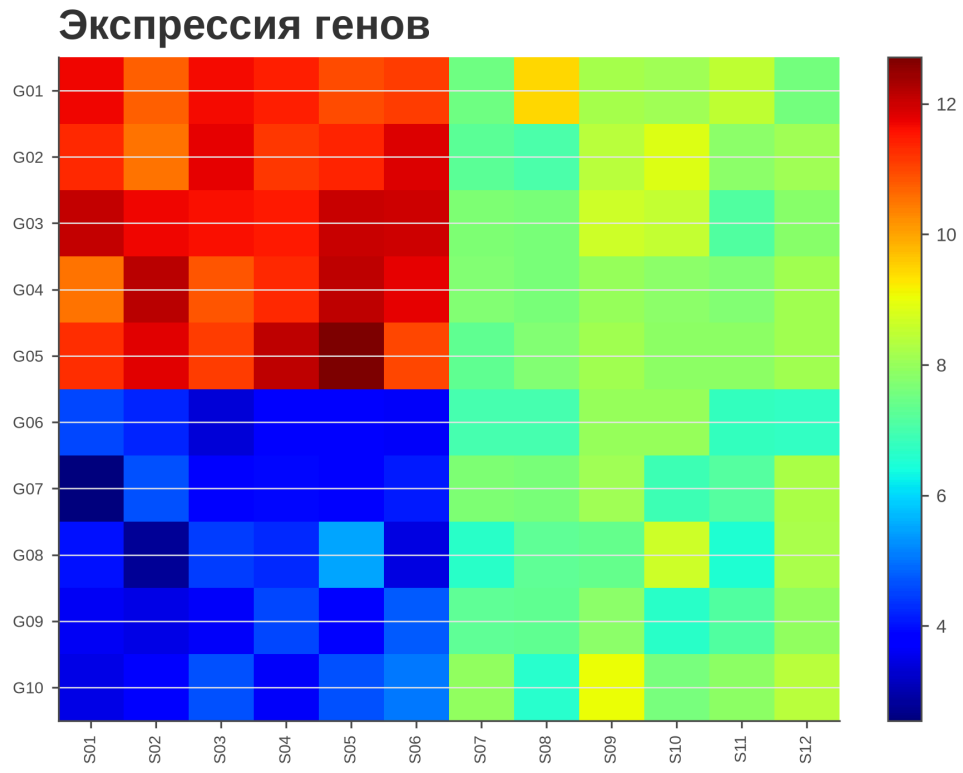


Рисунок 5.25.1. Плохой clustermap: без кластеризации и z-score, jet.

Код: `05_charts/25_clustermap/scripts/bad_plot.py`.

Ошибки: нет кластеризации, нет z-score, `jet`, нет дендрограмм, цвет не отражает отклонение от среднего.

5.25.4 Правильный график

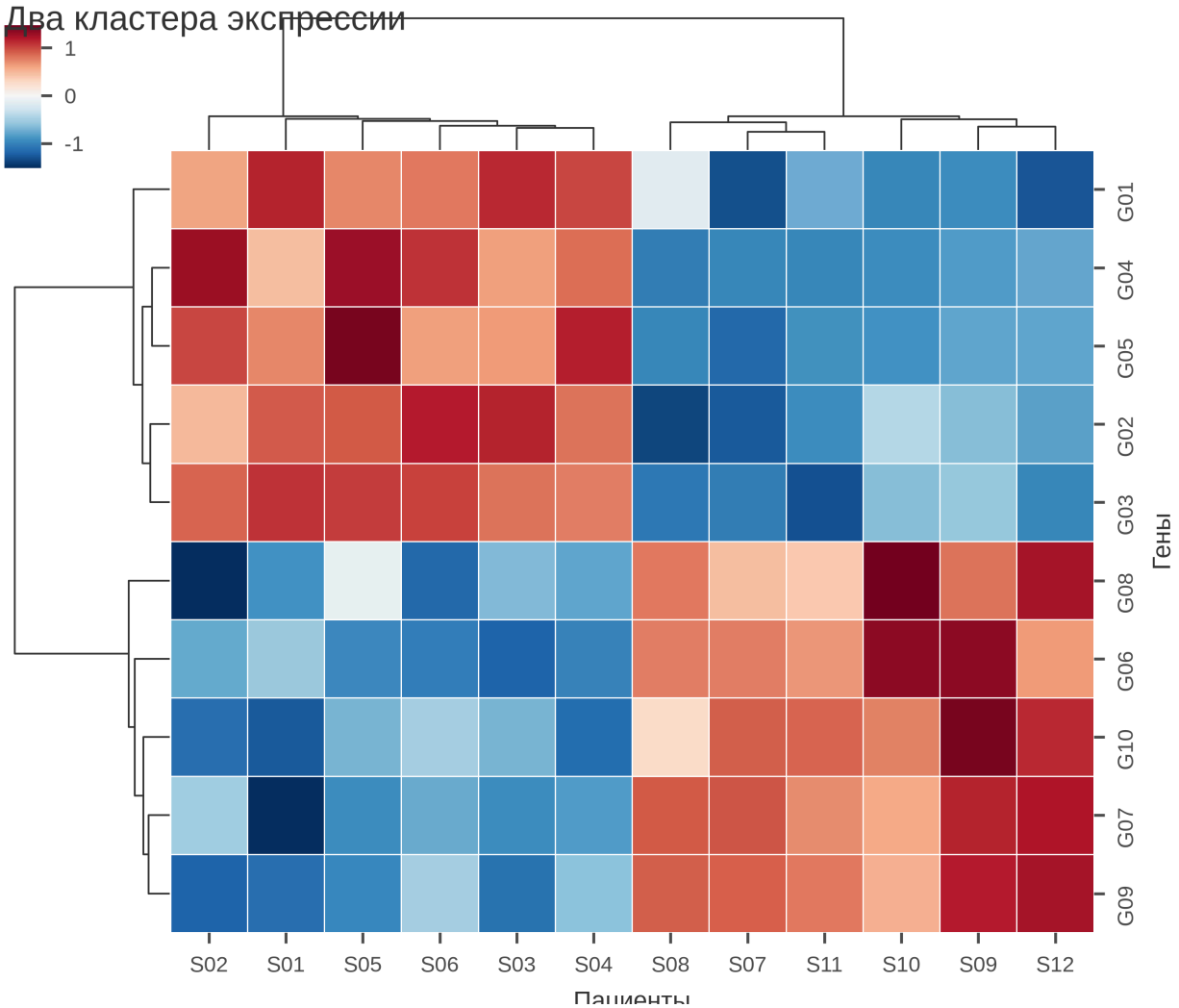


Рисунок 5.25.2. Правильный clustermap: z-score, дендрограммы, diverging RdBu_r.

Код: `05_charts/25_clustermap/scripts/good_plot.py`.

Улучшения: z-score по генам, `RdBu_r` с центром 0, дендрограммы, подписи, заголовок, 120 мм. Цвет — отклонение от среднего гена.

5.25.5 Чек-лист

- ☐ Понятно, что кластеризация — метод, а не истина.
- ☐ Указаны метрика/linkage или значение по умолчанию.
- ☐ z-score/нормировка осознанны.
- ☐ Палитра diverging с центром 0.
- ☐ Дендрограммы и подписи читаются.
- ☐ Нет `jet`.
- ☐ Указаны n и единицы.
- ☐ Проверено в 120 мм.

5.25.6 Как воспроизвести

```
uv sync
uv run python 05_charts/25_clustermap/scripts/make_data.py
uv run python 05_charts/25_clustermap/scripts/bad_plot.py
uv run python 05_charts/25_clustermap/scripts/good_plot.py
```

5.25.7 Что дальше

5.26 Corr matrix: корреляции.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Seaborn: `clustermap` — seaborn.pydata.org

5.26 Corr matrix: корреляционная матрица

Корреляционная матрица показывает парные связи многих переменных. Она полезна как **обзор**, но легко перегружается: верхний треугольник дублирует нижний, а цвет без коэффициентов и шкалы −1…1 обманчив.

5.26.1 Сфера применения

- обзор корреляций 5–15 биомаркеров;
- проверка мультиколлинеарности;
- выбор переменных для модели;
- поиск блоков связанных признаков.

Не подходит: причинность, малые выборки, нелинейные связи, много переменных (>20).

5.26.2 Данные

500 пациентов, 8 показателей: возраст, BMI, САД, глюкоза, HbA1c, ЛПНП, СРБ, eGFR. Корреляции — Пирсона.

Генератор: `05_charts/26_corr_matrix/scripts/make_data.py → data/26_dataset.csv`.

5.26.3 Плохой график

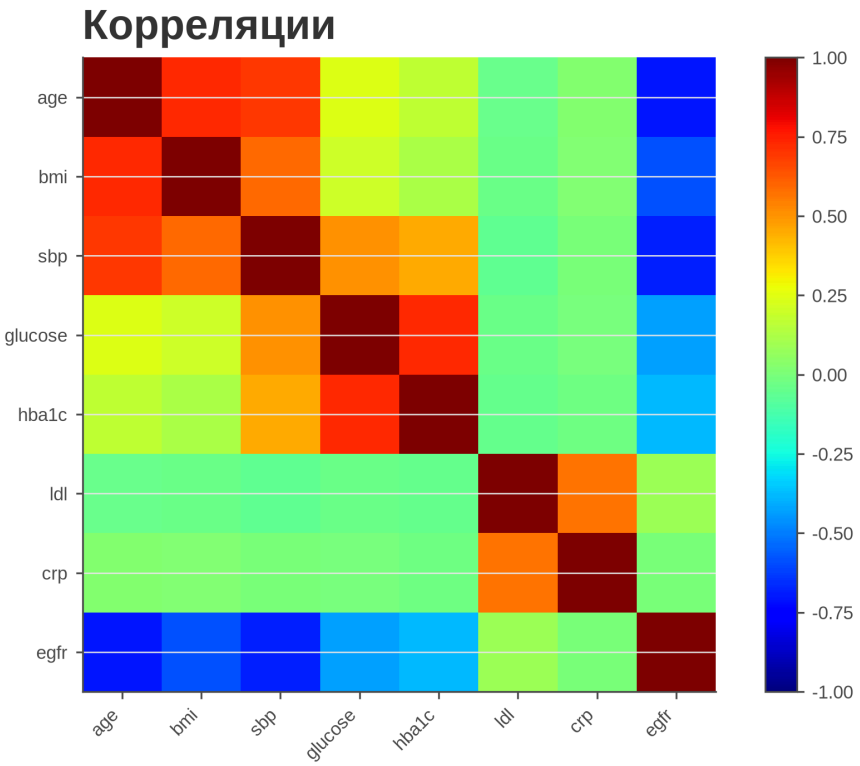


Рисунок 5.26.1. Плохая корреляционная матрица: полная, jet, без коэффициентов.

Код: `05_charts/26_corr_matrix/scripts/bad_plot.py`.

Ошибки: полная матрица с дублированием, `jet`, нет коэффициентов, нет шкалы −1…1, заголовок «Корреляции».

5.26.4 Правильный график

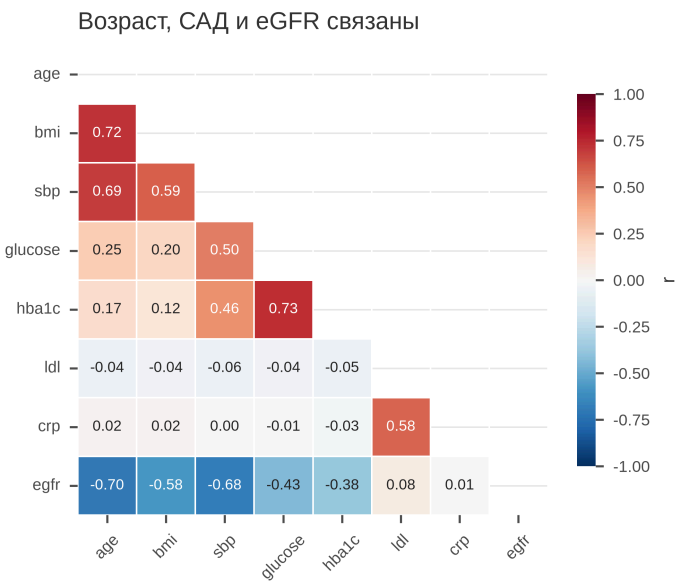


Рисунок 5.26.2. Правильная корреляционная матрица: нижний треугольник, RdBu_r, коэффициенты.

Код: `05_charts/26_corr_matrix/scripts/good_plot.py`.

Улучшения: нижний треугольник, `RdBu_r` с центром 0 и границами −1…1, коэффициенты в ячейках, colorbar «r», заголовок-вывод, 100 мм.

5.26.5 Чек-лист

- ☐ Показан только нижний/верхний треугольник.
- ☐ Шкала −1…1 и центр 0.
- ☐ Коэффициенты подписаны.
- ☐ Указан метод (Пирсон/Спирмен).
- ☐ Указаны n и единицы.
- ☐ Нет `jet`.
- ☐ Не интерпретируется как причинность.
- ☐ Проверено в 100 мм.

5.26.6 Как воспроизвести

```
uv sync
uv run python 05_charts/26_corr_matrix/scripts/make_data.py
uv run python 05_charts/26_corr_matrix/scripts/bad_plot.py
uv run python 05_charts/26_corr_matrix/scripts/good_plot.py
```

5.26.7 Что дальше

5.27 Kaplan–Meier: выживаемость.

Источники

- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Seaborn: `heatmap` — seaborn.pydata.org

5.27 Kaplan–Meier: выживаемость

Кривая Каплана–Мейера показывает долю пациентов без события во времени с учётом **цензурирования**. Это не доля выживших к фиксированному сроку: кривая пересчитывается в каждом событии, а цензурированные пациенты выходят из риска, не считаясь событиями.

5.27.1 Сфера применения

- время до смерти/рецидива/госпитализации;
- сравнение групп с цензурированием;
- оценка выживаемости к 12/24/36 месяцам;
- проверка пропорциональности рисков (лог-ранг).

Не подходит: нет цензурирования (ECDF), конкурирующие риски (5.28), сравнение средних без времени.

5.27.2 Данные

240 пациентов, две группы по 120, наблюдение до 36 месяцев; событие — смерть, остальные цензурированы.

Колонка	Смысл
`group`	контроль/лечение
`time_months`	время наблюдения
`event`	1 смерть, 0 цензурирование

Генератор: `05_charts/27_kaplan_meier/scripts/make_data.py` → `data/27_dataset.csv`.

5.27.3 Плохой график

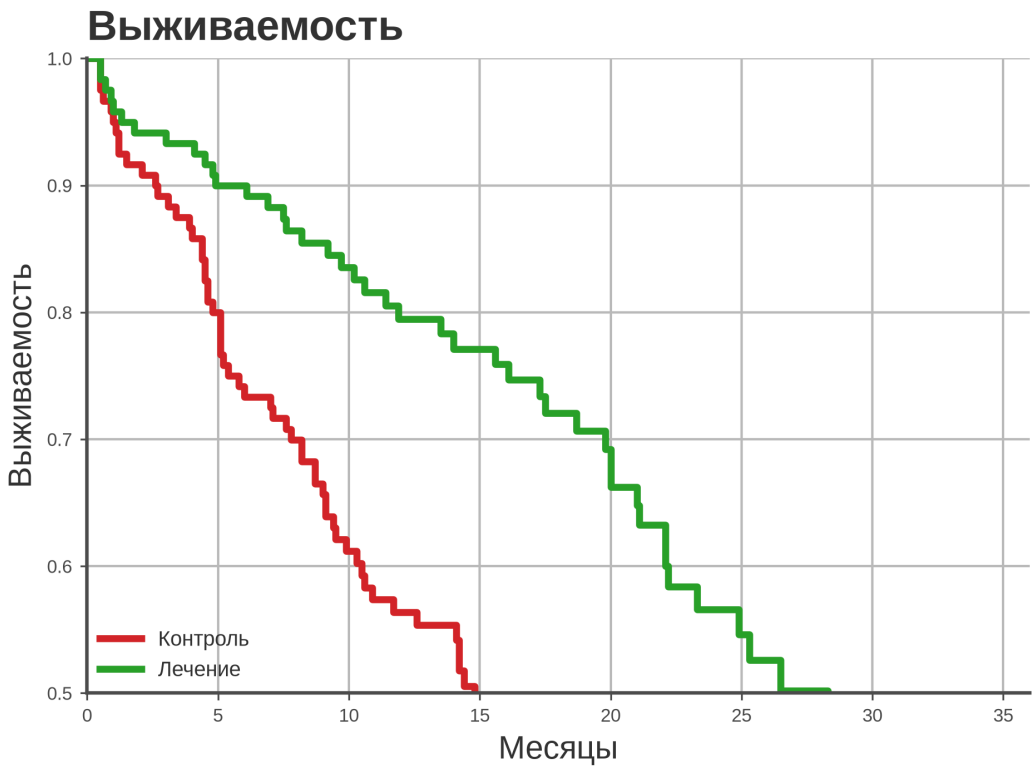


Рисунок 5.27.1. Плохая КМ: обрезанная ось 0,5–1,0, радуга, нет цензурирования и риска.

Код: `05_charts/27_kaplan_meier/scripts/bad_plot.py`.

Ошибки: ось 0,5–1,0 скрывает половину кривых; нет меток цензурирования, числа под риском, CI; радуга, тяжёлая сетка.

5.27.4 Правильный график

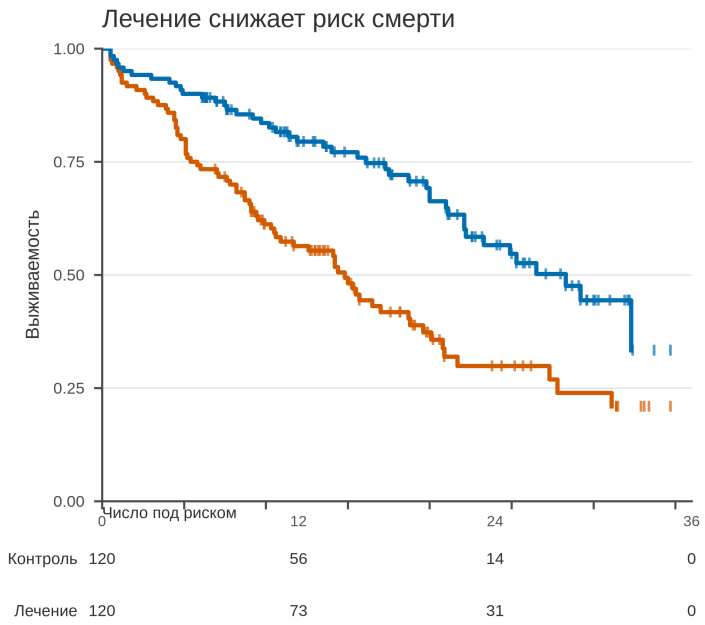


Рисунок 5.27.2. Правильная KM: две группы, метки цензурирования, риск-таблица, 95 мм.

Код: `05_charts/27_kaplan_meier/scripts/good_plot.py`.

Улучшения: полный диапазон 0–1, две группы, метки цензурирования, таблица «число под риском» на 0/12/24/36, единицы, 95 мм.

5.27.5 Чек-лист

- ☐ Указано событие и цензурирование.
- ☐ Ось Y 0–1.
- ☐ Метки цензурирования.
- ☐ Число под риском по времени.
- ☐ Указаны n и единицы времени.
- ☐ Нет радуги/тяжёлой сетки.
- ☐ Лог-ранг/HR — если есть сравнение.
- ☐ Проверено в 95 мм.

5.27.6 Как воспроизвести

```
uv sync
uv run python 05_charts/27_kaplan_meier/scripts/make_data.py
uv run python 05_charts/27_kaplan_meier/scripts/bad_plot.py
uv run python 05_charts/27_kaplan_meier/scripts/good_plot.py
```

5.27.7 Что дальше

5.28 Cumulative incidence: конкурирующие риски.

Источники

- Kaplan E. L., Meier P. Nonparametric Estimation from Incomplete Observations // *JASA*. 1958. — doi:10.1080/01621459.1958.10501452
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz

5.28 Cumulative incidence: конкурирующие риски

При конкурирующих рисках событие одного типа мешает событию другого. Обычная 1 − KM завышает риск, если считать конкурирующее событие цензурированием. Правильная оценка — кумулятивная заболеваемость (Aalen–Johansen): суммируются cause-specific hazards с учётом общей выживаемости.

5.28.1 Данные

300 пациентов, 36 месяцев; статус: 0 — цензурирован, 1 — кардиальная смерть, 2 — некардиальная смерть.

Генератор: `05_charts/28_cumulative_incidence/scripts/make_data.py → data/28_dataset.csv`.

5.28.2 Плохой график

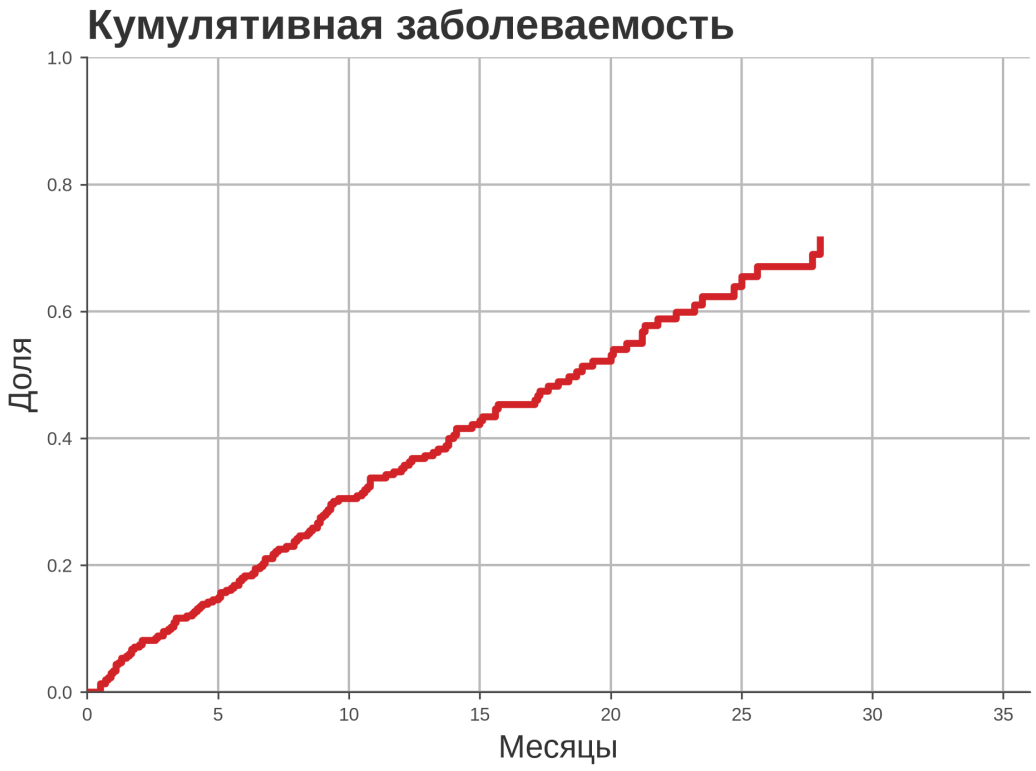


Рисунок 5.28.1. Плохая CIF: 1-KM без конкурирующего риска, одна кривая, тяжёлая сетка.

Код: `05_charts/28_cumulative_incidence/scripts/bad_plot.py`.

Ошибки: конкурирующая смерть считается цензурированием; риск завышен; нет второй кривой, n и единиц.

5.28.3 Правильный график

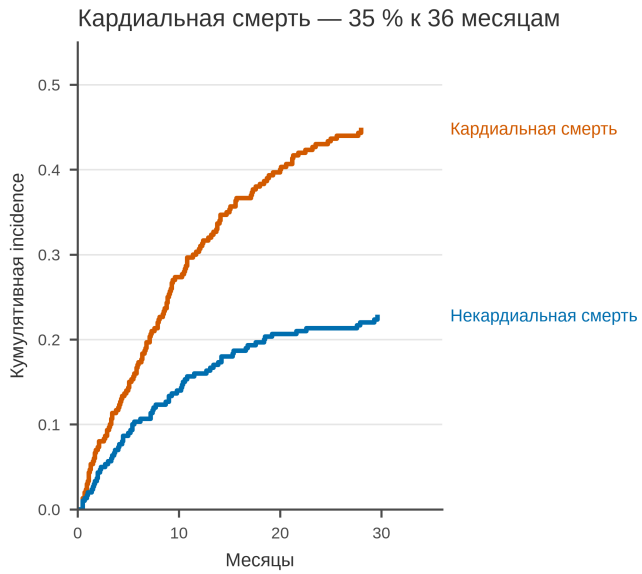


Рисунок 5.28.2. Правильная CIF: кардиальная и некардиальная смерть, 0–55 %, прямые подписи.

Код: `05_charts/28_cumulative_incidence/scripts/good_plot.py`.

Улучшения: две cause-specific CIF, общая выживаемость в знаменателе, полный диапазон, прямые подписи, 85 мм.

5.28.4 Чек-лист

- ☐ Конкурирующие события не считаются цензурированием.
- ☐ Показаны все конкурирующие исходы.
- ☐ Ось 0–разумный максимум.
- ☐ Указаны n, время, единицы.
- ☐ Прямые подписи кривых.
- ☐ Нет 1 − KM как «заболеваемости».
- ☐ Проверено в 85 мм.

5.28.5 Как воспроизвести

```
uv sync
uv run python 05_charts/28_cumulative_incidence/scripts/make_data.py
uv run python 05_charts/28_cumulative_incidence/scripts/bad_plot.py
uv run python 05_charts/28_cumulative_incidence/scripts/good_plot.py
```

5.28.6 Что дальше

5.29 ROC-кривая.

Источники

- Fine J. P., Gray R. J. A Proportional Hazards Model for the Subdistribution of a Competing Risk // *JASA*. 1999. — doi:10.1080/01621459.1999.10474144
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz

5.29 ROC: чувствительность и специфичность

ROC показывает компромисс TPR/FPR при всех порогах; AUC — площадь. AUC 0,5 — случайность, 1 — идеал. ROC полезна для баланса классов, но при редком исходе PR информативнее (5.30).

5.29.1 Данные

500 пациентов, score и бинарный исход; prevalence 31 %.

Генератор: `05_charts/29_roc/scripts/make_data.py → data/29_dataset.csv`.

5.29.2 Плохой график

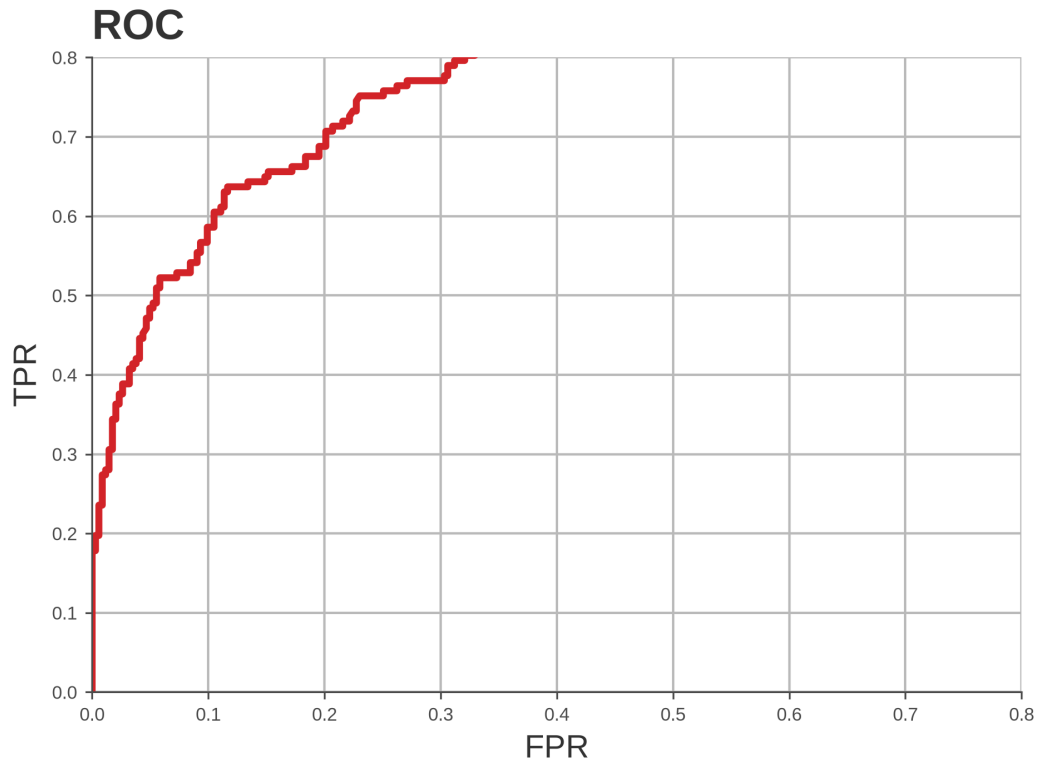


Рисунок 5.29.1. Плохая ROC: обрезанные оси 0–0,8, нет диагонали и AUC.

Код: `05_charts/29_roc/scripts/bad_plot.py`. Ошибки: обрезаны оси, нет диагонали, AUC/n, тяжёлая сетка.

5.29.3 Правильный график

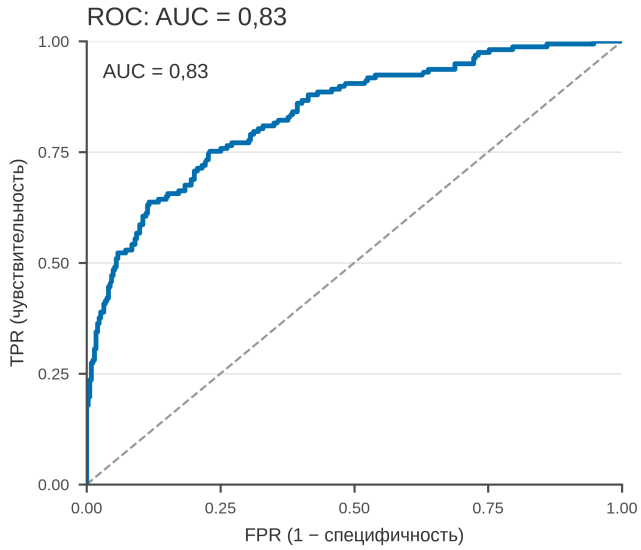


Рисунок 5.29.2. Правильная ROC: диагональ, AUC 0,84, полный диапазон.

Код: `05_charts/29_roc/scripts/good_plot.py`. Диагональ, AUC 0,84, оси 0–1, единицы, 85 мм.

5.29.4 Чек-лист

- ❐ Оси 0–1.
- ❐ Диагональ случайного классификатора.
- ❐ AUC и n.
- ❐ Указан порог/цель.
- ❐ Для редкого исхода — PR.

5.29.5 Как воспроизвести

```
uv sync
uv run python 05_charts/29_roc/scripts/make_data.py
uv run python 05_charts/29_roc/scripts/bad_plot.py
uv run python 05_charts/29_roc/scripts/good_plot.py
```

5.29.6 Что дальше

5.30 Precision–recall.

Источники

- Fawcett T. An Introduction to ROC Analysis // *Pattern Recognition Letters*. 2006. — doi:10.1016/j.patrec.2005.10.010

5.30 Precision–recall: редкий исход

При редком исходе ROC выглядит оптимистично: большое число TN маскирует ложные срабатывания. PR-кривая показывает precision (сколько предсказанных событий истинны) и recall (какую долю событий поймали).

5.30.1 Данные

600 пациентов, prevalence 8 %; score и событие.

Генератор: `05_charts/30_precision_recall/scripts/make_data.py → data/30_dataset.csv`.

5.30.2 Плохой график

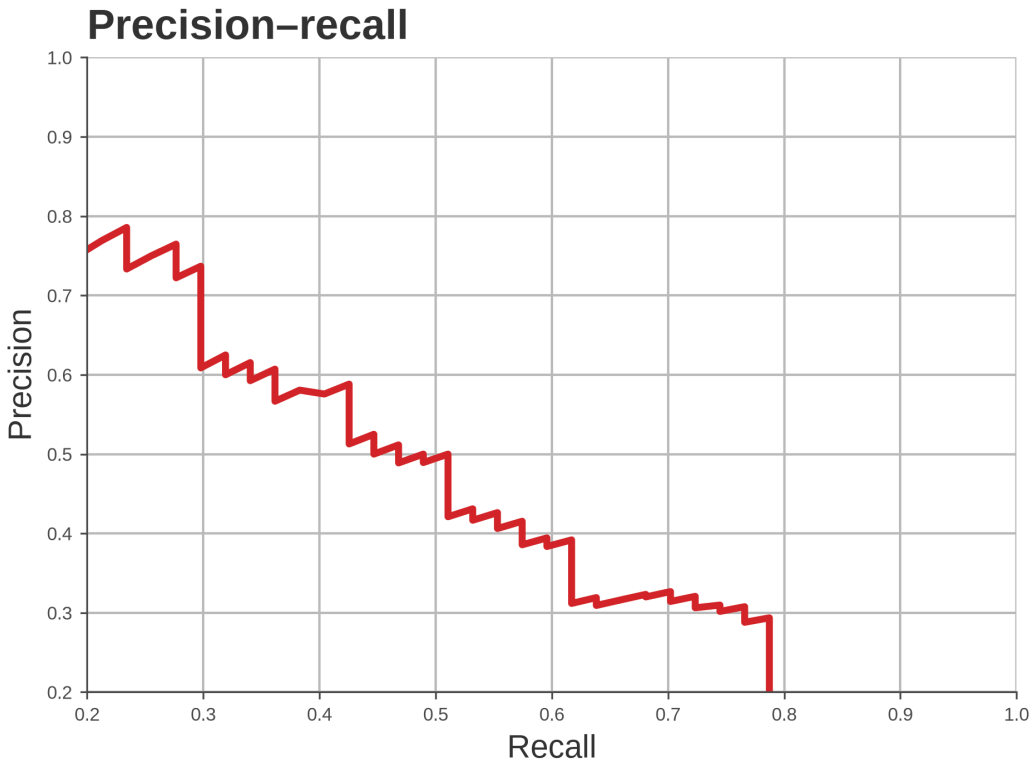


Рисунок 5.30.1. Плохой PR: обрезанные оси, нет baseline и AP.

Код: `05_charts/30_precision_recall/scripts/bad_plot.py`. Ошибки: оси 0,2–1, нет baseline prevalence, AP, n.

5.30.3 Правильный график

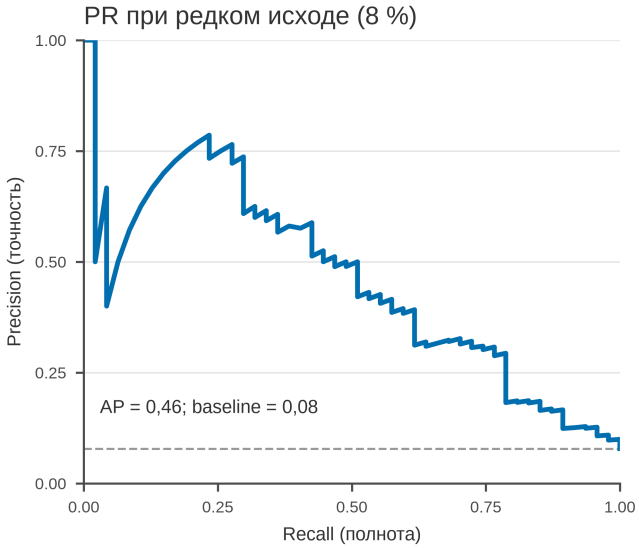


Рисунок 5.30.2. Правильный PR: baseline 8 %, AP 0,46, полный диапазон.

Код: `05_charts/30_precision_recall/scripts/good_plot.py`. Baseline, AP 0,46, оси 0–1, 85 мм.

5.30.4 Чек-лист

- ☐ Указана prevalence/baseline.
- ☐ Оси 0–1.
- ☐ AP или AUC-PR.
- ☐ n и событие.
- ☐ Для редкого исхода PR, не только ROC.

5.30.5 Как воспроизвести

```
uv sync
uv run python 05_charts/30_precision_recall/scripts/make_data.py
uv run python 05_charts/30_precision_recall/scripts/bad_plot.py
uv run python 05_charts/30_precision_recall/scripts/good_plot.py
```

5.30.6 Что дальше

5.31 Calibration.

Источники

- Saito T., Rehmsmeier M. The Precision-Recall Plot Is More Informative than the ROC Plot // *PLoS ONE*. 2015. — doi:10.1371/journal.pone.0118432

5.31 Calibration: соответствие прогноза и реальности

Калибровка отвечает: если модель говорит «риск 30 %», действительно ли у 30 % наступает событие? ROC/AUC этого не показывают. Идеальная калибровка — диагональ.

5.31.1 Данные

400 пациентов, прогнозируемый риск и бинарный исход; 8 бинов.

Генератор: `05_charts/31_calibration/scripts/make_data.py → data/31_dataset.csv`.

5.31.2 Плохой график

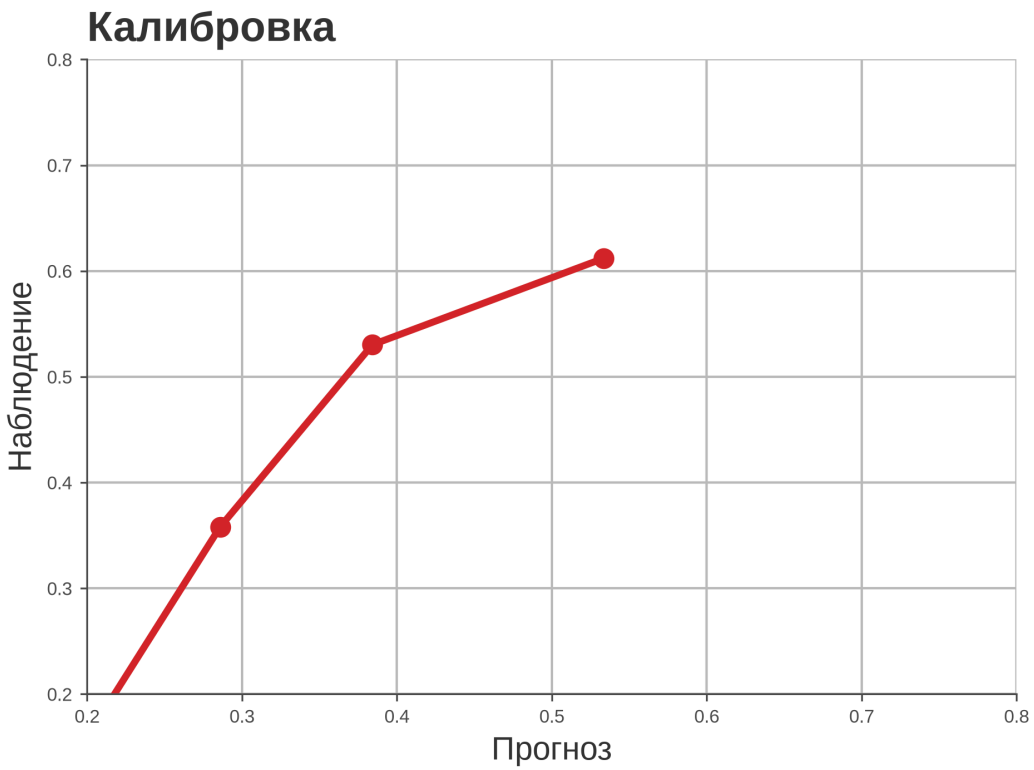


Рисунок 5.31.1. Плохая калибровка: обрезанные оси, нет диагонали и CI.

Код: `05_charts/31_calibration/scripts/bad_plot.py`. Ошибки: оси 0,2–0,8, нет диагонали, CI, Brier и n.

5.31.3 Правильный график

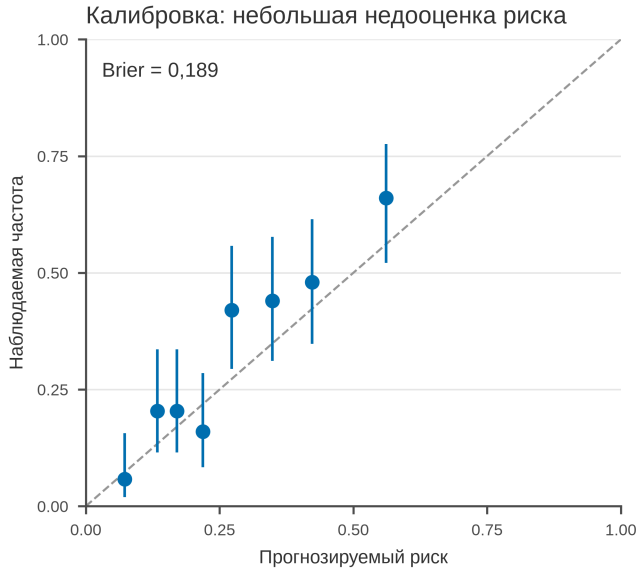


Рисунок 5.31.2. Правильная калибровка: диагональ, Wilson CI, Brier 0,19.

Код: `05_charts/31_calibration/scripts/good_plot.py`. Диагональ, 8 бинов, Wilson CI, Brier 0,19, 85 мм.

5.31.4 Чек-лист

- ☐ Диагональ идеальной калибровки.
- ☐ Бины и n.
- ☐ CI/uncertainty.
- ☐ Brier или calibration slope.
- ☐ Оси 0–1.
- ☐ Не путать с ROC/AUC.

5.31.5 Как воспроизвести

```
uv sync
uv run python 05_charts/31_calibration/scripts/make_data.py
uv run python 05_charts/31_calibration/scripts/bad_plot.py
uv run python 05_charts/31_calibration/scripts/good_plot.py
```

5.31.6 Что дальше

5.32 Caterpillar plot.

Источники

- Van Calster B. et al. Calibration: the Achilles Heel of Predictive Analytics // *BMC Medicine*. 2019. — doi:10.1186/s12916-019-1466-7

5.32 Caterpillar: упорядоченные оценки

Caterpillar plot — это точки оценок с доверительными интервалами, отсортированные по величине. Он показывает и ранг, и точность: нельзя сравнивать центры только по точке, если интервалы перекрываются.

5.32.1 Данные

30 центров, по 80 пациентов, осложнения. 95 % ДИ Уилсона.

Генератор: `05_charts/32_caterpillar/scripts/make_data.py → data/32_dataset.csv`.

5.32.2 Плохой график

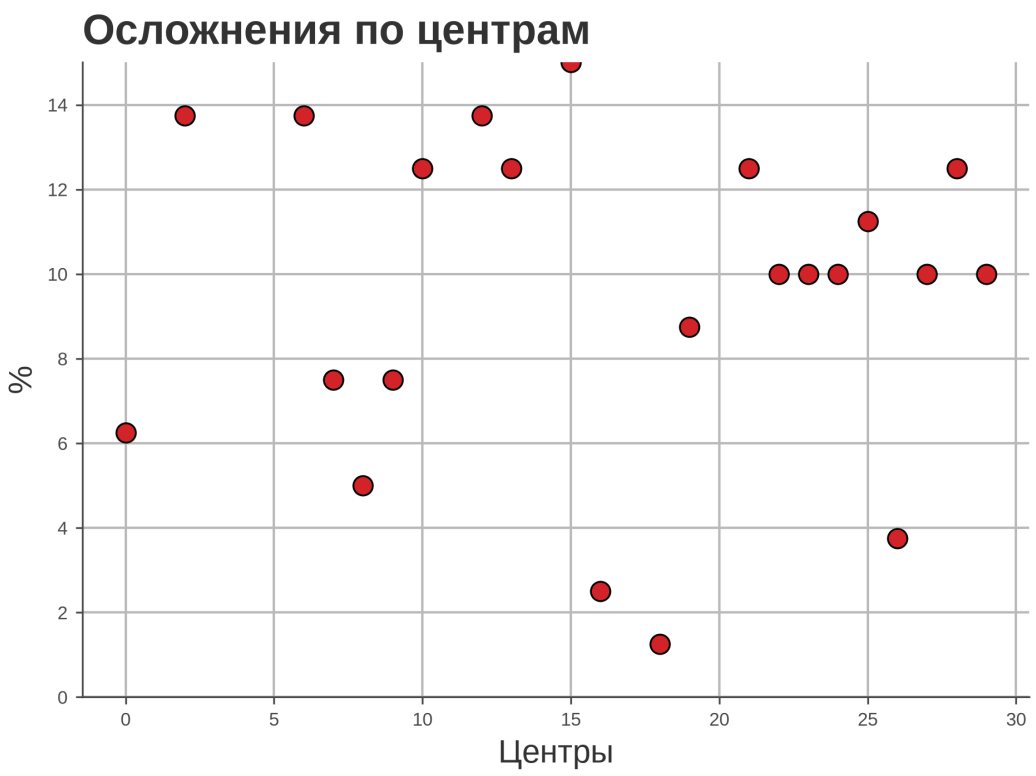


Рисунок 5.32.1. Плохой caterpillar: без сортировки и CI, обрезанная ось, радуга.

Код: `05_charts/32_caterpillar/scripts/bad_plot.py`. Ошибки: нет сортировки, CI, общей линии, n; обрезанная ось.

5.32.3 Правильный график

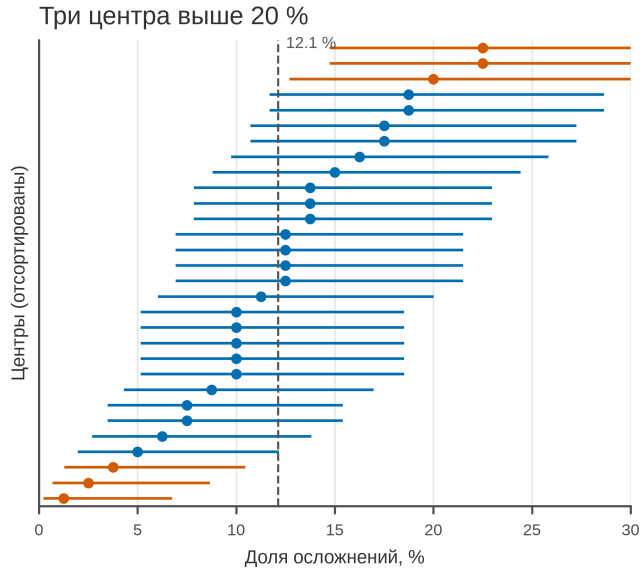


Рисунок 5.32.2. Правильный caterpillar: сортировка, Wilson CI, общая линия, крайние центры выделены.

Код: `05_charts/32_caterpillar/scripts/good_plot.py`. Сортировка, 95 % CI, общая линия, крайние центры акцентированы, 85 мм.

5.32.4 Чек-лист

- ❐ Сортировка по оценке.
- ❐ 95 % CI для каждой оценки.
- ❐ Общая/референсная линия.
- ❐ Указаны n и единицы.
- ❐ Пересечение CI не игнорируется.
- ❐ Крайние центры выделены осознанно.
- ❐ Проверено в 85 мм.

5.32.5 Как воспроизвести

```
uv sync
uv run python 05_charts/32_caterpillar/scripts/make_data.py
uv run python 05_charts/32_caterpillar/scripts/bad_plot.py
uv run python 05_charts/32_caterpillar/scripts/good_plot.py
```

5.32.6 Что дальше

5.33 Funnel plot.

Источники

- Spiegelhalter D. J. Funnel Plots for Comparing Institutional Performance // *Statistics in Medicine*. 2005. — doi:10.1002/sim.1970

5.33 Funnel: асимметрия и малые исследования

Funnel plot показывает эффект против точности (SE). Воронка — 95 % область вокруг pooled-эффекта. Асимметрия (малые исследования «выпадают» из воронки) — сигнал publication bias / small-study effects.

5.33.1 Данные

25 исследований, log OR и SE.

Генератор: `05_charts/33_funnel/scripts/make_data.py → data/33_dataset.csv`.

5.33.2 Плохой график

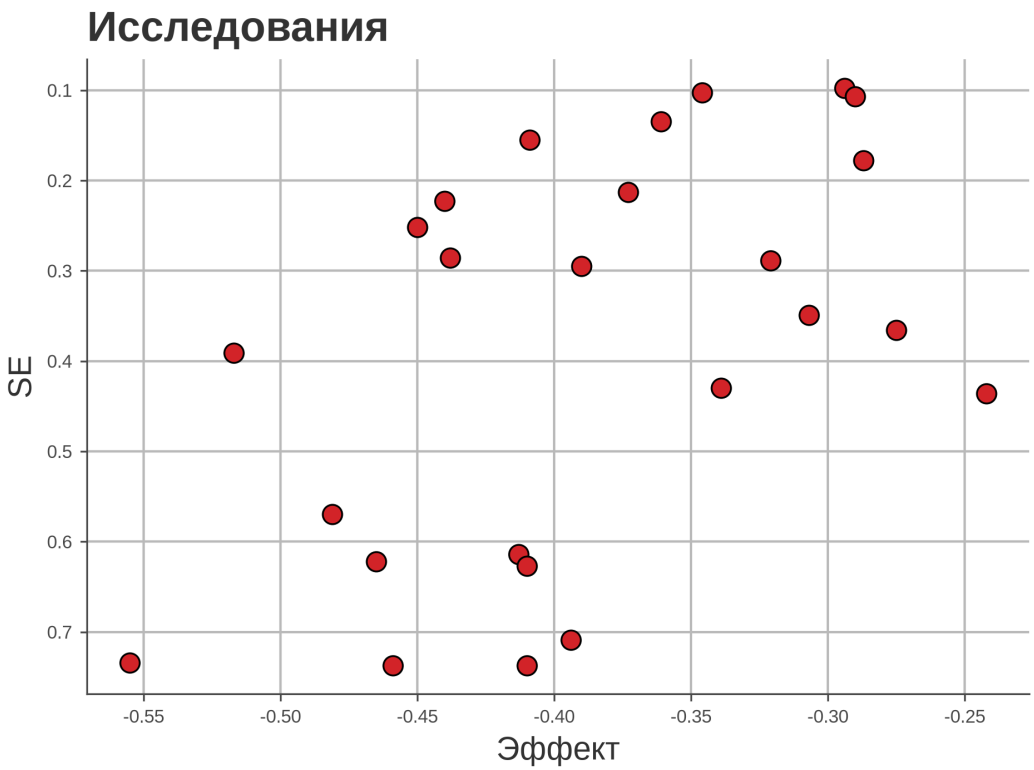


Рисунок 5.33.1. Плохой funnel: без воронки, pooled и подписей.

Код: `05_charts/33_funnel/scripts/bad_plot.py`. Ошибки: нет 95 % воронки, pooled-линии, осей эффекта/SE, n.

5.33.3 Правильный график

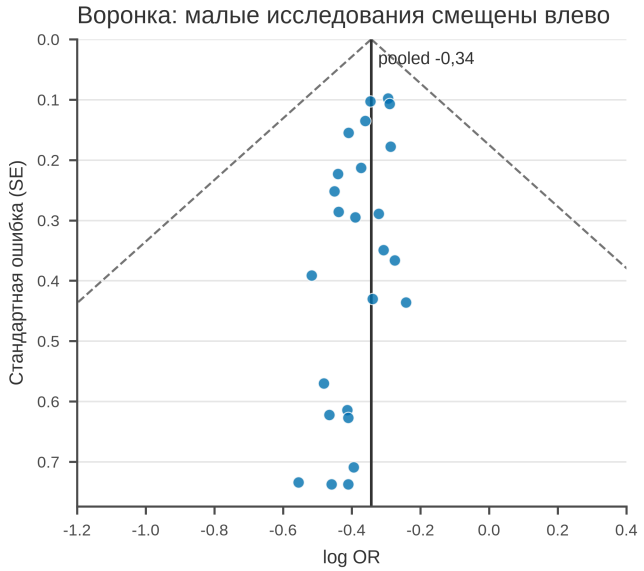


Рисунок 5.33.2. Правильный funnel: pooled, 95 % воронка, асимметрия.

Код: `05_charts/33_funnel/scripts/good_plot.py`. Pooled, 95 % воронка, SE по Y, точки, заголовок об асимметрии, 85 мм.

5.33.4 Чек-лист

- ☐ Эффект по X, SE/точность по Y (SE вниз).
- ☐ Pooled-эффект.
- ☐ 95 % воронка.
- ☐ Указаны k (число исследований).
- ☐ Асимметрия не подана как доказательство bias.
- ☐ Проверено в 85 мм.

5.33.5 Как воспроизвести

```
uv sync
uv run python 05_charts/33_funnel/scripts/make_data.py
uv run python 05_charts/33_funnel/scripts/bad_plot.py
uv run python 05_charts/33_funnel/scripts/good_plot.py
```

5.33.6 Итог раздела 5

Все 33 графика разобраны: данные → плохой пример → разбор → правильный пример. Каждый — со своим генератором, датасетом, SVG/PNG и Typst-дубликатом.

Источники

- Egger M. et al. Bias in Meta-Analysis Detected by a Simple, Graphical Test // *BMJ*. 1997. — doi:10.1136/bmj.315.7109.629
- Sterne J. A. C. et al. Recommendations for Examining and Interpreting Funnel Plot Asymmetry // *BMJ*. 2011. — doi:10.1136/bmj.d4002

6. Заключение

Как собрать всё вместе? Тридцать три раздела с графиками — это не список того, что «надо нарисовать». Это тренажёр одного навыка: **переводить научный вопрос в честную визуальную форму, которую можно проверить и воспроизвести**.

Здесь собраны принципы, алгоритм, таблица выбора, типичные ошибки и чек-листы, которые связывают все предыдущие главы в один рабочий процесс.

6.1 Семь принципов, которые остаются после всех графиков

1. Сначала вопрос, потом форма

График начинается не с `plt.plot`, а с предложения: «что читатель должен понять?» Если предложение нельзя сформулировать, рисунок ещё не готов. Форма (линия, точки, столбцы) следует из вопроса, а не из привычки.

Проверьте: назовите рисунок одним предложением с глаголом («Эффект насыщается после 20 мг»), а не существительным («Доза и эффект»).

2. Честность важнее красоты

Ось с нулём там, где длина кодирует величину; полный диапазон там, где обрезание искажает; площадь только там, где она пропорциональна величине. Красивый график, который врёт, — плохой график.

Проверьте: закройте подписи и спросите, можно ли по длине, площади или положению восстановить числа.

3. Данные важнее декора

Точки, распределения и неопределённость важнее теней, 3D и радуги. Если элемент не кодирует данные и не помогает чтению, его нужно удалить.

Проверьте: уберите элемент. Если смысл не изменился — элемент лишний.

4. Неопределённость — часть результата

Среднее без разброса, кривая без полосы, доля без интервала — это не «упрощение», а потеря информации. Ошибка оценки и разброс наблюдений — разные вещи; их нельзя подменять друг другом.

Проверьте: покажите, где на рисунке видно, насколько оценка точна.

5. Читаемость — это единицы, n и прямой путь

Единицы на осях, *n* в подписи, прямые подписи вместо легенды, где это возможно; десятичная запятая в русском тексте; шрифт, который не надо увеличивать.

Проверьте: посмотрите на рисунок 5 секунд. Понятны ли единицы, группы и направление эффекта?

6. Доступность — не «бонус», а условие

Палитра, различимая при дальтонизме; контраст; отсутствие кодирования только цветом; читаемость в градациях серого; текст, не наложенный на данные.

Проверьте: сделайте рисунок чёрно-белым. Осталась ли структура?

7. Воспроизводимость — критерий качества

Данные, скрипт, seed, версии пакетов, SVG и PNG, одинаковый результат при повторном запуске. Если рисунок нельзя воспроизвести, его нельзя проверить.

Проверьте: удалите `images/` и запустите генерацию снова. Получились ли те же файлы?

6.2 Универсальный алгоритм: двенадцать шагов

Шаг	Действие	Результат	Проверка
1	Сформулировать вопрос одним предложением	черновик заголовка	есть глагол и направление
2	Определить тип сравнения	trend / distribution / association / composition / effect	не более одного типа на рисунок
3	Проверить данные	n, пропуски, единицы, выбросы	нет «сырых» единиц
4	Выбрать семейство графика	раздел 5 по таблице 6.2	форма следует из вопроса

Шаг	Действие	Результат	Проверка
5	Сделать эскиз на бумаге	3–5 прямоугольников	понятно без цвета
6	Посчитать статистику	медиана/IQR, доля/CI, OR/HR	тип неопределённости назван
7	Написать код слоями	данные → стиль → график	данные не в литералах
8	Сравнить с плохим примером	список ошибок	ни одна не осталась
9	Проверить в конечном размере	85/120/170 мм	текст не меньше 6–7 pt
10	Проверить доступность	CVD, серый, контраст	смысл не только в цвете
11	Экспортировать SVG + PNG	вектор + растр	SVG без растровых вставок
12	Написать подпись и методы	что, n, статистика, ПО	подпись не повторяет заголовок

Обычно нужны три подхода. Первая итерация — «чтобы было». Вторая — «чтобы читалось». Третья — «чтобы нельзя было придраться». Если рисунок не прошёл третью, он ещё не готов.

6.3 Таблица выбора: задача → раздел

Задача	График	Раздел
Динамика одной или нескольких серий	line	5.01
События/дозы меняются скачком	step	5.02
Площадь — это интеграл (AUC, экспозиция)	area	5.03
Много групп во времени	small multiples	5.04
Связь двух непрерывных величин	scatter	5.05
Тысячи точек, плотность	hexbin	5.06
Сглаженная двумерная плотность	contour	5.07
Нелинейный тренд без формы	LOESS	5.08
Распределение одной величины	histogram	5.09
Точные квантили	ECDF	5.10
Сравнение 2–5 групп	box / violin / strip	5.11–5.13
Оценка и её точность	dot + CI	5.14
Абсолютные величины по категориям	bar	5.15
Эффект по подгруппам	forest	5.16
Изменение между двумя точками (агрегаты)	slope	5.17
Индивидуальные пары до/после	paired dot	5.18
Согласие двух методов	Bland–Altman	5.19
Состав целого	stacked / 100 % / pie / treemap	5.20–5.23
Матрица значений	heatmap	5.24
Матрица с кластерами	clustermap	5.25
Корреляции многих переменных	corr matrix	5.26
Выживаемость с цензурированием	Kaplan–Meier	5.27
Конкурирующие риски	cumulative incidence	5.28
Диагностическая способность	ROC	5.29
Редкий исход	precision–recall	5.30
Соответствие прогноза и реальности	calibration	5.31
Много центров/оценок	caterpillar	5.32
Асимметрия малых исследований	funnel	5.33

Таблица 6.2 — не догма. Один и тот же вопрос иногда допускает два графика; выбирайте тот, который **честнее отвечает на вопрос**, а не тот, который красивее выглядит в презентации.

6.4 Двенадцать типичных ошибок и противоядия

Ошибка	Почему плохо	Противоядие	Где
Обрезанная ось у столбцов/площадей	длина/площадь не пропорциональны	нулевая база или объяснение	1, 5.03, 5.15
`jet`/радуга для непрерывной шкалы	ложные границы, CVD	последовательная/дивергентная палитра	3, 5.06, 5.24
Столбцы для распределения	скрывают форму и выбросы	box/violin/strip/histogram	5.11–5.13
Pie для 8+ категорий	углы не сравнить	bar/100 % stacked/treemap	5.20–5.23
Среднее без разброса	нет неопределённости	SD/CI/точки	5.11, 5.14
SD вместо CI	не та неопределённость	Wilson/t/бутстрэп	5.14, 5.19
Тысячи непрозрачных точек	overplotting	hexbin/contour/alpha	5.05–5.07
Линейная ось для OR/RR/HR	асимметрия отношений	лог-шкала	5.16
1 − KM при конкурирующих рисках	завышенный риск	cumulative incidence	5.28
ROC при редком исходе	оптимистичная картина	precision–recall	5.30
Цвет — единственный носитель смысла	недоступно	форма/подпись/паттерн	3, 5.13
Нет единиц, n и подписи	невозможно прочитать	чек-лист 6.5	все

6.5 Чек-лист перед отправкой

Содержание

- ☐ Вопрос сформулирован одним предложением.
- ☐ На рисунке ровно одно сообщение.
- ☐ Сравнение корректно: группы, парные наблюдения, повторные измерения.
- ☐ Указаны *n* и единицы.
- ☐ Показана неопределённость (CI/SD/точки).
- ☐ Подпись объясняет статистику и метод.

Форма

- ☐ Нулевая база там, где длина/площадь кодирует величину.
- ☐ Оси не обрезают данные без объяснения.
- ☐ Палитра соответствует типу данных.
- ☐ Сетка не конкурирует с данными.
- ☐ Нет 3D, теней и радуги без смысла.
- ☐ Прямые подписи вместо легенды, где возможно.

Типографика и размер

- ☐ Рисунок проверен в конечном размере (85/120/170 мм).
- ☐ Минимальный кегль 6–7 pt.
- ☐ Нет наложений и обрезанных подписей.
- ☐ Десятичная запятая, единицы через пробел.
- ☐ Шрифт совпадает с требованием журнала.

Доступность

- ☐ Различимо при протанопии/дейтеранопии.
- ☐ Читаемо в градациях серого.
- ☐ Контраст текста и фона достаточен.
- ☐ Смысл не только в цвете.
- ☐ Есть alt-text/описание для публикации.

Воспроизводимость

- ☐ Данные лежат в `data/`, а не в коде.
- ☐ Скрипт запускается с чистого окружения.
- ☐ Seed зафиксирован.
- ☐ SVG и PNG сохранены.
- ☐ Повторный запуск даёт те же файлы.
- ☐ Версии пакетов зафиксированы (`uv.lock`).

6.6 Кто и что проверяет

Роль	**Что проверяет**	**Артефакт**
Автор	вопрос, данные, код, подпись	скрипт + черновик
Статистик	тест, CI, поправки, модель	таблица/методы
Дизайнер/редактор	сетка, шрифт, цвет, поля	PDF/PPT
Читатель	понятность за 5 секунд	финальный рисунок
Рецензент	соответствие журналу/гайдлайну	submission

Разделение простое: автор не должен быть единственным проверяющим. Свежий взгляд ловит то, что автор уже не видит.

6.7 Что автоматизировать, а что оставить человеку

Автоматизировать	**Оставить вручную**
генерацию данных и рисунков	выбор графика
единый стиль и палитры	формулировку сообщения
экспорт SVG/PNG	подпись и методы
проверку `ruff`	клиническую правдоподобность
детерминированность	доступность и alt-text
сборку PDF	финальную визуальную проверку
проверку ссылок	интерпретацию данных

Автоматизация не заменяет суждение. Она освобождает время для суждения.

6.8 Пять секунд на проверку

Покажите рисунок коллеге на пять секунд и спросите:

1. Что здесь главное?
2. Что с чем сравнивают?
3. В каких единицах?
4. Где неопределённость?
5. Что можно убрать?

Если ответы расходятся с задуманным — проблема не в зрителе.

6.9 Мини-экзамен: десять ситуаций

Ситуация	Что выбрать	Раздел
30 больниц, доля осложнений	caterpillar + CI	5.32
5 групп, распределение	box + точки / violin	5.11–5.12
Два метода измерения	Bland–Altman	5.19
Редкий исход (5 %)	precision–recall	5.30
Эффект по подгруппам	forest	5.16
Конкурирующие риски	cumulative incidence	5.28
20 000 точек	hexbin/contour	5.06–5.07
До/после у 25 пациентов	paired dot	5.18
4 доли целого	100 % stacked bar	5.21
Прогноз риска	calibration	5.31

6.10 Пример: «различаются ли центры по осложнениям?»

Разберём один путь целиком — без нового датасета, на примере 5.14.

Вопрос. Различается ли доля осложнений между шестью центрами, и насколько эти различия точны?

Данные. 600 пациентов, по 100 на центр; 88 осложнений; общая доля — 14,7 %. Для каждой доли нужен 95 % ДИ Уилсона.

Форма. Dot + CI, а не столбцы: сравниваются оценки с разной точностью, а не абсолютные объёмы. Столбцы с SD здесь дали бы ложное чувство точности.

Первая версия (плохая). Столбцы/точки с SD, обрезанная ось 0–15 %, радуга, без общей доли и n. Ошибки: SD вместо CI, скрытые центры, декор.

Финальная версия. Точки с 95 % ДИ Уилсона, сортировка по доле, общая линия 14,7 %, один цвет, n = 100. Интервалы пересекаются — значит, ранжировать центры без формального сравнения нельзя.

Подпись.

> Рисунок. Доля осложнений по шести центрам (n = 100 в каждом). Точки — оценки, усы — 95 % ДИ Уилсона; пунктир — общая доля 14,7 %. Интервалы пересекаются, поэтому различия между центрами не следует интерпретировать как рейтинг.

Урок. Один и тот же вопрос допускает разные формы; правильная форма показывает не только «сколько», но и «насколько точно».

6.11 Что дальше

Раздел 7 — подготовка рисунков к публикации: размеры, DPI, шрифты, CMYK, поля, требования журналов, права на изображения и сопроводительные документы. Раздел 8 — источники и программное обеспечение.

6.12 Коротко о главном

Качественный график = вопрос × честные данные × адекватная форма × неопределённость × читаемость × воспроизводимость.

Если хотя бы один множитель равен нулю, результат равен нулю. Красота — не отдельный множитель: она возникает как следствие честности и ясности.

Источники

- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz
- Cairo A. *The Truthful Art*. New Riders, 2016. — thefunctionalart.com
- Weissgerber T. L. et al. Beyond Bar and Line Graphs // *PLoS Biology*. 2015. — doi:10.1371/journal.pbio.1002128
- Cleveland W. S., McGill R. Graphical Perception // *JASA*. 1984. — doi:10.1080/01621459.1984.10478080

7. Публикация

Публикация — это не «сдать картинку», а передать редакции **воспроизводимый, соответствующий требованиям и однозначно читаемый** файл. Большинство отказов по рисункам связаны не с наукой, а с форматом: не та ширина, мелкий шрифт, RGB вместо CMYK, растр вместо вектора, отсутствие *n* и единиц.

Ниже — собственный датасет и журнальный рисунок, таблица настроек, модуль `journal_style.py`, интерактивный генератор темы, экспорт в SVG/PDF/PNG/TIFF и настройки для web/HTML.

7.1 Что требует журнал

Требования различаются, но проверять нужно одни и те же пункты:

- **физический размер** — ширина одной/двух колонок в мм;
- **шрифт** — семейство, кегль, встраивание/кривые;
- **разрешение** — 300–600 dpi для растра, вектор для линий;
- **формат** — PDF/SVG/EPS, TIFF, PNG; иногда только TIFF;
- **цвет** — RGB для онлайн, CMYK по запросу; sRGB;
- **читаемость** — 6–7 pt минимум, контраст, CVD;
- **подпись и методы** — что, *n*, статистика, ПО;
- **права** — лицензии на изображения и шрифты.

На практике: сначала читайте `author guidelines` журнала, затем используйте таблицу 7.1 как отправную точку. Не наоборот.

7.2 Данные и журнальный рисунок

Собственный датасет: 90 пациентов, три группы, САД на 0/4/8/12-й неделе. Генератор — `07_publication/scripts/make_data.py` → `data/07_publication_dataset.csv`.

Рисунок 7.1 — двухпанельный, 170 × 95 мм: A — средние САД с 95 % CI, B — изменение к 12-й неделе с 95 % CI. Шрифт 7 pt, панели A/B, прямые подписи, без заголовка внутри рисунка: сообщение несёт подпись.

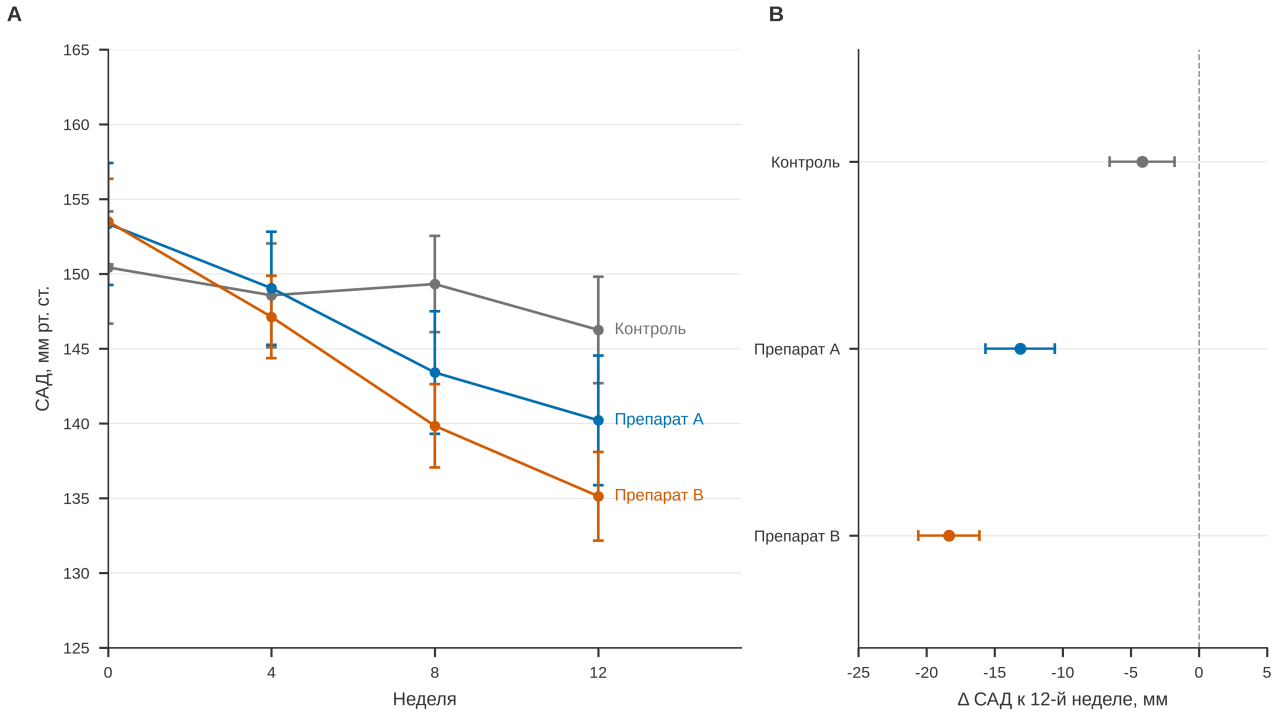


Рисунок 7.1. Журнальный рисунок 170 × 95 мм: панели A и B, 7 pt, 95 % CI, прямые подписи.

Решения, которые делают его «журнальным»:

1. **Точный размер 170 × 95 мм** — две колонки, без `bbox_inches="tight"`.
2. **Шрифт 7 pt** — читается в печати; подписи не меньше 6 pt.
3. **Две панели** с метками A/B; каждая отвечает на свой вопрос.
4. **95 % CI**, а не SD: показана точность среднего.
5. **Прямые подписи** вместо легенды в панели A.

6. **Цвета Okabe–Ito**; смысл не только в цвете.
7. **Нет заголовка** внутри рисунка — заголовок и вывод в подписи.
8. **Единицы и недели** на осях; $n = 30$ на группу — в подписи.

Пример подписи:

Рисунок 7.1. САД по группам за 12 недель. (A) Среднее ± 95 % ДИ; (B) изменение к 12-й неделе, среднее и 95 % ДИ. $n = 30$ на группу. CI рассчитаны как 1,96 × SEM (A) и t-интервал (B). Горизонтальная линия в B — отсутствие изменения. Данные: синтетический набор `07_publication_dataset.csv`; код: `plot_journal.py`.

7.3 Таблица рекомендованных настроек

Параметр	Печать / журнал	Web / HTML	Комментарий
Ширина	89 мм одна, 183 мм две	640–1200 px	проверьте колонки журнала
Поля	0, точный размер	CSS padding	не обрезайте `tight`
Шрифт	Arial/Helvetica, 6–8 pt	14–18 px	встройте или в кривые
Кегль осей	6–7 pt	14–16 px	минимум 6 pt
Линии	0,5–1,0 pt	1,5–2 px	тоньше — пропадут в печати
Маркеры	2–4 pt	6–8 px	не меньше 2 pt
Сетка	светлая, 0,4 pt	`#E6E6E6`, 1 px	не конкурирует
Разрешение	300–600 dpi	96/192 dpi (1x/2x)	вектор лучше растра
Форматы	PDF/SVG/EPS, TIFF	SVG/PNG/WebP	TIFF LZW для растра
Цвет	RGB, CMYK по запросу	sRGB	проверьте профиль
Фон	белый	белый/прозрачный	печать — белый
Заголовок	вне рисунка	HTML `h2`	подпись несёт вывод
Alt-text	в подписи/системе	`alt=""`	обязателен для web
Метаданные	Creator, без даты	—	воспроизводимость

7.4 journal_style.py: пресеты и экспорт

Модуль `07_publication/scripts/journal_style.py` содержит пресеты `nature`, `elsevier`, `ieee`, `web`, функцию `apply_journal_style()` и экспорт `export_journal()`.

```
from journal_style import apply_journal_style, export_journal

apply_journal_style("nature", base_pt=7)       # rcParams
fig, ax = plt.subplots(figsize=(170 / 25.4, 95 / 25.4))
...
export_journal(fig, "07_journal_figure",
               formats=("svg", "pdf", "png", "tiff"), dpi=600)
```

На что смотреть в экспорте:

- `bbox_inches=None` — физический размер остаётся ровно 170 мм;
- `svg.fonttype="path"` — текст в SVG не зависит от шрифтов у редакции;
- `pdf.fonttype=42` — TrueType, текст остаётся редактируемым;
- TIFF сохраняется с LZW-сжатием и 600 dpi;
- метаданные не содержат текущую дату, поэтому файлы воспроизводимы.

CMYK. Matplotlib работает в RGB. Если журнал требует CMYK, конвертируйте PDF/TIFF в Illustrator/Inkscape/Ghostscript и проверьте чёрный текст (не rich black) и предупреждения о цветовом профиле.

7.5 Генератор настроек в диалоге

Скрипт `07_publication/scripts/make_journal_style.py` задаёт вопросы и собирает готовый модуль темы.

```
uv run python 07_publication/scripts/make_journal_style.py --interactive
```

Диалог (Enter — значение по умолчанию):

```
Пресет [nature]: elsevier
Ширина одной колонки, мм [90.0]: 90
Ширина двух колонок, мм [190.0]: 190
Шрифт [Arial]: Helvetica
Базовый кегль, pt [8.0]: 8
DPI для растра [600]: 600
Форматы через запятую [svg,pdf,png,tiff]: pdf,tiff
Палитра (okabe/tol/grey) [okabe]: okabe
Ширина web-канвы, px [960]: 1200
Базовый кегль web, px [16]: 16
```

Результат: `build/journal_style_generated.py` и `.json`. Модуль можно импортировать:

```
from journal_style_generated import apply
apply()
```

Без диалога:

```
uv run python 07_publication/scripts/make_journal_style.py \
    --preset nature --output build/journal_style_generated.py
```

Сгенерированный файл — черновик. Сверьте ширину, шрифт и цвет с актуальными требованиями журнала.

7.6 Форматы: что и когда отправлять

Формат	Когда	Плюсы	Минусы
SVG	web, вектор	масштаб, малый размер	текст в кривых, нет CMYK
PDF	печать, вектор	шрифты, точный размер	нужен контроль профиля
TIFF	требование журнала	растр без потерь, LZW	большой файл
PNG	web/превью	прозрачность, точный px	не для печати линий
EPS	legacy	вектор	устаревает, шрифты
WebP	web	малый размер	не для журнала

Не отправляйте скриншот, PNG из PowerPoint и растр с 72 dpi. Векторные линии и текст должны оставаться векторными; растр — только для фото/полутонов.

7.7 Web/HTML: только настройки

Никаких интерактивных панелей — только параметры статического рисунка.

- **Канва:** 960 или 1200 px; высота 480–720 px.
- **Кегль:** 16 px база, 14 px минимум; 1x/2x.
- **Формат:** SVG для линий, PNG/WebP для растра; sRGB.
- **CSS:** `img { max-width: 100%; height: auto; }`.
- **Retina:** `srcset="fig.png 1x, fig@2x.png 2x"`.
- **Alt-text:** `alt="САД снижается..."`; не «рисунок 1».
- **Контраст:** ≥ 4,5:1 для текста; не только цвет.
- **Тёмная тема:** проверьте фон/линии; лучше отдавать светлый вариант.
- **Шрифты:** системный стек `Inter, Arial, sans-serif`.

```
<img src="07_journal_web.png"
     srcset="07_journal_web.png 1x, 07_journal_web@2x.png 2x"
     width="960" height="480"
     alt="САД снижается сильнее в группе препарата B к 12-й неделе"
     style="max-width:100%;height:auto">
```

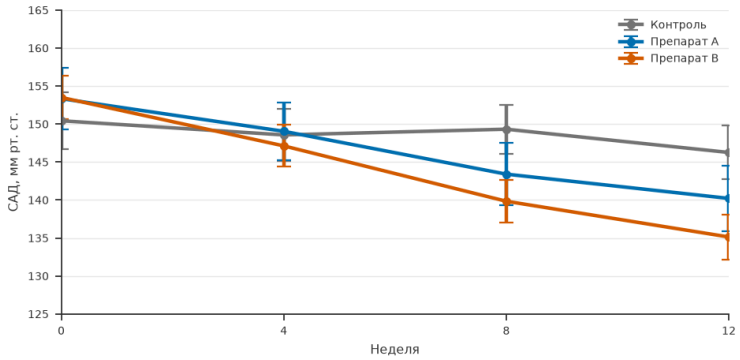


Рисунок 7.2. Web-версия 960 × 480 px: 16 px база, sRGB, PNG 1x/2x.

7.8 Подпись, методы, supplementary

Подпись должна позволять понять рисунок без текста статьи:

1. что показано;
2. *n* и единицы;
3. статистика/CI;
4. обозначения;
5. ПО и версия;
6. источник данных/код.

В методах укажите: matplotlib/seaborn/plotnine, версии, шрифт, ПО для конвертации CMYK, разрешение и формат.

Supplementary figures оформляйте тем же стилем, с нумерацией S1, S2…, отдельными файлами и alt-text.

7.9 Чек-лист перед отправкой

Размер и шрифты

- ☐ Ширина совпадает с колонкой журнала (мм).
- ☐ Кегль ≥ 6–7 pt; подписи не обрезаны.
- ☐ Шрифт встроен/в кривых; кириллица отображается.
- ☐ Нет `bbox_inches="tight"`, если нужен точный размер.

Цвет и печать

- ☐ RGB или CMYK по требованию.
- ☐ Проверка в градациях серого.
- ☐ CVD-безопасная палитра.
- ☐ Нет `jet` и 3D.

Форматы

- ☐ Вектор (PDF/SVG) для линий и текста.
- ☐ TIFF/PNG 300–600 dpi для растра.
- ☐ TIFF LZW, без потерь.
- ☐ Имена файлов по схеме журнала.

Содержание

- ☐ *n*, единицы, статистика.
- ☐ Панели A/B подписаны.
- ☐ Подпись не повторяет заголовок.
- ☐ Методы и ПО указаны.

Права и доступность

- ☐ Лицензии на изображения/шрифты.
- ☐ Alt-text для web.
- ☐ Контраст и читаемость.
- ☐ Нет информации только в цвете.

7.10 Частые причины отказа

Причина	Как исправить
неверная ширина	пересобрать в 89/183 мм, без tight
мелкий шрифт	7 pt минимум, проверить печать
растр вместо вектора	PDF/SVG для линий
72 dpi	300–600 dpi для растра
RGB вместо CMYK	конвертировать и проверить профиль
текст в растре	`pdf.fonttype=42`, SVG path
обрезанные оси	полный диапазон/объяснение
нет n и единиц	добавить в подпись/оси
цвет только	форма/подпись/паттерн
нет прав	лицензия/разрешение

7.11 Что дальше

Раздел 8 — источники: книги, статьи, руководства журналов и инструменты, на которых построена методичка.

Источники

- Nature Portfolio. Final submission guidelines — figures. — nature.com
- Elsevier. Artwork and media instructions. — elsevier.com
- IEEE. Author guidelines — graphics. — ieee.org
- Matplotlib: `savefig` — matplotlib.org
- Tufte E. R. *The Visual Display of Quantitative Information*. 1983. — edwardtufte.com
- Wilke C. O. *Fundamentals of Data Visualization*. 2019. — clauswilke.com/dataviz

8. Источники

Это скорее карта, чем библиография: что почитать, чем пользоваться и где сверять требования. Часть источников далека от Python — типографика, шрифты, генераторы диаграмм, экспорт, — но именно от них часто зависит, будет ли рисунок выглядеть профессионально.

8.1 Как пользоваться списком

1. **Сначала первоисточник.** Для требований журнала — `author guidelines`, для статистики — статья/учебник, для инструмента — официальная документация.
2. **Потом обзор.** Книги и руководства помогают выбрать метод, но не заменяют проверку актуальных требований.
3. **Ссылки стареют.** Проверяйте дату доступа и версию; для ПО указывайте версию в методах.
4. **Не цитируйте блог вместо стандарта.** Блог может объяснять, но не должен быть единственным основанием для норматива.

8.2 Визуализация и восприятие

- Tufte E. R. *The Visual Display of Quantitative Information*. Graphics Press, 1983. — edwardtufte.com. Классика про «чернила», пропорции и графическую честность.
- Tufte E. R. *Envisioning Information*. Graphics Press, 1990. — edwardtufte.com. Плотность, слои, малые кратные.
- Wilke C. O. *Fundamentals of Data Visualization*. O’Reilly, 2019. — clauswilke.com/dataviz. Лучший современный учебник по выбору формы.
- Cairo A. *The Truthful Art*. New Riders, 2016. — thefunctionalart.com. Честность, неопределённость, дизайн.
- Munzner T. *Visualization Analysis and Design*. CRC Press, 2014. — cs.ubc.ca. Формальная рамка «данные — задачи — кодирование».
- Ware C. *Information Visualization: Perception for Design*. 4th ed., Morgan Kaufmann, 2020. — elsevier.com. Почему глаз видит одно, а не другое.
- Cleveland W. S., McGill R. Graphical Perception // *JASA*. 1984. — doi:10.1080/01621459.1984.10478080. Экспериментальная база «положение лучше длины, длины лучше угла».
- Few S. *Show Me the Numbers*. Analytics Press, 2012. — perceptualedge.com. Деловая графика и таблицы.
- Kirk A. *Data Visualisation: A Handbook for Data Driven Design*. SAGE, 2019. — visualisingdata.com. Процесс дизайна, а не только код.
- Schwabish J. *Better Data Visualizations*. Columbia UP, 2021. — policyviz.com. Практические паттерны для отчётов и статей.
- Knaflic C. N. *Storytelling with Data*. Wiley, 2015. — storytellingwithdata.com. Контекст, нарратив, презентация.
- Bergstrom C. T., West J. D. *Calling Bullshit*. Random House, 2020. — callingbullshit.org. Как не обмануться графиком и статистикой.

8.3 Статистическая графика и отчёты

- Weissgerber T. L. et al. Beyond Bar and Line Graphs // *PLoS Biology*. 2015. — doi:10.1371/journal.pbio.1002128. Почему точки и распределения лучше столбцов.
- Cumming G. *Understanding the New Statistics*. Routledge, 2012. — routledge.com. Эффекты, CI, мета-анализ.
- Bland J. M., Altman D. G. Statistical Methods for Assessing Agreement // *The Lancet*. 1986. — doi:10.1016/S0140-6736(86)90837-8. Bland–Altman.
- Fawcett T. An Introduction to ROC Analysis // *Pattern Recognition Letters*. 2006. — doi:10.1016/j.patrec.2005.10.010. ROC, AUC, пороги.
- Saito T., Rehmsmeier M. The Precision-Recall Plot Is More Informative than the ROC Plot // *PLoS ONE*. 2015. — doi:10.1371/journal.pone.0118432. Редкие исходы.
- Van Calster B. et al. Calibration: the Achilles Heel of Predictive Analytics // *BMC Medicine*. 2019. — doi:10.1186/s12916-019-1466-7. Калибровка прогноза.
- Steyerberg E. W. *Clinical Prediction Models*. 2nd ed., Springer, 2019. — springer.com. Прогностические модели, валидация, калибровка.
- Harrell F. E. *Regression Modeling Strategies*. 2nd ed., Springer, 2015. — hbiostat.org. Регрессия, номограммы, оценка моделей.

- Kaplan E. L., Meier P. Nonparametric Estimation from Incomplete Observations // *JASA*. 1958. — doi:10.1080/01621459.1958.10501452. KM-оценка.
- Fine J. P., Gray R. J. A Proportional Hazards Model for the Subdistribution of a Competing Risk // *JASA*. 1999. — doi:10.1080/01621459.1999.10474144. Конкурирующие риски.
- Cochrane Handbook for Systematic Reviews of Interventions. — training.cochrane.org. Мета-анализ, forest/funnel, гетерогенность.
- Spiegelhalter D. *The Art of Statistics*. Pelican, 2019. — penguin.co.uk. Как думать о данных, а не только рисовать.

8.4 Цвет, доступность, восприятие

- Okabe M., Ito K. Color Universal Design (CUD). 2008. — jfly.uni-koeln.de/color. Палитра, различимая при CVD.
- ColorBrewer 2.0. — colorbrewer2.org. Качественные, последовательные, дивергентные шкалы.
- Tol P. Colour Schemes. — personal.sron.nl/~pault. Наборы bright/muted, проверенные на различимость.
- Smith N., van der Walt S. Viridis. — bids.github.io/colormap. Воспринимательно равномерная шкала.
- Crameri F. Scientific Colour Maps. — fabiocrameri.ch/colourmaps. Научные палитры, включая CVD-безопасные.
- W3C. Web Content Accessibility Guidelines (WCAG) 2.2. — w3.org/TR/WCAG22. Контраст, не-цветовые подсказки.
- WebAIM Contrast Checker. — webaim.org/resources/contrastchecker. Быстрая проверка контраста.
- Coblis Color Blindness Simulator. — color-blindness.com/coblis. Симуляция CVD.
- Color Oracle. — colororacle.org. Симулятор для рабочего стола.
- Chroma.js. — gka.github.io/chroma.js. Генератор цветовых шкал и интерполяций.
- Viz Palette. — projects.susielu.com/viz-palette. Проверка палитры для диаграмм.
- Coolors. — coolors.co. Быстрые палитры (проверяйте CVD вручную).
- Adobe Color. — color.adobe.com. Цветовые круги и гармонии.
- Colorgorical. — colorgorical.com. Генератор качественных палитр с учётом различимости.

8.5 Типографика и шрифты

- Bringhurst R. *The Elements of Typographic Style*. 4th ed., Hartley & Marks, 2012. — hartleyandmarks.com. Ритм, пропорции, детали набора.
- Butterick M. *Practical Typography*. — practicaltypography.com. Коротко и по делу: кегль, интерлиньяж, ширина строки.
- Lupton E. *Thinking with Type*. 2nd ed., Princeton Architectural Press, 2010. — thinkingwithtype.com. Типографическая система.
- Hochuli J. *Detail in Typography*. 2nd ed., Niggli, 2015. — niggli.ch. Микротипографика: кернинг, трекинг, переносы.
- Ruder E. *Typographie*. Niggli, 1967. — niggli.ch. Швейцарская школа, сетки.
- Gill E. *An Essay on Typography*. Godine, 1988. — godine.com. Классическое эссе о ремесле.
- Tschichold J. *The Form of the Book*. Hartley & Marks, 1991. — oakknoll.com. Книжная типографика и пропорции.
- Google Fonts. — fonts.google.com. Свободные шрифты с лицензиями.
- SIL Open Font License. — openfontlicense.org. Что можно делать со шрифтом.
- Font Squirrel. — fontsquirrel.com. Отборные freeware-шрифты.
- Fonts In Use. — fontsinuse.com. Примеры реальных пар и контекстов.
- Inter. — rsms.me/inter. Нейтральный UI/графический шрифт.
- IBM Plex. — ibm.com/plex. Корпоративная система с хорошей кириллицей.
- Atkinson Hyperlegible. — brailleinstitute.org/freefont. Повышенная различимость букв.
- Source Sans 3. — adobe-fonts.github.io/source-sans. Спокойный гротеск для интерфейсов и figure labels.
- Liberation Fonts. — github.com/liberationfonts. Метрические аналоги Arial/Times/Courier.
- DejaVu Fonts. — dejavu-fonts.github.io. Широкое покрытие символов.
- Fontjoy. — fontjoy.com. Генератор пар шрифтов (проверяйте вручную).
- Typescale. — typescale.com. Модульная шкала кеглей.
- Utopia. — utopia.fyi. Fluid type для web.
- Modular Scale. — modularscale.com. Пропорции типографики.

Таблица 8.1. Короткий список шрифтов

Шрифт	Где уместен	Лицензия	Почему
Inter	web, UI, подписи	OFL	нейтральный, хорошая кириллица
IBM Plex Sans	отчёты, web	OFL	корпоративный, полный набор
Source Sans 3	figure labels	OFL	спокойный, экономичный
Atkinson Hyperlegible	доступность	free	различимые буквы
Liberation Sans	замена Arial	OFL	метрики Arial
DejaVu Sans	символы, LaTeX	free	широкое покрытие
Noto Sans	мультиязычность	OFL	много письменностей
Libertinus Serif	тело, Typst	OFL	книжная антиква
Arial/Helvetica	журнальный стандарт	proprietary	привычность, но лицензия

8.6 Python и библиотеки

- Matplotlib. — matplotlib.org. Базовая графика, `savefig`, rcParams.
- Seaborn. — seaborn.pydata.org. Статистические графики, `clustermap`, `violinplot`.
- Plotnine. — plotnine.readthedocs.io. Grammar of graphics в Python.
- pandas. — pandas.pydata.org. Данные, группировки, пропуски.
- NumPy. — numpy.org. Массивы, интегрирование, seed.
- SciPy. — scipy.org. Статистика, t-интервалы, оптимизация.
- statsmodels. — statsmodels.org. LOWESS, регрессия, диагностика.
- squarify. — github.com/laserson/squarify. Treemap.
- adjustText. — github.com/Phlya/adjustText. Разведение подписей.
- Pillow. — python-pillow.org. TIFF/PNG, компрессия.
- colour-science. — colour-science.org. Цветовые пространства, конверсии.
- colorspacious. — colorspacious.readthedocs.io. CVD-симуляция и перцептивные пространства.
- palettable. — jiffyclub.github.io/palettable. ColorBrewer/Tol в Python.
- cmcrameri. — github.com/callumrollo/cmcrameri. Scientific colour maps в matplotlib.
- lifelines. — lifelines.readthedocs.io. Kaplan–Meier, Cox, log-rank.
- scikit-learn. — scikit-learn.org. ROC/PR, калибровка, модели.
- SHAP. — shap.readthedocs.io. Интерпретация моделей.
- ArviZ. — python.arviz.org. Байесовская диагностика.
- Plotly. — plotly.com/python. Интерактив и web-экспорт.
- Altair. — altair-viz.github.io. Vega-Lite из Python.
- Bokeh. — bokeh.org. Интерактивные web-графики.
- SciencePlots. — github.com/garrettj403/SciencePlots. Готовые journal-стили для matplotlib.
- brokenaxes. — github.com/bendichter/brokenaxes. Разрывы оси без обмана.
- matplotlib-scalebar. — github.com/ppinard/matplotlib-scalebar. Масштабные линейки для изображений.

8.7 Генераторы и low-code инструменты

- Datawrapper. — datawrapper.de. Быстрые публикационные графики и карты.
- Flourish. — flourish.studio. Инфографика и анимация.
- RAWGraphs. — rawgraphs.io. Много нестандартных типов из CSV.
- Tableau Public. — public.tableau.com. Исследовательская визуализация.
- Observable. — observablehq.com. Notebooks и D3/Plot.
- Vega-Lite. — vega.github.io/vega-lite. Декларативная грамматика.
- D3. — d3js.org. Полный контроль, высокая цена.
- Chart.js. — chartjs.org. Простые web-графики.
- ggplot2. — ggplot2.tidyverse.org. Эталон grammar of graphics.
- R Graph Gallery. — r-graph-gallery.com. Рецепты на R.
- Python Graph Gallery. — python-graph-gallery.com. Рецепты на Python.
- From Data to Viz. — data-to-viz.com. Дерево выбора графика.
- FT Visual Vocabulary. — github.com/Financial-Times/chart-doctor. «Какая задача — какой график».
- Data Viz Project. — datavizproject.com. Каталог форм.
- Quarto. — quarto.org. Документы и отчёты из кода.
- Typst. — typst.app. Вёрстка этой методички.
- PGFPlots. — pgfplots.sourceforge.net. Графики внутри LaTeX.

Таблица 8.2. Инструменты по задачам

Инструмент	Тип	Зачем
Datawrapper	online	быстрый график для публикации

Инструмент	Тип	Зачем
Flourish	online	инфографика и анимация
RAWGraphs	online	нестандартные типы из CSV
Observable Plot	JS	лаконичные web-графики
Vega-Lite	JS	декларативные спецификации
SciencePlots	Python	journal-стили matplotlib
ColorBrewer	palette	проверенные шкалы
Viz Palette	palette	проверка палитры
From Data to Viz	guide	выбор формы
Quarto	docs	отчёты и воспроизводимость

8.8 Публикация, экспорт, вектор

- Nature Portfolio. Final submission guidelines — figures. — nature.com.
- Elsevier. Artwork and media instructions. — elsevier.com.
- IEEE. Author guidelines — graphics. — journals.ieeeauthorcenter.ieee.org.
- ICMJE. Recommendations for the Conduct, Reporting, Editing, and Publication of Scholarly Work. — icmje.org.
- EQUATOR Network. — equator-network.org. Reporting guidelines.
- Inkscape. — inkscape.org. Векторная правка SVG/PDF.
- Adobe Illustrator. — adobe.com. CMYK, bleed, финальная правка.
- Affinity Designer. — affinity.serif.com. Альтернатива Illustrator.
- Scribus. — scribus.net. Открытая вёрстка.
- ImageMagick. — imagemagick.org. Пакетная обработка растров.
- Ghostscript. — ghostscript.com. PDF/PS, цветовые профили.
- qpdf. — qpdf.sourceforge.io. Проверка/починка PDF.
- veraPDF. — verapdf.org. Валидация PDF/A.
- ExifTool. — exiftool.org. Метаданные изображений.
- Poppler `pdffonts`. — poppler.freedesktop.org. Проверка встроенных шрифтов.
- SVGO. — github.com/svg/svgo. Оптимизация SVG.
- SVGOMG. — jakearchibald.github.io/svgomg. Визуальная оптимизация SVG.

8.9 Web-графика

- MDN. Responsive images. — developer.mozilla.org. `srcset`, `sizes`, `<picture>`.
- MDN. `<img>`. — developer.mozilla.org. `alt`, `width`, `height`, `loading`.
- web.dev. Learn Images. — web.dev/learn/images. Форматы, сжатие, responsive.
- Can I use. — caniuse.com. Поддержка WebP/AVIF/SVG.
- WCAG Quick Reference. — w3.org/WAI/WCAG22/quickref. Контраст и доступность.
- MDN. `prefers-color-scheme`. — developer.mozilla.org. Тёмная тема.
- Squoosh. — squoosh.app. Сжатие изображений.
- ImageOptim. — imageoptim.com. Оптимизация PNG/JPEG.
- Lighthouse. — developer.chrome.com. Производительность и доступность.
- axe. — deque.com/axe. Автоматическая проверка доступности.

8.10 Воспроизводимость и инфраструктура

- The Turing Way. — the-turing-way.netlify.app. Практики воспроизводимой науки.
- uv. — docs.astral.sh/uv. Окружение и lock-файл.
- Git. — git-scm.com. История и ревью.
- DVC. — dvc.org. Версии данных и моделей.
- Zenodo. — zenodo.org. DOI для данных/кода.
- Jupyter. — jupyter.org. Ноутбуки и исследование.
- Quarto. — quarto.org. Воспроизводимые документы.
- GNU Make. — gnu.org/software/make. Пайплайн «данные → рисунки → PDF».
- pytest. — docs.pytest.org. Тесты для кода и данных.
- Ruff. — docs.astral.sh/ruff. Линтер и форматтер.
- pre-commit. — pre-commit.com. Хуки перед коммитом.
- Docker. — docs.docker.com. Воспроизводимая среда.

8.11 Отчётные стандарты

- CONSORT. — consort-statement.org. РКИ.

- STROBE. — strobe-statement.org. Наблюдательные исследования.
- PRISMA. — prisma-statement.org. Систематические обзоры.
- SAMPL. — equator-network.org. Представление данных.
- TRIPOD. — tripod-statement.org. Прогностические модели.
- PROBAST. — probast.org. Риск смещения моделей.
- ICMJE. — icmje.org. Единые требования к рукописям.

8.12 Что не считать источником

- скриншоты чужих графиков без контекста;
- блоги без данных и методов;
- «красивые» галереи без проверки доступности;
- AI-ответы без первоисточника;
- устаревшие требования журнала;
- код без версии и лицензии.

Источник — это то, что можно проверить, датировать и процитировать.

8.13 Куда идти дальше

- **Выбор формы:** Wilke → From Data to Viz → раздел 5.
- **Статистика:** Cumming → Harrell/Steyerberg → раздел 6.
- **Цвет:** Okabe–Ito → ColorBrewer → WCAG.
- **Типографика:** Bringhurst → Practical Typography → шрифты.
- **Публикация:** guidelines журнала → раздел 7 → препресс.
- **Воспроизводимость:** The Turing Way → uv/Git/Make.

8.14 Как цитировать эту методичку

> GoodGraph. Методичка по научной графике на Python. Версия 0.1, 2026. Репозиторий: `GoodGraph`; сборка: `build/metodichka.pdf`.

Для ссылки на конкретный раздел указывайте номер и версию: например, «раздел 5.14, версия 0.1».

9. Приложения

R, Julia и электронные таблицы — краткий обзор и шесть графиков в каждой среде. Основная методичка построена на Python, но научная графика не начинается и не заканчивается им.

Чтобы сравнение было честным, все три примера используют **один и тот же учебный набор** `data/09_appendix_dataset.csv`: 150 пациентов, три группы, САД на 0/4/8/12-й неделе, возраст, BMI, глюкоза, время до события и категория нежелательного явления.

Это не учебники по R, Julia и Excel. Скорее — «карта входа»: какие шесть графиков там делаются естественно, где начинаются ограничения и что стоит автоматизировать, а что оставить руками.

A. Системы моделирования и анализа

Среда	Сильная сторона	Ключевые пакеты	Ограничения
R	статистика, Bioconductor, ggplot2, отчётные пакеты	ggplot2, dplyr, tidyr, survival, gridExtra, rmarkdown	синтаксис непривычен после Python; скорость на больших данных ниже
Julia	скорость, типы, дифференциальные уравнения, большие массивы	Plots, DataFrames, CSV, DifferentialEquations, Makie	экосистема меньше; предкомпиляция и версии требуют дисциплины
Excel / LibreOffice	быстрый взгляд, таблицы, ручная правка, отчёты	встроенные диаграммы, сводные таблицы, условное форматирование	воспроизводимость, сложные статистические графики, контроль версий
Python	автоматизация, ML, интеграция	matplotlib, seaborn, plotnine, pandas, statsmodels	требует кода и настройки окружения

R хорош там, где статистика уже живёт в R: модели, тесты, выживаемость, мета-анализ. `ggplot2` задаёт грамматику слоёв и облегчает faceting; `survival` и `gridExtra` закрывают KM и сборку панелей. Слабое место — воспроизводимость проектов: версии пакетов нужно фиксировать (`renv`), а не «у меня работает».

Julia выигрывает, когда важна скорость или математическая модель: дифференциальные уравнения, оптимизация, большие массивы. `Plots` даёт быстрый старт, `Makie` — более тонкий контроль. Плата — время на предкомпиляцию и более молодой набор пакетов; проект лучше держать в отдельном `Project.toml`/`Manifest.toml`.

Excel/LibreOffice незаменим для черновика, быстрой проверки и разговора с коллегами, которые не пишут код. Но как только нужны доверительные интервалы, KM, box plot с точками или единый стиль, таблица начинает проигрывать скрипту. Хорошая стратегия — считать в R/Julia/Python, а в Excel смотреть и обсуждать.

B. Шесть графиков в R

Скрипт: `09_appendices/r/six_plots.R`. Пакеты: `ggplot2`, `gridExtra`, `survival`. Запуск:

```
Rscript 09_appendices/r/six_plots.R
```

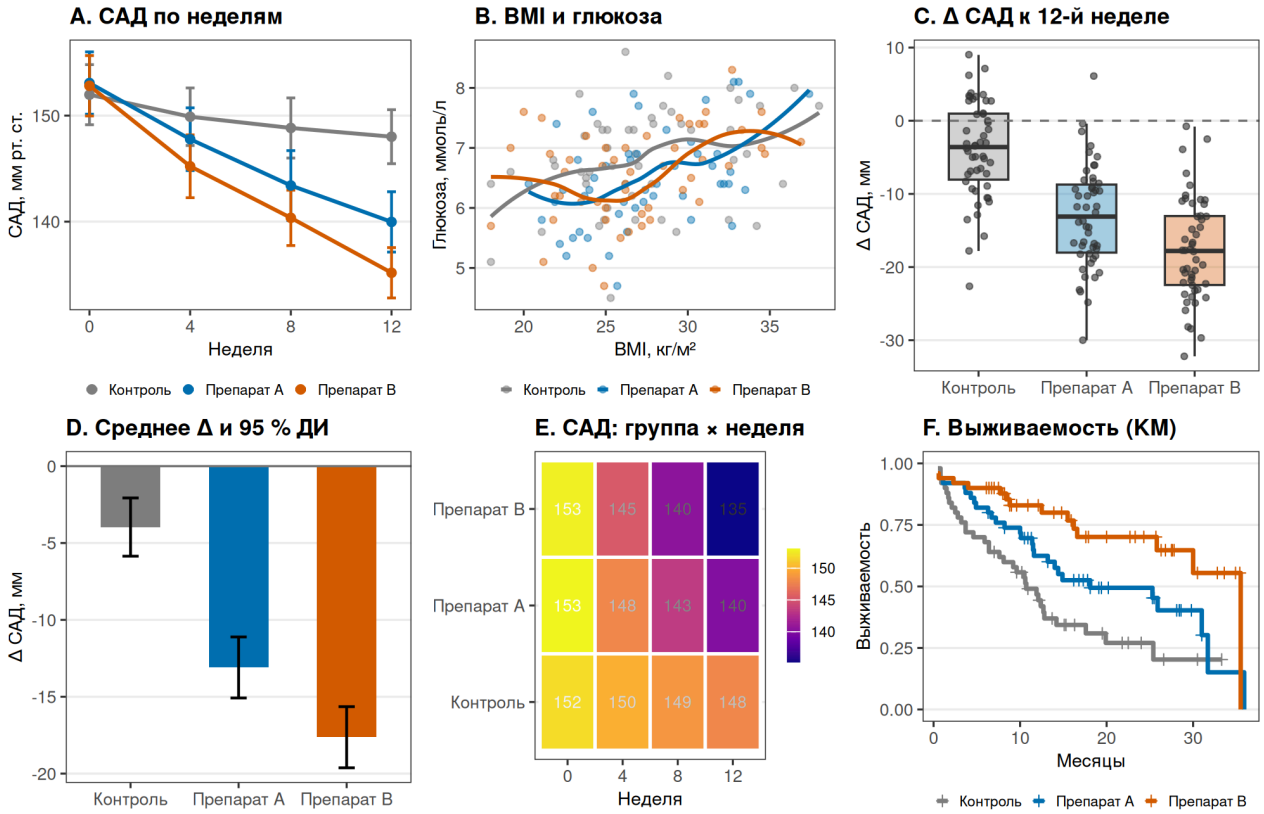


Рисунок 9.1. Шесть графиков в R: линия с 95 % CI, scatter с LOESS, box + точки, столбцы с CI, heatmap и Kaplan–Meier.

Что показано:

1. **Линия** — среднее САД по неделям с 95 % CI; группы различаются цветом.
2. **Scatter + LOESS** — BMI и глюкоза; точки полупрозрачны, шум не скрыт.
3. **Box + jitter** — индивидуальное изменение к 12-й неделе; видно разброс и выбросы.
4. **Столбцы с 95 % CI** — среднее изменение по группам, нулевая линия.
5. **Heatmap** — среднее САД по группе и неделе, значения в ячейках.
6. **Kaplan–Meier** — выживаемость с метками цензурирования, без «1 − KM».

Ключевой фрагмент:

```
agg <- aggregate(sbp_mm_hg ~ group + week, df,
                 function(x) c(mean = mean(x), sd = sd(x), n = length(x)))
agg <- do.call(data.frame, agg); agg$ci <- 1.96 * agg$sd / sqrt(agg$n)

ggplot(agg, aes(week, mean, color = group)) +
  geom_errorbar(aes(ymin = mean - ci, ymax = mean + ci), width = 0.35) +
  geom_line() + geom_point() +
  scale_color_manual(values = c("#7F7F7F", "#0072B2", "#D55E00")) +
  labs(x = "Неделя", y = "САД, мм рт. ст.")
```

Сильные места R здесь — `geom_ribbon`/`geom_errorbar`, `facet_wrap`, `survival::survfit` и `gridExtra` для панелей. Слабые — необходимость помнить про факторы, порядок уровней и `stringsAsFactors`.

C. Шесть графиков в Julia

Скрипт: `09_appendices/julia/six_plots.jl`. Проект: `09_appendices/julia_env/`. Запуск:

```
JULIA_DEPOT_PATH="$PWD/.julia:$HOME/.julia" \
julia --project=09_appendices/julia_env 09_appendices/julia/six_plots.jl
```

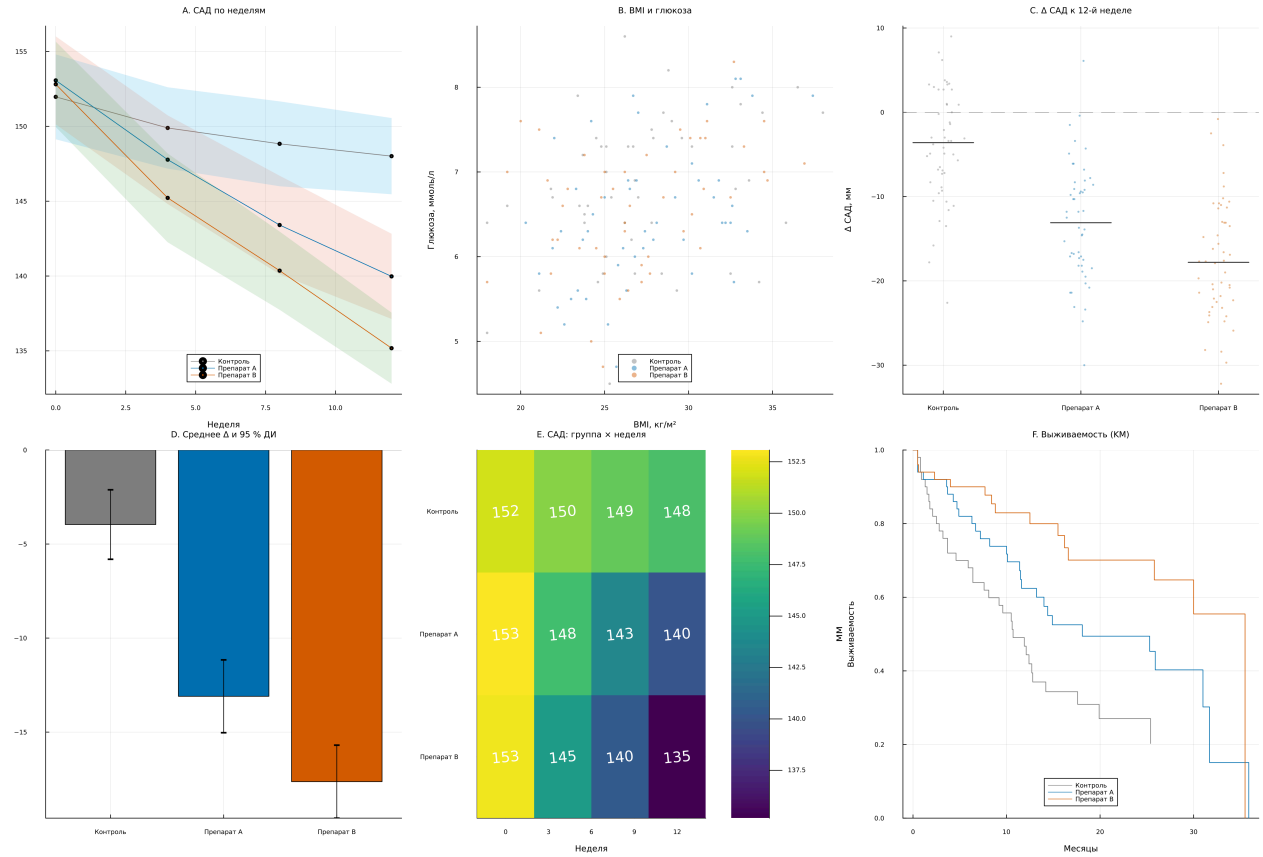


Рисунок 9.2. Шесть графиков в Julia: линия с ribbon, scatter, точки с медианой, столбцы с ошибками, heatmap и KM.

Что показано:

1. **Линия + ribbon** — среднее САД и 95 % CI; `ribbon` рисует полосу.
2. **Scatter** — BMI и глюкоза, цвет по группе; без ложного сглаживания.
3. **Точки + медиана** — индивидуальные изменения и медиана группы.
4. **Столбцы с ошибками** — среднее изменение и 95 % CI.
5. **Heatmap** — среднее САД, значения подписаны.
6. **Kaplan–Meier** — шаговая выживаемость по группам.

Ключевой фрагмент:

```
using Plots, DataFrames, CSV, Statistics
df = CSV.read("data/09_appendix_dataset.csv", DataFrame)
agg = combine(groupby(df, [:group, :week]),
              :sbp_mm_hg => mean => :mean,
              :sbp_mm_hg => std => :sd,
              :sbp_mm_hg => length => :n)
agg.ci = 1.96 .* agg.sd ./ sqrt.(agg.n)

plot(agg.week, agg.mean, group = agg.group, ribbon = agg.ci,
     marker = :circle, linecolor = [:gray, :blue, :orange])
```

Julia требует больше дисциплины к окружению: отдельный проект, `Manifest.toml`, предкомпиляция. Зато `Plots`/`Makie` хорошо ложатся на большие массивы и математические модели; для интерактивных отчётов чаще берут Makie или связку с Python.

D. Шесть графиков в Excel / LibreOffice

Книга: `09_appendices/excel/make_workbook.py` собирает `.xlsx` с шестью листами-графиками; данные лежат на скрытых листах. Запуск:

```
uv run python 09_appendices/excel/make_workbook.py
```

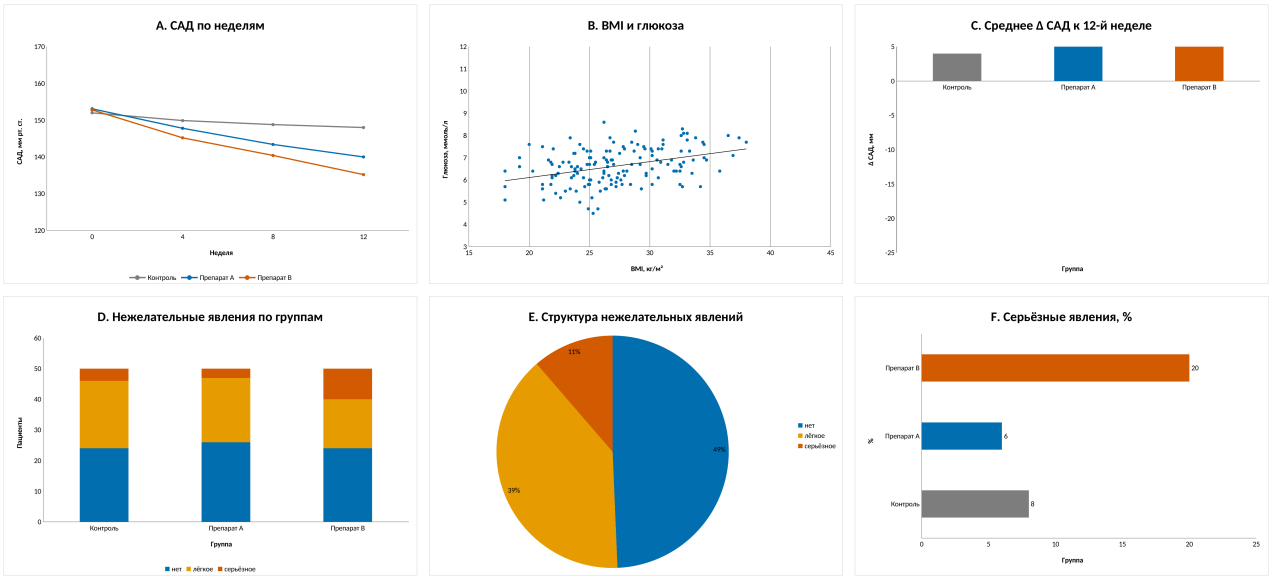


Рисунок 9.3. Шесть графиков в Excel: линия, scatter с трендом, столбцы, stacked, pie и bar.

Что показано:

1. **Линия** — среднее САД по неделям, три серии, ось 120–170.
2. **Scatter** — BMI и глюкоза, линейный тренд, без соединительных линий.
3. **Столбцы** — среднее изменение САД по группам, нулевая линия.
4. **Stacked columns** — нежелательные явления по группам.
5. **Pie** — структура явлений, 3 доли, проценты.
6. **Bar** — доля серьёзных явлений по группам, значения подписаны.

В Excel это делается через **Вставка → Диаграмма**:

График	**Тип Excel**	**Настройки**
Линия	Line	без 3D, маркеры, ось Y 120–170
Scatter	XY (Scatter)	только маркеры, линейный тренд
Столбцы	Column	нулевая линия, ось Y −25…5
Stacked	Stacked Column	100 % перекрытие, 3 серии
Pie	Pie	3 доли, проценты, без 3D
Bar	Bar	горизонтально, значения, ось 0–30

Excel силён в простых формах и быстрой правке. Но у него нет нормального box plot с точками, KM и heatmap «из коробки»; для них нужны надстройки, Python/R или ручная сборка. Плюс версии файла и «ручные» правки плохо проверяются в ревью — поэтому книгу лучше генерировать скриптом, а не править мышью.

E. Что выбрать

- **R** — если статистика, модели и отчёт уже в R; ggplot2 + survival.
- **Julia** — если важны скорость, типы и математическая модель.
- **Excel** — если нужен быстрый черновик, таблица или разговор с коллегами; для финальных журнальных рисунков — редко.
- **Python** — если нужен единый пайплайн «данные → рисунки → PDF» и автоматическая проверка, как в этой методичке.

Хорошая практика — не выбирать «одну среду навсегда», а разделять: расчёт в R/Julia/Python, просмотр в Excel, финальная вёрстка в Typst/LaTeX/Illustrator. Тогда сильные стороны каждой среды работают на результат, а не спорят друг с другом.

F. Перенос графика между средами

При переносе одного и того же рисунка из R в Julia, из Julia в Excel или из любой среды в журнал проверяйте не «похожесть картинки», а инварианты:

Что проверить	Почему это важно
Уровни групп и порядок	R и Julia по-разному сортируют факторы; порядок меняет легенду и цвета
Неделя как число	В Excel дата/текст легко превращается в категорию
Пропуски	`NA`, `missing` и пустая ячейка — разные вещи
Определение CI	95 % CI среднего, Уилсона или бутстрэп — не взаимозаменяемы
Шрифт и кириллица	Проверяйте встраивание и наличие глифов в PDF
Размер в мм	85/170 мм, а не «по ширине окна»
Цвет и печать	CVD, серый, белый фон, 3:1 для линий
Версии и seed	Без фиксации версий и seed рисунок не воспроизводится

Практический минимум: сохраните исходные данные, скрипт, версии пакетов и финальный PDF/SVG. Excel-файл без скрипта — это не воспроизводимый артефакт, а иллюстрация.

Источники

- R Core Team. *R: A Language and Environment for Statistical Computing*. — r-project.org
- Wickham H. *ggplot2: Elegant Graphics for Data Analysis*. 3rd ed., Springer, 2024. — ggplot2.tidyverse.org
- Therneau T. *A Package for Survival Analysis in R*. — cran.r-project.org
- Julia Language. *Documentation*. — docs.julialang.org
- Plots.jl. *Documentation*. — docs.juliaplots.org
- Microsoft. *Create a chart from start to finish*. — support.microsoft.com
- LibreOffice. *Charts*. — help.libreoffice.org